Introduction to Quant Finance

Advanced Module

Quant Club IIT KGP

Summer of Quant

Advanced Module Study Notes

July 27, 2026

Contents

1 Algorithmic Trading **16**
- 1.1 What is Backtesting? . . . 16
  - 1.1.1 Definition . . . 16
  - 1.1.2 Why Backtest? . . . 16
  - 1.1.3 The Backtesting Principle . . . 17
  - 1.1.4 Requirements of a Good Backtest . . . 17
- 1.2 Performance Metrics for Backtesting a Trading Strategy . . . 18
  - 1.2.1 Overview . . . 18
  - 1.2.2 Cumulative Return (Absolute Return) . . . 18
  - 1.2.3 Annualised Return . . . 18
  - 1.2.4 Annualised Volatility . . . 19
  - 1.2.5 Sharpe Ratio . . . 19
  - 1.2.6 Sortino Ratio . . . 20
  - 1.2.7 Beta . . . 20
  - 1.2.8 Maximum Drawdown . . . 21
  - 1.2.9 Summary of Performance Metrics . . . 22
- 1.3 Slippage and Latency . . . 22
  - 1.3.1 What is Slippage? . . . 22
  - 1.3.2 Why Slippage Occurs . . . 23
  - 1.3.3 Three Drivers of Slippage . . . 23
- 1.4 Transaction Cost Models . . . 24
  - 1.4.1 Components of Transaction Costs . . . 24
  - 1.4.2 Flat or Fixed Cost Model . . . 25
  - 1.4.3 Linear Cost Model . . . 25
  - 1.4.4 Quadratic Cost Model . . . 25
  - 1.4.5 Using Historical Execution Costs . . . 26
- 1.5 Backtesting Framework . . . 26
  - 1.5.1 Complete Workflow . . . 26
  - 1.5.2 Common Backtesting Pitfalls . . . 27
- 1.6 Libraries . . . 28
  - 1.6.1 Algorithmic Trading Libraries . . . 28
  - 1.6.2 Pyfolio . . . 29
  - 1.6.3 Zipline . . . 35
  - 1.6.4 Backtesting.py . . . 45
  - 1.6.5 Putting It Together . . . 52

2 Stochastic Processes **54**
- 2.1 Deterministic vs. Stochastic Models . . . 54
  - 2.1.1 Deterministic Models . . . 54
  - 2.1.2 Stochastic Models . . . 54
  - 2.1.3 Comparison . . . 54
- 2.2 Mathematical Foundations of Stochastic Processes . . . 55

2.2.1 Probability Spaces . . . 55
2.2.2 Measurable Functions . . . 56
2.2.3 Formal Definition . . . 56
2.2.4 Sample Paths . . . 57
2.2.5 Stationarity . . . 57
2.2.6 Stationary and Independent Increments . . . 58
2.2.7 Filtrations: Information Through Time . . . 59
2.2.8 Adapted Processes . . . 59
2.2.9 Moment-Generating Functions . . . 60
2.2.10 Characteristic Functions . . . 60
2.2.11 Gaussian Random Variables . . . 60
2.2.12 Random Vectors and Covariance Matrices . . . 61
2.2.13 Gaussian Random Vectors . . . 61
2.2.14 Martingales . . . 62
2.3 Concept Map . . . 63

3 Markov Chains **64**
3.1 Motivation . . . 64
3.1.1 Why the Independence Assumption Can Fail . . . 64
3.2 Core Structure and Probability Propagation . . . 65
3.2.1 Definition and Markov Property . . . 65
3.2.2 Transition Matrices . . . 65
3.2.3 State-Distribution Vectors . . . 66
3.2.4 Chapman–Kolmogorov Equations . . . 67
3.2.5 Twelve-Month Loan Transition Probabilities . . . 67
3.3 State Classification and Long-Run Behaviour . . . 68
3.3.1 Accessibility, Communication, and Irreducibility . . . 68
3.3.2 Recurrence, Transience, and Absorption . . . 68
3.3.3 Periodicity . . . 68
3.3.4 Stationary Distributions and Ergodicity . . . 69
3.4 Model Design and Estimation . . . 69
3.4.1 Choosing States in Quantitative Finance . . . 69
3.4.2 Maximum Likelihood Estimation: General Idea . . . 70
3.4.3 Deriving the Markov-Chain MLE . . . 70
3.4.4 Loan-Portfolio MLE Example . . . 71
3.4.5 Assumptions Behind the Estimator . . . 71
3.4.6 Large-Sample Properties of the MLE . . . 72
3.5 Limitations and Extensions . . . 72

4 Random Walk **73**
4.1 Simple Symmetric Random Walk . . . 73
4.1.1 Definition and Rule . . . 73
4.1.2 Coin-Flip Analogy . . . 73
4.1.3 The Growing Tree of Possible Positions . . . 73
4.2 Counting Random-Walk Paths . . . 74
4.2.1 Paths Between Two Space–Time Points . . . 74
4.2.2 Total Number of Paths . . . 74
4.2.3 Number of Paths to a Fixed Endpoint . . . 75

4.2.4 A Useful Recursion . . . 75
4.2.5 Translation Invariance . . . 75
4.3 Return Probabilities . . . 76
4.4 The Reflection Principle . . . 76
4.4.1 Why the Reflection Principle Works . . . 77
4.4.2 Counting Paths That Avoid a Barrier . . . 77
4.5 Interpretation and Limitations in Finance . . . 78
4.6 Random Walk as the Discrete Precursor to Brownian Motion . . . 78

5 Brownian Motion **80**
5.1 Standard Brownian Motion . . . 80
5.1.1 History . . . 80
5.1.2 Definition . . . 80
5.1.3 Existence . . . 81
5.1.4 Brownian Motion as a Limit of Random Walks . . . 82
5.1.5 Finite-Dimensional Distributions . . . 82
5.1.6 Simple Transformations . . . 83
5.1.7 Irregularity . . . 84
5.1.8 The Markov Property and Stopping Times . . . 85
5.1.9 The Reflection Principle . . . 86
5.1.10 Martingales . . . 87
5.1.11 Maximums and Hitting Times . . . 87
5.1.12 Zeros and Arcsine Laws . . . 89
5.1.13 The Law of the Iterated Logarithm . . . 91
5.1.14 Computational Exercise . . . 91
5.2 Brownian Motion with Drift and Scaling . . . 92
5.2.1 Definition . . . 92
5.2.2 Finite-Dimensional Distributions . . . 92
5.2.3 Transformations . . . 93
5.2.4 The Markov Property and Stopping Times . . . 94
5.3 The Brownian Bridge . . . 95
5.3.1 Definition and Constructions . . . 95
5.3.2 The General Brownian Bridge . . . 97
5.3.3 Applications: The Empirical Distribution Function . . . 98
5.4 Geometric Brownian Motion . . . 99
5.4.1 Definition . . . 99
5.4.2 Distributions . . . 100
5.4.3 Moments . . . 100
5.4.4 Asymptotic Behavior . . . 101
5.4.5 Martingale Property . . . 101
5.5 Modelling Stock Price . . . 102
5.6 Itô's Lemma and Girsanov's Theorem . . . 102
5.7 Factors Affecting Option Prices . . . 102
5.8 Black–Scholes . . . 102
5.9 Deriving the Black–Scholes Equation . . . 102
5.10 Options Pricing . . . 102
5.11 Binomial Model . . . 102
5.12 Option Greeks . . . 102

6 Monte Carlo Simulation **103**
6.1 The Big Picture . . . 103
6.2 Foundations: Random Variables . . . 103
6.2.1 The Normal Random Variable . . . 104
6.2.2 Mean and Variance . . . 104
6.3 The Law of Large Numbers . . . 104
6.3.1 Dice-Roll Example . . . 105
6.3.2 Estimating Other Statistics . . . 105
6.3.3 Recovering an Entire Distribution . . . 106
6.3.4 Randomness Versus Uncertainty . . . 106
6.4 What Monte Carlo Simulation Is . . . 106
6.4.1 Bernoulli and Exponential Examples . . . 106
6.4.2 Geometric Example: Rolls Until the First Six . . . 107
6.5 Extending the LLN to Stochastic Processes . . . 107
6.6 Brownian Motion: A Quick Recap . . . 108
6.7 Worked Example: Estimating a Fair Game Value . . . 108
6.7.1 Estimating Event Probabilities . . . 109
6.8 Applying Monte Carlo to Option Pricing . . . 110
6.8.1 The Black–Scholes Equation and Two Pricing Views . . . 110
6.8.2 Why Simulation Is Valid . . . 110
6.8.3 Euler–Maruyama Discretization . . . 110
6.8.4 General Monte Carlo Pricing Recipe . . . 111
6.8.5 Deriving the Black–Scholes Formula . . . 111
6.8.6 Monte Carlo European Call: Numerical Example . . . 112
6.8.7 Path-Dependent and Exotic Payoffs . . . 113
6.9 Analytical and Monte Carlo Methods . . . 114
6.10 Learning Fast Pricing Functions from Simulation . . . 114
6.11 Challenges in Real-World Finance . . . 115
6.12 Summary . . . 115

7 Modelling Stock Prices **117**
7.1 Why Stock Prices Need a Stochastic Model . . . 117
7.1.1 Visual Motivation . . . 117
7.2 Brownian Motion: Recap . . . 117
7.2.1 Definition and Historical Origin . . . 117
7.2.2 Single and Multiple Sample Paths . . . 118
7.2.3 Distribution at a Fixed Time . . . 118
7.3 Why Standard Brownian Motion Is Not a Stock-Price Model . . . 118
7.4 Brownian Motion with Drift . . . 119
7.4.1 Constructing the Model . . . 119
7.4.2 Distribution . . . 119
7.5 Geometric Brownian Motion . . . 120
7.5.1 Exponentiation and Positivity . . . 120
7.5.2 The GBM Stochastic Differential Equation . . . 120
7.5.3 Illustrative Paths . . . 121
7.5.4 Distribution of Log Returns . . . 121
7.6 Estimating GBM Parameters from Data . . . 121
7.6.1 Taking Logarithms . . . 121

7.6.2 Eliminating the Initial Price . . . 122
7.6.3 Mean and Variance . . . 122
7.6.4 Sample Estimators . . . 122
7.6.5 Marks & Spencer Workflow . . . 123
7.6.6 Illustrative Parameter Estimates . . . 123
7.6.7 Validation by Simulation . . . 123
7.6.8 Limitations of GBM . . . 123
7.7 Financial Interpretation of Log Differences . . . 124
7.7.1 From a Difference to a Ratio . . . 124
7.7.2 Taylor Approximation . . . 124
7.7.3 Simple and Log Returns . . . 125
7.8 Summary of the Stock-Price Modelling Journey . . . 125

8 Itô's Lemma and Girsanov's Theorem 126
8.1 Motivation . . . 126
8.2 From Taylor's Formula to Stochastic Calculus . . . 126
8.2.1 The Ordinary Taylor Expansion . . . 126
8.2.2 An Ordinary-Calculus Example . . . 126
8.2.3 Why Brownian Motion Is Different . . . 127
8.3 Itô's Lemma . . . 127
8.3.1 Example: Squared Brownian Motion . . . 128
8.4 Solving Geometric Brownian Motion . . . 128
8.5 Girsanov's Theorem . . . 129
8.5.1 Changing the Probability Measure . . . 129
8.5.2 Risk-Neutral Stock Dynamics . . . 129
8.6 How the Ideas Connect . . . 130
8.7 Key Formula Reference . . . 130
8.8 Core Takeaways . . . 130

9 Factors Affecting Option Prices 132
9.1 Option Price Fundamentals . . . 132
9.1.1 The Option Price Equation . . . 132
9.1.2 Intrinsic Value . . . 132
9.1.3 Extrinsic Value (Time Value) . . . 133
9.2 The Six Key Factors Affecting Option Prices . . . 133
9.2.1 Underlying Price (Related Greek: Delta) . . . 133
9.2.2 Strike Price . . . 134
9.2.3 Time Until Expiration (Related Greek: Theta) . . . 135
9.2.4 Volatility of the Underlying (Related Greek: Vega) . . . 135
9.2.5 Dividends . . . 136
9.2.6 Interest Rates (Related Greek: Rho) . . . 136
9.3 Volatility — A Deep Dive . . . 137
9.3.1 Volatility as a Measure of Risk . . . 137
9.3.2 Historical Volatility vs. Implied Volatility . . . 138
9.3.3 Computing Historical Volatility: Step-by-Step . . . 138
9.4 A History of Volatility Models . . . 139
9.4.1 Pre-Black-Scholes: Bachelier's Model (1900) . . . 140
9.4.2 Black-Scholes Model (1973) — The Cornerstone . . . 140

9.4.3 The Volatility Surface: Theory vs. Reality . . . 141
9.4.4 The Stochastic Volatility Era (Post-1973) . . . 142
9.4.5 Asset Class Specific Volatility Models . . . 142
9.4.6 Rough Volatility — The Frontier . . . 144
9.5 Volatility Modeling Techniques . . . 146
9.5.1 Historical Approaches (Backward-Looking) . . . 146
9.5.2 Geometric Brownian Motion (GBM) as a Volatility Framework . . . 146
9.6 The ARCH Model . . . 147
9.6.1 Motivation: The Problem with Constant Variance . . . 147
9.6.2 The ARCH Framework . . . 148
9.6.3 ARCH(1): The Basic Model . . . 148
9.6.4 ARCH(2) and ARCH(p): Higher Order Models . . . 149
9.6.5 Configuring ARCH: ACF and PACF of Squared Residuals . . . 150
9.6.6 Python Implementation: ARCH(1) . . . 150
9.7 The GARCH Model . . . 151
9.7.1 From ARCH to GARCH: Adding Persistence . . . 151
9.7.2 GARCH(1,1) — The Workhorse . . . 151
9.7.3 GARCH(p,q) — General Form . . . 152
9.7.4 GARCH Configuration: ACF and PACF . . . 152
9.7.5 Heavy Tails in GARCH . . . 153
9.7.6 Python Implementation: GARCH(1,1) . . . 153
9.7.7 GARCH Family — Extensions . . . 154
9.7.8 ARCH vs. GARCH vs. GBM: When to Use Which . . . 154
9.8 Interest Rate Modeling . . . 154
9.8.1 Why Interest Rates Need Their Own Models . . . 154
9.8.2 Properties of Interest Rate Processes . . . 155
9.8.3 Stochastic Differential Equations (SDEs) — The Mathematical Tool 155
9.8.4 Specifying the Drift: Mean Reversion . . . 156
9.8.5 Specifying the Volatility: The CEV Framework . . . 156
9.8.6 One-Factor Interest Rate Models . . . 157
9.8.7 The Full One-Factor Mean-Reverting Model . . . 158
9.8.8 Arbitrage-Free vs. Equilibrium Models . . . 158
9.9 The Vasicek Interest Rate Model . . . 160
9.9.1 Overview . . . 160
9.9.2 The Vasicek SDE . . . 160
9.9.3 Mean Reversion Mechanics . . . 160
9.9.4 Discrete Form and OLS Estimation . . . 161
9.9.5 Forecasting Mean with the Vasicek Model . . . 162
9.9.6 Volatility Forecasting with the Vasicek Model . . . 162
9.9.7 Applications of the Vasicek Model . . . 163
9.9.8 Limitations of the Vasicek Model . . . 164
9.10 The Vasicek Model for Credit Risk . . . 164
9.10.1 The Problem: Portfolio-Level Default Risk . . . 164
9.10.2 The Gaussian Copula and Factor Structure . . . 164
9.10.3 Loss Distribution and Worst-Case Default Rate . . . 165
9.10.4 The Worst-Case Default Rate (WCDR) Formula . . . 166
9.10.5 Numerical Example . . . 166
9.10.6 Capital Requirement Calculation . . . 167

9.10.7 Summary of Vasicek Credit Model Assumptions . . . 168
9.11 Summary and Quick Reference . . . 168
9.11.1 The Six Factors at a Glance . . . 168
9.11.2 Volatility Modeling Landscape . . . 169
9.11.3 Interest Rate Models at a Glance . . . 169
9.11.4 Key Formulas Reference . . . 170

10 Black–Scholes Model **171**
10.1 Overview . . . 171
10.2 Historical Origins and Development . . . 171
10.2.1 Options before Black–Scholes: Thales of Miletus (circa 600 BC) . . . 171
10.2.2 Louis Bachelier and the random walk (1900) . . . 171
10.2.3 Brownian motion: Robert Brown and Einstein . . . 172
10.2.4 Ed Thorp: from blackjack to Wall Street (1950s–1960s) . . . 173
10.2.5 Black, Scholes, and Merton (1973) . . . 173
10.2.6 Real-world use cases after Black-Scholes . . . 174
10.2.7 Jim Simons and Renaissance Technologies . . . 175
10.3 Options Basics (Building Blocks) . . . 176
10.3.1 Worked numeric example (call option) . . . 176
10.3.2 Three advantages of options . . . 176
10.3.3 Options as insurance — long put and long call . . . 177
10.3.4 Short call example . . . 177
10.4 What the Black-Scholes Model Does . . . 177
10.5 Key Input Variables . . . 177
10.6 The Black-Scholes Formula . . . 178
10.7 Assumptions of the Black-Scholes Model . . . 179
10.8 The Mathematics Behind the Model . . . 179
10.8.1 Geometric Brownian Motion (GBM) . . . 179
10.8.2 Warm-up: the deterministic case . . . 179
10.8.3 Bringing back randomness: Itô's Lemma . . . 180
10.8.4 Solving Geometric Brownian Motion with Itô's Lemma . . . 180
10.9 Deriving the Black-Scholes Equation: Replicating Portfolios and Delta Hedging . . . 181
10.9.1 The core idea . . . 181
10.9.2 Setting up the portfolio (formal version) . . . 181
10.9.3 Solving for A (delta) and B . . . 181
10.9.4 The Black-Scholes equation and formula . . . 182
10.9.5 Alternative framing: hedging a short option position directly . . . 182
10.10 Option Greeks . . . 183
10.11 Volatility Skew . . . 184
10.12 Benefits and Limitations . . . 184
10.12.1 Benefits . . . 184
10.12.2 Limitations . . . 185
10.13 Quick-Reference Summary Table: Key People . . . 185
10.14 Summary . . . 186

11 Deriving the Black–Scholes Equation **187**
11.1 Overview . . . 187
11.2 Derivation 1 — The PDE Approach (Replicating Portfolio / Delta Hedging) 187
11.2.1 Step 1 — Stock Price Dynamics . . . 187
11.2.2 Step 2 — Defining the Derivative's Value . . . 187
11.2.3 Step 3 — Constructing a Riskless Portfolio . . . 188
11.2.4 Step 4 — Eliminating Risk (Delta Hedging) . . . 188
11.2.5 Step 5 — Applying the No-Arbitrage Principle . . . 189
11.2.6 Step 6 — The Black-Scholes Equation . . . 189
11.3 Derivation 2 — The Risk-Neutral Pricing / Expectation Approach . . . 189
11.3.1 Recap: How the Underlying Evolves . . . 189
11.3.2 Two Approaches to Pricing: Risky vs. Risk-Neutral . . . 190
11.3.3 The Risk-Neutral Pricing Formula . . . 190
11.3.4 Expanding the Expectation . . . 190
11.3.5 Writing the Expectation as an Integral . . . 191
11.3.6 The Problem: This Is a "Normal" Expectation, Not a Risk-Neutral One . . . 191
11.3.7 The Resulting Master Pricing Equation . . . 191
11.4 Recap: The Black-Scholes-Merton Formula . . . 192
11.5 Understanding Volatility in the Model . . . 192
11.5.1 Worked Comparison: Oil ETF vs. Cow (Live Cattle) ETF . . . 192
11.5.2 Volatility as Both an Implied Input and a Standard Deviation . . . 193
11.6 High-Level, Intuitive Reading of the Formula . . . 193
11.6.1 Discounting to Present Value . . . 194
11.6.2 The Probability Terms . . . 194
11.7 Worked Numerical Example . . . 194
11.8 Practice Problem: Backing Out Implied Volatility . . . 195
11.9 Delta Hedging — Eliminating Risk in Practice . . . 196
11.10 Criticisms of the Black-Scholes-Merton Model . . . 196
11.10.1 The Volatility Smile . . . 197
11.11 Advanced Technical Notes (Notation and Modeling Choices) . . . 197
11.11.1 The Payoff Functions, Formally . . . 198
11.11.2 Closed-Form via the Error Function (erf) . . . 198
11.12 Summary . . . 199

12 Option Pricing **200**
12.1 Options Pricing — Overview . . . 200
12.1.1 What Is Options Pricing? . . . 200
12.1.2 The Goal of Options Pricing . . . 200
12.1.3 Common Option Pricing Models . . . 200
12.1.4 Recommended Study Approach . . . 200
12.2 Foundational Concepts in Derivative Pricing . . . 201
12.2.1 The Core Intuition: A Simple Analogy . . . 201
12.2.2 Arbitrage . . . 201
12.2.3 Replication . . . 201
12.2.4 Risk Neutrality . . . 202
12.2.5 Value vs. Price — A Critical Distinction . . . 202

12.2.6 Monetary vs. Non-Monetary Benefits (and Costs) of Holding the Underlying . . . 203
12.2.7 The Cost-of-Carry Model . . . 203
12.3 Forward and Futures Contracts — Detailed Pricing and Valuation . . . 203
12.3.1 Key Definitions . . . 203
12.3.2 The No-Arbitrage Approach . . . 204
12.3.3 Forward Pricing (No Interim Cash Flows) . . . 204
12.3.4 Forward Mispricing and Arbitrage . . . 204
12.3.5 Valuing a Forward Contract (No Cash Flows on Underlying) . . . 205
12.3.6 Equity Forward Contracts . . . 206
12.3.7 Forward and Future Contracts on Fixed-Income Bonds . . . 207
12.3.8 Futures Contracts on Fixed-Income Bonds . . . 207
12.3.9 Interest Rate Forward and Futures Contracts . . . 208
12.3.10 Currency Forward and Futures Contracts . . . 210
12.3.11 Futures Contracts — General Note . . . 211
12.3.12 Practice Questions (Forward Valuation) . . . 211
12.4 Options — Basic Concepts, Moneyness, and Valuation . . . 211
12.4.1 Recap of Basic Option Concepts . . . 211
12.4.2 Moneyness . . . 212
12.4.3 Intrinsic Value and Time Value . . . 212
12.4.4 Theoretical Bounds for Option Values . . . 213
12.4.5 Six Key Factors Affecting European Option Values . . . 213
12.5 Put-Call Parity . . . 214
12.5.1 The Replication Argument . . . 214
12.5.2 Rearranging: Synthetic Positions . . . 215
12.5.3 Worked Example . . . 215
12.5.4 Put-Call-Forward Parity . . . 215

13 Binomial Model 216
13.6 The Binomial Option Pricing Model — Core Theory . . . 216
13.6.1 What Is the Binomial Model, and Why Use It? . . . 216
13.6.2 Key Inputs and Notation . . . 216
13.6.3 The One-Step Binomial Tree — Setup . . . 217
13.6.4 Deriving the Risk-Neutral Probability (Full Algebra) . . . 217
13.6.5 Worked Numeric Example (One-Step Tree, Discrete Compounding) 218
13.6.6 Alternative Method: The Replicating Portfolio (Delta-Hedging) Approach . . . 218
13.6.7 An Equivalent One-Period Model — Alternate Notation (as taught in the derivatives-basics lecture) . . . 219
13.6.8 American vs. European Options in the Binomial Framework . . . 220
13.7 The Binomial Model — Full Generalization and Multi-Step Trees . . . 220
13.7.1 Comparing Black-Scholes and the Binomial Model . . . 220
13.7.2 Why the Actual Probability of Up/Down Doesn't Matter — A Clean Illustration . . . 220
13.7.3 Binomial Calculation by Replicating Portfolio . . . 221
13.7.4 Full Mathematical Generalization . . . 222
13.7.5 Multi-Step (Two-Step, Three-Step, ...) Binomial Trees . . . 223
13.7.6 Worked Example #1 — Two-Step Binomial Tree (Put Option) . . . 224

13.7.7 Worked Example #2 — Three-Step Binomial Tree (European Put) 224
13.7.8 When Is the Binomial Model the Right Choice? 225
13.7.9 Effectiveness and Limitations of the Binomial Model 225
13.7.10 Other Applications of the Binomial Model (Beyond Plain Options) 225
13.7.11 Embedded Options 226
13.8 Consolidated Formula Cheat-Sheet 226
13.9 Sources Referenced in These Notes 227

14 Option Greeks 228
14.1 Why We Need the Greeks 228
14.1.1 Quick refresher on options 229
14.1.2 Context: the Black–Scholes–Merton (BSM) model 229
14.2 Delta (Δ) — Directional Sensitivity 230
14.2.1 Definition 230
14.2.2 Sign and range 230
14.2.3 BSM formula 230
14.2.4 Delta as a hedge ratio 231
14.2.5 Worked examples 231
14.2.6 Portfolio Delta 231
14.2.7 Delta of Forwards and Futures 231
14.2.8 Covered vs Naked Positions — Why Hedging Matters 231
14.2.9 Worked Delta-Hedging Examples 232
14.3 Gamma (Γ) — The Acceleration of Delta 233
14.3.1 Definition 233
14.3.2 Behavior 233
14.3.3 Why Gamma matters 233
14.3.4 Worked Examples 233
14.3.5 Gamma-Neutral Hedging 234
14.3.6 The Delta–Theta–Gamma Relationship (BSM PDE) 234
14.4 Theta (Θ) — Time Decay 234
14.4.1 Definition 234
14.4.2 Sign convention 235
14.4.3 Behavior 235
14.4.4 Worked Examples 235
14.5 Vega (ν) — Volatility Sensitivity 235
14.5.1 Definition 235
14.5.2 Implied Volatility vs. Historical Volatility 236
14.5.3 Behavior 236
14.5.4 Worked Examples 236
14.6 Rho (ρ) — Interest Rate Sensitivity 236
14.6.1 Definition 236
14.6.2 Sign convention and behavior 236
14.6.3 Practical significance 237
14.7 Summary Table of the Five Greeks 237
14.8 Practical Risk Management With the Greeks (Applied Trading Framework) 237
14.8.1 The core mental model: "Risk = Greeks" 237
14.8.2 Two pre-trade steps, before you ever place an order 237
14.8.3 Choosing a strike: let current Implied Volatility decide 238

14.8.4 Greek behavior and biggest concern, by moneyness . . . 238
14.8.5 Worked example: managing an ATM position as conditions drift . 239
14.8.6 A cautionary tale on *not* doing this . . . 239
14.9 What Actually Drives Volatility: Supply, Demand, and Market Participants 240
14.9.1 Hedgers . . . 240
14.9.2 Speculators . . . 240
14.9.3 Arbitrageurs . . . 240
14.9.4 What causes volatility to fall . . . 240
14.9.5 Greed, fear, and news . . . 241
14.9.6 "Buying" and "Selling" volatility — trader shorthand . . . 241
14.10 Classifying Options Strategies by Direction-Dependence . . . 241
14.11 Key Takeaways . . . 242

15 Sentiment Analysis: Opinion Mining 244
15.1 Introduction . . . 244
15.2 Growth and Applications . . . 244
15.3 Books and General Resources . . . 244
15.4 Selected Research Papers . . . 244
15.5 Training Resources . . . 245

16 Datasets for Self-Practice 246
16.1 Review and Social-Media Datasets . . . 246
16.2 Sentiment-Analysis Lexicons . . . 246
16.2.1 Loughran–McDonald Master Dictionary (1993–2023) . . . 246

17 Time Series Analysis: Handling Volatility 248
17.1 Why Should We Care About Modelling Volatility? . . . 248
17.2 How Do We Measure Volatility? . . . 248
17.3 Background for ARCH/GARCH Models . . . 249
17.3.1 The Efficient Market Hypothesis (EMH) . . . 249
17.3.2 Returns vs. Prices . . . 249
17.3.3 Volatility Clustering . . . 250
17.4 Heteroskedasticity — The Core Problem . . . 250
17.5 The ARCH Model . . . 250
17.5.1 What Does "ARCH" Mean? . . . 250
17.5.2 Why Use an ARCH Model? . . . 251
17.5.3 ARCH Model Statement and Derivation . . . 251
17.5.4 Interpretation of the ARCH Model . . . 252
17.5.5 Formal Model Statement (Alternative Notation) . . . 252
17.6 The GARCH Model . . . 253
17.6.1 What Does "GARCH" Mean? . . . 253
17.6.2 GARCH Model Statement and Derivation . . . 253
17.6.3 Interpretation of GARCH Coefficients . . . 254
17.6.4 Alternative Notation (used in the second source article) . . . 254
17.7 Model Identification: Using ACF and PACF . . . 254
17.7.1 General Procedure . . . 254
17.7.2 Worked Example (S&P 500 Returns) . . . 254
17.8 Formal Testing for ARCH/GARCH Effects: The McLeod-Li Test . . . 255
17.9 Simulating ARCH and GARCH Processes in Python . . . 256

17.9.1 Simulating an ARCH(1) Process . . . 256
17.9.2 Squared ARCH(1) Series . . . 256
17.9.3 Simulating a GARCH(1,1) Process . . . 256
17.9.4 Squared GARCH(1,1) Series . . . 257
17.9.5 Fitting a GARCH(1,1) Model to the Simulated Series . . . 257
17.10 Applying GARCH to Real Data (S&P 500) . . . 258
17.10.1 Data Import . . . 258
17.10.2 Fitting GARCH(1,1) on Absolute Returns . . . 258
17.11 Forecasting Volatility . . . 259
17.11.1 N-Step-Ahead Forecast . . . 259
17.11.2 One-Step-Ahead Rolling Forecast . . . 260
17.11.3 Comparison . . . 260
17.12 Model Diagnostics . . . 260
17.12.1 Jarque-Bera Test (Normality of Residuals) . . . 261
17.12.2 Implication — Heavy Tails in Financial Data . . . 261
17.13 Model Extensions and Related Techniques . . . 262
17.13.1 Combining ARIMA with GARCH . . . 262
17.14 Key Takeaways / Summary . . . 263
17.15 References . . . 264

18 Sequence-to-Sequence Modelling in Finance 265
18.1 What is Sequence-to-Sequence (Seq2Seq) Modelling? . . . 265
18.1.1 How is Seq2Seq used in Time Series? . . . 265
18.2 Recurrent Neural Networks (RNNs) — Foundations . . . 266
18.2.1 From Feedforward Networks to Recurrence . . . 266
18.2.2 Formal Definition . . . 266
18.2.3 Visualizing the Recurrence ("Network Fold") . . . 267
18.2.4 Types of RNNs (by Input/Output Shape) . . . 268
18.2.5 Applications of RNNs . . . 268
18.3 The Vanishing (and Exploding) Gradient Problem . . . 268
18.3.1 What Is It? . . . 268
18.3.2 Approaches to Mitigate It . . . 269
18.4 Gated Recurrent Units: LSTM and GRU . . . 269
18.4.1 Long Short-Term Memory (LSTM) . . . 269
18.4.2 Gated Recurrent Unit (GRU) . . . 270
18.4.3 LSTM vs. GRU — Similarities and Differences . . . 271
18.5 Empirical Comparison: LSTM vs. GRU vs. tanh-RNN . . . 272
18.5.1 Goal . . . 272
18.5.2 Datasets and Tasks . . . 272
18.5.3 Model Setup . . . 272
18.5.4 Results . . . 273
18.5.5 Key Findings . . . 273
18.6 Practical Implementation: RNN/LSTM for Time-Series Forecasting (Python) 274
18.6.1 Step 1 — Load and Prepare the Data . . . 274
18.6.2 Step 2 — Feature Scaling (Normalization) . . . 274
18.6.3 Step 3 — Build a Windowed (Sliding-Window) Dataset . . . 274
18.6.4 Step 4 — Build a Stacked LSTM Network . . . 275
18.6.5 Step 5 — Compile and Train . . . 275

18.6.6 Step 6 — Prepare Test Data and Predict . . . 276
18.6.7 Step 7 — Visualize Results . . . 276
18.7 Fusion Transformers — Combining Multiple Data Sources . . . 277
18.7.1 Core Idea . . . 277
18.7.2 Applications to Time Series (and Finance Specifically) . . . 277
18.8 The Temporal Fusion Transformer (TFT) . . . 278
18.8.1 Motivation . . . 278
18.8.2 Types of Inputs TFT Can Handle . . . 278
18.8.3 Why This Architecture Suits Volatile Financial Series . . . 279
18.8.4 Architecture (Conceptual Summary) . . . 279
18.9 TFT Implementation Walkthrough (PyTorch / pytorch-forecasting) . . . 280
18.9.1 Setup and Imports . . . 280
18.9.2 Data and Exploratory Data Analysis (EDA) . . . 280
18.9.3 Feature Engineering . . . 281
18.9.4 Building the TimeSeriesDataSet . . . 282
18.9.5 Baseline Comparison . . . 283
18.9.6 Model Instantiation and Training . . . 283
18.9.7 Evaluation and Interpretation . . . 284
18.10Summary — Tying It Back to "Handling Volatility" in Finance . . . 285
18.11References / Further Reading (mentioned in course materials) . . . 286

Algorithmic Trading

Chapter 1

Algorithmic Trading

1.1 What is Backtesting?

1.1.1 Definition

Backtesting is the process of simulating a trading strategy on **historical market data** to estimate how it would have performed in the past. It is the main quantitative screening step before paper trading or committing real capital.

> **Core idea.** If this strategy had been used over the previous N years with the information available at each point in time, what returns, risks, and losses would it have produced?

1.1.2 Why Backtest?

Without a backtest, a trader has little evidence that a strategy is viable. A well-designed backtest provides the following benefits.

Benefit	Explanation
Risk-free evaluation	Tests a strategy on historical observations without risking actual capital.
Confidence building	Shows whether favourable results persist across time periods and market regimes.
Rejection or refinement	Reveals weak assumptions and parameters before money is committed.
Quantitative measurement	Turns a qualitative trading idea into measurable returns, risks, and costs.

1.1.3 The Backtesting Principle

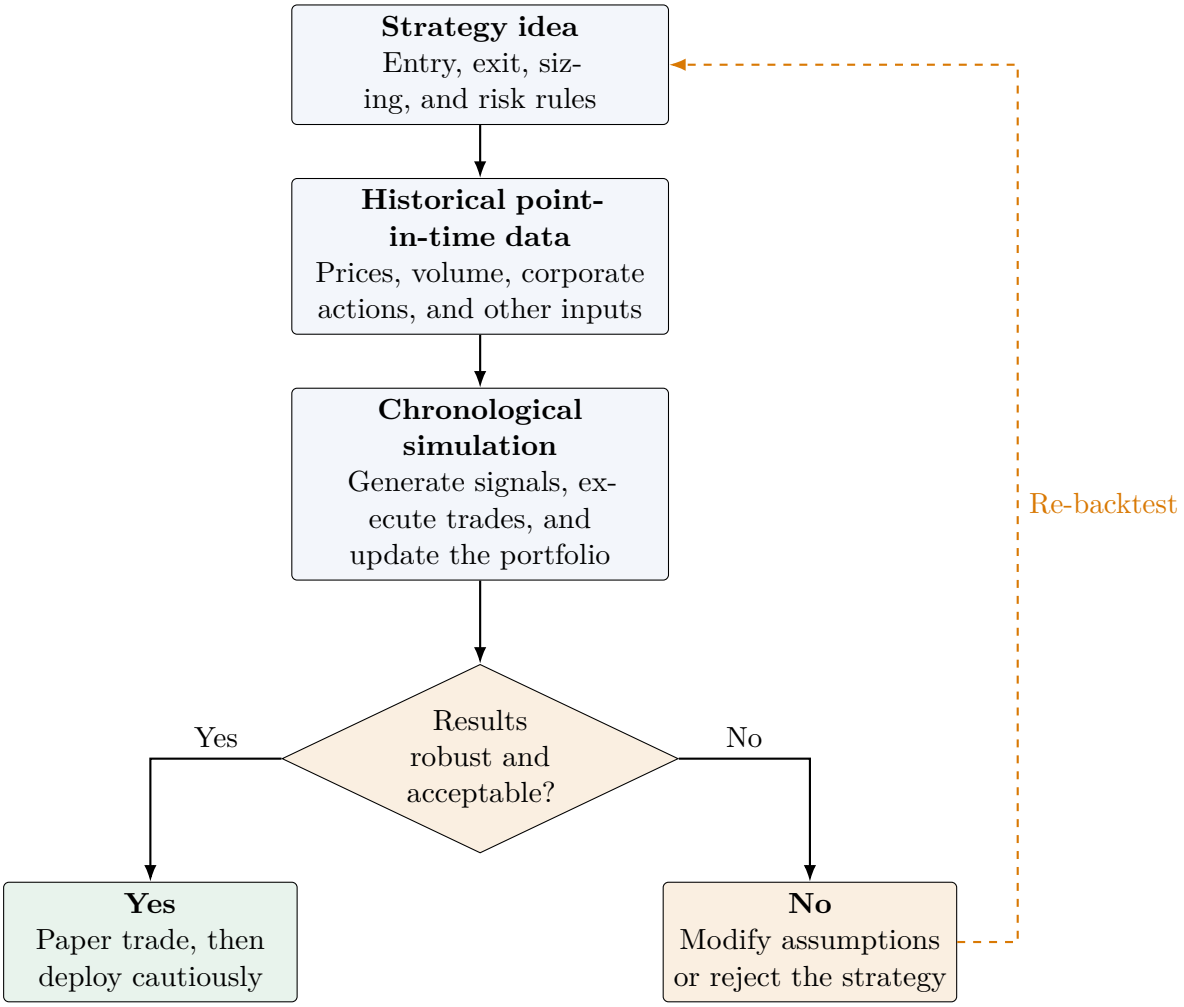


Figure 1.1: The basic backtesting and refinement cycle.

1.1.4 Requirements of a Good Backtest

1. **Quality historical data:** clean, complete, correctly adjusted, and free from survivorship bias.
2. **Realistic simulation:** transaction costs, bid–ask spreads, slippage, latency, liquidity, and market impact are represented.
3. **Appropriate time horizon:** the test covers several market regimes, including rising, falling, volatile, and sideways markets.
4. **Rigorous evaluation:** several return and risk measures are considered rather than raw profit alone.

Warning: overfitting. A strategy that looks exceptional on the data used to design it may merely have been tuned to historical noise. Reserve unseen data for out-of-sample testing and use walk-forward validation before drawing conclusions.

1.2 Performance Metrics for Backtesting a Trading Strategy

1.2.1 Overview

Performance metrics permit objective comparisons among strategies. **No single metric gives a complete assessment**; returns, risk, and risk-adjusted performance should be examined together.

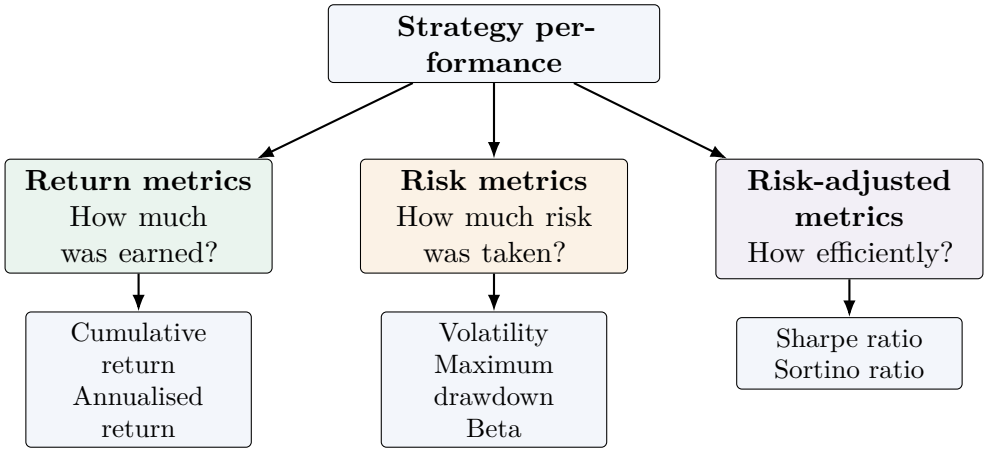


Figure 1.2: Three complementary views of strategy performance.

1.2.2 Cumulative Return (Absolute Return)

Definition. The total percentage gain or loss between the beginning and end of the backtest, irrespective of its duration:

$$\text{Cumulative return} = \frac{V_f - V_0}{V_0} \times 100\%,$$

where V_0 and V_f are the initial and final portfolio values.

Example.
If a portfolio grows from Rs.1,00,000 to Rs.1,45,000, then

$$\frac{1{,}45{,}000 - 1{,}00{,}000}{1{,}00{,}000} \times 100\% = \mathbf{45}\%.$$

Limitation. Cumulative return ignores time. A 45% return over ten years is not comparable with a 45% return earned in one year unless both are annualised.

1.2.3 Annualised Return

Definition. The annualised return, or compound annual growth rate (CAGR), is the constant yearly rate that produces the observed cumulative return over the same period:

$$\text{Annualised return} = (1 + \text{cumulative return})^{365/N} - 1,$$

where N is the number of calendar days in the backtest. If the duration is expressed as T years, the exponent is $1/T$.

Example.
A cumulative return of 45% over 547 days gives

$$(1.45)^{365/547} - 1 \approx (1.45)^{0.667} - 1 \approx \mathbf{28.1}\%.$$

Strategy	**Duration**	**Cumulative return**	**Annualised return**
A	3 years	60%	$(1.60)^{1/3} - 1 = 16.96\%$
B	1.5 years	45%	$(1.45)^{2/3} - 1 \approx 28.1\%$

Strategy B has the lower cumulative return but the higher annualised return because it earned its gain in half the time.

1.2.4 Annualised Volatility

Definition. Volatility measures the dispersion of periodic returns. For daily data it is commonly annualised as

$$\sigma_{\text{annual}} = \sigma_{\text{daily}}\sqrt{252},$$

where 252 is the conventional number of trading days in a year.

The square-root factor arises because independent return variances add over time: annual variance is $252\sigma^2_{\text{daily}}$, so annual standard deviation is $\sqrt{252}\sigma_{\text{daily}}$.

Example.
For a daily standard deviation of 1.3%,

$$0.013\sqrt{252} = 0.206 \approx \mathbf{20.6}\%.$$

Higher volatility means that realised returns are less stable. Thus, two strategies with equal annualised returns can have very different risk profiles.

1.2.5 Sharpe Ratio

Definition. The Sharpe ratio measures excess return per unit of total volatility:

$$\text{Sharpe ratio} = \frac{R_p - R_f}{\sigma_p},$$

where R_p is the portfolio return, R_f is the risk-free return, and σ_p is portfolio volatility. All three quantities must use the same time scale.

Sharpe ratio	**Typical interpretation**
< 0	The strategy earned less than the risk-free asset.
0 to 1	Positive but weak risk-adjusted performance.
1 to 2	Good risk-adjusted performance.
2 to 3	Very good risk-adjusted performance.
> 3	Exceptional in a backtest; verify carefully for overfitting and omitted risks.

Worked example.
For an annual strategy return of 18%, an annual risk-free rate of 5%, and annualised volatility of 12%,

$$\text{Sharpe ratio} = \frac{18\% - 5\%}{12\%} = \frac{13\%}{12\%} = \mathbf{1.08}.$$

Limitations.

- Upside and downside volatility are penalised equally.
- The measure can understate tail risk when returns are skewed or heavy-tailed.
- Results are sensitive to the sampling frequency, annualisation convention, and chosen risk-free rate.

1.2.6 Sortino Ratio

Definition. The Sortino ratio replaces total volatility with downside deviation, so only returns below a chosen target T are penalised:

$$\text{Sortino ratio} = \frac{R_p - R_f}{\sigma_{\text{down}}}, \qquad \sigma_{\text{down}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} \min(0, R_i - T)^2}.$$

The target may be zero, the risk-free rate, or another minimum acceptable return.

Example.

For periodic returns $5\%, -3\%, 8\%, -2\%, 4\%$ and $T = 0$,

$$\sigma_{\text{down}} = \sqrt{\frac{(-3)^2 + (-2)^2}{5}} = \sqrt{2.6} = 1.61\%.$$

If the annual return is 12%, the risk-free rate is 4%, and the downside deviation has been expressed on the same annual scale, then

$$\text{Sortino ratio} = \frac{12\% - 4\%}{1.61\%} = 4.97.$$

The Sortino ratio is especially useful for asymmetric strategies, but a high value should not be interpreted without examining rare tail losses.

1.2.7 Beta

Definition. Beta measures the sensitivity of a strategy to a market benchmark:

$$\beta = \frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)},$$

where R_p and R_m are portfolio and market returns.

Beta	**Meaning**	**Example**
$\beta < 0$	Tends to move opposite to the market	Inverse or short strategies
$\beta = 0$	No linear market exposure	Market-neutral strategy
$0 < \beta < 1$	Moves with the market, but less strongly	Defensive portfolio
$\beta = 1$	Has the same market sensitivity as the benchmark	Index-tracking portfolio
$\beta > 1$	Amplifies market movements	Leveraged or aggressive portfolio

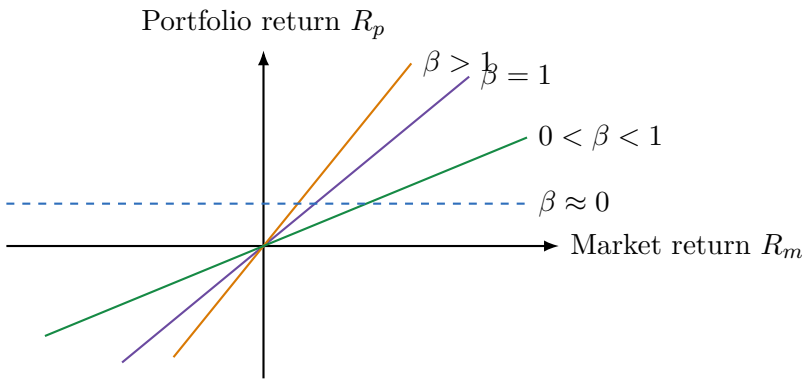


Figure 1.3: Beta is the slope of the portfolio–market return relationship.

A common single-factor decomposition is

$$R_p = \alpha + \beta R_m + \varepsilon,$$

where α is benchmark-independent performance and ε is unexplained variation. A high-return strategy with $\beta = 1.5$ may mostly represent leveraged market exposure rather than a distinct trading edge.

1.2.8 Maximum Drawdown

Definition. Maximum drawdown (MDD) is the largest peak-to-trough decline in portfolio value before a new peak is reached. Its positive magnitude is

$$\text{MDD} = \max_t \left(\frac{P_t - V_t}{P_t} \right) \times 100\%, \qquad P_t = \max_{s \le t} V_s.$$

Worked example.
Consider the portfolio path

$$\text{Rs. } 1{,}00{,}000 \to \text{Rs. } 1{,}50{,}000 \to \text{Rs. } 90{,}000 \to \text{Rs. } 1{,}20{,}000 \to \text{Rs. } 70{,}000 \to \text{Rs. } 1{,}60{,}000.$$

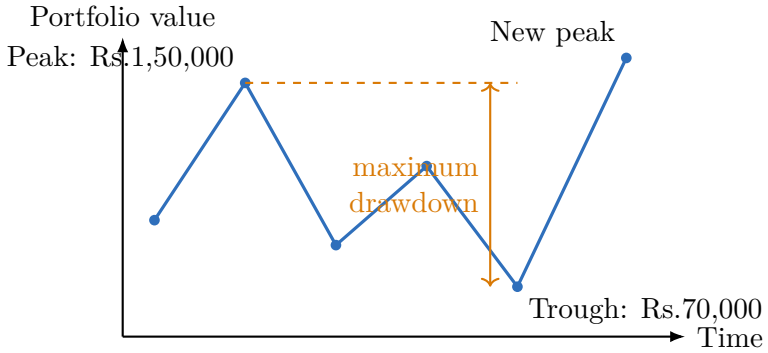


Figure 1.4: Maximum drawdown is measured from the preceding peak, not from the final value.

Thus,

$$\text{MDD} = \frac{1{,}50{,}000 - 70{,}000}{1{,}50{,}000} \times 100\% = \mathbf{53.33}\%.$$

The interim recovery to Rs.1,20,000 does not reset the peak because it does not exceed Rs.1,50,000.

A 50% loss requires a 100% subsequent gain merely to break even. MDD therefore gives an intuitive capital-preservation and psychological stress measure that volatility alone does not provide.

Two useful related measures are

$$\text{Calmar ratio} = \frac{\text{annualised return}}{|\text{MDD}|}, \qquad \text{Recovery factor} = \frac{\text{net profit}}{|\text{MDD}|}.$$

1.2.9 Summary of Performance Metrics

Metric	**Formula**	**Measures**	**Preferred direction**
Cumulative return	$(V_f - V_0)/V_0$	Total gain or loss	Higher
Annualised return	$(1 + CR)^{365/N} - 1$	Compounded yearly growth	Higher
Annualised volatility	$\sigma_d\sqrt{252}$	Dispersion of returns	Lower, generally
Sharpe ratio	$(R_p - R_f)/\sigma_p$	Excess return per unit of total risk	Higher
Sortino ratio	$(R_p - R_f)/\sigma_{\text{down}}$	Excess return per unit of downside risk	Higher
Beta	$\text{Cov}(R_p, R_m)/\text{Var}(R_m)$	Market sensitivity	Depends on objective
Maximum drawdown	$(P - T)/P$	Worst peak-to-trough decline	Lower

1.3 Slippage and Latency

1.3.1 What is Slippage?

Slippage is the difference between the price observed when a signal is generated and the price at which the order is actually filled:

$$\text{Slippage magnitude} = |\text{execution price} - \text{signal price}|\,.$$

For a buy order, execution above the signal price is adverse; for a sell order, execution below the signal price is adverse.

> **Example.** An algorithm generates a buy signal for HDFC Bank at Rs.1,650.00. The order is filled at Rs.1,651.25, producing adverse slippage of Rs.1.25 per share.

Slippage is a real transaction cost and can turn a small theoretical edge into a loss.

1.3.2 Why Slippage Occurs

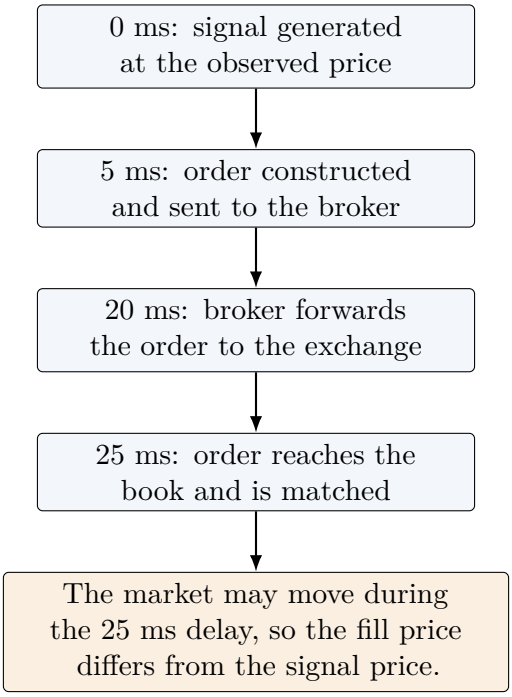


Figure 1.5: A simplified signal-to-execution latency chain.

1.3.3 Three Drivers of Slippage

Underlying Asset Volatility

The farther a price can move per unit of time, the larger the range of possible execution prices during a fixed delay.

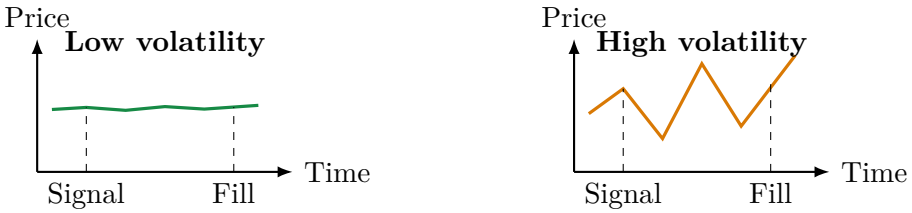


Figure 1.6: The same execution delay creates more slippage uncertainty in a volatile asset.

> **Rule of thumb.** Higher short-horizon volatility generally implies higher expected slippage, all else equal.

Latency

Latency is the delay between signal generation and execution. It includes:

- **algorithmic latency**: time used to calculate the signal, perform checks, and construct the order; and
- **communication and execution latency**: time used to send the order through the broker, reach the exchange, queue, and obtain a fill.

Hence,

$$\text{total latency} = \text{processing latency} + \text{communication and execution latency.}$$

Strategy type	Sensitivity	Reason
High-frequency trading	Extremely high	Signals may decay within microseconds or milliseconds.
Intraday algorithms	Moderate to high	Opportunities often last seconds or minutes.
Swing or position trading	Usually lower	Signals are intended to persist for days or weeks.

This is why latency-sensitive firms may colocate servers near exchange matching engines and use specialised networking or hardware.

Strategy Type

Strategy	Typical impact	Reason
Momentum	Often adverse	The strategy buys into rising prices or sells into falling prices.
Mean reversion	Can be less adverse, but is not guaranteed to be favourable	The price may continue moving before reverting.
Market making	Two-sided and complex	Limit-order fills, queue position, spread capture, and adverse selection all matter.

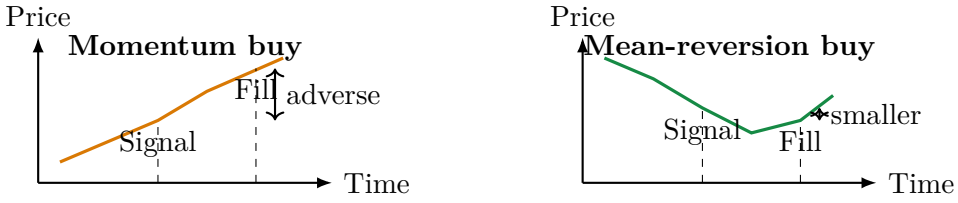


Figure 1.7: Illustrative, not guaranteed, slippage patterns for two strategy types.

1.4 Transaction Cost Models

1.4.1 Components of Transaction Costs

Transaction costs include every cost caused by executing and maintaining a trade. Omitting them can materially overstate backtested performance. Important components include:

- brokerage commissions;
- exchange fees, stamp duty, and taxes;
- the bid–ask spread;
- slippage and latency effects; and
- market impact, especially for orders large relative to available liquidity.

1.4.2 Flat or Fixed Cost Model

A fixed model assigns the same monetary cost to every trade:

$$\text{total transaction cost} = C_{\text{fixed}} N_{\text{trades}}.$$

For a fee of Rs.20 per trade and 500 trades,

$$\text{total cost} = \text{Rs. } 20 \times 500 = \text{Rs. } 10{,}000.$$

Advantages.

It is simple, computationally inexpensive, and can represent genuinely fixed per-order fees.

Disadvantages.

It ignores trade size, liquidity, spread, volatility, slippage, and market impact. It is therefore only a rough approximation for small and relatively uniform orders.

1.4.3 Linear Cost Model

A linear model makes cost proportional to traded notional V:

$$\text{cost} = cV.$$

For a 0.1% cost rate on a trade worth Rs.5,00,000,

$$\text{cost} = 0.001 \times \text{Rs. } 5{,}00{,}000 = \text{Rs. } 500.$$

A piecewise-linear specification may include a minimum charge, different brokerage tiers, or different spread assumptions across size ranges. Linear models are more realistic than fixed models for percentage fees, but they still miss nonlinear market impact.

1.4.4 Quadratic Cost Model

A simple nonlinear approximation is

$$\text{cost} = c_1 V + c_2 V^2,$$

or, in a purely quadratic specification, cost $= cV^2$. The quadratic term approximates the increasing marginal cost of consuming more levels of the order book.

For example, a purchase of 100 shares may be absorbed at the best ask, whereas a purchase of 10,000 shares may consume several price levels. The trader's own demand then raises the average execution price.

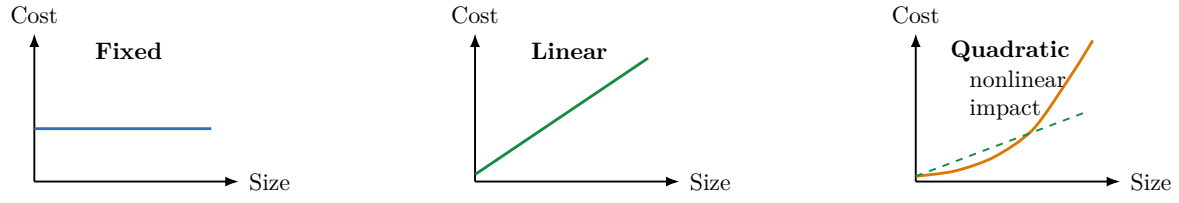


Figure 1.8: Fixed, linear, and quadratic transaction-cost approximations.

Advantages.

A calibrated nonlinear model can represent market impact more realistically when order size is material relative to liquidity.

Disadvantages.

The coefficients must be estimated for each asset, venue, order type, and market regime. A quadratic form is still an approximation; using it without calibration does not guarantee accuracy.

1.4.5 Using Historical Execution Costs

A more data-driven cost model can be constructed as follows:

1. collect fill prices, decision prices, order sizes, timestamps, and contemporaneous market conditions from executed orders;
2. estimate slippage as a function of order size, volatility, liquidity, spread, participation rate, and latency; and
3. apply the fitted model to simulated orders in the backtester.

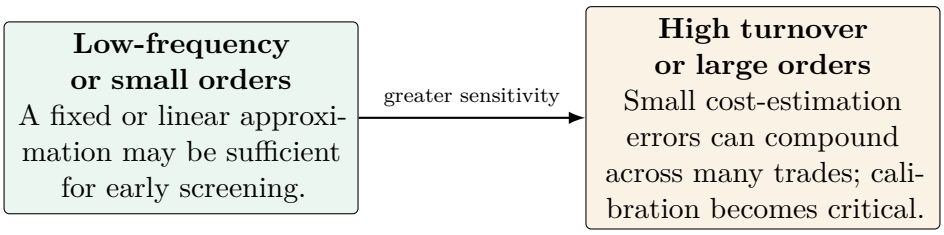


Figure 1.9: The required cost-model detail grows with turnover and market impact.

For a high-turnover strategy, even a few basis points of omitted cost per trade can consume the entire forecast edge.

1.5 Backtesting Framework

1.5.1 Complete Workflow

A rigorous backtest follows a staged process.

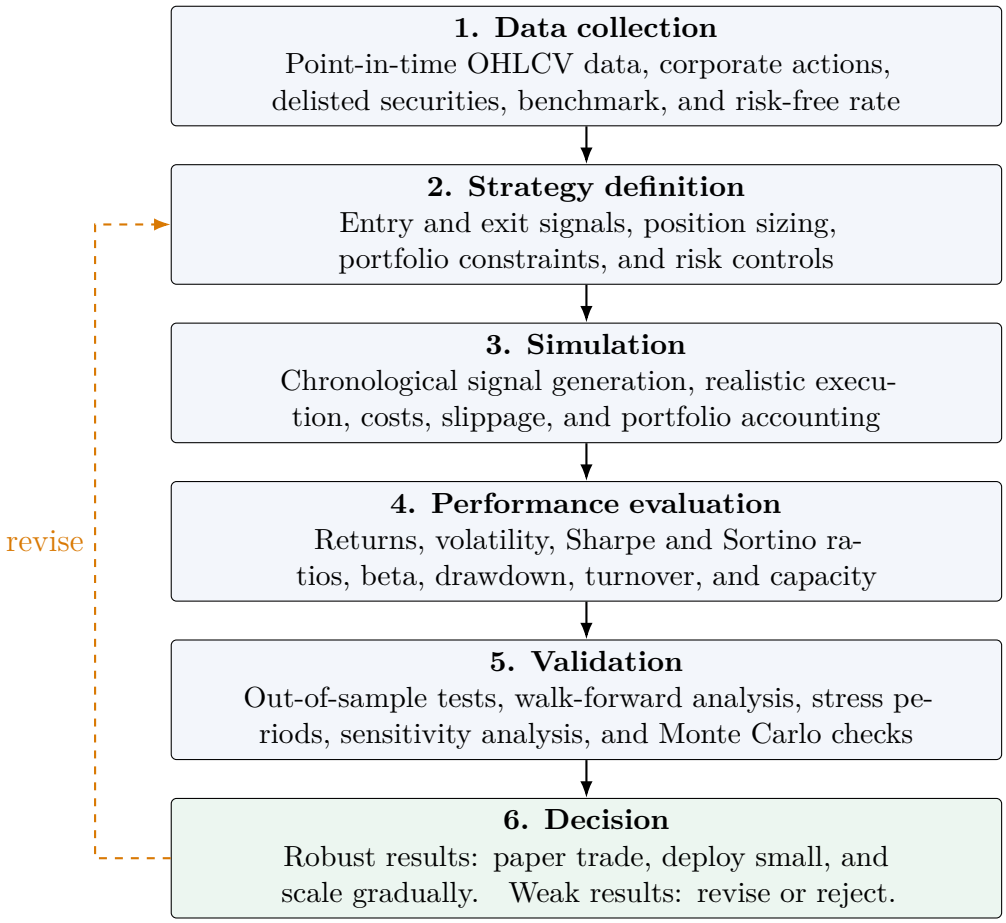


Figure 1.10: A complete backtesting and validation workflow.

1.5.2 Common Backtesting Pitfalls

Look-Ahead Bias

Look-ahead bias occurs when the strategy uses information that was not available at the simulated decision time. For example, today's closing price cannot generate a trade executed earlier during today's session.

Mitigation: enforce strict timestamp ordering and delay execution until after every required input would genuinely have been observable.

Survivorship Bias

Survivorship bias occurs when a historical universe contains only securities that still exist today. Failed, delisted, or removed securities are omitted, which can make past results look artificially strong.

Mitigation: use point-in-time index constituents and include all securities that were investable on each historical date.

Overfitting or Curve Fitting

Overfitting occurs when parameters are tuned so aggressively that the strategy captures noise specific to the development sample.

Mitigation: use a small number of economically justified parameters, reserve out-of-sample data, perform walk-forward validation, and test parameter sensitivity.

Ignoring Transaction Costs

Ignoring commissions, taxes, spreads, slippage, and market impact can turn an unprofitable strategy into an apparently profitable one. For example, a 0.1% round-trip cost repeated 1,000 times equals 100% of the repeatedly traded notional, although its exact effect on portfolio return depends on position size and turnover.

Mitigation: calibrate costs to the asset, venue, order type, liquidity, trade size, and strategy frequency.

Latency Blindness

Latency blindness assumes an instantaneous fill at the signal price. In practice, both the market and the available queue can change before an order arrives.

Mitigation: delay fills realistically and model order type, queue position, partial fills, and slippage at the relevant time scale.

1.6 Libraries

1.6.1 Algorithmic Trading Libraries

> **Note on sources:** These notes are compiled from the official documentation and tutorials for each library (linked below), since these are the canonical, most reliable, and most current references. I was not able to transcribe the linked YouTube playlists directly (no video/audio access), but the concepts, functions, and worked examples they'd typically cover for these libraries are the same ones documented in the official tutorials, which are used here as the primary source. If you want specific clarifications that came from the videos and aren't reflected here, let me know what topic/timestamp and I can look into it further.

Sources used:

- Pyfolio: https://quantopian.github.io/pyfolio/ (Overview + all 6 tutorial notebooks)
- Zipline (maintained fork *zipline-reloaded*): https://zipline.ml4trading.io/ (Tutorial + full API reference)
- Zipline-trader (Alpaca-focused live-trading fork): https://zipline-trader.readthedocs.io/en/latest/beginner-tutorial.html
- Backtesting.py: https://kernc.github.io/backtesting.py/doc/backtesting/ (API docs, `backtesting.backtesting`, `backtesting.lib`) + Quick Start User Guide

---

Quick Orientation: Which Library Does What?

Library	Role	Scope	Best for

Zipline	Event-driven **backtesting simulator** (runs a strategy bar-by-bar)	Multi-asset, portfolio-level	Writing and *simulating* a trading algorithm against historical data, including realistic order execution, slippage, commissions
Pyfolio	**Performance & risk analytics** (post-hoc analysis)	Works on the *output* (returns/ positions/ transactions) of a backtest (e.g. from Zipline) or even a single asset's returns	Generating professional "tear sheets" — the statistics and plots used to *evaluate* a strategy after it has been run
Backtesting.py	Lightweight, single-asset **backtesting + optimization** framework	One tradeable asset at a time; not built for multi-asset portfolio rebalancing	Quickly prototyping, backtesting, and optimizing indicator/signal-based strategies with an intuitive, minimal API and interactive plots

In short: you might **write and run** a strategy in **Zipline**, then **analyze its performance** with **Pyfolio** — or, for simpler single-asset strategy iteration, use **Backtesting.py** end-to-end (it has its own built-in stats and plotting, so it doesn't need Pyfolio).

1.6.2 Pyfolio

What It Is

Pyfolio is a Python library for **performance and risk analysis of financial portfolios**, originally developed by Quantopian Inc. It integrates well with **Zipline** (Zipline produces the returns/positions/transactions data that Pyfolio analyzes), but it can just as easily analyze the returns of a single asset compared to a benchmark, or returns from any other source.

Installation:

```
pip install pyfolio
```

(Bayesian tear sheets additionally require `pip install theano` and `pip install pymc3`.)

The Core Concept: Tear Sheets

At the heart of Pyfolio is the **tear sheet** — a collection of individual plots and statistics tables that together give a comprehensive picture of a strategy's performance. Rather

than compute metrics one by one, you call one function and get a full battery of analysis at once.

The main tear-sheet-generating functions are:

Function	Purpose
`pf.create_returns_tear_sheet(returns,...)`	Core statistics + plots based on a returns series alone (works even for a single stock)
`pf.create_full_tear_sheet(returns, positions=None,transactions=None,...)`	The "everything" tear sheet: returns stats + positions + transactions + (optionally) round trips + slippage sweeps
`pf.create_position_tear_sheet(returns,positions,sector_mappings=None)`	Analysis of holdings over time (top positions, sector allocation)
`pf.create_round_trip_tear_sheet(returns,positions,transactions,sector_mappings=None)`	Analysis of individual "round trip" trades (win rate, profit factor, holding durations, etc.)

Required Input Data

Pyfolio tear sheets are built from up to three pandas objects:

- **`returns`** — a `pd.Series` of daily percentage returns, datetime-indexed.
- **`positions`** — a `pd.DataFrame` of daily dollar values held per symbol (plus a `'cash'` column), datetime-indexed.
- **`transactions`** — a `pd.DataFrame` of individual trades executed (datetime-indexed), used for transaction-cost/slippage and round-trip analysis.

Getting a Single Stock's Returns

```
import pyfolio as pf
stock_rets = pf.utils.get_symbol_rets('FB')
pf.create_returns_tear_sheet(stock_rets, live_start_date='2015-12-1')
```

`live_start_date` splits the analysis into **in-sample** (before that date) and **out-of-sample** (after) — useful for checking whether a strategy's live performance matches its backtested performance.

Getting Data From a Zipline Backtest

If you ran a strategy in Zipline and saved the results (`results.pickle`), Pyfolio can extract the three required objects directly:

```
import pandas as pd
import pyfolio as pf

results = pd.read_pickle('results.pickle')
returns, positions, transactions = pf.utils.extract_rets_pos_txn_from_zipline(
    results)
```

The Statistics Table (What Each Metric Means)

Every `create_*_tear_sheet` call prints a summary table like this (columns split into **All / In-sample / Out-of-sample** if `live_start_date` is given):

Metric	Meaning
Annual return	Compounded annualized return
Cumulative returns	Total return over the whole period
Annual volatility	Annualized standard deviation of daily returns
Sharpe ratio	Risk-adjusted return: excess return per unit of total volatility
Calmar ratio	Annual return divided by max drawdown — rewards strategies with shallow drawdowns
Stability	R^2 of a linear fit to the log cumulative returns curve — how smooth/consistent the equity growth is
Max drawdown	Largest peak-to-trough decline in cumulative returns
Omega ratio	Probability-weighted ratio of gains vs. losses relative to a threshold
Sortino ratio	Like Sharpe, but only penalizes *downside* volatility (not upside swings)
Skew	Asymmetry of the returns distribution (positive skew = more extreme gains than losses)
Kurtosis	"Fat-tailedness" of the returns distribution (high kurtosis = more extreme outlier days)
Tail ratio	Ratio of the size of the right tail (gains) to the left tail (losses), e.g. at the 95th vs. 5th percentile
Daily value at risk	Estimated daily loss threshold not expected to be exceeded except in the worst X% of cases
Gross leverage	Total absolute exposure (long + short) relative to portfolio value
Daily turnover	How much of the portfolio is traded each day, as a fraction of portfolio value
Alpha	Return not explained by the benchmark (skill/edge)
Beta	Sensitivity of the strategy's returns to the benchmark's returns

Below the summary table, tear sheets also show a **"Worst drawdown periods"** table listing each major drawdown's net decline (%), peak date, valley date, recovery date, and duration (in days) — this makes it easy to see not just *how deep* losses got but *how long* the strategy stayed underwater.

Plots Generated

Depending on which tear sheet function is called, Pyfolio can generate:

- Cumulative returns plot (linear and log scale), vs. benchmark
- Returns plot (per-period)
- Rolling 6-month volatility
- Rolling 6-month Sharpe ratio
- Top 5 drawdown periods (highlighted on the cumulative returns curve)
- Drawdown "underwater" plot (depth of drawdown over time)
- Monthly returns heatmap (%) and annual returns bar chart
- Distribution of monthly returns (histogram)
- Return quantiles box plot
- Stress event table (mean/min/max returns during historical stress periods like the 2008 Lehman crisis, Aug '07, Mar '08, Sept '08, the 2009 recovery, etc.) — lets you see how the strategy would have behaved during known crisis windows
- Top 10 long/short positions of all time, and all positions ever held (with sector breakdown, if `sector_mappings` is supplied)

Worked Example: Full Zipline Algorithm → Pyfolio Tear Sheet

This is the canonical workflow: run a Zipline algorithm, save its output, then feed that output into Pyfolio.

```
import pyfolio as pf
import zipline
%load_ext zipline
!zipline ingest   # download/prepare pricing data bundle first
```

```
%%zipline --start 2004-1-1 --end 2010-1-1 -o results.pickle

# An "On-Line Portfolio Moving Average Reversion" (OLMAR) strategy
import numpy as np
from zipline.finance import commission, slippage

STOCKS = ['AMD', 'CERN', 'COST', 'DELL', 'GPS', 'INTC', 'MMM']

def initialize(algo, eps=1, window_length=5):
    algo.stocks = STOCKS
    algo.sids = [algo.symbol(s) for s in algo.stocks]
    algo.m = len(algo.stocks)
    algo.b_t = np.ones(algo.m) / algo.m   # start with an equal-weighted
   portfolio
    algo.eps = eps                         # reversion sensitivity parameter
    algo.window_length = window_length
    algo.set_commission(commission.PerShare(cost=0))
    algo.set_slippage(slippage.FixedSlippage(spread=0))

def handle_data(algo, data):
    # Compute each asset's price relative to its recent moving average
```

```
    mavgs = data.history(algo.sids, 'price', algo.window_length, '1d').mean()
    x_tilde = np.array([mavgs[sid] / data.current(sid, "price") for sid in
    algo.sids])
    # ... mean-reversion portfolio update logic (project onto the simplex
    # of valid portfolio weights) ...
    rebalance_portfolio(algo, data, new_weights)

def rebalance_portfolio(algo, data, desired_port):
    for i, sid in enumerate(algo.sids):
        algo.order_target_percent(sid, desired_port[i])
```

Then extract the results and feed them straight to Pyfolio:

```
import pandas as pd
results = pd.read_pickle('results.pickle')
returns, positions, transactions = pf.utils.extract_rets_pos_txn_from_zipline(
    results)

# One plot at a time...
pf.plot_drawdown_periods(returns, top=5).set_xlabel('Date')

# ...or the full tear sheet, including round-trip trade analysis
pf.create_full_tear_sheet(returns, positions=positions, transactions=
    transactions,
                          live_start_date='2009-10-22', round_trips=True)
```

If you'd rather not reveal which specific symbols your strategy traded (e.g. when sharing a tear sheet publicly), pass `hide_positions=True`:

```
pf.create_full_tear_sheet(returns, positions=positions, transactions=
    transactions,
                          live_start_date='2009-10-22', hide_positions=True)
```

Sector Analysis

To break down PnL and position exposure by **sector** instead of just by symbol, pass a `sector_mappings` dictionary (symbol → sector name):

```
sect_map = {'COST': 'Consumer Goods', 'INTC': 'Technology', 'CERN': '
    Healthcare',
            'GPS': 'Technology', 'MMM': 'Construction', 'DELL': 'Technology',
    'AMD': 'Technology'}

pf.create_position_tear_sheet(returns, positions, sector_mappings=sect_map)
# or, for round trips:
pf.create_round_trip_tear_sheet(returns, positions, transactions,
    sector_mappings=sect_map)
```

This adds a "Profitability (PnL / PnL total) per name" table aggregated at the **sector** level in addition to the per-symbol table, which is useful for spotting concentration risk (e.g. "is my strategy's profit really coming from one sector?").

Round-Trip Trade Analysis

A **round trip trade** begins when a new long or short position is opened, and ends when it is completely or partially closed. Round-trip analysis exists to answer: *is this strategy repeatedly profitable, or did a few lucky trades carry the whole equity curve?*

How it works under the hood: Pyfolio reconstructs your full portfolio from the `transactions` you provide. When a new trade occurs, it checks whether shares of that symbol are already held from an earlier purchase; if so, it computes the PnL, return, and duration of that "round trip." At the very end of the data, any still-open positions are synthetically closed out (using the last available price) so that their PnL is captured too.

```
pf.create_round_trip_tear_sheet(returns, positions, transactions,
    sector_mappings=sect_map)
```

This produces four groups of statistics, each broken down into **All trades / Short trades / Long trades**:

Stat group	Key fields	Meaning
Summary stats	Total # round trips, Percent profitable, Winning/Losing/Even round trips	Basic win-rate bookkeeping
PnL stats	Total profit, Gross profit, Gross loss, Profit factor, Avg./Largest winning & losing trade	Profit factor = gross profit / |gross loss|; a value > 1 means the strategy makes more from winners than it loses from losers
Duration stats	Avg./Median/Longest/ Shortest duration	How long, on average, capital is tied up in a trade
Return stats	Avg./Median returns (all / winning / losing), largest winning/losing trade (%)	Same PnL picture but expressed as % return per round trip rather than $

There's also a **per-symbol breakdown** of all the above (so you can see, e.g., that one ticker in your universe was actually a net drag on performance), and a **"Profitability per name"** bar chart.

Under the hood, manually:

```
rts = pf.round_trips.extract_round_trips(
    transactions, portfolio_value=positions.sum(axis='columns') / (returns +
    1))
pf.round_trips.print_round_trip_stats(rts)
```

`rts` is itself a DataFrame with one row per round trip: `close_dt`, `long` (bool), `open_dt`, `pnl`, `rt_returns`, `symbol`, `duration`, `returns`.

Slippage Sensitivity Analysis

Slippage is the difference between the price you expected to trade at and the price you actually got — it eats into real-world returns in a way backtests often underestimate.

Pyfolio's transactions tear sheet can generate **"slippage sweep"** plots showing how the strategy's equity curve would look under a range of additional per-dollar slippage assumptions.

```
pf.create_full_tear_sheet(returns, positions, transactions, slippage=5)
```

Here, `slippage` is specified in **basis points** (1% = 100 bps). Passing `slippage=5` applies 5 bps of *additional* slippage on top of whatever is already baked into the returns you passed in, and shows the resulting performance for the main tear sheet plots.

Important nuance: the slippage sweep plots themselves always show a *range* of slippage scenarios (e.g. 0, 1, 2, 5, 10 bps) for comparison — the `slippage=N` argument only affects the *main* returns/plots elsewhere in the tear sheet, not the sweep itself. If your `returns` already reflect realistic slippage (e.g. from a proper Zipline backtest with a slippage model), pass `slippage=0` so you don't double-count, while still getting the sweep plot for comparison purposes.

Bayesian Tear Sheet

Pyfolio also offers a **Bayesian** analysis (requires PyMC3 + Theano) that models strategy returns probabilistically rather than with simple point-estimate statistics — e.g. giving credible intervals for the Sharpe ratio rather than a single number, and modeling the probability distribution of future drawdowns. This is more advanced/experimental and is best explored via the linked example notebook rather than summarized here, since it's essentially a from-scratch probabilistic model rather than a fixed formula.

1.6.3 Zipline

What It Is and Current Status

Zipline is an **open-source, event-driven algorithmic trading simulator** written in Python, originally built by Quantopian as the engine behind their backtesting and live-trading platform. When Quantopian shut down in late 2020, its original documentation domain (`zipline.io`) expired.

Important: the documentation used here is for **`zipline-reloaded`**, the actively maintained community fork (maintained by Stefan Jansen, author of *Machine Learning for Algorithmic Trading*) — this is the version you should actually install today, via:

```
pip install zipline-reloaded
# or
conda install -c conda-forge zipline-reloaded
```

There is also a separate fork, **`zipline-trader`**, aimed specifically at connecting Zipline to the Alpaca brokerage for **live trading**, not just backtesting — its beginner tutorial content is functionally identical to the core Zipline tutorial (same `initialize`/`handle_data` API), but it adds bundle ingestion via Alpaca, a YAML config file, and a "Going Live" mode.

Key stated benefits of Zipline:

- **Realistic**: models slippage, transaction costs, and order delays.
- **Stream-based**: processes each time-bar event individually, which helps avoid look-ahead bias.

- **Batteries included**: common transforms (e.g. moving averages) and risk stats (e.g. Sharpe) are available out of the box.
- **PyData integration**: input/output are Pandas DataFrames.

The Core Algorithm Structure

Every Zipline algorithm is built around **two required functions**:

```
def initialize(context):
    ...  # called once, before the backtest starts

def handle_data(context, data):
    ...  # called once per event (trading bar)
```

- **`initialize(context)`** runs once at the very start. `context` is a persistent namespace object — anything you store on it (`context.i = 0`, `context.asset = ...`) is available in every later call.
- **`handle_data(context, data)`** runs once per bar (once per day, for daily data; once per minute, for minute data). `data` is a `BarData` object giving you access to current and historical price/volume info for your universe.

Two additional optional functions:

- **`before_trading_start(context, data)`** — runs once per day, before the market opens (good place to run/refresh a Pipeline).
- **`analyze(context, perf)`** — runs once, automatically, after the whole backtest finishes; `perf` is the full performance DataFrame, useful for custom plotting.

Minimal Example (`buyapple.py`)

```
from zipline.api import order, record, symbol

def initialize(context):
    pass

def handle_data(context, data):
    order(symbol('AAPL'), 10)  # buy 10 shares of AAPL every bar
    record(AAPL=data.current(symbol('AAPL'), 'price'))
```

- `order()` takes an asset object and a signed integer amount — positive to buy/cover, negative to sell/short.
- `record()` saves a named value on every bar; after the run finishes, these appear as columns in the performance DataFrame (letting you plot e.g. your own custom indicator alongside portfolio value).

The `BarData` Object (`data`)

Passed into `handle_data`/`before_trading_start`, this is your window into current and historical prices.

`data.current(assets, fields)`

Returns the *current* bar's value(s) for one or more assets/fields.

- Valid `fields`: `price`, `open`, `high`, `low`, `close`, `volume`, and `last_traded`.
- `"price"` = last known close, forward-filled if no trade happened this bar, and split/dividend-adjusted.
- `"open"/"high"/"low"/"close"` = NaN if there was no trade this bar.
- `"volume"` = 0 if no trade this bar.
- Return type depends on inputs: scalar (1 asset, 1 field) → `pd.Series` (1 asset+many fields, or many assets+1 field) → `pd.DataFrame` (many assets+many fields).

`data.history(assets, fields, bar_count, frequency)`

Returns a **trailing window** of historical data — this is how you get access to "yesterday's" or "last N days'" prices, adjusted for splits/dividends/mergers as of *now* (so no look-ahead bias creeps in via unadjusted historical prices).

- `frequency`: `'1d'` for daily bars, `'1m'` for minute bars (needs minute-resolution data ingested).
- `bar_count`: how many historical bars to return.
- Shape of the return value follows the same "scalar / Series / DataFrame" logic as `.current()`, with the extra wrinkle that multi-asset + multi-field requests return a `MultiIndex` DataFrame.

`data.can_trade(assets)` / `data.is_stale(assets)`

- `can_trade()`: True only if the asset is alive, its exchange is open right now, and there's a known last price for it — use this before ordering to avoid errors on delisted/inactive assets.
- `is_stale()`: True if the asset is alive but simply hasn't traded in the current bar (helps distinguish "no recent data" from "doesn't exist").

Dual Moving Average Example (Using `history`)

This is the canonical "beyond the basics" tutorial example, and it introduces the idea of **precomputing indicators from a rolling history window**:

```
from zipline.api import order_target, record, symbol

def initialize(context):
    context.i = 0
    context.asset = symbol('AAPL')

def handle_data(context, data):
    context.i += 1
    if context.i < 300:
        return  # skip until we have a full 300-bar window

    short_mavg = data.history(context.asset, 'price', bar_count=100, frequency
    ="1d").mean()
    long_mavg  = data.history(context.asset, 'price', bar_count=300, frequency
    ="1d").mean()

    if short_mavg > long_mavg:
        order_target(context.asset, 100)   # go/stay long 100 shares
    elif short_mavg < long_mavg:
```

```
        order_target(context.asset, 0)     # flatten the position

    record(AAPL=data.current(context.asset, 'price'),
           short_mavg=short_mavg, long_mavg=long_mavg)

def analyze(context, perf):
    import matplotlib.pyplot as plt
    fig = plt.figure()
    ax1 = fig.add_subplot(211); perf.portfolio_value.plot(ax=ax1)
    ax2 = fig.add_subplot(212); perf['AAPL'].plot(ax=ax2)
    perf[['short_mavg', 'long_mavg']].plot(ax=ax2)
    plt.show()
```

Logic: this is a classic momentum "golden cross / death cross" strategy — go long when the short-term average crosses above the long-term average (recent prices trending up relative to the longer trend), and exit when it crosses back below.

A subtlety worth remembering: `order_target()` (like all the "target" order functions) does **not** account for orders you've already placed but which haven't filled yet. Calling `order_target(asset, 10)` twice in a row (before either fills) results in **20 shares total**, not 10 — because Zipline doesn't yet know the first order will succeed when it processes the second.

Running an Algorithm: Three Interfaces

A. Command Line

```
zipline ingest -b quandl      # first: download/prepare a data bundle
zipline run -f my_algo.py --start 2016-1-1 --end 2018-1-1 -o results.pickle --
    no-benchmark
```

Key `zipline run` flags:

Flag	Purpose
`-f/-algofile`	Path to your algorithm's `.py` file
`-b/-bundle`	Which ingested data bundle to use (default `quandl`)
`-s/-start`, `-e/-end`	Backtest date range
`-data-frequency`	`daily` (default) or `minute`
`-capital-base`	Starting cash (default 10,000,000)
`-o/-output`	Where to save the resulting performance DataFrame (pickle format)
`-benchmark-symbol` / `-benchmark-sid` / `-benchmark-file`	What to use as the benchmark for alpha/beta calculations
`-no-benchmark`	Skip a benchmark entirely (alpha/beta won't be computed)
`-trading-calendar`	Which market calendar to simulate against (default NYSE)

Zipline first calls `initialize()`, then streams historical bars through `handle_data()` day by day (or minute by minute). After `handle_data()` returns each bar, Zipline looks

at any orders placed and tries to fill them against that bar's volume — applying the configured commission and slippage models, so the true cost is more than just `price` × `shares`.

B. Jupyter Notebook Magic

```
%load_ext zipline
```

```
%%zipline --start 2016-1-1 --end 2018-1-1 -o perf_variable
from zipline.api import symbol, order, record

def initialize(context):
    pass

def handle_data(context, data):
    order(symbol('AAPL'), 10)
    record(AAPL=data.current(symbol('AAPL'), 'price'))
```

Same CLI-style flags apply, but the algorithm lives inline in the notebook cell, and `-o name` creates a Python variable (rather than writing a file) holding the resulting DataFrame.

C. `run_algorithm()` — for Running Like a Normal Script/IDE

```
from zipline import run_algorithm
import pandas as pd
import pandas_datareader.data as web

start = pd.Timestamp('2014'); end = pd.Timestamp('2018')
sp500 = web.DataReader('SP500', 'fred', start, end).SP500
benchmark_returns = sp500.pct_change()

result = run_algorithm(
    start=start.tz_localize('UTC'), end=end.tz_localize('UTC'),
    initialize=initialize, handle_data=handle_data,
    capital_base=100000, benchmark_returns=benchmark_returns,
    bundle='quandl', data_frequency='daily')
```

This is the most flexible option since it's just a Python function call — good for running inside an IDE, a script, or a testing/CI pipeline.

The Performance DataFrame

Whichever way you run it, Zipline returns/saves a `pd.DataFrame` with **one row per trading day (or minute)**. Key columns include:

Column	Meaning
`portfolio_value`	Total account value (cash + holdings) at that point
`returns`	Period return
`algorithm_period_return` / `benchmark_period_return`	Cumulative return so far, strategy vs. benchmark

`alpha`, `beta`, `sharpe`, `sortino`	Rolling risk/performance stats, computed automatically
`algo_volatility`, `benchmark_volatility`	Rolling annualized volatility
`max_drawdown`	Running max drawdown to date
`positions`	List of currently held positions (dicts, with sid, amount, cost basis, last sale price, etc.)
`transactions`	List of any fills that happened this bar
`orders`	List of any orders created/updated this bar
`ending_cash`, `ending_value`, `ending_exposure`	Cash and market value of holdings at end of bar
`longs_count` / `shorts_count`, `long_value` / `short_value`	Breakdown of long vs. short exposure
`gross_leverage` / `net_leverage`	Leverage measures
Any name you passed to `record()`	Your custom tracked values (e.g. `AAPL`, `short_mavg`)

Placing Orders

Function	Semantics
`order(asset, amount, limit_price=None, stop_price=None, style=None)`	Order a **fixed number of shares** (+ = buy/cover, − = sell/short)
`order_value(asset, value, ...)`	Order enough shares to transact a **fixed dollar value**. Equivalent to `order(asset, value / current_price)`
`order_percent(asset, percent, ...)`	Order enough shares to allocate a given **% of current portfolio value** to the asset
`order_target(asset, target, ...)`	Order the difference needed so the position reaches a **target share count**
`order_target_value(asset, target, ...)`	Same, but target is a **dollar value** (for Futures, this becomes target *exposure*, since futures have no "value" per se)
`order_target_percent(asset, target, ...)`	Same, but target is a **% of portfolio value**

Critical caveat repeated in the docs for all four "target" functions: none of them account for your own still-open orders. Calling the same `order_target*` twice before the first fills will double the effect, since each call only looks at your *current filled* position, not your pending orders.

Execution Styles (how/when an order fills)

- **`MarketOrder`** (default) — filled at the current/next available market price.
- **`LimitOrder(limit_price)`** — filled only at `limit_price` or better.

- `StopOrder(stop_price)` — becomes a market order once price crosses `stop_price` (in the direction that would normally hurt you, i.e. this is a protective/breakout trigger).
- `StopLimitOrder(limit_price, stop_price)` — combination: becomes a *limit* order once the stop price is triggered.

You can pass `limit_price=` or `stop_price=` directly to `order()` instead of constructing a style object. Do not combine either shortcut with an explicit `style=` argument.

Managing Orders

- `get_order(order_id)` — look up a specific order.
- `get_open_orders(asset=None)` — all open orders (dict of asset → list, or a list if `asset` given).
- `cancel_order(order_param)` — cancel by ID or Order object.
- **Cancel policies.** Use `set_cancel_policy(cancel_policy)`. Built-ins are `EODCancel()` (cancel all open orders at end of day; minute-frequency simulations only) and `NeverCancel()` (the default; orders are Good-Til-Canceled).

Looking Up Assets

- `symbol(symbol_str, country_code=None)` — look up a single Equity by ticker (as of the "symbol lookup date," since tickers can be reused by different companies over time).
- `symbols(*args)` — look up several tickers at once, returns a list.
- `future_symbol(symbol)` — look up a Future contract by its symbol.
- `sid(sid)` — look up any Asset by its permanent integer ID (doesn't change over time, unlike ticker symbols).
- `set_symbol_lookup_date(dt)` — change the reference date used to resolve ticker symbols to assets.

Trading Controls (Safety Rails)

These raise a `TradingControlException` if violated — designed to prevent a bug (or, on a live account, a fat-finger) from doing serious damage:

- `set_do_not_order_list(restricted_list)` — assets that can never be ordered.
- `set_long_only()` — disallow short positions entirely.
- `set_max_leverage(max_leverage)` — cap on account leverage.
- `set_max_order_count(max_count)` — cap on number of orders per day.
- `set_max_order_size(asset=None, max_shares=None, max_notional=None)` — cap the size of any single order.
- `set_max_position_size(asset=None,max_shares=None,max_notional=None)` — cap the size of any resulting position (note: splits/dividends or price moves can still push you slightly over after the fact).

Commission and Slippage Models

These model the two main sources of "backtest looks better than reality" error.

Commission (`set_commission(us_equities=..., us_futures=...)`), can only be set inside `initialize()`:

Model	Formula
`commission.PerShare(cost=0.001, min_trade_cost=0.0)`	Flat $ per share traded, with an optional minimum per trade. **This is Zipline's default for equities.**
`commission.PerTrade(cost=0.0)`	Flat $ per trade regardless of size (multi-fill orders charge the full amount on the first fill)
`commission.PerDollar(cost=0.0015)`	$ cost proportional to dollar value transacted (default 0.15% of value)

For custom logic, subclass `CommissionModel` and implement `calculate(order,transaction)`.

Slippage (`set_slippage(us_equities=..., us_futures=...)`):

Model	Formula / Behavior
`slippage.FixedSlippage(spread=0.0)`	Buys fill at `close+spread/2`; sells fill at `close-spread/2`. No volume/liquidity limit — will always fill in full as soon as *any* trading occurs, even if your order is bigger than the historical volume (unrealistic for large orders, but simple).
`slippage.VolumeShareSlippage(volume_limit=0.025,price_impact=0.1)`	Models slippage as a function of how much of the bar's volume your order represents. Buys fill at `price*(1+price_impact*volume_share**2)`; sells at `price*(1-price_impact*volume_share**2)`, where `volume_share` is the fraction of the bar's volume your order fills (capped at `volume_limit`, e.g. 2.5% of the bar's volume by default). This is more realistic since larger orders (relative to available liquidity) get worse fills.

You can also subclass `SlippageModel` and implement `process_order(data, order)` to build fully custom fill logic (it can even raise `LiquidityExceeded` to signal "no more of this order fills this bar").

Scheduling Recurring Logic

Rather than checking "is it a certain day/time?" manually inside `handle_data` every bar, you can register a function to run automatically on a schedule:

```
from zipline.api import schedule_function, date_rules, time_rules

schedule_function(my_rebalance_func,
                  date_rule=date_rules.week_start(),
                  time_rule=time_rules.market_open(minutes=30))
```

- **date_rules** provides `every_day()`, `month_start(days_offset=0)`, `month_end(days_offset=0)`, `week_start(days_offset=0)`, and `week_end(days_offset=0)`.
- **time_rules** provides `every_minute`, `market_open(offset=None,hours=None,minutes=None)`, and `market_close(offset=None,hours=None,minutes=None)`. With no offset, the default trigger is one minute after open or one minute before close.

This is the idiomatic way to implement e.g. "rebalance my portfolio on the first trading day of each month" without manual date-checking logic.

Benchmark, Recording, and Misc

- `set_benchmark(benchmark_asset)` — set which asset to use as the return benchmark (for alpha/beta); dividends on the benchmark are automatically reinvested in the calculation.
- `record(**kwargs)` — track custom named values per bar (shows up as columns in the performance DataFrame and in `analyze()`).
- `get_environment(field='platform')` — introspect the run itself: `'arena'` (backtest vs. live), `'data_frequency'`, `'start'`/`'end'`, `'capital_base'`, `'platform'`, or `'*'` for everything as a dict.
- `fetch_csv(url, ...)` — pull in an arbitrary external CSV (e.g. a sentiment index or alternative dataset) and make it queryable through `data`, keyed by date (and optionally by symbol).

The Pipeline API (Cross-Sectional Computations)

A **Pipeline** lets you efficiently compute the same calculation (a "Factor," "Filter," or "Classifier") across your **entire tradeable universe** at once, once per day — much faster and more memory-efficient than looping over each asset yourself inside `handle_data`. This is the mechanism used for things like "rank all 8,000 US stocks by 20-day momentum" or "select all stocks whose 10-day average volume exceeds $1M."

Registering and reading a pipeline:

```
from zipline.api import attach_pipeline, pipeline_output

def initialize(context):
    pipe = Pipeline(columns={...}, screen=my_filter)
    attach_pipeline(pipe, 'my_pipeline')

def before_trading_start(context, data):
    context.pipeline_data = pipeline_output('my_pipeline')
```

- `attach_pipeline(pipeline, name, chunks=None, eager=True)` — registers a pipeline to run once per trading day.
- `pipeline_output(name)` — fetches that day's already-computed results as a DataFrame (rows = assets that passed the `screen`, columns = the pipeline's declared `columns`).

Core Pipeline building blocks:

- **`Filter`** — a Boolean-output term (e.g. "10-day VWAP < $20"). Filters are commonly built via comparison operators on Factors (`<`, `<=`, `>`, `>=`, `!=`, `.eq()`), and combined with `&` (and), `|` (or), `~` (not). A Filter set as a Pipeline's `screen` drops non-passing asset/date rows from the output entirely.
- **`Factor`** — a numerical (or date) output term; the most common Pipeline term. Factors support arithmetic (`+`, `-`, `*`, `/`), and useful transformation methods:
  - `.rank(method='ordinal',ascending=True,mask=None,groupby=None)` — cross-sectional rank each day
  - `.zscore(mask=None, groupby=None)` — standardize `(row - mean) / std` each day
  - `.demean(mask=None, groupby=None)` — subtract the row mean
  - `.winsorize(min_percentile, max_percentile, ...)` — clip extreme values to a percentile band (limits outlier influence without fully discarding them)
  - `.quantiles(bins, mask=None)` — bucket each day's cross-section into `bins` quantile groups
  - Rolling cross-sectional statistical methods against a target series include `.pearsonr(target,correlation_length,mask=None)`, `.spearmanr(...)`, and `.linear_regression(target,regression_length,mask=None)`.
- **`CustomFactor`** — the base class for writing your **own** Factor. You override `compute(self, today, assets, out, *inputs)`, where `inputs` are NumPy arrays shaped `(window_length, num_assets)` for each declared input column, and you write your result into `out`. Example:

```
class TenDayRange(CustomFactor):
    inputs = [USEquityPricing.high, USEquityPricing.low]
    window_length = 10

    def compute(self, today, assets, out, highs, lows):
        out[:] = np.nanmax(highs, axis=0) - np.nanmin(lows, axis=0)
```

Why this matters: the Pipeline engine handles all the tricky bookkeeping (aligning windows across assets with different listing dates, forward-filling NaNs for not-yet-listed stocks, batching computation efficiently) so your `compute()` function can just focus on the math.

The Blotter

A **blotter** is the component that actually turns your placed orders into transactions each bar — it books the trade, applying the active slippage and commission models. Zipline's default is `SimulationBlotter`, and you generally don't need to touch this directly unless you're building a custom execution model; it's exposed mainly for extensibility (e.g. to log every order/fill for custom analysis, or to implement custom order-batching logic via `batch_order()`).

1.6.4 Backtesting.py

What It Is

Backtesting.py is a small, fast, dependency-light (Pandas/NumPy/Bokeh) Python framework purpose-built for backtesting **single-asset** strategies driven by technical indicators/signals — it deliberately does **not** try to support multi-asset portfolio rebalancing or stock-picking across a universe (that's Zipline's job). Its selling point is a tiny, memorable API, fast vectorized execution, and beautiful **interactive** (Bokeh-based) plots and heatmaps, all built in — no need to pair it with something like Pyfolio.

Installation:

```
pip install backtesting
```

Data Format

You bring your own OHLC(V) data as a `pandas.DataFrame` with columns `'Open'`, `'High'`, `'Low'`, `'Close'`, and optionally `'Volume'`. If you're missing some columns (e.g. you only have close prices), you can fake them: `df['Open'] = df['High'] = df['Low'] = df['Close']`. The index should ideally be a **datetime index**; otherwise a simple monotonic range index works too. Extra columns (e.g. a sentiment score) are allowed and remain accessible to your strategy.

The package ships a small sample dataset for experimentation:

```
from backtesting.test import GOOG   # daily OHLCV for Google, 2004--2013
```

Defining a Strategy

A strategy subclasses `Strategy` and overrides two methods:

```
from backtesting import Strategy
from backtesting.lib import crossover
import pandas as pd

def SMA(values, n):
    return pd.Series(values).rolling(n).mean()

class SmaCross(Strategy):
    n1 = 10   # class variables => automatically optimizable parameters
    n2 = 20

    def init(self):
        # Precompute indicators in a vectorized way, over the FULL dataset
        self.sma1 = self.I(SMA, self.data.Close, self.n1)
        self.sma2 = self.I(SMA, self.data.Close, self.n2)

    def next(self):
        # Called once per bar; only "sees" data up to the current bar
        if crossover(self.sma1, self.sma2):
            self.position.close()
            self.buy()
        elif crossover(self.sma2, self.sma1):
            self.position.close()
```

```
            self.sell()
```

- **`init()`** runs once, before simulation starts. This is where you declare/precompute indicators — you have access to the *entire* dataset here (for efficient vectorized computation), but nothing you do here can leak into `next()`'s trading decisions in a look-ahead sense, because...
- **`next()`** is called once per bar as the simulation steps forward in time. Inside `next()`, `self.data` and any `self.I()`-declared indicator arrays are **truncated to only what would have been known so far** — i.e. `array[-1]` is always "now," `array[-2]` is "one bar ago," etc. This is the mechanism that prevents look-ahead bias.
- **Important limitation:** Backtesting.py cannot simulate trading *within* a candle. New orders are filled at the **next bar's open** (or the *current* bar's close, if you set `trade_on_close=True`). If you need intra-candle decisions (day-trading logic), you need more granular (e.g. hourly/minute) source data instead.
- **`self.I(func,*args,name=None,plot=True,overlay=None,color=None,scatter=False,**kwargs)`** wraps an indicator-computing function so that (a) it is automatically plotted with a sensible legend/label, and (b) it participates in the same gradually revealed windowing as `self.data`. Rolling indicators typically front-pad with NaN for their warm-up period — **the backtest does not actually begin trading until the first bar where *all* declared indicators are non-NaN** (e.g. bar 201 if you use a 200-bar moving average), which can shift results if you change indicator lengths.

Why use `crossover()` instead of manual comparisons?

```
from backtesting.lib import crossover
crossover(self.sma1, self.sma2)   # True exactly on the bar sma1 crosses above
    sma2
```

This is equivalent to (but much more readable than) writing:

```
self.sma1[-2] < self.sma2[-2] and self.sma1[-1] > self.sma2[-1]
```

Running a Backtest

```
from backtesting import Backtest

bt = Backtest(GOOG, SmaCross, cash=10_000, commission=.002)
stats = bt.run()
```

`Backtest(data, strategy, ...)` Constructor Parameters

Parameter	Meaning
`cash`	Starting capital (default 10,000)

`spread`	Constant bid-ask spread, as a fraction of price (e.g. `0.0002` for a 0.2‰ forex spread)
`commission`	Commission rate. A single float (e.g. `0.01` = 1%, applied at **both** entry and exit), a tuple `(fixed, relative)` (e.g. `(100, .01)` = min $100 + 1%), or a callable `func(order_size, price) -> float` for fully custom commission logic. Negative values model a market-maker rebate.
`margin`	Required margin ratio for leveraged trading, e.g. `0.02` for 50:1 leverage (1/leverage)
`trade_on_close`	If `True`, market orders fill at the *current* bar's close instead of the *next* bar's open
`hedging`	If `True`, long and short positions can coexist; if `False` (default), opposite orders close existing trades first, FIFO-style
`exclusive_orders`	If `True`, every new order automatically closes out the prior trade first — enforces "at most one open trade (long or short) at a time," letting you skip manual `self.position.close()` calls
`finalize_trades`	If `True`, any trade still open at the very end of the backtest is force-closed on the last bar so it's included in the statistics (otherwise it's excluded as "unrealized")

There's also `FractionalBacktest` (in `backtesting.lib`) for assets like Bitcoin where you need to trade fractional units — it works by rescaling price/volume by a `fractional_unit` (default: 1 satoshi) so the underlying whole-share engine can still be used.

The `run()` Output — Statistics Explained

`bt.run()` returns a `pd.Series` of results. Example output (annotated):

Stat	Meaning
`Start`, `End`, `Duration`	Backtest date range and span
`Exposure Time [%]`	% of the time the strategy held *any* position (long or short)
`Equity Final [$]` / `Equity Peak [$]`	Final and highest account value reached
`Return [%]`	Total return over the period
`Buy & Hold Return [%]`	What you'd have made just holding the asset the whole time — the natural benchmark
`Return (Ann.) [%]`	Annualized return

`Volatility (Ann.) [%]`	Annualized volatility of the equity curve
`CAGR [%]`	Compound annual growth rate
`Sharpe Ratio` / `Sortino Ratio` / `Calmar Ratio`	Same definitions as in the Pyfolio section above
`Alpha [%]` / `Beta`	Performance/sensitivity relative to buy & hold
`Max. Drawdown [%]` / `Avg. Drawdown [%]`	Depth of worst / typical drawdown
`Max. Drawdown Duration` / `Avg. Drawdown Duration`	How long the worst / a typical drawdown lasted
`# Trades`	Total number of round-trip trades taken
`Win Rate [%]`	% of trades that were profitable
`Best Trade [%]` / `Worst Trade [%]` / `Avg. Trade [%]`	Individual trade return extremes and average
`Max. Trade Duration` / `Avg. Trade Duration`	How long trades are typically held
`Profit Factor`	Gross profit / gross loss (same meaning as in Pyfolio)
`Expectancy [%]`	Average expected % return per trade
`SQN`	Van Tharp's **System Quality Number** — a measure combining trade count, average PnL, and PnL standard deviation, used as **Backtesting.py's default metric to maximize during optimization**
`Kelly Criterion`	The theoretically optimal fraction of capital to risk per trade, per the Kelly formula
`_strategy`	The Strategy instance (and its parameter values) used for this run
`_equity_curve`	Full DataFrame: `Equity`, `DrawdownPct`, `DrawdownDuration` per bar
`_trades`	Full DataFrame: one row per closed trade, with entry/exit bar, price, PnL, return %, duration, and (helpfully) the value of every declared indicator at entry and exit

Trading Inside `next()`: Orders, Positions, and Trades

Placing Orders

```
self.buy(size=.9999, limit=None, stop=None, sl=None, tp=None, tag=None)
self.sell(size=.9999, limit=None, stop=None, sl=None, tp=None, tag=None)
```

- `size`: if between 0 and 1, interpreted as a **fraction of currently available liquidity** (cash + unrealized P/L − used margin) to commit; if ≥ 1, an **absolute number of units**.

- **`limit/stop`**: set these to place limit/stop(-limit) orders instead of a plain market order.
- **`sl/tp`**: stop-loss / take-profit price — automatically creates a linked contingent order that closes the resulting Trade once that price is touched. These are also **writable properties on an existing `Trade`** (`trade.sl = new_price`), letting you implement trailing stops by updating them inside `next()`.
- **`tag`**: an arbitrary marker you can attach to track a specific order/trade through to the results (e.g. to tell apart different entry signals in `_trades`).
- **Caution:** `self.sell(size=.1)` does **not** automatically close an existing `self.buy(size=.1)` position. Netting occurs only with `exclusive_orders=True`, or when price, spread, and commission are all zero and match exactly. To deliberately exit, use `Position.close()` or `Trade.close()` instead of placing an opposite order.
- Unless `trade_on_close=True`, market orders fill on the **next** bar's open; limit/stop orders fill whenever their trigger condition is met.

`Order` (pending, not yet filled)

Properties: `is_long`, `is_short`, `is_contingent` (True for SL/TP bracket orders auto-placed against an open Trade), `limit`, `stop`, `size`, `sl`, `tp`, `tag`. Method: `cancel()`.

`Position` (`self.position`) — your current net holding

Truthy/falsy in boolean context (`if self.position:`). Properties: `is_long`, `is_short`, `size` (negative if short), `pl` (cash P/L), `pl_pct` (% P/L). Method: `close(portion=1.0)` — closes `portion` of every active trade.

`Trade` (`self.trades` = open, `self.closed_trades` = settled)

Properties: `size`, `entry_bar` / `exit_bar`, `entry_price` / `exit_price`, `entry_time` / `exit_time`, `is_long` / `is_short`, and `pl` / `pl_pct` (commissions appear once closed). It also exposes `value`, writable `sl` / `tp`, and the inherited `tag`. Method: `close(portion=1.0)`.

Helper Functions (`backtesting.lib`)

Function	Purpose
`crossover(series1, series2)`	True if `series1` just crossed **above** `series2` on this bar
`cross(series1, series2)`	True if the two series crossed in **either** direction
`barssince(condition, default=inf)`	How many bars since `condition` was last True
`quantile(series, quantile=None)`	If `quantile=None`, returns the *rank* (0–1) of the series' last value among its history; if given a value in (0,1), returns the *value* at that quantile

`resample_apply(rule, func, series, *args, agg=None, **kwargs)`	Apply an indicator function to `series` after resampling it to a different (usually coarser) timeframe — e.g. compute a **daily** SMA(10) inside an **hourly** strategy, then forward-fill it back down to hourly resolution, all with correct handling to avoid look-ahead bias (`label='right'` on the resample)
`plot_heatmaps(heatmap, agg='max', ...)`	Plot a grid of 2D heatmaps from an `optimize(..., return_heatmap=True)` result, one per pair of parameters, aggregating any remaining dimensions with `agg`
`compute_stats(*,stats,data,trades=None,risk_free_rate=0.0)`	Recompute summary statistics from a filtered subset of trades. For example, use only long trades with `compute_stats(stats=stats,trades=stats._trades[stats._trades.Size>0],data=GOOG)`.
`random_ohlc_data(example_data, frac=1.0, random_state=None)`	Generates synthetic OHLC data with similar descriptive statistics to real `example_data` — useful for Monte Carlo robustness/stress-testing of a strategy against data it wasn't fit to

Composable Base Strategies

- **`SignalStrategy`** — subclass this and call `super().init()`, then `self.set_signal(entry_size,exit_portion=None)` with precomputed entry/exit signal arrays (positive = long, negative = short). This supports a **vectorized** strategy instead of manually issuing orders inside `next()`.
- **`TrailingStrategy`** — adds an automatic **ATR-based trailing stop-loss**. Call `set_trailing_sl(n_atr=6)` to trail the price by `n_atr` average-true-range units, or `set_trailing_pct(pct=0.05)` to trail by a percentage (internally converted to ATR units), and optionally `set_atr_periods(periods=100)` to change the ATR lookback.
- **`MultiBacktest`** — runs the *same* strategy across **multiple instruments in parallel** (e.g. `MultiBacktest([EURUSD, BTCUSD], SmaCross)`), returning a DataFrame of stats per instrument — useful for checking whether a strategy is robust across different assets, not overfit to one.

Optimization

```
stats = bt.optimize(
    n1=range(5, 30, 5),
    n2=range(10, 70, 5),
    maximize='Equity Final [$]',
    constraint=lambda p: p.n1 < p.n2)
```

- Any keyword whose name matches a Strategy class variable becomes a **parameter sweep**; pass a list/range of candidate values.
- `maximize`: either the name of a column from the `run()` stats (e.g. `'Equity Final [$]'`, `'Sharpe Ratio'`), or a custom function `f(stats) -> float` to maximize. Default: Van Tharp's **SQN**.
- `method`: `"grid"` (default) — exhaustive (or, with `max_tries`, randomized) search over the full parameter grid; or `"sambo"` — model-based ("SAMBO") optimization that intelligently searches for a near-optimal combination in at most `max_tries` evaluations (default 200), much cheaper than grid search for large parameter spaces.
- `constraint`: a function over a parameter combination that returns `True` if that combination is admissible to test (e.g. rejecting `n1 >= n2`, which would be a meaningless "moving average" crossover).
- `return_heatmap=True`: additionally returns a `pd.Series` with a MultiIndex over every admissible parameter combination and its resulting score — feed this into `plot_heatmaps()` to visualize the whole parameter landscape (not just the single best point), which helps you judge whether the "best" result is a robust plateau or an overfit spike.
- `random_state`: set for reproducible optimization runs.
- After optimizing, `stats._strategy` gives you the winning **Strategy instance**, e.g. `<Strategy SmaCross(n1=10,n2=15)>`.

Overfitting warning (explicit in the docs): optimizing on a single in-sample period can make performance look far better than it will be live — always be skeptical of a heavily-tuned single backtest result, and prefer checking whether nearby parameter combinations (visible in the heatmap) perform similarly well, rather than only trusting the single "best" point.

Plotting

```
bt.plot(plot_volume=True, plot_pl=True, plot_drawdown=False, plot_trades=True,
    ...)
```

Produces an interactive **Bokeh** HTML chart (opened in your browser by default) with:

- The OHLC candlestick chart (optionally with larger-timeframe candles "superimposed," e.g. daily candles over hourly data)
- All indicators declared via `self.I()`, automatically placed either overlaying price or in their own subplot
- Equity curve (`plot_equity`) or cumulative return curve (`plot_return`) — mutually complementary views of the same thing (equity = return + 100%)
- Profit/loss markers (`plot_pl`), trade volume (`plot_volume`), a separate drawdown-depth chart (`plot_drawdown`)
- "Tractor beam" hash-marks between each trade's entry and exit (`plot_trades`)
- `resample=True` automatically downsamples very long OHLC series so Bokeh doesn't choke on plotting >10,000 candles, while still aggregating equity/drawdown/trade data sensibly for the coarser view

Worked Example Summary (Google Stock, SMA Crossover)

Running the `SmaCross` strategy (10/20-day SMA) on ~9 years of Google daily data with `cash=10_000, commission=.002`:

- Untuned (`n1=10, n2=20`): ~**463%** return vs. **607%** buy & hold, Sharpe ≈ 0.60, Max Drawdown ≈ −34%.
- After optimizing `n1` over 5–30 and `n2` over 10–70 (constrained to `n1 < n2`), maximizing **Equity Final [$]**: the best combination (`n1=10, n2=15`) achieves ~**678%** return — beating buy & hold this time — but at a *worse* max drawdown (−44.5%) and lower Sharpe than you might hope, illustrating the classic **return-vs-risk (and overfitting) trade-off** that any optimization needs to be checked against.

1.6.5 Putting It Together

A realistic research pipeline combining these three tools might look like:

1. **Prototype and iterate quickly** on a single-asset idea in **Backtesting.py** — cheap to run, built-in optimizer, and good interactive plots for fast feedback.
2. Once the strategy needs a **realistic multi-asset universe, Pipeline-based stock selection, or more nuanced order/commission/slippage modeling**, port it to **Zipline**.
3. Use **Pyfolio** on the Zipline output (or on any returns series, really) to get a rigorous, standardized **tear sheet** — the in-sample/out-of-sample split, drawdown analysis, round-trip trade quality, and stress-period behavior are exactly the kind of due-diligence checks you'd want before trusting a strategy with real capital.

Derivatives

Chapter 2

Stochastic Processes

This chapter develops the probability and information structures used to describe random systems through time. It begins by distinguishing stochastic models from deterministic ones, then builds the formal language needed for random walks, Markov chains, Brownian motion, and derivative pricing.

2.1 Deterministic vs. Stochastic Models

2.1.1 Deterministic Models

Definition.
A deterministic model predicts outputs with **complete certainty** for a given set of inputs and parameters.

Characteristics.

- No randomness is involved.
- Repeating the process with the same inputs **always** yields the same result.

Examples.

- Calculating the position of the moon in its orbit.
- Computing the cost of electricity consumed from the units used and the price per unit.

2.1.2 Stochastic Models

Definition.
A stochastic model includes elements of **uncertainty and randomness** in its outputs.

Characteristics.

- The same inputs can produce **different outputs** because of underlying randomness.
- Outputs depend on **probability distributions**, not fixed formulas.

Examples.

- **Stock-price prediction:** affected by unpredictable news, market trends, and investor behaviour.
- **Crop-yield estimation:** influenced by weather, pests, and soil quality.

2.1.3 Comparison

Deterministic Models	Stochastic Models
Predictable outputs	Random outputs

No uncertainty	Includes uncertainty
The same inputs always produce the same result	The same inputs may produce different results
Suitable when all variables are known	Suitable for systems influenced by randomness

Why stock prices are stochastic, not deterministic. The cost of electricity is deterministic because it depends on fixed, known inputs: units consumed and the rate per unit. Stock prices, by contrast, are influenced by many unpredictable factors, so their evolution over time must be modelled as a **stochastic process**.

2.2 Mathematical Foundations of Stochastic Processes

Before defining a stochastic process formally, we need the mathematical structure on which random outcomes and their probabilities are defined.

2.2.1 Probability Spaces

A **probability space** is a triple

$$(\Omega, \mathcal{F}, \mathbb{P}),$$

with the following components.

1. **Sample space Ω.** This is the set of every possible outcome of the experiment.
2. **Sigma algebra $\mathcal{F}$.** This is a collection of subsets of Ω, called **events**, to which probabilities can be assigned.
3. **Probability measure $\mathbb{P}$.** This assigns a number $\mathbb{P}(A) \in [0, 1]$ to every event $A \in \mathcal{F}$.

Sigma-algebra requirements.

A collection $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ is a sigma algebra if

1. $\Omega \in \mathcal{F}$;
2. if $A \in \mathcal{F}$, then $A^c \in \mathcal{F}$;
3. if $A_1, A_2, \ldots \in \mathcal{F}$, then $\bigcup_{k=1}^{\infty} A_k \in \mathcal{F}$.

These conditions also imply that $\varnothing \in \mathcal{F}$ and that countable intersections of measurable events belong to $\mathcal{F}$.

Probability-measure requirements.

For events in $\mathcal{F}$, a probability measure satisfies

$$\mathbb{P}(A) \geq 0, \qquad \mathbb{P}(\Omega) = 1,$$

and, for pairwise disjoint events $A_1, A_2, \ldots$,

$$\mathbb{P}\left(\bigcup_{k=1}^{\infty} A_k\right) = \sum_{k=1}^{\infty} \mathbb{P}(A_k).$$

Example: rolling a fair die.
The sample space is

$$\Omega = \{1, 2, 3, 4, 5, 6\}.$$

Define

$$A_{\text{odd}} = \{1, 3, 5\}, \qquad A_{\text{even}} = \{2, 4, 6\}.$$

If we record only whether the result is odd or even, the relevant sigma algebra is

$$\mathcal{F}_{\text{parity}} = \{\varnothing, \Omega, A_{\text{odd}}, A_{\text{even}}\}.$$

For a fair die,

$$\mathbb{P}(A) = \frac{|A|}{6}, \qquad A \subseteq \Omega,$$

so $\mathbb{P}(A_{\text{odd}}) = \mathbb{P}(A_{\text{even}}) = 1/2$. If every individual outcome must be distinguishable, one normally uses the full power set $\mathcal{F} = \mathcal{P}(\Omega)$.

2.2.2 Measurable Functions

Let $(E, \mathcal{E})$ and $(F, \mathcal{G})$ be measurable spaces. A function $f : E \to F$ is **measurable** if

$$B \in \mathcal{G} \quad \Longrightarrow \quad f^{-1}(B) = \{x \in E : f(x) \in B\} \in \mathcal{E}.$$

In words, every measurable event in the output space can be traced back to a measurable event in the input space. This condition is what makes expressions such as

$$\mathbb{P}(f \in B) = \mathbb{P}\Big(f^{-1}(B)\Big)$$

well defined.

For $\mathbb{R}^d$, the usual sigma algebra is the **Borel sigma algebra** $\mathcal{B}(\mathbb{R}^d)$, generated by all open subsets of $\mathbb{R}^d$.

2.2.3 Formal Definition

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, let T be an index set, and let $(E, \mathcal{E})$ be a measurable state space. A **stochastic process** is a family

$$X = (X_t)_{t \in T}$$

such that, for every fixed $t \in T$,

$$X_t : (\Omega, \mathcal{F}) \longrightarrow (E, \mathcal{E})$$

is a measurable random variable. Equivalently, it can be viewed as a map

$$X : T \times \Omega \longrightarrow E, \qquad (t, \omega) \longmapsto X_t(\omega).$$

Common choices are

$$T = \{0, 1, 2, \ldots\} \quad \text{or} \quad T = [0, \infty),$$

for discrete and continuous time respectively, and

$$E = \mathbb{R}^d, \qquad \mathcal{E} = \mathcal{B}(\mathbb{R}^d),$$

for a d-dimensional process.

Dimension	Process	Interpretation
One-dimensional	$X_t \in \mathbb{R}$	A stock price, interest rate, or one coordinate of a particle.
Two-dimensional	$X_t = (X_t^{(1)}, X_t^{(2)}) \in \mathbb{R}^2$	The position of a particle moving in a plane.
Three-dimensional	$X_t = (X_t^{(1)}, X_t^{(2)}, X_t^{(3)}) \in \mathbb{R}^3$	The position of a particle moving through physical space.

Continuous time does not imply continuous paths. Even when $T = [0, \infty)$, the function $t \mapsto X_t(\omega)$ may jump. A continuous-time process may therefore have continuous paths, jump paths, or more irregular behaviour.

Stock-price example.
If S_t denotes the price of a stock at time t, then each S_t is a random variable. Before the market evolves, S_t is unknown; after an outcome ω is realised, $S_t(\omega)$ is the observed price at time t.

2.2.4 Sample Paths

The two inputs t and ω have different roles:

- fixing t gives a random variable $\omega \mapsto X_t(\omega)$;
- fixing ω gives an ordinary function $t \mapsto X_t(\omega)$, called a **sample path** or **realisation**.

Different outcomes $\omega_1, \omega_2, \ldots$ generally generate different paths.

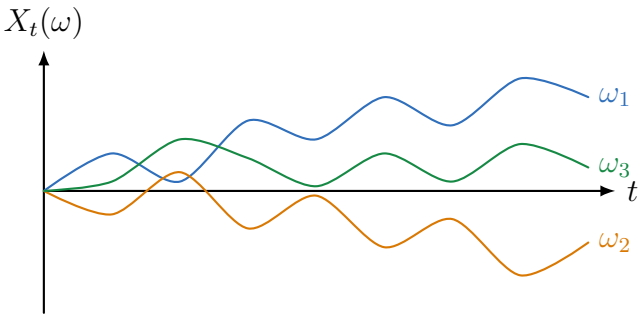


Figure 2.1: Different outcomes generate different sample paths of the same stochastic process.

2.2.5 Stationarity

A process $X = (X_t)$ is **strictly stationary** if shifting the time origin does not change any of its finite-dimensional distributions. Formally, for every n, every $t_1, \ldots, t_n$, and every admissible shift h,

$$(X_{t_1+h}, \ldots, X_{t_n+h}) \overset{d}{=} (X_{t_1}, \ldots, X_{t_n}).$$

The process therefore has the same probabilistic behaviour after a time shift.

A second-order process is **weakly stationary** if

$$\mathbb{E}[X_t] = m$$

is constant and

$$\mathrm{Cov}(X_s, X_t) = C(t-s)$$

depends only on the time lag.

> **Important distinction.** Stationarity does not say that the future is independent of the past. It says that the distributional rules are unchanged by shifting time. Dependence may still be present.

2.2.6 Stationary and Independent Increments

These are properties of changes in the process rather than of its absolute level.

Stationary increments.

The process has stationary increments if the distribution of

$$X_{t+h} - X_t$$

depends only on the interval length h, not on the starting time t. When $X_0 = 0$, this can be written

$$X_{t+h} - X_t \overset{d}{=} X_h.$$

Independent increments.

For every sequence

$$0 \le t_0 < t_1 < \cdots < t_n,$$

the increments

$$X_{t_1} - X_{t_0}, \quad X_{t_2} - X_{t_1}, \quad \ldots \quad , X_{t_n} - X_{t_{n-1}}$$

are independent random variables. Thus changes over non-overlapping time intervals do not influence one another.

For example, if $t_1 < t_2 \le t_3 < t_4$, independent increments imply that

$$X_{t_2} - X_{t_1} \quad \text{and} \quad X_{t_4} - X_{t_3}$$

are independent.

Property	Meaning
Stationary process	The joint distribution of the levels is unchanged by a common time shift.
Stationary increments	The distribution of a change depends only on the length of its interval.
Independent increments	Changes over disjoint time intervals are independent.

Standard Brownian motion has stationary and independent increments, but it is not itself stationary because $\mathrm{Var}(W_t) = t$ changes with time.

2.2.7 Filtrations: Information Through Time

A **filtration** is an increasing family of sub-sigma algebras

$$(\mathcal{F}_t)_{t\in T}$$

such that

$$s \le t \quad \Longrightarrow \quad \mathcal{F}_s \subseteq \mathcal{F}_t \subseteq \mathcal{F}.$$

The sigma algebra $\mathcal{F}_t$ represents all information available by time t. Information accumulates as time passes, so later sigma algebras are larger. The quadruple

$$(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\in T}, \mathbb{P})$$

is called a **filtered probability space**.

Three-coin-flip example.

Suppose a coin is flipped three times. Then

$$\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}.$$

Let C_k denote the result of flip k, and define the natural filtration

$$\mathcal{F}_k = \sigma(C_1, \ldots, C_k).$$

Time	Information atoms	Interpretation	Events
0	Ω	No flip has been observed. Only impossible and certain events are distinguishable.	2
1	$A_H = \{HHH, HHT, HTH, HTT\}$, $A_T = \{THH, THT, TTH, TTT\}$	The first flip is known.	$2^2 = 4$
2	$A_{HH}, A_{HT}, A_{TH}, A_{TT}$, according to the first two flips	The first two flips are known.	$2^4 = 16$
3	The eight singleton outcomes	All three flips are known.	$2^8 = 256$

Each sigma algebra consists of every union of its information atoms. For example,

$$\mathcal{F}_0 = \{\varnothing, \Omega\}, \qquad \mathcal{F}_1 = \{\varnothing, \Omega, A_H, A_T\}.$$

If the realised sequence is HTH, then the information narrows from Ω at time 0, to A_H after the first flip, to A_{HT} after the second, and finally to the singleton $\{HTH\}$.

2.2.8 Adapted Processes

A process $X = (X_t)$ is **adapted** to the filtration $(\mathcal{F}_t)$ if

$$X_t \text{ is } \mathcal{F}_t\text{-measurable for every } t.$$

Adaptedness means that the value X_t can be determined from information available at time t; it does not use future information.

For a stock-price process S_t, adaptation means that events such as

$$\{S_t \le s\}$$

belong to $\mathcal{F}_t$. The market can observe the current and past prices, but not future prices.

2.2.9 Moment-Generating Functions

For a real random variable X, the **moment-generating function** (MGF) is

$$M_X(u) = \mathbb{E}\Big[e^{uX}\Big],$$

for those real u at which the expectation is finite. Using

$$e^{uX} = \sum_{k=0}^{\infty} \frac{u^k X^k}{k!},$$

and interchanging expectation and summation when justified gives

$$M_X(u) = \sum_{k=0}^{\infty} \frac{u^k}{k!} \mathbb{E}[X^k].$$

Hence, if the MGF exists on an open interval around zero,

$$M_X^{(k)}(0) = \mathbb{E}[X^k].$$

In particular,

$$\mathbb{E}[X] = M_X'(0), \qquad \mathbb{E}[X^2] = M_X''(0),$$

and

$$\mathrm{Var}(X) = M_X''(0) - \Big(M_X'(0)\Big)^2.$$

The MGF need not exist away from zero for every distribution. For a Gaussian random variable $X \sim \mathcal{N}(\mu, \sigma^2)$, however,

$$M_X(u) = \exp\!\left(\mu u + \frac{1}{2}\sigma^2 u^2\right).$$

2.2.10 Characteristic Functions

The **characteristic function** of a real random variable X is

$$\varphi_X(u) = \mathbb{E}\Big[e^{iuX}\Big], \qquad u \in \mathbb{R},$$

where $i^2 = -1$. Since $|e^{iuX}| = 1$, the characteristic function exists for every real-valued random variable and every real u, even when the MGF does not exist. It is the Fourier transform of the probability distribution of X and uniquely determines that distribution.

When the corresponding moments exist,

$$\varphi_X^{(k)}(0) = i^k \mathbb{E}[X^k].$$

2.2.11 Gaussian Random Variables

A Gaussian or normal random variable with mean μ and variance σ^2 is written

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

and has density

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\!\left[-\frac{(x-\mu)^2}{2\sigma^2}\right].$$

Its characteristic function is

$$\boxed{\varphi_X(u) = \exp\!\left(i\mu u - \frac{1}{2}\sigma^2 u^2\right).}$$

For a standard normal variable $Z \sim \mathcal{N}(0,1)$, this becomes

$$\varphi_Z(u) = e^{-u^2/2}.$$

Derivation.
Substituting the Gaussian density into the definition gives

$$\varphi_X(u) = \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{\infty} \exp\left(iux - \frac{(x-\mu)^2}{2\sigma^2} \right) dx.$$

With $x = \mu + \sigma z$,

$$\varphi_X(u) = e^{i\mu u} \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \exp\left(-\frac{z^2}{2} + iu\sigma z \right) dz.$$

Completing the square,

$$-\frac{z^2}{2} + iu\sigma z = -\frac{(z - iu\sigma)^2}{2} - \frac{\sigma^2 u^2}{2}.$$

The remaining shifted Gaussian integral contributes 1 after normalization, which yields

$$\varphi_X(u) = e^{i\mu u} e^{-\sigma^2 u^2/2}.$$

2.2.12 Random Vectors and Covariance Matrices

An n-dimensional random vector is

$$\boldsymbol{X} = (X_1, \ldots, X_n)^{\mathsf{T}}.$$

Its mean is the vector

$$\boldsymbol{\mu} = \mathbb{E}[\boldsymbol{X}] = \begin{pmatrix} \mathbb{E}[X_1] \\ \vdots \\ \mathbb{E}[X_n] \end{pmatrix},$$

and its covariance matrix is

$$\boldsymbol{\Sigma} = \mathrm{Cov}(\boldsymbol{X}), \qquad \Sigma_{jk} = \mathrm{Cov}(X_j, X_k) = \mathbb{E}[(X_j - \mu_j)(X_k - \mu_k)]\,.$$

The matrix $\boldsymbol{\Sigma}$ is symmetric and positive semidefinite. Its diagonal entries are variances, while its off-diagonal entries measure linear co-movement between components.

The multivariate characteristic function is

$$\varphi_{\boldsymbol{X}}(\boldsymbol{u}) = \mathbb{E}\left[e^{i\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X}} \right], \qquad \boldsymbol{u} \in \mathbb{R}^n.$$

2.2.13 Gaussian Random Vectors

A random vector $\boldsymbol{X}$ is **jointly Gaussian** if every linear combination

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X}$$

is a one-dimensional Gaussian random variable. If

$$\boldsymbol{X} \sim \mathcal{N}_n(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

then

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X} \sim \mathcal{N}\left(\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\mu}, \boldsymbol{u}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{u} \right),$$

and

$$\boxed{\varphi_{\boldsymbol{X}}(\boldsymbol{u}) = \exp\Big(i\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\mu} - \frac{1}{2}\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{u}\Big)}.$$

If $X_1, \ldots, X_n$ are **independent** Gaussian variables, then the vector is jointly Gaussian. Indeed,

$$\begin{aligned}\varphi_{\boldsymbol{X}}(\boldsymbol{u}) &= \mathbb{E}\left[\prod_{k=1}^{n} e^{iu_k X_k}\right] \\ &= \prod_{k=1}^{n} \mathbb{E}[e^{iu_k X_k}] \\ &= \exp\left(i\sum_{k=1}^{n} u_k\mu_k - \frac{1}{2}\sum_{k=1}^{n} u_k^2\sigma_k^2\right).\end{aligned}$$

Important qualification. Gaussian marginal distributions alone do not guarantee that a vector is jointly Gaussian. Independence of Gaussian components is sufficient, but without independence or the joint-Gaussian assumption, every component may be normal while some linear combinations are not.

2.2.14 Martingales

Let $(\Omega, \mathcal{F}, (\mathcal{F}_t), \mathbb{P})$ be a filtered probability space. An adapted process $X = (X_t)$ is a **martingale** if

$$\mathbb{E}[|X_t|] < \infty$$

for every t, and, whenever $s \le t$,

$$\boxed{\mathbb{E}[X_t \mid \mathcal{F}_s] = X_s}.$$

Thus, given all information currently available, the conditional mean of a future value is the present value. A martingale is often described as a mathematical model of a fair game.

Suppose the observed value at time s is $X_s = 100$. Many future paths may rise or fall, but their conditional average at a later time t is 100. If new information arrives at a later time u and the realised value is $X_u = 104$, then the updated conditional average of later values becomes 104. Conditional expectations change as the information set grows.

Random-walk example.

Let

$$X_n = \sum_{k=1}^{n} U_k, \qquad \mathbb{P}(U_k = 1) = \mathbb{P}(U_k = -1) = \frac{1}{2},$$

and let

$$\mathcal{F}_n = \sigma(U_1, \ldots, U_n).$$

Because U_{n+1} is independent of $\mathcal{F}_n$ and has mean zero,

$$\begin{aligned}\mathbb{E}[X_{n+1} \mid \mathcal{F}_n] &= \mathbb{E}[X_n + U_{n+1} \mid \mathcal{F}_n] \\ &= X_n + \mathbb{E}[U_{n+1}] \\ &= X_n.\end{aligned}$$

Therefore the simple symmetric random walk is a martingale with respect to its natural filtration.

Submartingales and supermartingales.
An integrable adapted process is a **submartingale** if

$$\mathbb{E}[X_t \mid \mathcal{F}_s] \geq X_s, \qquad s \leq t,$$

and a **supermartingale** if

$$\mathbb{E}[X_t \mid \mathcal{F}_s] \leq X_s.$$

The terminology refers to conditional drift, not to monotone sample paths: a submartingale can fall on individual paths, and a supermartingale can rise.

For a random walk with

$$\mathbb{P}(U_k = 1) = p, \qquad \mathbb{P}(U_k = -1) = 1 - p,$$

the expected increment is $2p - 1$. The process is a submartingale when $p > 1/2$, a martingale when $p = 1/2$, and a supermartingale when $p < 1/2$.

Connection to option pricing.
Under a risk-neutral probability measure $\mathbb{Q}$, it is generally the **discounted** tradable asset price, rather than the undiscounted price, that is modelled as a martingale. With a deterministic short rate r,

$$\widetilde{S}_t = e^{-rt} S_t$$

satisfies

$$\mathbb{E}^{\mathbb{Q}}\left[\widetilde{S}_t \mid \mathcal{F}_s\right] = \widetilde{S}_s, \qquad s \leq t.$$

Equivalently,

$$S_s = e^{-r(t-s)} \mathbb{E}^{\mathbb{Q}}[S_t \mid \mathcal{F}_s].$$

This conditional-expectation principle is central to no-arbitrage derivative pricing.

2.3 Concept Map

Concept	Role
Probability space	Specifies outcomes, measurable events, and their probabilities.
Stochastic process	Provides a time-indexed family of random variables.
Sample path	Shows one realised evolution after an outcome ω is fixed.
Stationary increments	Makes the distribution of a change depend only on elapsed time.
Independent increments	Separates randomness across disjoint intervals.
Filtration	Records the information available through time.
Adaptedness	Ensures the process uses no future information.
MGF and characteristic function	Encode a distribution and, when valid, its moments.
Covariance matrix	Describes the scale and linear dependence of a random vector.
Martingale	Expresses conditional fairness relative to a filtration.

Chapter 3

Markov Chains

Markov chains provide a middle ground between two extreme modelling choices. An i.i.d. model assumes that successive observations are completely independent, while a fully general time-series model may require the entire past to predict the next observation. A Markov chain retains a local form of dependence: the next state depends on the current state, but not on the full history once the current state is known.

3.1 Motivation

3.1.1 Why the Independence Assumption Can Fail

A random variable represents a distribution of possible outcomes rather than one deterministic number. Examples include

$$X \sim \mathcal{N}(\mu, \sigma^2), \qquad Y \sim \text{Bin}(n, p), \qquad Z \sim \text{Bernoulli}(p).$$

Repeatedly drawing from the same distribution is often modelled by an i.i.d. sequence. This assumes both that the distribution remains unchanged and that one observation contains no information about the next. In finance, these assumptions can fail because return distributions drift, volatility changes, and credit or market states persist through time.

An i.i.d. dice-roll portfolio.

Suppose a portfolio value V_t is independently set each day by a fair die:

$$\mathbb{P}(V_t = k) = \frac{1}{6}, \qquad k = 1, \ldots, 6.$$

Then the values are time-invariant and independent. Knowing today's value does not change tomorrow's distribution. This is mathematically convenient, but it cannot express persistence or restrictions on how states evolve.

Loan-delinquency motivation.

Consider four possible states for an individual loan:

$$\mathcal{S} = \{C, D_{30}, D_{60}, D_{90}\},$$

where C denotes current, D_{30} denotes 30–59 days late, D_{60} denotes 60–89 days late, and D_{90} denotes at least 90 days late. If each month's state were drawn independently from a fixed categorical distribution, a current loan could jump directly to D_{90} in one month. That transition is inconsistent with the way delinquency ages.

The empirical proportion of delinquent loans,

$$\widehat{p} = \frac{\text{number of delinquent loans}}{\text{total number of loans}} = \frac{1}{n}\sum_{\ell=1}^{n} \mathbf{1}_{\{\text{loan } \ell \text{ is delinquent}\}},$$

describes a cross-section at one time, but does not describe how individual loans move between states. A transition model is needed.

3.2 Core Structure and Probability Propagation

3.2.1 Definition and Markov Property

Let $X_0, X_1, \ldots$ take values in a finite state space $\mathcal{S} = \{1, \ldots, m\}$. The process is a **Markov chain** if, for every admissible history,

$$\boxed{\mathbb{P}(X_{t+1} = j \mid X_t = i, X_{t-1} = i_{t-1}, \ldots, X_0 = i_0) = \mathbb{P}(X_{t+1} = j \mid X_t = i)}.$$

This is the **Markov property**. It is often called memorylessness, but the name should be interpreted carefully: the chain may be strongly dependent through time. The statement is that the present state contains all the historical information used by the model to predict the next state.

For the loan example,

$$\begin{aligned}&\mathbb{P}(X_{t+1} = D_{60} mid X_t = D_{30}, X_{t-1} = C)\\ &\quad = \mathbb{P}(X_{t+1} = D_{60} \mid X_t = D_{30}).\end{aligned}$$

A chain is **time homogeneous** if the conditional probabilities do not depend on the calendar time:

$$\mathbb{P}(X_{t+1} = j \mid X_t = i) = p_{ij} \quad \text{for every } t.$$

Markov chains may also have countably infinite state spaces, but the notebook and the applications below focus on finite, time-homogeneous chains.

3.2.2 Transition Matrices

The one-step transition probabilities are collected in the matrix

$$P = (p_{ij})_{i,j=1}^{m}, \qquad p_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

Every transition matrix is **row stochastic**:

$$p_{ij} \geq 0, \qquad \sum_{j=1}^{m} p_{ij} = 1 \quad \text{for each } i.$$

The row identifies the current state and the column identifies the next state.

For the delinquency model, the structural zero pattern is

$$P = \begin{array}{c|cccc} & C & D_{30} & D_{60} & D_{90} \\ \hline C & p_{11} & p_{12} & 0 & 0 \\ D_{30} & p_{21} & p_{22} & p_{23} & 0 \\ D_{60} & p_{31} & 0 & p_{33} & p_{34} \\ D_{90} & p_{41} & 0 & 0 & p_{44} \end{array}.$$

Missing arrows or zero entries encode transitions that the model declares impossible in one step.

The numerical matrix used throughout the notebook is

$$P = \begin{pmatrix} 0.99 & 0.01 & 0 & 0 \\ 0.25 & 0.70 & 0.05 & 0 \\ 0.15 & 0 & 0.80 & 0.05 \\ 0.01 & 0 & 0 & 0.99 \end{pmatrix}.$$

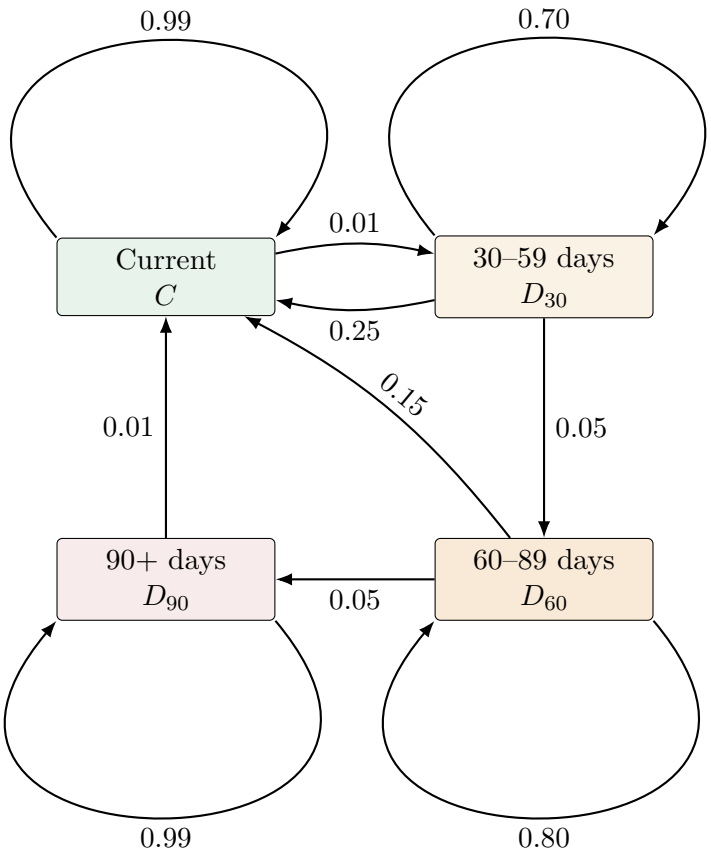


Figure 3.1: The loan-delinquency Markov chain. An omitted arrow represents a zero one-step transition probability.

The matrix describes the evolution of **each loan**, not the portfolio as a single four-state object. The portfolio distribution is obtained by aggregating the state probabilities of its individual loans.

3.2.3 State-Distribution Vectors

Let

$$\boldsymbol{\pi}(t) = \Big(\pi_1(t), \ldots, \pi_m(t)\Big)$$

be a row vector, where

$$\pi_i(t) = \mathbb{P}(X_t = i), \qquad \sum_{i=1}^{m} \pi_i(t) = 1.$$

The one-step evolution follows from the law of total probability:

$$\pi_j(t+1) = \sum_{i=1}^{m} \pi_i(t) p_{ij}.$$

In matrix form,

$$\boxed{\boldsymbol{\pi}(t+1) = \boldsymbol{\pi}(t)P}.$$

For a portfolio of N statistically identical loans, the expected number in state j at time t is

$$\mathbb{E}[N_j(t)] = N\pi_j(t).$$

This expectation formula does not require the loans to be mutually independent, although independence strongly affects portfolio-level uncertainty and confidence intervals.

3.2.4 Chapman–Kolmogorov Equations

Define the n-step transition probability

$$p_{ij}^{(n)} = \mathbb{P}(X_{t+n} = j \mid X_t = i).$$

Conditioning on an intermediate state k gives the **Chapman–Kolmogorov equation**

$$\boxed{p_{ij}^{(a+b)} = \sum_{k=1}^{m} p_{ik}^{(a)} p_{kj}^{(b)}}.$$

Here m is the number of states, while a and b are elapsed numbers of steps. In matrix form,

$$P^{(a+b)} = P^{(a)} P^{(b)}.$$

For a time-homogeneous chain, $P^{(n)} = P^n$, and therefore

$$\boldsymbol{\pi}(t) = \boldsymbol{\pi}(0) P^t.$$

3.2.5 Twelve-Month Loan Transition Probabilities

For the loan matrix above, direct matrix multiplication gives

$$P^{12} \approx \begin{pmatrix} 0.95984 & 0.03168 & 0.00665 & 0.00183 \\ 0.89358 & 0.04109 & 0.03208 & 0.03324 \\ 0.69191 & 0.02031 & 0.07201 & 0.21576 \\ 0.11048 & 0.00272 & 0.00037 & 0.88644 \end{pmatrix}.$$

Thus a currently performing loan has probability

$$\mathbb{P}(X_{12} = D_{90} \mid X_0 = C) = (P^{12})_{C,D_{90}} \approx 0.001826,$$

or approximately 0.1826%, of being at least 90 days delinquent after 12 months under the model.

Starting State	**Current**	**30–59**	**60–89**	**90+**
Current	95.98%	3.17%	0.66%	0.18%
30–59 days	89.36%	4.11%	3.21%	3.32%
60–89 days	69.19%	2.03%	7.20%	21.58%
90+ days	11.05%	0.27%	0.04%	88.64%

For the illustrative initial portfolio distribution

$$\boldsymbol{\pi}(0) = (0.50, 0.20, 0, 0.30),$$

the genuine 12-month distribution is

$$\boldsymbol{\pi}(12) = \boldsymbol{\pi}(0) P^{12} \approx (0.69178, 0.02487, 0.00985, 0.27349).$$

Therefore the expected 90+ delinquency proportion after 12 months is approximately 27.349%. For a portfolio of 1,000 loans, the corresponding expected state counts are approximately

$$(692, 25, 10, 273).$$

Notebook correction. One notebook cell describes its result as a 12-month forecast but computes P^{10000}. Its reported 3.8462% is the long-run stationary 90+ proportion, not the 12-month proportion for the stated initial distribution.

3.3 State Classification and Long-Run Behaviour

3.3.1 Accessibility, Communication, and Irreducibility

A state j is **accessible** from state i, written $i \to j$, if

$$(P^n)_{ij} > 0$$

for some $n \geq 0$. States i and j **communicate** if $i \to j$ and $j \to i$. Communication partitions the state space into communication classes.

A chain is **irreducible** if every pair of states communicates. In an irreducible finite chain, there is no permanently isolated regime: every state can eventually reach every other state with positive probability.

For the loan matrix, the chain is irreducible. For example, $C \to D_{30}$, $D_{30} \to D_{60}$, and $D_{60} \to D_{90}$, while every delinquency state can return to C. Some transitions require several steps even though their one-step probability is zero.

3.3.2 Recurrence, Transience, and Absorption

Starting from state i, define the first return time

$$T_i^+ = \inf\{n \geq 1 : X_n = i\}.$$

The state is **recurrent** if

$$\mathbb{P}_i(T_i^+ < \infty) = 1,$$

and **transient** if this probability is strictly less than 1. A transient state has a positive probability of never being revisited after it is left.

A state i is **absorbing** if

$$p_{ii} = 1.$$

Once the chain enters an absorbing state, it can never leave. The loan matrix has no absorbing state because every diagonal entry is less than 1, although the states C and D_{90} are highly persistent. Because the displayed chain is finite and irreducible, all four states are positive recurrent.

3.3.3 Periodicity

The period of state i is

$$d(i) = \gcd\{n \geq 1 : (P^n)_{ii} > 0\}.$$

A state is **aperiodic** if $d(i) = 1$. In an irreducible chain all states have the same period. A positive self-transition $p_{ii} > 0$ implies aperiodicity for that state; therefore the loan chain is aperiodic because every state has a self-loop.

3.3.4 Stationary Distributions and Ergodicity

A probability row vector $\boldsymbol{\pi}^*$ is a **stationary distribution** if

$$\boxed{\boldsymbol{\pi}^* P = \boldsymbol{\pi}^*, \qquad \sum_{i=1}^{m} \pi_i^* = 1}.$$

If $X_0 \sim \boldsymbol{\pi}^*$, then $X_t \sim \boldsymbol{\pi}^*$ at every later time.

For a finite irreducible and aperiodic chain, the stationary distribution is unique and

$$\boldsymbol{\pi}(0) P^t \longrightarrow \boldsymbol{\pi}^* \qquad \text{as } t \to \infty$$

for every initial distribution. This convergence property is commonly called **ergodicity** in the finite-state setting.

Solving the stationary equations for the loan matrix gives

$$\boldsymbol{\pi}^* = \left(\frac{120}{130}, \frac{4}{130}, \frac{1}{130}, \frac{5}{130}\right) \approx (0.92308, 0.03077, 0.00769, 0.03846).$$

The long-run 90+ delinquency proportion is therefore 3.8462%.

The evolution from the notebook's mixed initial distribution is summarized below.

Month	Current	30–59	60–89	90+
0	50.000%	20.000%	0.000%	30.000%
1	54.800%	14.500%	1.000%	29.700%
2	58.324%	10.698%	1.525%	29.453%
12	69.178%	2.487%	0.985%	27.349%
50	77.115%	2.552%	0.631%	19.701%
∞	92.308%	3.077%	0.769%	3.846%

The convergence is slow because D_{90} has a 99% monthly self-transition probability. Thus 50 months illustrates movement toward the steady state, but does not yet closely approximate it.

Steady-state warning. The limiting distribution is meaningful only if the transition mechanism remains stable. Economic cycles, seasonality, policy changes, underwriting changes, and external shocks can make the true transition matrix vary through time.

3.4 Model Design and Estimation

3.4.1 Choosing States in Quantitative Finance

State definitions are problem dependent. Common choices include

- **volatility regimes:** low, medium, and high, using a measure such as the VIX or realised volatility;
- **market trends:** bullish, bearish, and sideways, using index returns or trend indicators;
- **liquidity regimes:** tight, normal, and wide spreads, using quoted or effective bid–ask spreads;
- **credit states:** current and progressively older delinquency buckets.

Thresholds should be chosen before estimating transitions, or selected using a properly separated training sample. Poor thresholds can discard important information or create states with too few observed transitions.

3.4.2 Maximum Likelihood Estimation: General Idea

Maximum likelihood estimation chooses the parameter value under which the observed data have the greatest probability or density. For example, if $Y_1, \ldots, Y_n$ are modelled as i.i.d. $\mathcal{N}(\mu, \sigma^2)$, then maximizing the Gaussian likelihood gives

$$\widehat{\mu} = \frac{1}{n}\sum_{k=1}^{n} Y_k, \qquad \widehat{\sigma}^2 = \frac{1}{n}\sum_{k=1}^{n}(Y_k - \widehat{\mu})^2.$$

As more representative observations become available, these estimates typically stabilize near the data-generating parameters. Early estimates can be volatile because they are based on small samples.

The same principle estimates a Markov transition matrix: instead of fitting the parameters of one density, we fit every row of conditional transition probabilities.

3.4.3 Deriving the Markov-Chain MLE

Suppose the observed state sequence is

$$x_0, x_1, \ldots, x_T,$$

and let $\nu_i = \mathbb{P}(X_0 = i)$ be the initial distribution. Under the time-homogeneous Markov model, the likelihood is

$$L(P) = \nu_{x_0} \prod_{t=0}^{T-1} p_{x_t x_{t+1}}.$$

Define the transition counts

$$n_{ij} = \sum_{t=0}^{T-1} \mathbf{1}_{\{x_t = i, x_{t+1} = j\}}, \qquad n_i = \sum_{j=1}^{m} n_{ij}.$$

Then

$$L(P) = \nu_{x_0} \prod_{i=1}^{m}\prod_{j=1}^{m} p_{ij}^{n_{ij}},$$

and, ignoring the initial term when it does not involve P,

$$\ell(P) = \log L(P) = \text{constant} + \sum_{i=1}^{m}\sum_{j=1}^{m} n_{ij} \log p_{ij}.$$

Each row is maximized subject to

$$\sum_{j=1}^{m} p_{ij} = 1, \qquad p_{ij} \geq 0.$$

For row i, use the Lagrangian

$$\mathcal{L}_i = \sum_{j=1}^{m} n_{ij} \log p_{ij} + \lambda_i \left(1 - \sum_{j=1}^{m} p_{ij}\right).$$

The first-order condition is

$$\frac{\partial \mathcal{L}_i}{\partial p_{ij}} = \frac{n_{ij}}{p_{ij}} - \lambda_i = 0,$$

so $p_{ij} = n_{ij}/\lambda_i$. Summing over j yields $\lambda_i = n_i$, and therefore

$$\boxed{\widehat{p}_{ij} = \frac{n_{ij}}{n_i} = \frac{\text{observed transitions } i \to j}{\text{all observed departures from } i}}.$$

The MLE is simply the observed transition frequency within each row. If $n_i = 0$, that row cannot be estimated from the data without additional structure or smoothing.

For multiple loan histories, the counts n_{ij} are aggregated across all loans and all adjacent observation times. The chain is fitted to each loan's state transitions, not directly to the aggregate portfolio proportions.

3.4.4 Loan-Portfolio MLE Example

The notebook generates 1,000 loan histories over 12 months using the matrix

$$P = \begin{pmatrix} 0.99 & 0.01 & 0 & 0 \\ 0.25 & 0.70 & 0.05 & 0 \\ 0.15 & 0 & 0.80 & 0.05 \\ 0.01 & 0 & 0 & 0.99 \end{pmatrix}$$

and an initial distribution

$$(0.85, 0.08, 0.05, 0.02).$$

One simulated cross-section contains 865 current loans, 67 loans 30–59 days late, 46 loans 60–89 days late, and 22 loans at least 90 days late. Sampling variation explains why these counts are not exactly equal to their expected values $(850, 80, 50, 20)$.

Aggregating observed transitions gives the MLE

$$\widehat{P} \approx \begin{pmatrix} 0.99198 & 0.00802 & 0 & 0 \\ 0.23325 & 0.71712 & 0.04963 & 0 \\ 0.15504 & 0 & 0.77132 & 0.07364 \\ 0.00545 & 0 & 0 & 0.99455 \end{pmatrix}.$$

The estimates are close to the generating matrix. Rows or transitions with fewer observations show greater sampling error; for example, the estimated $D_{60} \to D_{90}$ probability differs more noticeably from its true value than the frequently observed current-state probabilities.

3.4.5 Assumptions Behind the Estimator

The interpretation and large-sample reliability of $\widehat{P}$ depend on several assumptions.

1. **Markov property.** The current state contains all past information relevant to the next transition.
2. **Time homogeneity.** The matrix P is constant across the sample and forecast horizon.
3. **Sufficient state visitation.** Every row of interest has enough departures, and rare transitions are observed often enough to be estimated.
4. **Ergodicity for one long history.** The chain must explore its relevant states so empirical transition frequencies converge to stable probabilities.
5. **Cross-sectional independence or an appropriate dependence model.** Treating different loans as independent ignores shared exposure to unemployment, interest rates, geography, underwriting standards, and other macroeconomic shocks.

Time homogeneity and cross-loan independence are often the most questionable assumptions in credit applications. Violating them can make standard errors too small and long-horizon forecasts misleading.

3.4.6 Large-Sample Properties of the MLE

Under suitable regularity, identifiability, and ergodicity conditions, the transition-frequency estimator has the following properties.

- **Consistency:** $\widehat{p}_{ij} \to p_{ij}$ as the number of observed departures from state i grows.
- **Asymptotic normality:** conditional on n_i, row i behaves like a multinomial estimate. Writing $\boldsymbol{p}_i = (p_{i1}, \ldots, p_{im})^{\mathsf{T}}$,

$$\sqrt{n_i}(\widehat{\boldsymbol{p}}_i - \boldsymbol{p}_i) \xrightarrow{d} \mathcal{N}\Big(\mathbf{0}, \operatorname{diag}(\boldsymbol{p}_i) - \boldsymbol{p}_i\boldsymbol{p}_i^{\mathsf{T}}\Big).$$

- **Asymptotic efficiency:** under standard conditions, the MLE achieves the usual information-bound efficiency within the specified model.
- **Invariance:** if $g(P)$ is a function of the transition parameters, its MLE is $g(\widehat{P})$. For example, an estimated n-step matrix is $\widehat{P}^{,n}$.

The law of large numbers explains why empirical probabilities tend to move toward their true values as the data set grows. It does not guarantee that a misspecified Markov model converges to the real economic mechanism.

> **MLE versus maximum entropy.** Maximum likelihood and maximum entropy are distinct principles. The MLE maximizes the likelihood of the observed data. It is not generally a maximum-entropy estimator unless a separate entropy argument and appropriate constraints establish that connection.

3.5 Limitations and Extensions

The Markov assumption is a substantial improvement over unconditional independence, but it remains a simplification. Important limitations include

- dependence on events further in the past than the current state;
- transition probabilities that vary with economic conditions;
- common shocks that couple many loans or assets;
- imperfect, noisy, or arbitrarily discretized state definitions;
- unobserved regimes that influence the observed process.

Natural extensions include higher-order Markov chains, non-homogeneous transition matrices, covariate-dependent transition models, regime-switching models, and hidden Markov models. In quantitative finance, Markov chains are often the first step toward models of volatility regimes, liquidity regimes, credit migration, and latent market states.

> **Core modelling lesson.** Full independence is frequently too strong. Markov chains replace it with local conditional dependence, enabling probabilities and expectations to be propagated with a transition matrix. The resulting model is useful only to the extent that its states, transition rules, and stability assumptions reflect the real system.

Chapter 4

Random Walk

A random walk is a concrete discrete-time stochastic process and, when its current position is taken as the state, a Markov chain. The simple symmetric case below turns the preceding abstract ideas into explicit paths, probabilities, and barrier calculations before leading naturally to Brownian motion.

4.1 Simple Symmetric Random Walk

4.1.1 Definition and Rule

A **simple symmetric random walk** is one of the simplest stochastic processes. At every discrete time step, there is:

- a 50% probability of moving $+1$, one unit to the right;
- a 50% probability of moving -1, one unit to the left.

Update rule.

$$x(n+1) = x(n) \pm 1.$$

Each outcome, $+1$ or -1, is equally likely; this is why the walk is called symmetric.

4.1.2 Coin-Flip Analogy

Coin-Flip Outcome	Movement
Heads	Move $+1$
Tails	Move -1

4.1.3 The Growing Tree of Possible Positions

Time n	Possible Positions
0	$\{0\}$
1	$\{-1, +1\}$
2	$\{-2, 0, +2\}$
3	$\{-3, -1, +1, +3\}$

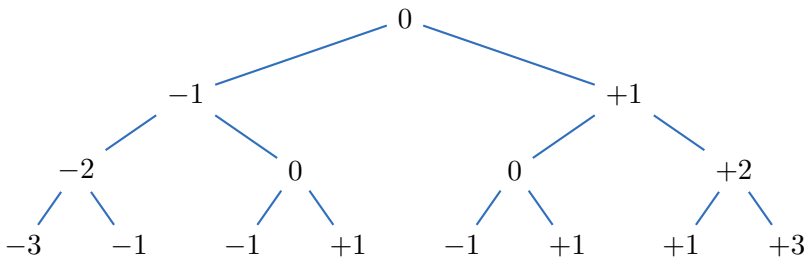


Figure 4.1: The first three steps of a simple symmetric random walk.

Key intuition. As time progresses, the number of possible paths and reachable positions increases, making the path increasingly unpredictable even though the movement rule remains simple. In the limit of many suitably scaled small steps, this behaviour leads to Brownian motion.

4.2 Counting Random-Walk Paths

The combinatorial approach to random walks treats every sequence of upward and downward steps as a path. This viewpoint is especially useful because, when the steps are independent and equally likely, calculating a probability reduces to counting the paths in the corresponding event.

4.2.1 Paths Between Two Space–Time Points

Let

$$\alpha = (m, p), \qquad \beta = (n, q), \qquad m < n,$$

be two points in the space–time plane. A random-walk path from α to β is a function

$$\pi : \{m, m+1, \ldots, n\} \longrightarrow \mathbb{Z}$$

such that

$$\pi(m) = p, \qquad \pi(n) = q, \qquad |\pi(k+1) - \pi(k)| = 1.$$

Thus each step changes the position by exactly one unit. Denote the set of all such paths by

$$\mathrm{PATH}(\alpha, \beta).$$

For paths starting at the origin, define

$$N_{n,x} := \Big|\mathrm{PATH}\Big((0,0),(n,x)\Big)\Big|.$$

4.2.2 Total Number of Paths

At each of n time steps there are two possible moves, $+1$ or -1. Consequently, the total number of length-n paths starting at the origin is

$$2^n.$$

Under the simple symmetric random-walk model, every one of these paths has probability 2^{-n}.

4.2.3 Number of Paths to a Fixed Endpoint

Suppose a path reaches position x after n steps. If u is the number of upward steps and d the number of downward steps, then

$$u + d = n, \qquad u - d = x.$$

Solving these equations gives

$$u = \frac{n+x}{2}, \qquad d = \frac{n-x}{2}.$$

The path is determined once the u time indices containing upward steps are selected. Hence

$$N_{n,x} = \begin{cases} \binom{n}{(n+x)/2}, & |x| \le n \text{ and } n+x \text{ is even}, \\ 0, & \text{otherwise.} \end{cases}$$

The parity condition is important: after an even number of steps the walk can reach only even positions, while after an odd number of steps it can reach only odd positions. Dividing the path count by 2^n gives the endpoint distribution

$$\boxed{\mathbb{P}(X_n = x) = 2^{-n} N_{n,x} = 2^{-n} \binom{n}{(n+x)/2}},$$

whenever the endpoint is reachable.

Example: returning to the origin after ten steps.
To finish at the starting position, the path must contain five upward and five downward steps. Therefore

$$\mathbb{P}(X_{10} = 0) = \frac{N_{10,0}}{2^{10}} = \frac{\binom{10}{5}}{2^{10}} = \frac{252}{1024} = \frac{63}{256} \approx 0.2461.$$

4.2.4 A Useful Recursion

Every path ending at (n, x) must arrive on its final step either from $(n-1, x-1)$ or from $(n-1, x+1)$. The two possibilities are disjoint, so

$$N_{n,x} = N_{n-1,x-1} + N_{n-1,x+1}, \qquad N_{0,0} = 1.$$

This recurrence is the same addition rule that generates Pascal's triangle, which explains the appearance of binomial coefficients in the endpoint formula.

4.2.5 Translation Invariance

Shifting a path horizontally or vertically does not change its pattern of upward and downward steps. Therefore

$$\Big|\mathrm{PATH}\big((a,p),(b,q)\big)\Big| = N_{b-a,q-p}.$$

Only the elapsed time $b - a$ and the net displacement $q - p$ matter.

Example.
A path from $(2,3)$ to $(15,2)$ has length 13 and displacement -1. Translating its starting point to the origin gives

$$\left|\mathrm{PATH}\Big((2,3),(15,2)\Big)\right| = N_{13,-1} = \binom{13}{6} = 1716.$$

4.3 Return Probabilities

A simple symmetric random walk can return to the origin only after an even number of steps. At time $2n$, exactly n steps must be upward and n must be downward, so

$$\boxed{\mathbb{P}(X_{2n}=0) = \frac{\binom{2n}{n}}{2^{2n}}}.$$

For $n=5$, this reproduces the ten-step probability $63/256$. Stirling's approximation further gives

$$\mathbb{P}(X_{2n}=0) \sim \frac{1}{\sqrt{\pi n}} \qquad \text{as } n \to \infty.$$

Thus the probability of being at the origin at one particular late time decreases to zero, but only at the relatively slow rate $n^{-1/2}$.

4.4 The Reflection Principle

Path counting becomes more subtle when a path is required to touch, cross, or avoid a barrier. The reflection principle converts such a constrained count into an unconstrained endpoint count.

Let L be the horizontal line at integer height h, and let

$$\alpha = (m,p), \qquad \beta = (n,q),$$

lie strictly on the same side of L. Reflect α across L to obtain

$$\alpha' = (m, 2h-p).$$

Reflection principle. The number of paths from α to β that touch or cross L is equal to the total number of unrestricted paths from the reflected point α' to β:

$$\#\{\alpha \to \beta \text{ paths meeting } L\} = \left|\mathrm{PATH}(\alpha',\beta)\right|.$$

Using translation invariance, this count can be written explicitly as

$$|\mathrm{PATH}(\alpha',\beta)| = N_{n-m,q-(2h-p)} = N_{n-m,q-2h+p}.$$

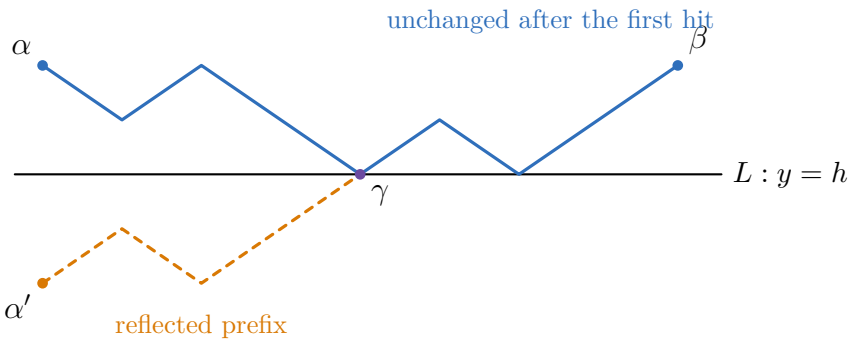


Figure 4.2: Reflecting the path up to its first meeting point γ with the barrier produces a path from α' to β.

4.4.1 Why the Reflection Principle Works

Let $\mathcal{S}$ be the set of paths from α to β that meet L. For any $\pi \in \mathcal{S}$, let

$$\tau = \min\{k : \pi(k) = h\}$$

be the first time the path meets the barrier. Define a new path by reflecting the part before this first meeting:

$$(f\pi)(k) = \begin{cases} 2h - \pi(k), & m \le k \le \tau, \\ \pi(k), & \tau \le k \le n. \end{cases}$$

The transformed path starts at α', passes through the same first meeting point $\gamma = (\tau, h)$, and then follows the original path to β.

This construction is a bijection:

1. Every constrained path produces a path from α' to β.
2. Since α' and β lie on opposite sides of L, every path between them must meet L.
3. Reflecting the prefix of that path up to its first meeting with L recovers the unique original path.

Therefore the constrained and reflected path sets have the same size.

4.4.2 Counting Paths That Avoid a Barrier

The total number of paths from $\alpha = (m, p)$ to $\beta = (n, q)$ is $N_{n-m,q-p}$. Subtracting the number that meet L gives

$$\boxed{\#\{\alpha \to \beta \text{ paths avoiding } L\} = N_{n-m,q-p} - N_{n-m,q-2h+p}}.$$

This formula is the basic tool for questions about positivity, maximum levels, and first-passage events.

Example: paths that remain strictly positive.

Count the ten-step paths from $(0, 0)$ to $(10, 4)$ that satisfy $X_k > 0$ for every $k = 1, \ldots, 10$. The first step must be upward, so the problem reduces to paths from $(1, 1)$ to $(10, 4)$ that

avoid the line $L : y = 0$. Thus

$$\begin{aligned}\#\{\text{strictly positive paths ending at } 4\} &= N_{9,3} - N_{9,5} \\ &= \binom{9}{6} - \binom{9}{7} \\ &= 84 - 36 \\ &= 48.\end{aligned}$$

Since there are 2^{10} total ten-step paths, the corresponding unconditional probability is

$$\frac{48}{2^{10}} = \frac{3}{64}.$$

4.5 Interpretation and Limitations in Finance

A scaled random walk can be used as a first model of price changes:

$$S_n = S_0 + aX_n,$$

where a is the monetary size of one price movement. The model captures the idea that short-term changes are uncertain and that many possible price paths may emerge from the same initial value. It also gives exact formulas for endpoint probabilities and barrier events.

However, the simple symmetric model is not a realistic complete model of an asset price:

- it assumes equal probabilities of upward and downward moves;
- it assumes independent increments of constant size;
- it permits negative values when applied directly to prices;
- it does not account for changing volatility, jumps, or market microstructure.

These limitations motivate the continuous-time Brownian-motion and geometric Brownian-motion models developed in the following sections.

4.6 Random Walk as the Discrete Precursor to Brownian Motion

Let $\boldsymbol{X} = (X_0, X_1, X_2, \ldots)$ be the symmetric simple random walk defined by

$$X_n = \sum_{i=1}^{n} U_i,$$

where $\boldsymbol{U} = (U_1, U_2, \ldots)$ is a sequence of independent random variables satisfying

$$\mathbb{P}(U_i = 1) = \mathbb{P}(U_i = -1) = \frac{1}{2} \qquad \text{for every } i.$$

Key statistical properties.

$$\mathbb{E}[X_n] = 0, \qquad \mathrm{Var}(X_n) = n.$$

Because X is the partial-sum process of an i.i.d. sequence, it has **stationary, independent increments**, although only in discrete time.

Central Limit Theorem connection.

$$\frac{X_n}{\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1) \qquad \text{as } n \to \infty.$$

Constructing a continuous-time process.

For $h, d \in (0, \infty)$, define the continuous-time jump process

$$X_{h,d}(t) = dX_{\lfloor t/h \rfloor}.$$

Its jumps occur at $\{0, h, 2h, \ldots\}$, and every jump has size $\pm d$.

We want to let $h \downarrow 0$, producing smaller time steps, and $d \downarrow 0$, producing smaller spatial jumps, simultaneously. These limits cannot be chosen arbitrarily because

$$\mathbb{E}[X_{h,d}(t)] = 0, \qquad \mathrm{Var}[X_{h,d}(t)] = d^2 \left\lfloor \frac{t}{h} \right\rfloor \approx \frac{d^2 t}{h}.$$

The critical scaling.

If $d = \sqrt{h}$, the Central Limit Theorem implies that the distribution of $X_{h,d}(t)$ converges to a normal distribution with mean 0 and variance t as $h \downarrow 0$:

$$\boxed{d = \sqrt{h} \quad \Longrightarrow \quad X_{h,d}(t) \xrightarrow{d} \mathcal{N}(0,t)}.$$

The mathematical bridge. When the random walk's spatial step is the square root of its time step, the scaled discrete process converges to standard Brownian motion. Its limiting paths are continuous and appear smooth at a distance, but are almost surely nowhere differentiable.

Chapter 5

Brownian Motion

5.1 Standard Brownian Motion

5.1.1 History

In 1827 the botanist Robert Brown observed that tiny pollen particles suspended in water move continuously but very erratically. In his "miracle year" of 1905, Albert Einstein explained this phenomenon physically: the particles are constantly bombarded by the surrounding water molecules, and this bombardment is what produces the jittery motion. Einstein's explanation was one of the pieces of evidence that helped establish the atomic theory of matter. It was not until 1918 that Norbert Wiener gave a mathematically rigorous construction of Brownian motion as a stochastic process; for this reason the process is also called the *Wiener process*.

Brownian motion, together with the Bernoulli trials process and the Poisson process, is one of the three cornerstone processes of probability theory. Each is built from an idealized list of assumptions, each generates a rich mathematical theory, and each serves as a building block for many other stochastic processes used throughout physics, statistics, and economics.

Intuition 5.1.1. Think of a single pollen grain being struck from every direction, many thousands of times per second, by water molecules moving in random directions. No single collision moves the particle very far, but the cumulative effect of a huge number of tiny independent pushes produces, via the Central Limit Theorem, a displacement that is approximately normally distributed over any fixed time interval. Brownian motion is the mathematical idealization of this limiting behavior.

5.1.2 Definition

Definition 5.1.2 (Standard Brownian Motion)**.** A *standard Brownian motion* is a random process $X = \{X_t : t \in [0, \infty)\}$ with state space $\mathbb{R}$ that satisfies the following properties:

(1) $X_0 = 0$ (with probability 1).

(2) X has *stationary increments*: for $s, t \in [0, \infty)$ with $s < t$, the distribution of $X_t - X_s$ is the same as the distribution of X_{t-s}.

(3) X has *independent increments*: for $t_1, t_2, \ldots, t_n \in [0, \infty)$ with $t_1 < t_2 < \cdots < t_n$, the random variables

$$X_{t_1},\ X_{t_2} - X_{t_1},\ \ldots,\ X_{t_n} - X_{t_{n-1}}$$

are independent.

(4) X_t is normally distributed with mean 0 and variance t, for each $t \in (0, \infty)$.

(5) With probability 1, $t \mapsto X_t$ is continuous on $[0, \infty)$.

Interpreting the assumptions physically.

(1) We may measure the position of the particle starting at $t = 0$ from its own current location, so $X_0 = 0$ is simply a convention (it is occasionally useful to relax this and let X_0 take other values).

(2) Property (2) is a statement of *time homogeneity*: the physical mechanism jostling the particle (collisions with water molecules) does not change over time, so the distribution of the displacement over an interval $[s, t]$ depends only on the length $t - s$ of the interval, not on where the interval starts.

(3) Property (3) is an idealization that would hold approximately if we sampled the process at times that are large compared to the (extremely small) time between individual molecular collisions.

(4) Property (4) follows heuristically from the Central Limit Theorem: the displacement at time t is the sum of a huge number of tiny, independent contributions, so it should be approximately normal. The mean being 0 reflects *spatial homogeneity* — the particle is no more likely to be pushed right than left. Because of stationary, independent increments, one can show quite generally that $\mathrm{Var}(X_t) = \sigma^2 t$ for some constant $\sigma^2 > 0$; by rescaling time we may take $\sigma^2 = 1$, which gives *standard* Brownian motion. (Section 5.2 treats the general case $\sigma^2 \neq 1$.)

(5) Continuity of the sample paths is essential since we are modeling the physical position of a particle as a continuous function of time.

Remark 5.1.3. Properties (2) and (4) together force the mean and variance functions of X to be linear in t. Indeed, stationary and independent increments imply that the characteristic function of X_t satisfies a Cauchy-type functional equation in t, whose only continuous solutions are exponentials of linear functions of t — this is exactly what singles out the normal family with mean and variance proportional to t.

5.1.3 Existence

Of course, before working with such a process we should ask whether one actually exists satisfying all five defining properties.

Theorem 5.1.4 (Existence of Standard Brownian Motion)**.** *There exists a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ and a stochastic process $X = \{X_t : t \in [0, \infty)\}$ on this space satisfying the definition above.*

Proof sketch. The defining assumptions determine a consistent family of finite-dimensional distributions (given explicitly in Section 5.1.5 below). By the Kolmogorov Existence Theorem, there is a stochastic process $U = \{U_t : t \in [0, \infty)\}$ having these finite-dimensional distributions. The process U need not have continuous paths, however, so a further construction is required.

Recall that a *binary rational* (or *dyadic rational*) in $[0, \infty)$ is a number of the form $k/2^n$ with $k, n \in \mathbb{N}$. Let $\mathbb{D}_+$ denote the set of all binary rationals; $\mathbb{D}_+$ is countable but dense in $[0, \infty)$. For $n \in \mathbb{N}_+$, define $X_n(t) = U_t$ whenever t is a binary rational of the form $k/2^n$, and define $X_n(t)$ at other points by linear interpolation between the two nearest such binary rationals. One can show that $X_n(t) \to U(t)$ as $n \to \infty$ for every $t \in \mathbb{D}_+$, and that with probability 1 this convergence is uniform on $\mathbb{D}_+ \cap [0, T]$ for every $T > 0$. Hence U restricted to $\mathbb{D}_+$ is continuous with probability 1.

Finally, define $X_t = \lim_{s \to t,\, s \in \mathbb{D}_+} U_s$ for $t \in [0, \infty)$; the limit exists because U is continuous on $\mathbb{D}_+$ with probability 1. The resulting process $X = \{X_t : t \in [0, \infty)\}$ is

continuous on $[0,\infty)$ with probability 1 and has the same finite-dimensional distributions as U, hence satisfies the definition. $\square$

5.1.4 Brownian Motion as a Limit of Random Walks

The physical picture of a particle being kicked by molecules suggests approximating Brownian motion by a random walk with very small time steps and very small spatial jumps.

Let $X = (X_0, X_1, X_2, \dots)$ be the symmetric simple random walk, so that

$$X_n = \sum_{i=1}^{n} U_i,$$

where $U = (U_1, U_2, \dots)$ is an i.i.d. sequence with $\mathbb{P}(U_i = 1) = \mathbb{P}(U_i = -1) = \frac{1}{2}$. Since X is the partial-sum process of an i.i.d. sequence, $\mathbb{E}(X_n) = 0$ and $\mathrm{Var}(X_n) = n$, and X has stationary, independent increments (in discrete time). By the Central Limit Theorem, $X_n/\sqrt{n} \to N(0,1)$ in distribution as $n \to \infty$.

For $h, d \in (0,\infty)$, form the continuous-time jump process

$$X_{h,d} = \{d\, X_{\lfloor t/h \rfloor} : t \in [0,\infty)\}, \tag{18.1.1}$$

which jumps at times $0, h, 2h, \dots$ by amounts $\pm d$. We would like to send $h \downarrow 0$ and $d \downarrow 0$, but not arbitrarily: note that $\mathbb{E}[X_{h,d}(t)] = 0$ while $\mathrm{Var}[X_{h,d}(t)] = d^2 \lfloor t/h \rfloor \approx d^2 t/h$. Thus, if we take $d = \sqrt{h}$, the Central Limit Theorem shows that the distribution of $X_{h,d}(t)$ converges to $N(0,t)$ as $h \downarrow 0$. More generally, one can verify that *all* of the defining properties of standard Brownian motion emerge in this limit — so Brownian motion is, in a precise sense, the diffusive scaling limit of the simple random walk.

Remark 5.1.5. This is the discrete precursor of Donsker's invariance principle (the "functional CLT"): properly rescaled random walks converge, as processes, to Brownian motion.

5.1.5 Finite-Dimensional Distributions

Let $X = \{X_t : t \in [0,\infty)\}$ be standard Brownian motion. By property (4) of the definition, X_t has probability density function

$$f_t(x) = \frac{1}{\sqrt{2\pi t}} \exp\!\left(-\frac{x^2}{2t}\right), \qquad x \in \mathbb{R}. \tag{18.1.2}$$

This family of one-dimensional densities, together with stationary independent increments, determines *all* finite-dimensional distributions of X:

Proposition 5.1.6. *If $t_1, t_2, \dots, t_n \in (0,\infty)$ with $0 < t_1 < t_2 < \cdots < t_n$, then $(X_{t_1}, X_{t_2}, \dots, X_{t_n})$ has joint density*

$$f_{t_1,\dots,t_n}(x_1,\dots,x_n) = f_{t_1}(x_1)\, f_{t_2-t_1}(x_2-x_1) \cdots f_{t_n-t_{n-1}}(x_n - x_{n-1}), \qquad (x_1,\dots,x_n) \in \mathbb{R}^n. \tag{18.1.3}$$

Proof. This follows directly from the independence of the increments $X_{t_1}, X_{t_2} - X_{t_1}, \dots, X_{t_n} - X_{t_{n-1}}$, together with stationarity, which lets each increment's density be written as $f_{t_i - t_{i-1}}$. $\square$

Proposition 5.1.7 (Brownian motion is a Gaussian process)**.** *X is a Gaussian process with mean function $m(t) = 0$ for $t \in [0, \infty)$ and covariance function*

$$c(s,t) = \min\{s,t\}, \qquad s,t \in [0,\infty).$$

Proof. X is a Gaussian process because X_t is normally distributed for every t and X has stationary, independent increments (so that every finite linear combination of values of X is a sum of independent normal random variables, hence normal). The mean is 0 by assumption. For the covariance, suppose $s \le t$. Since X_s and $X_t - X_s$ are independent,

$$\mathrm{Cov}(X_s, X_t) = \mathrm{Cov}\left[X_s,\ X_s + (X_t - X_s)\right] = \mathrm{Var}(X_s) + 0 = s = \min\{s,t\}.$$

□

Since a Gaussian process's finite-dimensional (multivariate normal) distributions are completely determined by its mean and covariance functions, this theorem gives an equivalent, and very compact, characterization: **standard Brownian motion is precisely a continuous Gaussian process with mean** 0 **and covariance** $\min\{s,t\}$**.** Note also that the correlation function is

$$\mathrm{Corr}(X_s, X_t) = \frac{\min\{s,t\}}{\sqrt{st}} = \sqrt{\frac{\min\{s,t\}}{\max\{s,t\}}}, \qquad (s,t) \in (0,\infty)^2. \tag{18.1.5}$$

Proposition 5.1.8 (Higher moments and MGF)**.** *For $n \in \mathbb{N}$ and $t \in [0,\infty)$,*

$$\mathbb{E}(X_t^{2n}) = 1 \cdot 3 \cdots (2n-1)\, t^n = \frac{(2n)!\, t^n}{n!\, 2^n}, \qquad \mathbb{E}(X_t^{2n+1}) = 0.$$

Moreover, X_t has moment generating function

$$\mathbb{E}\left(e^{uX_t}\right) = e^{tu^2/2}, \qquad u \in \mathbb{R}. \tag{18.1.6}$$

Proof. These are standard results for the normal distribution, since $X_t \sim N(0,t)$. □

5.1.6 Simple Transformations

Several elementary transformations preserve standard Brownian motion; these reveal important structural symmetries.

Proposition 5.1.9 (Reflection)**.** *Let $Y_t = -X_t$ for $t \ge 0$. Then $Y = \{Y_t : t \ge 0\}$ is also a standard Brownian motion.*

Proof. Y is Gaussian with mean $\mathbb{E}(-X_t) = 0$ and covariance $\mathrm{Cov}(-X_s, -X_t) = \mathrm{Cov}(X_s, X_t) = \min\{s,t\}$; continuity of Y follows from that of X. □

Proposition 5.1.10 (Restarting / spatial-temporal homogeneity)**.** *Fix $s \in [0,\infty)$ and define $Y_t = X_{s+t} - X_s$ for $t \ge 0$. Then $Y = \{Y_t : t \ge 0\}$ is also a standard Brownian motion.*

Proof. Since X has stationary, independent increments, Y is equivalent in distribution to X, and Y inherits continuity from X. □

This shows that Brownian motion is both *temporally* and *spatially* homogeneous: restarting the clock and resetting the origin at any fixed time produces an equivalent process. This is the seed of the Markov property developed in Section 5.1.8.

Proposition 5.1.11 (Time reversal)**.** *Fix $T > 0$ and define $Y_t = X_{T-t} - X_T$ for $0 \le t \le T$. Then $Y = \{Y_t : t \in [0, T]\}$ is also a standard Brownian motion on $[0, T]$.*

Proof. Y is Gaussian since it is a linear combination of values of X. Clearly $\mathbb{E}(Y_t) = 0$. For $s, t \in [0, T]$ with $s \le t$,

$$\begin{aligned}\operatorname{Cov}(Y_s, Y_t) &= \operatorname{Cov}(X_{T-s} - X_T,\ X_{T-t} - X_T)\\ &= \operatorname{Cov}(X_{T-s}, X_{T-t}) - \operatorname{Cov}(X_{T-s}, X_T)\\ &\quad - \operatorname{Cov}(X_T, X_{T-t}) + \operatorname{Cov}(X_T, X_T)\\ &= (T-t) - (T-s) - (T-t) + T = s.\end{aligned}$$

Continuity of Y follows from that of X. □

Proposition 5.1.12 (Self-similarity)**.** *Let $a > 0$ and define $Y_t = \frac{1}{a} X_{a^2 t}$ for $t \ge 0$. Then $Y = \{Y_t : t \ge 0\}$ is also a standard Brownian motion.*

Proof. Y is Gaussian, $\mathbb{E}(Y_t) = \frac{1}{a}\mathbb{E}(X_{a^2 t}) = 0$, and for $s \le t$,

$$\operatorname{Cov}(Y_s, Y_t) = \operatorname{Cov}\Big(\tfrac{1}{a} X_{a^2 s}, \tfrac{1}{a} X_{a^2 t}\Big) = \frac{1}{a^2} \min\{a^2 s, a^2 t\} = s.$$

Y is continuous since X is. □

The graph of Y is obtained from the graph of X by scaling the time axis by a^2 and the spatial axis by a — the temporal scale factor is the *square* of the spatial scale factor, a direct echo of the random-walk scaling $d = \sqrt{h}$ seen above. Equivalently, "zooming in or out" on the graph of X reproduces a statistically identical picture: Brownian motion is self-similar and has a fractal quality. This is a first hint that, although continuous, $t \mapsto X_t$ is extremely irregular (Section 5.1.7).

Proposition 5.1.13 (Time inversion)**.** *Let $Y_0 = 0$ and $Y_t = t X_{1/t}$ for $t > 0$. Then $Y = \{Y_t : t \ge 0\}$ is also a standard Brownian motion.*

Proof. Y is Gaussian; $\mathbb{E}(Y_t) = t\,\mathbb{E}(X_{1/t}) = 0$ for $t > 0$; and for $0 < s < t$,

$$\operatorname{Cov}(Y_s, Y_t) = \operatorname{Cov}(s X_{1/s}, t X_{1/t}) = st\ \operatorname{Cov}(X_{1/s}, X_{1/t}) = st \cdot \frac{1}{t} = s.$$

Since $t \mapsto X_t$ is continuous on $[0, \infty)$ with probability 1, $t \mapsto Y_t$ is continuous on $(0, \infty)$ with probability 1. Continuity at $t = 0$ requires $t X_{1/t} \to 0$ as $t \downarrow 0$, equivalently $X_s / s \to 0$ as $s \to \infty$; this holds with probability 1 by the Law of the Iterated Logarithm (Section 5.1.13). □

5.1.7 Irregularity

The defining properties already hint that a standard Brownian path cannot be smooth. We now make this precise.

Consider the difference quotient at t,

$$\frac{X_{t+h} - X_t}{h}. \tag{18.1.11}$$

By stationary increments, if $h > 0$ the numerator has the same distribution as X_h, while if $h < 0$ it has the same distribution as $-X_{-h}$, which in turn has the same distribution as

X_{-h}. In either case the difference quotient has the same distribution as $X_{|h|}/h$, a normal random variable with mean 0 and variance $|h|/h^2 = 1/|h|$. As $h \to 0$ this variance diverges to ∞, so the difference quotient does not even converge in distribution — the weakest possible mode of convergence. This already suggests X cannot be differentiable at t.

The self-similarity transformation reinforces this: if X were differentiable at t, the chain rule applied to $Y_t = \frac{1}{a}X_{a^2t}$ would give $Y'(t) = aX'(a^2t)$ — yet Y is again a standard Brownian motion for *every* $a > 0$, so "zooming in" should reveal the same roughness at every scale, not a well-defined slope. These heuristics are made rigorous in the following theorem.

Theorem 5.1.14. *With probability 1, X is nowhere differentiable on $[0, \infty)$.*

Theorem 5.1.15 (Hölder continuity)**.** *Standard Brownian motion has Hölder exponent $\frac{1}{2}$: X is Hölder continuous with exponent α for every $\alpha < \frac{1}{2}$, but is not Hölder continuous with exponent α for any $\alpha > \frac{1}{2}$.*

In particular X is not Lipschitz continuous — again showing it is not differentiable.

Theorem 5.1.16 (Hausdorff dimension of the graph)**.** *The graph of standard Brownian motion has Hausdorff dimension $\frac{3}{2}$.*

This sits exactly midway between the dimension of a smooth curve (dimension 1) and that of the plane (dimension 2), quantifying just how "space-filling" and jagged the Brownian path is.

Theorem 5.1.17 (Infinite total variation)**.** *Suppose $a, b \in \mathbb{R}$ with $a < b$. Then the total variation of X on $[a, b]$ is ∞.*

Remark 5.1.18. This last fact is the reason ordinary (Riemann–Stieltjes) integration cannot be used to define integrals with respect to Brownian motion, and is one of the main motivations for Itô's stochastic integral, which appears below in the discussion of martingales and in the construction of the Brownian bridge (Section 5.3) and geometric Brownian motion (Section 5.4).

5.1.8 The Markov Property and Stopping Times

Let $\mathcal{F}_t = \sigma\{X_s : 0 \le s \le t\}$ be the σ-algebra generated by the process up to time t; the family $\mathcal{G} = \{\mathcal{F}_t : t \in [0, \infty)\}$ is called a *filtration*. Recall that a Markov process is one for which the future is independent of the past given the present; the stationary, independent increments property immediately gives Brownian motion the Markov property. (To view X as a Markov process it is sometimes convenient to relax Assumption (1) and allow X_0 to take an arbitrary value in $\mathbb{R}$.)

Theorem 5.1.19. *Standard Brownian motion is a time-homogeneous Markov process with transition probability density*

$$p_t(x, y) = f_t(y - x) = \frac{1}{\sqrt{2\pi t}} \exp\left[-\frac{(y-x)^2}{2t}\right], \qquad t \in (0, \infty),\ x, y \in \mathbb{R}. \tag{18.1.12}$$

Proof. Fix $s \in [0, \infty)$. This follows because $\{X_{s+t} - X_s : t \in [0, \infty)\}$ is itself a standard Brownian motion, independent of $\mathcal{F}_s$. □

Proposition 5.1.20 (Diffusion equations)**.** *The transition density p satisfies the* forward equation

$$\frac{\partial}{\partial t}p_t(x,y) = \frac{1}{2}\frac{\partial^2}{\partial y^2}p_t(x,y) \tag{18.1.13}$$

and the backward equation

$$\frac{\partial}{\partial t}p_t(x,y) = \frac{1}{2}\frac{\partial^2}{\partial x^2}p_t(x,y). \tag{18.1.14}$$

Proof. Direct calculus applied to the explicit Gaussian formula for $p_t(x,y)$. □

These names reflect which variable the spatial derivative is taken in: the forward equation differentiates with respect to y, the state *forward* at time t; the backward equation differentiates with respect to x, the state *backward* at time 0.

Definition 5.1.21 (Stopping time)**.** A random time τ taking values in $[0,\infty]$ is a *stopping time* with respect to X if $\{\tau \le t\} \in \mathcal{F}_t$ for every $t \in [0,\infty)$. The σ-algebra of events determinable up to time τ is

$$\mathcal{F}_\tau = \{B \in \mathcal{F} : B \cap \{\tau \le t\} \in \mathcal{F}_t \text{ for all } t \ge 0\}. \tag{18.1.15}$$

An important example: for $x \in \mathbb{R}$, let $\tau_x = \inf\{t \ge 0 : X_t = x\}$, the first time the process hits state x. This is a stopping time, since whether $\tau_x \le t$ can be determined by observing the path up to time t.

Theorem 5.1.22 (Strong Markov property)**.** *Suppose τ is a stopping time and define $Y_t = X_{\tau+t} - X_\tau$ for $t \ge 0$. Then $Y = \{Y_t : t \ge 0\}$ is a standard Brownian motion and is independent of $\mathcal{F}_\tau$.*

This is the generalization, from fixed times to stopping times, of the restarting/homogeneity property established earlier.

5.1.9 The Reflection Principle

Definition 5.1.23. Let τ be a stopping time for X. Define

$$W_t = \begin{cases} X_t, & 0 \le t < \tau \\ 2X_\tau - X_t, & \tau \le t < \infty. \end{cases} \tag{18.1.16}$$

The graph of W is obtained from the graph of X by reflecting across the horizontal line $x = X_\tau$ after time τ. In particular, if $\tau = \tau_a$ is the first hitting time of a level $a > 0$, the graph of W is obtained by reflecting across the line $x = a$ after time τ_a.

Theorem 5.1.24. *The reflected process $W = \{W_t : t \ge 0\}$ is also a standard Brownian motion.*

Intuition 5.1.25. Because the strong Markov property says that X "forgets" everything except its current value once it reaches the stopping time τ, and because a standard Brownian motion is symmetric about its current level (reflecting $Y_t = -X_t$ is again a standard Brownian motion), reflecting the future path after τ produces a process with exactly the same law as the original.

The reflection principle is the key tool used below to compute the distribution of the running maximum of Brownian motion.

5.1.10 Martingales

As usual, let $\mathcal{F}_t = \sigma\{X_s : 0 \le s \le t\}$ and $\mathcal{G} = \{\mathcal{F}_t : t \in [0,\infty)\}$.

Proposition 5.1.26. *X is a martingale with respect to $\mathcal{G}$.*

Proof. This holds for any process with stationary, independent increments and mean 0. For $s < t$, since X_s is $\mathcal{F}_s$-measurable and $X_t - X_s$ is independent of $\mathcal{F}_s$,

$$\mathbb{E}(X_t \mid \mathcal{F}_s) = \mathbb{E}[X_s + (X_t - X_s) \mid \mathcal{F}_s] = X_s + \mathbb{E}(X_t - X_s) = X_s. \tag{18.1.17}$$

□

Proposition 5.1.27. *Let $Y_t = X_t^2 - t$ for $t \in [0,\infty)$. Then $Y = \{Y_t : t \ge 0\}$ is a martingale with respect to $\mathcal{G}$.*

Proof. For $s < t$,

$$Y_t = X_t^2 - t = [X_s + (X_t - X_s)]^2 - t = X_s^2 + 2X_s(X_t - X_s) + (X_t - X_s)^2 - t. \tag{18.1.18}$$

Since X_s is $\mathcal{F}_s$-measurable and $X_t - X_s$ is independent of $\mathcal{F}_s$,

$$\mathbb{E}(Y_t \mid \mathcal{F}_s) = X_s^2 + 2X_s\,\mathbb{E}(X_t - X_s) + \mathbb{E}[(X_t - X_s)^2] - t. \tag{18.1.19}$$

But $\mathbb{E}(X_t - X_s) = 0$ and $\mathbb{E}[(X_t - X_s)^2] = \mathrm{Var}(X_t - X_s) = t - s$, so $\mathbb{E}(Y_t \mid \mathcal{F}_s) = X_s^2 - s = Y_s$. □

Remark 5.1.28. These two martingales are the first two members of an infinite hierarchy: for standard Brownian motion, $\exp(uX_t - u^2t/2)$ is also a martingale for every $u \in \mathbb{R}$ (this follows from the moment generating function computed above), and its Taylor coefficients in u generate an entire family of polynomial martingales in X_t and t (Hermite-type martingales), of which X_t and $X_t^2 - t$ are the first two.

5.1.11 Maximums and Hitting Times

For $y \in [0,\infty)$, recall $\tau_y = \inf\{t \ge 0 : X_t = y\}$ (with $\tau_0 = 0$). For $t \in [0,\infty)$, let

$$Y_t = \max\{X_s : 0 \le s \le t\},$$

the running maximum of X on $[0,t]$; Y_t is well defined because X is continuous, and $Y_0 = 0$. The processes $\{\tau_y : y \ge 0\}$ and $\{Y_t : t \ge 0\}$ are, in a precise sense, inverses of one another:

Proposition 5.1.29. *For $t, y \in (0,\infty)$, $\tau_y \le t$ if and only if $Y_t \ge y$.*

Proof. Both events say exactly that the process hits state y somewhere in $[0,t]$; this uses the continuity of X and the fact that $X_0 = 0$. □

Thus the distribution of Y_t for each t determines the distribution of τ_y for each y, and vice versa.

Theorem 5.1.30 (Distribution of the hitting time)**.** *For $y > 0$, τ_y has the same distribution as y^2/Z^2, where Z is standard normal. The density is*

$$g_y(t) = \frac{y}{\sqrt{2\pi t^3}} \exp\left(-\frac{y^2}{2t}\right), \qquad t \in (0,\infty). \tag{18.1.20}$$

Proof. Let $t > 0$. Since $X_t \geq y \implies Y_t \geq y \implies \tau_y \leq t$,

$$\mathbb{P}(X_t \geq y) = \mathbb{P}(X_t \geq y,\, \tau_y \leq t) = \mathbb{P}(X_t \geq y \mid \tau_y \leq t)\,\mathbb{P}(\tau_y \leq t). \tag{18.1.21}$$

By the strong Markov property, $s \mapsto X(\tau_y + s) - y$ is another standard Brownian motion, so by symmetry $\mathbb{P}(X_t \geq y \mid \tau_y \leq t) = \frac{1}{2}$. Hence

$$\mathbb{P}(\tau_y \leq t) = 2\,\mathbb{P}(X_t \geq y) = \frac{2}{\sqrt{2\pi t}} \int_y^\infty e^{-x^2/2t}\,\mathrm{d}x = \frac{2}{\sqrt{2\pi}} \int_{y/\sqrt{t}}^\infty e^{-z^2/2}\,\mathrm{d}z, \tag{18.1.22}$$

using the substitution $z = x/\sqrt{t}$; the right side is $\mathbb{P}(y^2/Z^2 \leq t)$. Differentiating with respect to t gives the density g_y. □

The distribution of τ_y is called the *Lévy distribution* with scale parameter y^2.

Theorem 5.1.31 (Recurrence)**.** *Standard Brownian motion is* recurrent*: $\mathbb{P}(\tau_y < \infty) = 1$ for every $y \in \mathbb{R}$.*

Proof. For $y > 0$, letting $t \to \infty$ in (18.1.22) gives $\mathbb{P}(\tau_y < \infty) = \frac{2}{\sqrt{2\pi}} \int_0^\infty e^{-z^2/2}\,\mathrm{d}z = 1$, since this is the integral of the full standard normal density over its positive half. For $y < 0$, symmetry (τ_y has the same distribution as τ_{-y}) gives the same conclusion; trivially $\tau_0 = 0$. □

Corollary 5.1.32. *With probability 1, X visits every point of $\mathbb{R}$.*

Proof. By continuity, if X reaches $y > 0$ it visits every point of $[0, y]$, and symmetrically for $y < 0$. So the event that X visits every point of $\mathbb{R}$ contains $\bigcap_{n=1}^\infty (\{\tau_n < \infty\} \cap \{\tau_{-n} < \infty\})$, a countable intersection of probability-1 events, hence itself has probability 1. □

Theorem 5.1.33 (Null recurrence)**.** *Standard Brownian motion is* null recurrent*: $\mathbb{E}(\tau_y) = \infty$ for every $y \neq 0$.*

Proof. By symmetry take $y > 0$. From the tail formula for τ_y,

$$\mathbb{E}(\tau_y) = \int_0^\infty \mathbb{P}(\tau_y > t)\,\mathrm{d}t = \frac{2}{\sqrt{2\pi}} \int_0^\infty \int_0^{y/\sqrt{t}} e^{-z^2/2}\,\mathrm{d}z\;\mathrm{d}t. \tag{18.1.24}$$

Reversing the order of integration,

$$\mathbb{E}(\tau_y) = \frac{2}{\sqrt{2\pi}} \int_0^\infty \int_0^{y^2/z^2} \mathrm{d}t\, e^{-z^2/2}\,\mathrm{d}z = \frac{2y^2}{\sqrt{2\pi}} \int_0^\infty \frac{1}{z^2} e^{-z^2/2}\,\mathrm{d}z. \tag{18.1.25}$$

Restricting the last integral to $[0, 1]$ and using $e^{-z^2/2} \geq e^{-1/2}$ there gives $\mathbb{E}(\tau_y) \geq \frac{2y^2 e^{-1/2}}{\sqrt{2\pi}} \int_0^1 \frac{1}{z^2}\,\mathrm{d}z = \infty$. □

Recurrent but null recurrent is the classic hallmark of one-dimensional Brownian motion: it is guaranteed to return to (indeed, to hit) every level, yet the expected waiting time to do so is infinite.

Proposition 5.1.34. *The process $\{\tau_x : x \in [0, \infty)\}$ has stationary, independent increments.*

Proof. By the temporal/spatial homogeneity of X and the strong Markov property: for $0 \le x < y$, continuity forces X to reach x before y, so $\tau_y = \tau_x + (\tau_y - \tau_x)$. But $\tau_y - \tau_x$ is the hitting time of $y - x$ for the process $t \mapsto X(\tau_x + t) - x$, which is itself a standard Brownian motion independent of $\mathcal{F}(\tau_x)$. Hence $\tau_y - \tau_x$ is independent of $\mathcal{F}(\tau_x)$ and has the same distribution as τ_{y-x}. □

Corollary 5.1.35. *The family of densities $\{g_x : x \in (0,\infty)\}$ is closed under convolution: $g_x * g_y = g_{x+y}$ for $x, y \in (0,\infty)$.*

Theorem 5.1.36 (Distribution of the running maximum)**.** *For $t > 0$, Y_t has the same distribution as $|X_t|$ — the* half-normal distribution *with scale parameter t — with density*

$$h_t(y) = \sqrt{\frac{2}{\pi t}} \exp\left(-\frac{y^2}{2t}\right), \qquad y \in [0,\infty). \tag{18.1.27}$$

Moreover $\mathbb{E}(Y_t) = \sqrt{2t/\pi}$ and $\mathrm{Var}(Y_t) = t(1 - 2/\pi)$.

Proof. From the inverse relation between τ_y and Y_t, together with the distribution of τ_y,

$$\mathbb{P}(Y_t \ge y) = \mathbb{P}(\tau_y \le t) = 2\,\mathbb{P}(X_t \ge y) = \mathbb{P}(|X_t| \ge y), \qquad y \ge 0,$$

so Y_t and $|X_t|$ have the same distribution; the density and moments then follow from the half-normal (folded normal) distribution. □

5.1.12 Zeros and Arcsine Laws

For $0 \le s < t$, let $E(s,t) = \{X_u = 0 \text{ for some } u \in (s,t)\}$, the event that X has a zero in (s,t).

Theorem 5.1.37.

$$\mathbb{P}[E(s,t)] = 1 - \frac{2}{\pi} \arcsin\left(\sqrt{\frac{s}{t}}\right). \tag{18.1.29}$$

Proof. Conditioning on X_s and using symmetry,

$$\mathbb{P}[E(s,t)] = \int_{-\infty}^{\infty} \mathbb{P}[E(s,t) \mid X_s = x] f_s(x)\,\mathrm{d}x = 2\int_{-\infty}^{0} \mathbb{P}[E(s,t) \mid X_s = x] f_s(x)\,\mathrm{d}x. \tag{18.1.30}$$

By spatial-temporal homogeneity, for $x > 0$, $\mathbb{P}[E(s,t) \mid X_s = -x] = \mathbb{P}(\tau_x < t - s)$ (a process at $-x$ hitting 0 before time $t - s$ is equivalent to a process at 0 reaching x before time $t - s$). Substituting the density g_x of τ_x gives, after the change of variables $v = \sqrt{u/s}$,

$$\mathbb{P}[E(s,t)] = \frac{2}{\pi}\int_0^{\sqrt{t/s-1}} \frac{1}{v^2+1}\,\mathrm{d}v = \frac{2}{\pi}\arctan\left(\sqrt{\frac{t}{s}-1}\right) = 1 - \frac{2}{\pi}\arcsin\left(\sqrt{\frac{s}{t}}\right). \tag{18.1.33}$$

□

In particular, $\mathbb{P}[E(0,t)] = 1$ for every $t > 0$, so with probability 1, X has a zero in every interval $(0,t)$. In fact:

Corollary 5.1.38. *For every $t > 0$, X has infinitely many zeros in $(0,t)$ with probability 1.*

Proof. The event of infinitely many zeros in $(0,t)$ equals $\bigcap_{n=1}^{\infty} E(0,t/n)$, a countable intersection of probability-1 events. □

Note that $\mathbb{P}[E(s,t)]$ depends only on the ratio s/t; in particular $\mathbb{P}[E(s,t)] = \mathbb{P}[E(1/t,1/s)] = \mathbb{P}[E(cs,ct)]$ for every $c>0$. So, for example, the chance of at least one zero in $(2,5)$ equals the chance of a zero in $(1/5,1/2)$, in $(6,15)$, and in $(200,500)$.

Definition 5.1.39. For $t>0$, let $Z_t = \max\{s\in[0,t] : X_s = 0\}$, the time of the *last* zero of X before t.

Theorem 5.1.40 (Arcsine law for the last zero)**.** *Z_t has the* arcsine distribution *with parameter t:*

$$H_t(s) = \frac{2}{\pi}\arcsin\left(\sqrt{\frac{s}{t}}\right), \quad 0\le s\le t; \qquad h_t(s) = \frac{1}{\pi\sqrt{s(t-s)}}, \quad 0<s<t. \tag{18.1.34, 18.1.35}$$

Moreover $\mathbb{E}(Z_t) = t/2$ and $\mathrm{Var}(Z_t) = t^2/8$.

Proof. For $0\le s<t$, $\{Z_t\le s\}$ is the same event as $\{X$ has no zero in $(s,t)\} = E(s,t)^c$, so $H_t(s) = 1-\mathbb{P}[E(s,t)] = \frac{2}{\pi}\arcsin\sqrt{s/t}$; differentiating gives h_t. The mean is the midpoint $t/2$ by symmetry. □

Remark 5.1.41. The density h_t is U-shaped and symmetric about $t/2$, so the *most* likely locations for the last zero are near the two endpoints 0 and t, and the *least* likely location is the midpoint — a famously counter-intuitive fact about Brownian motion (and, e.g., about random walks / gambler's-ruin type processes more generally: long "winning streaks" near the start or end of an interval are more typical than a change of sign near the middle).

Theorem 5.1.42 (Structure of the zero set)**.** *Let $Z = \{t\in[0,\infty) : X_t = 0\}$. With probability 1:*

(1) Z is closed.

(2) $\lambda(Z) = 0$ (Lebesgue measure zero).

(3) Z is nowhere dense.

(4) Z is perfect (has no isolated points).

Moreover, Z has Hausdorff dimension $\frac{1}{2}$.

Proof. (1) Z is the preimage of the closed set $\{0\}$ under the (a.s. continuous) map $t\mapsto X_t$, so is closed a.s. (2) For each fixed t, $\mathbb{P}(t\in Z) = \mathbb{P}(X_t=0) = 0$ since X_t has a continuous distribution; by Fubini,

$$\mathbb{E}[\lambda(Z)] = \mathbb{E}\left[\int_0^\infty \mathbf{1}_Z(t)\,\mathrm{d}\lambda(t)\right] = \int_0^\infty \mathbb{P}(t\in Z)\,\mathrm{d}\lambda(t) = 0,$$

so $\mathbb{P}[\lambda(Z)=0]=1$. (3) A closed set of Lebesgue measure zero automatically has empty interior. (4) If $s\in Z$, temporal-spatial homogeneity makes $t\mapsto X_{s+t}$ another standard Brownian motion, which (by the zero results above) has a zero in $(s,s+1/n)$ for every n, with probability 1; hence s is not isolated in Z. □

Remark 5.1.43. This theorem gives one of the cleanest illustrations of just how strange the zero set of Brownian motion is: it is uncountable (being perfect and nonempty), yet has Lebesgue measure zero, is nowhere dense, and sits "dimensionally" exactly halfway between a point and a line.

5.1.13 The Law of the Iterated Logarithm

We know $X_t \sim N(0,t)$, so $\sqrt{t}$ gives a rough sense of the size of X_t for large t. The precise almost-sure growth envelope is given by the celebrated Law of the Iterated Logarithm:

Theorem 5.1.44 (Law of the Iterated Logarithm)**.** *With probability 1,*

$$\limsup_{t\to\infty} \frac{X_t}{\sqrt{2t\ln\ln t}} = 1. \tag{18.1.37}$$

Remark 5.1.45. By symmetry ($-X$ is also standard Brownian motion) the corresponding $\liminf$ equals -1. The function $\sqrt{2t\ln\ln t}$ grows just slightly faster than $\sqrt{t}$ — this is precisely the borderline envelope that the path touches infinitely often (from above and below) but never permanently escapes.

5.1.14 Computational Exercise

Exercise 5.1.46. For a standard Brownian motion $X = \{X_t : t \geq 0\}$, explicitly find the probability density function, the covariance matrix, and the correlation matrix of $(X_{0.5}, X_1, X_{2.3})$.

Solution sketch. By the finite-dimensional distribution formula (18.1.3), the joint density is the product

$$f_{0.5,1,2.3}(x_1,x_2,x_3) = f_{0.5}(x_1)\, f_{0.5}(x_2-x_1)\, f_{1.3}(x_3-x_2),$$

with each factor of the Gaussian form (18.1.2) using the appropriate variance (0.5, 0.5, and 1.3 respectively). The covariance matrix has (i,j) entry $\min(t_i,t_j)$ for $(t_1,t_2,t_3) = (0.5,1,2.3)$:

$$\Sigma = \begin{pmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 1 \\ 0.5 & 1 & 2.3 \end{pmatrix},$$

and the correlation matrix has (i,j) entry $\sqrt{\min(t_i,t_j)/\max(t_i,t_j)}$:

$$R = \begin{pmatrix} 1 & \sqrt{0.5} & \sqrt{0.5/2.3} \\ \sqrt{0.5} & 1 & \sqrt{1/2.3} \\ \sqrt{0.5/2.3} & \sqrt{1/2.3} & 1 \end{pmatrix}.$$

□

5.2 Brownian Motion with Drift and Scaling

5.2.1 Definition

We now relax the restriction $\sigma^2 = 1$ (and mean 0) that characterized *standard* Brownian motion, obtaining a two-parameter family.

Definition 5.2.1. Suppose $\mu \in \mathbb{R}$ and $\sigma \in (0, \infty)$. *Brownian motion with drift parameter μ and scale parameter σ* is a random process $X = \{X_t : t \in [0, \infty)\}$ with state space $\mathbb{R}$ satisfying:

(1) $X_0 = 0$ (with probability 1).
(2) Stationary increments: the distribution of $X_t - X_s$ $(s < t)$ is the same as that of X_{t-s}.
(3) Independent increments, as before.
(4) $X_t \sim N(\mu t, \sigma^2 t)$ for $t \in [0, \infty)$.
(5) With probability 1, $t \mapsto X_t$ is continuous on $[0, \infty)$.

As with standard Brownian motion, stationary independent increments force the mean and variance of X_t to be linear functions of t; we are simply no longer insisting the slope of the mean be 0 or that the variance slope be 1.

Proposition 5.2.2 (Relation to standard Brownian motion)**.** *(a) If $Z = \{Z_t\}$ is standard Brownian motion and $\mu \in \mathbb{R}$, $\sigma \in (0, \infty)$, then $X_t = \mu t + \sigma Z_t$ defines a Brownian motion with drift μ and scale σ.*

(b) Conversely, if X is Brownian motion with drift μ and scale σ, then $Z_t = (X_t - \mu t)/\sigma$ is a standard Brownian motion.

Proof. It is straightforward to check that X and Z as defined satisfy the respective sets of defining properties (linearity of the transformations preserves the Gaussian, stationary-independent-increment structure; the mean and variance transform exactly as required). □

In differential form, part (a) is written

$$dX_t = \mu\, dt + \sigma\, dZ_t, \qquad X_0 = 0. \tag{18.2.1}$$

Remark 5.2.3. This is our first example of a stochastic differential equation (SDE): a deterministic drift term $\mu\, dt$ plus a random noise term $\sigma\, dZ_t$ driven by standard Brownian motion. The same template — deterministic drift plus a Brownian noise term, possibly with state-dependent coefficients — underlies both the Brownian bridge (Section 5.3) and geometric Brownian motion (Section 5.4).

5.2.2 Finite-Dimensional Distributions

Since $X_t \sim N(\mu t, \sigma^2 t)$, the density is

$$f_t(x) = \frac{1}{\sigma\sqrt{2\pi t}} \exp\left[-\frac{1}{2\sigma^2 t}(x - \mu t)^2\right], \qquad x \in \mathbb{R}. \tag{18.2.2}$$

As before, this family determines all finite-dimensional distributions:

Proposition 5.2.4. *If $0 < t_1 < \cdots < t_n$ then $(X_{t_1}, \ldots, X_{t_n})$ has density*

$$f_{t_1,\ldots,t_n}(x_1, \ldots, x_n) = f_{t_1}(x_1)\, f_{t_2-t_1}(x_2 - x_1) \cdots f_{t_n-t_{n-1}}(x_n - x_{n-1}), \qquad (x_1, \ldots, x_n) \in \mathbb{R}^n. \tag{18.2.3}$$

Proof. As before, this follows from independence and stationarity of the increments. □

Proposition 5.2.5. *X is a Gaussian process with mean function $m(t) = \mu t$ and covariance function $c(s,t) = \sigma^2 \min\{s,t\}$.*

Proof. Gaussianity follows from the construction $X_t = \mu t + \sigma Z_t$ with Z standard Brownian motion, itself Gaussian. The mean and covariance follow from stationary independent increments; note that μ and σ^2 are exactly the mean and variance of X_1. □

The correlation function

$$\operatorname{Corr}(X_s, X_t) = \frac{\sigma^2 \min\{s,t\}}{\sigma\sqrt{s}\,\sigma\sqrt{t}} = \sqrt{\frac{\min\{s,t\}}{\max\{s,t\}}}, \qquad (s,t) \in (0,\infty)^2, \tag{18.2.4}$$

is independent of the parameters μ, σ — exactly the same as for standard Brownian motion, since correlation is a standardized (scale-free) measure of association.

5.2.3 Transformations

Proposition 5.2.6 (Scaling in time and space)**.** *Let $a \in \mathbb{R} \setminus \{0\}$ and $b \in (0,\infty)$, and define $Y_t = aX_{bt}$ for $t \geq 0$. Then $Y = \{Y_t\}$ is a Brownian motion with drift parameter $ab\mu$ and scale parameter $|a|\sqrt{b}\,\sigma$.*

Proof. Y is Gaussian; $\mathbb{E}(Y_t) = a\mathbb{E}(X_{bt}) = ab\mu t$; and $\operatorname{Cov}(Y_s, Y_t) = a^2 \operatorname{Cov}(X_{bs}, X_{bt}) = a^2\sigma^2 \min\{bs, bt\} = a^2 b\sigma^2 \min\{s,t\}$, matching a Brownian motion with drift $ab\mu$ and scale $|a|\sqrt{b}\,\sigma$ (note $(|a|\sqrt{b}\,\sigma)^2 = a^2 b\sigma^2$). Continuity carries over from X. □

Suppose $a > 0$ (so we scale purely temporally and spatially, without reflection). To *preserve* the drift parameter μ (when $\mu \neq 0$) we would need $ab = 1$; to preserve the scale parameter σ we would need $a\sqrt{b} = 1$. In general we cannot satisfy both simultaneously unless $\mu = 0$, which gives a useful special case:

Proposition 5.2.7 (Self-similarity with zero drift)**.** *Suppose X has drift parameter 0 and scale parameter $\sigma > 0$, and let $c > 0$. Then $Y_t = \frac{1}{c} X_{c^2 t}$ is again a Brownian motion with drift 0 and scale σ.*

Proposition 5.2.8 (Restarting)**.** *Fix $s \geq 0$ and let $Y_t = X_{s+t} - X_s$. Then Y is a Brownian motion with the same drift μ and scale σ.*

Proof. Y is Gaussian; $\mathbb{E}(Y_t) = \mathbb{E}(X_{s+t}) - \mathbb{E}(X_s) = \mu(s+t) - \mu s = \mu t$; and for $r \leq t$,

$$\begin{aligned} \operatorname{Cov}(Y_r, Y_t) &= \operatorname{Cov}(X_{s+r} - X_s,\ X_{s+t} - X_s) \\ &= \operatorname{Cov}(X_{s+r}, X_{s+t}) - \operatorname{Cov}(X_{s+r}, X_s) - \operatorname{Cov}(X_s, X_{s+t}) + \operatorname{Cov}(X_s, X_s) \qquad (18.2.6) \\ &= \sigma^2(s+r) - \sigma^2 s - \sigma^2 s + \sigma^2 s = \sigma^2 r. \qquad (18.2.7) \end{aligned}$$

Continuity of Y follows from that of X. □

This restarting property is the analogue, for general drift and scale, of the temporal-spatial homogeneity seen for standard Brownian motion, and again anticipates the Markov property.

5.2.4 The Markov Property and Stopping Times

As before let $\mathcal{F}_t = \sigma\{X_s : 0 \le s \le t\}$.

Theorem 5.2.9. *X is a time-homogeneous Markov process with transition density*

$$p_t(x,y) = f_t(y-x) = \frac{1}{\sigma\sqrt{2\pi t}} \exp\Big[-\frac{1}{2\sigma^2 t}(y-x-\mu t)^2\Big], \qquad t \in (0,\infty),\ x,y \in \mathbb{R}. \tag{18.2.8}$$

Proof. Fix $s \ge 0$. The process $\{X_{s+t} - X_s : t \ge 0\}$ is another Brownian motion with drift μ and scale σ, independent of $\mathcal{F}_s$; the transition density follows. □

Proposition 5.2.10 (Diffusion equations)**.** *The transition density satisfies the forward equation*

$$\frac{\partial}{\partial t}p_t(x,y) = -\mu\frac{\partial}{\partial y}p_t(x,y) + \frac{1}{2}\sigma^2\frac{\partial^2}{\partial y^2}p_t(x,y) \tag{18.2.9}$$

and the backward equation

$$\frac{\partial}{\partial t}p_t(x,y) = \mu\frac{\partial}{\partial x}p_t(x,y) + \frac{1}{2}\sigma^2\frac{\partial^2}{\partial x^2}p_t(x,y). \tag{18.2.10}$$

Proof. Direct calculus, exactly as in the standard case but now carrying the extra drift term through the chain rule. □

Just as in the standard case, $\tau_x = \inf\{t \ge 0 : X_t = x\}$ is a stopping time, $\mathcal{F}_\tau$ is defined analogously to (18.1.15), and Brownian motion with drift and scaling retains the strong Markov property:

Theorem 5.2.11 (Strong Markov property)**.** *If τ is a stopping time and $Y_t = X_{\tau+t} - X_\tau$, then Y is a Brownian motion with the same drift and scale parameters, and is independent of $\mathcal{F}_\tau$.*

5.3 The Brownian Bridge

5.3.1 Definition and Constructions

In its most common formulation, the *Brownian bridge* process is obtained by taking a standard Brownian motion on $[0,1]$ and conditioning on the event $X_1 = 0$. Since we already have $X_0 = 0$, the process is "tied down" at both ends, and the path in between forms a (very jagged) bridge. We first give an axiomatic definition and then several equivalent constructions from standard Brownian motion.

Definition 5.3.1 (Brownian bridge)**.** A *Brownian bridge* is a stochastic process $X = \{X_t : t \in [0,1]\}$ with state space $\mathbb{R}$ satisfying:

(1) $X_0 = 0$ and $X_1 = 0$ (each with probability 1).
(2) X is a Gaussian process.
(3) $\mathbb{E}(X_t) = 0$ for $t \in [0,1]$.
(4) $\operatorname{Cov}(X_s, X_t) = \min\{s,t\} - st$ for $s,t \in [0,1]$.
(5) With probability 1, $t \mapsto X_t$ is continuous on $[0,1]$.

So a Brownian bridge is simply a continuous Gaussian process pinned to 0 at both endpoints, with a specific covariance structure. We now verify existence via several explicit constructions from a standard Brownian motion $Z = \{Z_t : t \in [0,\infty)\}$.

Theorem 5.3.2 (Construction 1)**.** *Let Z be standard Brownian motion and define $X_t = Z_t - tZ_1$ for $t \in [0,1]$. Then X is a Brownian bridge.*

Proof. $X_0 = Z_0 = 0$ and $X_1 = Z_1 - Z_1 = 0$. Linear combinations of the X_t's reduce to linear combinations of the Z_t's, hence are normal, so X is Gaussian. $\mathbb{E}(X_t) = \mathbb{E}(Z_t) - t\mathbb{E}(Z_1) = 0$. For $s,t \in [0,1]$,

$$\begin{aligned}\operatorname{Cov}(X_s, X_t) &= \operatorname{Cov}(Z_s - sZ_1,\ Z_t - tZ_1)\\ &= \operatorname{Cov}(Z_s, Z_t) - t\operatorname{Cov}(Z_s, Z_1) - s\operatorname{Cov}(Z_1, Z_t)\\ &\quad + st\operatorname{Cov}(Z_1, Z_1)\\ &= \min\{s,t\} - st - st + st = \min\{s,t\} - st.\end{aligned}$$

Continuity of X follows from that of Z. □

Theorem 5.3.3 (Reverse construction 1)**.** *Suppose X is a Brownian bridge and Z is a standard normal random variable independent of X. Let $Z_t = X_t + tZ$ for $t \in [0,1]$. Then $Z = \{Z_t : t \in [0,1]\}$ is a standard Brownian motion on $[0,1]$.*

Proof. $Z_0 = X_0 = 0$. Linear combinations reduce to linear combinations of X's and Z, hence are normal; Z is Gaussian. $\mathbb{E}(Z_t) = \mathbb{E}(X_t) + t\mathbb{E}(Z) = 0$. For $s \le t$,

$$\begin{aligned}\operatorname{Cov}(Z_s, Z_t) &= \operatorname{Cov}(X_s + sZ,\ X_t + tZ)\\ &= \operatorname{Cov}(X_s, X_t) + t\operatorname{Cov}(X_s, Z) + s\operatorname{Cov}(X_t, Z) + st\operatorname{Var}(Z)\\ &= (\min\{s,t\} - st) + 0 + 0 + st = \min\{s,t\}.\end{aligned}$$

Continuity carries over from X. □

Theorem 5.3.4 (Construction 2)**.** *Let Z be standard Brownian motion, set $X_1 = 0$, and define*

$$X_t = (1-t)\, Z\Big(\frac{t}{1-t}\Big), \qquad t \in [0,1). \tag{18.3.1}$$

Then $X = \{X_t : t \in [0,1]\}$ is a Brownian bridge.

Proof. $X_0 = Z_0 = 0$, and by definition $X_1 = 0$. X is Gaussian since it is a (time-changed) linear transformation of Z. For $t \in [0,1)$,

$$\mathbb{E}(X_t) = (1-t)\,\mathbb{E}\Big[Z\Big(\frac{t}{1-t}\Big)\Big] = 0. \tag{18.3.2}$$

If $s < t$ in $[0,1)$ then $s/(1-s) < t/(1-t)$, so

$$\operatorname{Cov}(X_s, X_t) = \operatorname{Cov}\Big[(1-s)Z\Big(\tfrac{s}{1-s}\Big),\, (1-t)Z\Big(\tfrac{t}{1-t}\Big)\Big] = (1-s)(1-t)\frac{s}{1-s} = s(1-t). \tag{18.3.3}$$

By symmetry, $\operatorname{Cov}(X_s, X_t) = \min\{s,t\}(1-\max\{s,t\}) = \min\{s,t\} - st$. Finally, $t \mapsto X_t$ is continuous with probability 1 on $[0,1)$, and, with probability 1, $X_t = (1-t)Z[t/(1-t)] \to 0$ as $t \uparrow 1$ (by the Law of the Iterated Logarithm applied to Z), matching $X_1 = 0$. □

Theorem 5.3.5 (Reverse construction 2)**.** *Suppose X is a Brownian bridge. Define*

$$Z_t = (1+t)\, X\Big(\frac{t}{1+t}\Big), \qquad t \in [0,\infty). \tag{18.3.4}$$

Then $Z = \{Z_t : t \in [0,\infty)\}$ is a standard Brownian motion.

Proof. $Z_0 = X_0 = 0$; linear combinations reduce to linear combinations of X's, so Z is Gaussian; $\mathbb{E}(Z_t) = (1+t)\mathbb{E}[X(t/(1+t))] = 0$. If $s < t$ in $[0,1]$ then $s/(1+s) < t/(1+t)$, and

$$\begin{aligned} \operatorname{Cov}(Z_s, Z_t) &= \operatorname{Cov}\Big[(1+s)X\Big(\tfrac{s}{1+s}\Big), (1+t)X\Big(\tfrac{t}{1+t}\Big)\Big] \\ &= (1+s)(1+t)\left[\frac{s}{1+s} - \frac{s}{1+s}\frac{t}{1+t}\right] = s. \end{aligned} \tag{18.3.6}$$

Continuity of Z follows from continuity of X. □

Theorem 5.3.6 (Conditioning construction)**.** *Let X be a standard Brownian motion on $[0,\infty)$. Then, conditioned on $X_1 = 0$, the process $\{X_t : t \in [0,1]\}$ is a Brownian bridge.*

Proof sketch. Continuity survives conditioning, and the conditioned process remains Gaussian, since for $s < t$ in $[0,1]$, (X_t, X_1) has a joint normal distribution determined by the mean/covariance of X; by standard multivariate-normal computations, the conditional law of X_t given $X_1 = 0$ is $N(0, t(1-t))$, and the conditional law of (X_s, X_t) given $X_1 = 0$ is bivariate normal with mean $(0,0)$ and $\operatorname{Cov}(X_s, X_t \mid X_1 = 0) = s(1-t)$. □

Theorem 5.3.7 (Stochastic-integral construction)**.** *Let Z be standard Brownian motion, set $X_1 = 1$, and define*

$$X_t = (1-t)\int_0^t \frac{1}{1-s}\,\mathrm{d}Z_s, \qquad t \in [0,1). \tag{18.3.7}$$

Then $X = \{X_t : t \in [0,1]\}$ is a Brownian bridge (started at 1; the analogous construction pinned at 0 replaces the boundary value accordingly).

Proof. $X_0 = 0$ and $X_1 = 0$ (or 1, by definition of the endpoint). Since the integrand $1/(1-s)$ is deterministic, the Itô integral $\int_0^t \frac{1}{1-s}\,\mathrm{d}Z_s$ is Gaussian, so X is a Gaussian process. By the martingale inequality for stochastic integrals, $X_t \to 0$ as $t \uparrow 1$ with probability 1, and X is continuous on $[0,1)$ with probability 1 (basic properties of Itô integrals). $\mathbb{E}(X_t) = 0$ since the stochastic integral has mean 0. For $s \le t$ in $[0,1]$, using independence of $\int_0^s$ and $\int_s^t$ increments and the Itô isometry,

$$\mathrm{Cov}(X_s, X_t) = (1-s)(1-t)\ \mathrm{Var}\Big(\int_0^s \frac{1}{1-u}\,\mathrm{d}Z_u\Big) \tag{18.3.9}$$

$$= (1-s)(1-t)\int_0^s \frac{1}{(1-u)^2}\,\mathrm{d}u = (1-s)(1-t)\left(\frac{1}{1-s} - 1\right) = (1-t)s. \tag{18.3.10}$$

□

In differential form, this construction can be written

$$dX_t = \frac{X_t}{1-t}\,\mathrm{d}t + \mathrm{d}Z_t, \qquad X_0 = 0. \tag{18.3.11}$$

Remark 5.3.8. This SDE is illuminating: the drift term $X_t/(1-t)$ grows without bound as $t \to 1^-$ whenever $X_t \neq 0$, and it always points *back toward zero* (opposite in sign to X_t multiplied by... actually here the drift pushes X_t away from zero when $X_t \neq 0$, but the shrinking coefficient $(1-t)$ in the original definition (18.3.7) exactly compensates, forcing $X_t \to 0$ as $t \to 1$). This is a first example of the type of self-correcting SDE used throughout diffusion theory to force a process to a prescribed terminal value.

Proposition 5.3.9. *For the Brownian bridge X, $X_t \sim N(0, t(1-t))$ for $t \in [0,1]$. In particular the variance increases and then decreases on $[0,1]$, reaching a maximum of $1/4$ at $t = 1/2$, and vanishing at the two endpoints $t = 0, 1$ (as it must, since $X_0 = X_1 = 0$ deterministically).*

5.3.2 The General Brownian Bridge

The constructions above all produce the *standard* Brownian bridge, pinned at 0 at both ends. It is simple to generalize to a bridge from a to b for arbitrary $a, b \in \mathbb{R}$.

Theorem 5.3.10. *Let $Z = \{Z_t : t \in [0,1]\}$ be a standard Brownian bridge, and for $a, b \in \mathbb{R}$ define*

$$X_t = (1-t)a + tb + Z_t, \qquad t \in [0,1].$$

Then $X = \{X_t : t \in [0,1]\}$ is a Brownian bridge process from a to b.

Any of the constructions above for the standard bridge can similarly be modified to produce a general bridge; the general characterizing properties are:

Definition 5.3.11 (General Brownian bridge)**.** The Brownian bridge process $X = \{X_t : t \in [0,1]\}$ from a to b satisfies:

(1) $X_0 = a$ and $X_1 = b$ (each with probability 1).
(2) X is a Gaussian process.
(3) $\mathbb{E}(X_t) = (1-t)a + tb$ for $t \in [0,1]$.
(4) $\mathrm{Cov}(X_s, X_t) = \min\{s,t\} - st$ for $s, t \in [0,1]$.
(5) With probability 1, $t \mapsto X_t$ is continuous on $[0,1]$.

5.3.3 Applications: The Empirical Distribution Function

We turn to one of the most fundamental problems in statistics — and to the Brownian bridge's most celebrated application.

Suppose T is a real-valued random variable with unknown distribution function $F(t) = \mathbb{P}(T \le t)$. To estimate F, sample $\mathbf{T} = (T_1, T_2, \dots)$, a sequence of independent copies of T. For $n \in \mathbb{N}_+$ and $t \in \mathbb{R}$, the natural estimator based on the first n observations is

$$F_n(t) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(T_i \le t), \tag{18.3.12}$$

the *empirical distribution function*. Since $\mathbf{1}(T_1 \le t), \mathbf{1}(T_2 \le t), \dots$ are i.i.d. Bernoulli trials with success probability $F(t)$, standard sample-mean theory gives:

Proposition 5.3.12. *For fixed $t \in \mathbb{R}$:*

(1) $\mathbb{E}[F_n(t)] = F(t)$ *(unbiasedness).*

(2) $\mathrm{Var}[F_n(t)] = F(t)[1 - F(t)]/n$ *(consistency).*

(3) $F_n(t) \to F(t)$ *as* $n \to \infty$ *with probability 1 (strong law of large numbers).*

(4) $\sqrt{n}\,[F_n(t) - F(t)]$ *has mean* 0*, variance* $F(t)[1 - F(t)]$*, and converges to the corresponding normal distribution as* $n \to \infty$ *(central limit theorem).*

Fact (4) describes the behavior at a single, fixed t. To understand the *whole function* $t \mapsto F_n(t)$, consider the special case $T \sim \mathrm{Uniform}(0, 1)$, so $F(t) = t$ for $t \in [0, 1]$, and form the stochastic process

$$X_n(t) = \sqrt{n}\,[F_n(t) - t], \qquad t \in [0, 1]. \tag{18.3.13}$$

Theorem 5.3.13. *X_n has mean function* 0 *and variance function $t(1 - t)$, and for each fixed t, $X_n(t)$ converges in distribution to $N(0, t(1 - t))$ as $n \to \infty$. Moreover, the covariance function of X_n coincides* exactly *with that of the Brownian bridge:*

$$\mathrm{Cov}[X_n(s), X_n(t)] = \min\{s, t\} - st, \qquad s, t \in [0, 1]. \tag{18.3.16}$$

Proof. Suppose $s \le t$. From basic properties of covariance,

$$\mathrm{Cov}[X_n(s), X_n(t)] = n\ \mathrm{Cov}[F_n(s), F_n(t)] = \frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{n} \mathrm{Cov}[\mathbf{1}(T_i \le s), \mathbf{1}(T_j \le t)]. \tag{18.3.14}$$

For $i \ne j$, $\mathbf{1}(T_i \le s)$ and $\mathbf{1}(T_j \le t)$ are independent, hence have covariance 0. For $i = j$,

$$\begin{aligned} \mathrm{Cov}[\mathbf{1}(T_i \le s), \mathbf{1}(T_i \le t)] &= \mathbb{P}(T_i \le s,\, T_i \le t) - \mathbb{P}(T_i \le s)\mathbb{P}(T_i \le t) \\ &= \mathbb{P}(T_i \le s) - \mathbb{P}(T_i \le s)\mathbb{P}(T_i \le t) \\ &= s - st. \end{aligned} \tag{18.3.15}$$

using $s \le t$ and $F(u) = u$. Summing the n diagonal terms and dividing by n gives $\mathrm{Cov}[X_n(s), X_n(t)] = s - st = \min\{s, t\} - st$. □

Remark 5.3.14 (Why this matters). This result is the seed of the theory of *empirical processes*. Because the covariance structure of X_n already matches the Brownian bridge exactly for every n, and because $X_n(t)$ converges to a normal distribution pointwise, Donsker's theorem shows that the entire process X_n converges (in an appropriate functional sense) to a genuine Brownian bridge as $n \to \infty$. This is the theoretical foundation for goodness-of-fit statistics such as the Kolmogorov–Smirnov and Cramér–von Mises tests, whose limiting null distributions are functionals (such as the supremum or the integral of the square) of a standard Brownian bridge.

5.4 Geometric Brownian Motion

5.4.1 Definition

Geometric Brownian motion (GBM) is the process most commonly used to model quantities that must remain positive and grow (or decay) multiplicatively subject to random noise — populations, and, famously, stock prices.

Definition 5.4.1 (Geometric Brownian motion)**.** Suppose $Z = \{Z_t : t \in [0, \infty)\}$ is standard Brownian motion, $\mu \in \mathbb{R}$, and $\sigma \in (0, \infty)$. Let

$$X_t = \exp\left[\left(\mu - \frac{\sigma^2}{2}\right) t + \sigma Z_t\right], \qquad t \in [0, \infty). \tag{18.4.1}$$

The process $X = \{X_t : t \in [0, \infty)\}$ is *geometric Brownian motion* with drift parameter μ and volatility parameter σ.

Equivalently, $X_t = \exp(U_t)$ where

$$U_t = \left(\mu - \frac{\sigma^2}{2}\right) t + \sigma Z_t, \qquad t \in [0, \infty), \tag{18.4.2}$$

is an ordinary Brownian motion with drift parameter $\mu - \sigma^2/2$ and scale parameter σ (Section 5.2). So geometric Brownian motion is simply the *exponential* of Brownian motion with drift and scaling. Because $\exp(\cdot) > 0$ always, $X_t > 0$ for all t — precisely why GBM is used to model quantities (population sizes, prices) that cannot be negative. Note $X_0 = 1$; for $x_0 > 0$, the process $\{x_0 X_t\}$ is geometric Brownian motion *starting at* x_0.

Intuition 5.4.2 (Why $\mu - \sigma^2/2$?). One might expect the "drift" inside the exponential to simply be μ, matching the deterministic growth rate. The extra $-\sigma^2/2$ is precisely the correction needed so that $\mathbb{E}(X_t) = e^{\mu t}$, matching ordinary deterministic exponential growth at rate μ (see the moments below). Intuitively, $\mathbb{E}[e^{\sigma Z_t}] = e^{\sigma^2 t/2}$ (from the Brownian motion moment generating function), which is *larger* than $e^0 = 1$ because of Jensen's inequality applied to the convex exponential function — averaging a convex function of a mean-zero random variable overshoots the function of the average. Subtracting $\sigma^2/2$ from the drift exactly cancels this convexity bias, so that the mean of X_t grows at the "clean" rate μ even though the typical (median, or almost-sure long-run) behavior of X_t is governed by $\mu - \sigma^2/2$, not μ (see the asymptotic behavior below).

Theorem 5.4.3 (Stochastic differential equation)**.** *Geometric Brownian motion satisfies*

$$dX_t = \mu X_t \, dt + \sigma X_t \, dZ_t. \tag{18.4.3}$$

The deterministic part of this SDE, $dX_t = \mu X_t \, dt$, is exactly the ODE for exponential growth or decay at rate μ; the random term $\sigma X_t \, dZ_t$ superimposes proportional (multiplicative) noise, whose intensity scales with the current level X_t — larger populations/prices experience larger absolute fluctuations, but the same *relative* fluctuations, which is the defining feature of genuinely multiplicative noise.

5.4.2 Distributions

Theorem 5.4.4 (Lognormal distribution)**.** *For $t \in (0,\infty)$, X_t has the* lognormal distribution *with parameters $(\mu - \sigma^2/2)t$ and $\sigma\sqrt{t}$; its density is*

$$f_t(x) = \frac{1}{\sqrt{2\pi t}\,\sigma x} \exp\left(-\frac{[\ln x - (\mu - \sigma^2/2)t]^2}{2\sigma^2 t}\right), \qquad x \in (0,\infty). \tag{18.4.4}$$

Moreover:

(1) f_t increases and then decreases, with mode at $x = \exp[(\mu - \frac{3}{2}\sigma^2)t]$.

(2) f_t is concave upward, then downward, then upward again, with inflection points at $x = \exp\Big[(\mu - \sigma^2)t \pm \frac{1}{2}\sigma\sqrt{\sigma^2 t^2 + 4t}\Big]$.

Proof. Since $U_t = (\mu - \sigma^2/2)t + \sigma Z_t \sim N\Big((\mu - \sigma^2/2)t,\, \sigma^2 t\Big)$, and $X_t = e^{U_t}$, X_t is lognormal with these two parameters by definition; the PDF and its shape properties follow directly from the standard theory of the lognormal distribution. □

Remark 5.4.5. Because X_t is a (nonlinear) transformation of a Gaussian process rather than a linear one, *geometric* Brownian motion itself is **not** a Gaussian process, in sharp contrast to standard Brownian motion and Brownian motion with drift and scaling.

Proposition 5.4.6 (Distribution and quantile functions)**.** *For $t \in (0,\infty)$,*

$$F_t(x) = \Phi\left[\frac{\ln x - (\mu - \sigma^2/2)t}{\sigma\sqrt{t}}\right], \qquad x \in (0,\infty), \tag{18.4.5}$$

where Φ is the standard normal distribution function, and

$$F_t^{-1}(p) = \exp\Big[(\mu - \sigma^2/2)t + \sigma\sqrt{t}\,\Phi^{-1}(p)\Big], \qquad p \in (0,1). \tag{18.4.6}$$

Proof. Both follow directly from the corresponding CDF and quantile function of the lognormal distribution. □

5.4.3 Moments

Theorem 5.4.7. *For $n \in \mathbb{N}$ and $t \in [0,\infty)$,*

$$\mathbb{E}(X_t^n) = \exp\left\{\left[n\mu + \frac{\sigma^2}{2}(n^2 - n)\right] t\right\}. \tag{18.4.7}$$

Proof. This follows from the moment formula for the lognormal distribution. □

As a function of the moment order n, the dominant term inside the exponential is $\sigma^2 n^2/2$: if $n > 1 - 2\mu/\sigma^2$, then $n\mu + \frac{\sigma^2}{2}(n^2 - n) > 0$, so $\mathbb{E}(X_t^n) \to \infty$ as $t \to \infty$. Setting $n = 1, 2$ gives the mean and variance:

Corollary 5.4.8. *For $t \in [0,\infty)$,*

(1) $\mathbb{E}(X_t) = e^{\mu t}$.

(2) $\mathrm{Var}(X_t) = e^{2\mu t}\left(e^{\sigma^2 t} - 1\right)$.

The mean function $m(t) = \mathbb{E}(X_t) = e^{\mu t}$ satisfies precisely the deterministic part of the SDE (18.4.3). If $\mu > 0$, $m(t) \to \infty$; if $\mu = 0$, $m(t) \equiv 1$; if $\mu < 0$, $m(t) \to 0$, as $t \to \infty$.

5.4.4 Asymptotic Behavior

Although the *mean* of X_t is governed entirely by μ, the *almost-sure, pathwise* long-run behavior is governed by the combination $\mu - \sigma^2/2$ — this is the key qualitative fact about GBM, and the source of the "volatility drag" well known in mathematical finance.

Theorem 5.4.9 (Asymptotic behavior)**.** *(1) If $\mu > \sigma^2/2$, then $X_t \to \infty$ as $t \to \infty$, with probability 1.*

(2) If $\mu < \sigma^2/2$, then $X_t \to 0$ as $t \to \infty$, with probability 1.

(3) If $\mu = \sigma^2/2$, then X_t has no limit as $t \to \infty$, with probability 1.

Proof. These results follow from the Law of the Iterated Logarithm applied to Z_t: asymptotically, the linear term $(\mu - \sigma^2/2)t$ dominates the fluctuating term σZ_t (which only grows like $\sqrt{t \ln \ln t}$), so the sign of $\mu - \sigma^2/2$ determines whether $U_t = \ln X_t \to +\infty$, $-\infty$, or oscillates without limit. □

Remark 5.4.10 (Volatility drag). This is exactly why, in a model of stock prices, one can have a positive expected return ($\mu > 0$) while the price nonetheless converges to 0 almost surely — this happens whenever $0 < \mu < \sigma^2/2$. Increasing volatility σ (holding μ fixed) can flip the almost-sure long-run trend from growth to decay even though it leaves the expected value $\mathbb{E}(X_t) = e^{\mu t}$ completely unchanged. The expectation is dominated by increasingly rare, increasingly enormous upward excursions, while the "typical" path decays.

5.4.5 Martingale Property

Theorem 5.4.11. *If $\mu = 0$, geometric Brownian motion X is a martingale with respect to the filtration $\mathcal{G} = \{\mathcal{F}_t : t \geq 0\}$ generated by the underlying standard Brownian motion Z, where $\mathcal{F}_t = \sigma\{Z_s : 0 \leq s \leq t\}$.*

Proof from stochastic integrals. When $\mu = 0$, X satisfies $dX_t = \sigma X_t \, dZ_t$, so

$$X_t = 1 + \sigma \int_0^t X_s \, \mathrm{d}Z_s, \qquad t \geq 0. \tag{18.4.8}$$

Any process defined as (the initial value plus) a stochastic integral is automatically a martingale, under the usual integrability conditions on the integrand — which hold here. □

Direct proof. Let $\mathcal{F}_t = \sigma\{Z_s : 0 \leq s \leq t\}$ and let $0 \leq s \leq t$. Writing $Z_t = Z_s + (Z_t - Z_s)$ to exploit stationary independent increments,

$$X_t = \exp\left[-\frac{\sigma^2}{2}t + \sigma Z_s + \sigma(Z_t - Z_s)\right]. \tag{18.4.9}$$

Since Z_s is $\mathcal{F}_s$-measurable and $Z_t - Z_s$ is independent of $\mathcal{F}_s$,

$$\mathbb{E}(X_t \mid \mathcal{F}_s) = \exp\left(-\frac{\sigma^2}{2}t + \sigma Z_s\right) \mathbb{E}\{\exp[\sigma(Z_t - Z_s)]\}. \tag{18.4.10}$$

But $Z_t - Z_s \sim N(0, t-s)$, so by the moment generating function of the normal distribution,

$$\mathbb{E}\{\exp[\sigma(Z_t - Z_s)]\} = \exp\left[\frac{\sigma^2}{2}(t-s)\right]. \tag{18.4.11}$$

Substituting,

$$\mathbb{E}(X_t \mid \mathcal{F}_s) = \exp\left(-\frac{\sigma^2}{2}s + \sigma Z_s\right) = X_s. \tag{18.4.12}$$

□

Remark 5.4.12. The martingale property when $\mu = 0$ is the mathematical heart of risk-neutral option pricing in the Black–Scholes framework: under the risk-neutral measure, the discounted price process is modeled as a driftless ($\mu = 0$) geometric Brownian motion, guaranteeing it is a martingale and hence arbitrage-free in the relevant sense.

5.5 Modelling Stock Price

5.6 Itô's Lemma and Girsanov's Theorem

5.7 Factors Affecting Option Prices

5.8 Black–Scholes

5.9 Deriving the Black–Scholes Equation

5.10 Options Pricing

5.11 Binomial Model

5.12 Option Greeks

Chapter 6

Monte Carlo Simulation

This chapter synthesizes the course material on Brownian motion for quantitative finance, the probabilistic justification of Monte Carlo simulation, option pricing by simulation, and its relationship with the Black–Scholes model. The emphasis is on concepts, mathematics, intuition, and worked numerical examples rather than implementation code.

6.1 The Big Picture

Monte Carlo simulation answers the question *What is the average or expected outcome of this random process?* by repeatedly sampling from the process and averaging the resulting values, rather than by solving for the answer analytically.

This matters in quantitative finance because many problems, from valuing a simple wager to pricing an exotic derivative, reduce to computing an expectation. Expectations of complicated random processes can be very hard, or impossible, to calculate in closed form, while simulating those processes is often comparatively straightforward. The mathematical foundation of the method is the **Law of Large Numbers**.

6.2 Foundations: Random Variables

A random variable X is characterized by equivalent descriptions of its distribution, including the following.

- **Cumulative distribution function:**

$$F_X(a) = \mathbb{P}(X \leq a) = \int_{-\infty}^{a} f_X(x)\ \mathrm{d}x.$$

- **Probability mass or density function:**

$$f_X(a) = \mathbb{P}(X = a)$$

  in the discrete case, while in the continuous case

$$f_X(a) = \frac{\mathrm{d}}{\mathrm{d}a}\mathbb{P}(X \leq a).$$

- **Characteristic function:**

$$\varphi_X(t) = \mathbb{E}[e^{itX}] = \int_{-\infty}^{\infty} e^{itx} f_X(x)\ \mathrm{d}x.$$

Although X is written like an ordinary variable, it represents an entire distribution of possible outcomes. Individual draws differ, but a large sample histogram reveals the distribution governing the randomness. A single draw cannot generally be predicted; probability describes the relative likelihood of its possible outcomes. This is why uncertainty requires statistical and simulation-based tools.

6.2.1 The Normal Random Variable

If $X \sim \mathcal{N}(\mu, \sigma^2)$, then

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right], \qquad F_X(x) = \int_{-\infty}^{x} f_X(u)\ \mathrm{d}u,$$

and its characteristic function is

$$\varphi_X(t) = \exp\left(i\mu t - \frac{1}{2}\sigma^2 t^2\right).$$

The characteristic function is the Fourier-transform representation of the same distribution:

$$f(x) \xrightarrow{\mathcal{F}} \varphi(t) \xrightarrow{\mathcal{F}^{-1}} f(x).$$

The inverse transformation recovers the original density, so moving to the frequency domain changes the representation rather than the underlying probability law.

6.2.2 Mean and Variance

The expected value is

$$\mathbb{E}[X] = \sum_x x\, \mathbb{P}(X = x)$$

for a discrete variable and

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x)\ \mathrm{d}x$$

for a continuous variable. Its variance is

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

For $X \sim \mathcal{N}(\mu, \sigma^2)$, the mean is μ and the variance is σ^2.

The normal distribution appears repeatedly because the Central Limit Theorem states that, under broad conditions, standardized sample averages approach a normal distribution as the sample size grows, regardless of the shape of the original population distribution. This connects probability theory to empirical statistics and explains why Brownian motion, constructed from Gaussian increments, naturally models the accumulation of many small random shocks.

6.3 The Law of Large Numbers

The Law of Large Numbers explains why Monte Carlo simulation works: the average of sufficiently many independent, identically distributed draws approaches the underlying expected value.

Theorem 6.3.1 (Weak Law of Large Numbers)**.** *Let $X_1, X_2, \ldots$ be i.i.d. random variables with finite mean μ, and let*

$$\overline{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Then, for every $\varepsilon > 0$,

$$\lim_{n\to\infty} \mathbb{P}\Big(\Big|\overline{X}_n - \mu\Big| > \varepsilon\Big) = 0.$$

Thus $\overline{X}_n$ converges to μ in probability.

Theorem 6.3.2 (Strong Law of Large Numbers)**.** *Under the standard integrability assumptions,*

$$\mathbb{P}\Big(\lim_{n\to\infty} \overline{X}_n = \mu\Big) = 1.$$

Thus the sample mean converges to μ almost surely.

The strong law gives the stronger pathwise statement: with probability one, the particular sequence of running averages that is observed converges to the true mean.

6.3.1 Dice-Roll Example

For a fair six-sided die,

$$\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5,$$

and

$$\begin{aligned}\mathrm{Var}(X) &= \frac{(1-3.5)^2+(2-3.5)^2+(3-3.5)^2+(4-3.5)^2+(5-3.5)^2+(6-3.5)^2}{6} \\ &= \frac{17.5}{6} \approx 2.917.\end{aligned}$$

Simulated running estimates of both quantities converge to these theoretical values.

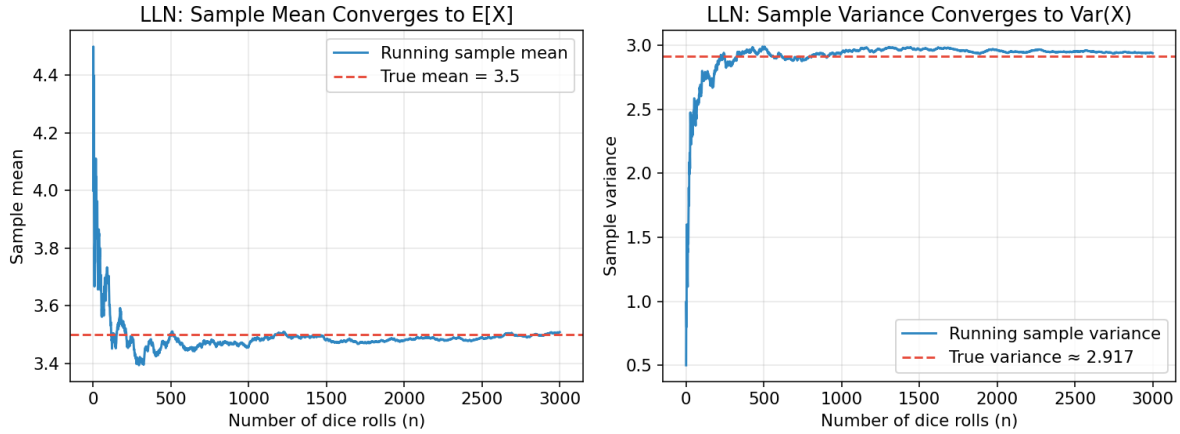


Figure 6.1: Convergence of the running sample mean and sample variance for fair die rolls.

6.3.2 Estimating Other Statistics

For observations $X_1, \ldots, X_n$ with sample mean $\overline{X}$, common Monte Carlo estimators are

$$\mathbb{E}[X] \approx \frac{1}{n}\sum_{i=1}^{n} X_i, \qquad \mathrm{Var}(X) \approx \frac{1}{n-1}\sum_{i=1}^{n}(X_i-\overline{X})^2,$$

$$\mathrm{Skew}(X) \approx \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i-\overline{X})^3}{\left[\frac{1}{n}\sum_{i=1}^{n}(X_i-\overline{X})^2\right]^{3/2}},$$

and

$$\mathrm{Kurt}(X) \approx \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i-\overline{X})^4}{\left[\frac{1}{n}\sum_{i=1}^{n}(X_i-\overline{X})^2\right]^{2}}.$$

An estimator can be **biased**, meaning its expected value differs systematically from the parameter being estimated. The $n-1$ denominator in the sample-variance estimator corrects the downward bias that would result from using n after estimating the mean from the same observations. Bias may sometimes be introduced deliberately for stability, but it must be understood and assessed.

6.3.3 Recovering an Entire Distribution

The indicator $\mathbf{1}\{X_i \le x\}$ is a Bernoulli random variable whose mean is the true CDF value $F(x)$. Consequently, the empirical distribution function

$$F_n(x) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{X_i \le x\} \xrightarrow{\text{a.s.}} F(x) \qquad (n \to \infty).$$

Thus simulation can recover probabilities and an entire distribution, not only a few moments. The same idea later estimates probabilities of financial events.

6.3.4 Randomness Versus Uncertainty

The LLN assumes repeated draws from a fixed distribution. Real financial distributions are not directly observable and are often non-stationary: their centers, spreads, tails, and dependence structures can change over time. Simulation therefore offers clean convergence under an assumed model, whereas markets contain **uncertainty** about the model itself as well as **randomness** conditional on the model.

6.4 What Monte Carlo Simulation Is

Definition 6.4.1 (Monte Carlo simulation)**.** Monte Carlo simulation approximates an expectation through repeated random sampling:

1. generate many independent samples from the distribution or process of interest;
2. calculate the desired quantity for every sample;
3. average the calculated values.

Important practical considerations include using a sufficiently large sample size, generating randomness from the correct target distribution, employing variance-reduction methods when computation is expensive, and validating against an analytical solution whenever one is available. Sample sizes of 10,000 or more are common starting points, although the required size depends on the payoff variance and the desired precision.

6.4.1 Bernoulli and Exponential Examples

Distribution	**Analytical** $\mathbb{E}[X]$	**Naive MC estimate,** $n = 100{,}000$
Bernoulli, $p = 0.7$	0.7000	0.7011
Exponential, $\lambda = 2.0$	$1/\lambda = 0.5000$	0.5023

Even a naive simulation with 100,000 samples produces estimates within roughly 0.001–0.002 of the analytical values for these examples.

6.4.2 Geometric Example: Rolls Until the First Six

Let X be the number of fair-die rolls required to obtain the first six. Then X has a geometric distribution with $p = 1/6$:

$$\mathbb{P}(X = k) = p(1-p)^{k-1}, \qquad k = 1, 2, \ldots.$$

Using the geometric-series identity

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}, \qquad |x| < 1,$$

the mean is

$$\begin{aligned}\mathbb{E}[X] &= \sum_{k=1}^{\infty} k\, p(1-p)^{k-1} \\ &= p\, \frac{\mathrm{d}}{\mathrm{d}x} \left(\sum_{k=0}^{\infty} x^k \right) \Bigg|_{x=1-p} \\ &= p\, \frac{\mathrm{d}}{\mathrm{d}x} \left(\frac{1}{1-x} \right) \Bigg|_{x=1-p} = \frac{1}{p}.\end{aligned}$$

For $p = 1/6$, the analytical mean is 6.

n	**Monte Carlo estimate**
100	6.3900
1,000	6.0260
10,000	6.0170
100,000	6.0099

The estimates approach 6, but improvement slows as n grows. This reflects the characteristic Monte Carlo convergence rate $O(n^{-1/2})$. The same recipe extends to multidimensional asset paths and models combining different distributions.

6.5 Extending the LLN to Stochastic Processes

A stochastic process is a collection of random variables indexed by time or another parameter. Some processes have memory, while others are memoryless. For example, white noise

$$X_t = \varepsilon_t, \qquad \varepsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2),$$

is a sequence of independent Gaussian shocks.

Under a fixed process model, the LLN applies across independent paths. At a fixed time t,

$$\frac{1}{N} \sum_{i=1}^{N} X_i(t) \xrightarrow{N \to \infty} \mathbb{E}[X(t)],$$

and for an increment over Δt,

$$\frac{1}{N} \sum_{i=1}^{N} \big[X_i(t + \Delta t) - X_i(t) \big] \xrightarrow{N \to \infty} \mathbb{E}[X(t + \Delta t) - X(t)].$$

Thus an **ensemble average**, taken across many independent paths at a fixed time, converges to the theoretical process mean. This differs from a **time average**, calculated along a single long path. Equality of time and ensemble averages depends on **ergodicity**. This distinction is important in applications such as the Kelly criterion, where the experience of a single wealth path can differ substantially from the ensemble average.

6.6 Brownian Motion: A Quick Recap

Standard Brownian motion $W = (W_t)_{t \geq 0}$ satisfies

1. $W_0 = 0$;
2. increments after time s are independent of the history up to s;
3. $W_t - W_s \sim \mathcal{N}(0, t-s)$ for $0 \leq s < t$;
4. sample paths are almost surely continuous and nowhere differentiable.

For a partition $0 = t_0 < t_1 < \cdots < t_n = t$, the path value can be reconstructed from its increments:

$$W_t = \sum_{k=1}^{n} \left(W_{t_k} - W_{t_{k-1}} \right).$$

Hence a complete path can be generated by accumulating many small Gaussian increments. If only a single terminal value is required, one can instead sample $W_T \sim \mathcal{N}(0, T)$ directly.

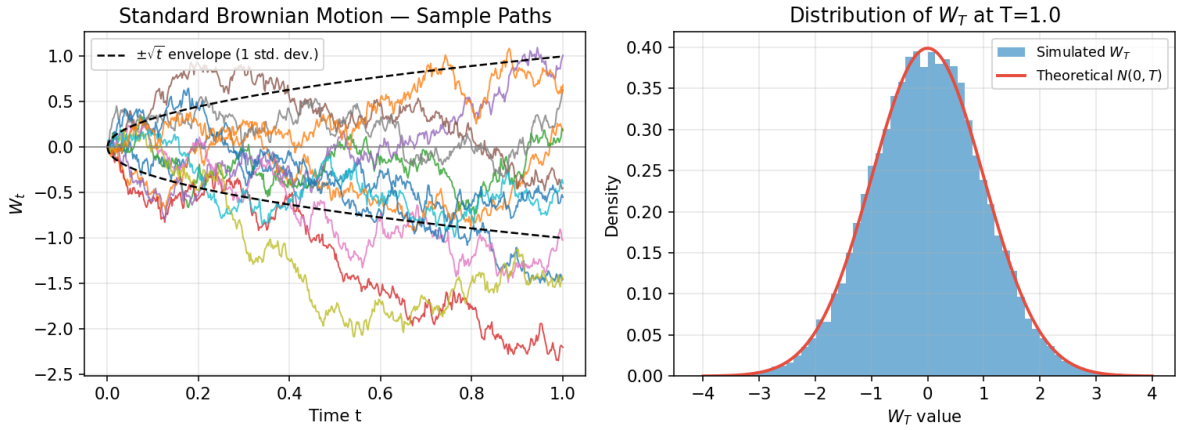


Figure 6.2: Standard Brownian sample paths and the empirical terminal distribution at $T = 1$.

The paths fan outward from zero, while the dashed $\pm\sqrt{t}$ envelope shows one standard deviation of spread. Across many independent paths, the empirical terminal distribution agrees with the theoretical $\mathcal{N}(0, T)$ law. This is the fixed-time ensemble convergence discussed in the previous section.

6.7 Worked Example: Estimating a Fair Game Value

Consider the following game.

1. Roll a fair six-sided die.
2. After an odd roll, flip a coin with a 60% probability of heads. Heads pays \$1000; tails loses \$500.
3. After an even roll, flip a fair coin. Heads loses \$500; tails pays \$1000.

The fair value is the break-even price that produces zero expected profit or loss. The two branches contribute

$$\mathbb{E}[\text{odd branch}] = \frac{1}{2}\Big[0.6(1000) + 0.4(-500)\Big] = 200$$

and

$$\mathbb{E}[\text{even branch}] = \frac{1}{2}\Big[0.5(-500) + 0.5(1000)\Big] = 125.$$

Therefore

$$\boxed{\mathbb{E}[\text{game payoff}] = \$325.00.}$$

The same Monte Carlo method remains valid when the expectation cannot be calculated analytically. A casino or market maker seeking a positive edge would charge more than \$325, such as \$330; a price of \$320 would instead favor the player. Repeated-play wealth paths show how even a small edge accumulates over many games.

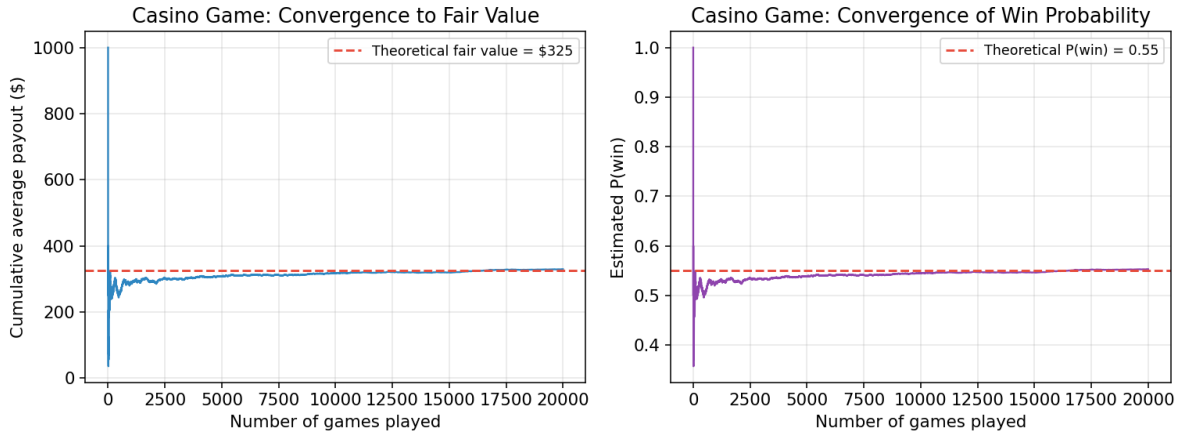


Figure 6.3: Monte Carlo convergence of the casino game's average payoff and win probability.

6.7.1 Estimating Event Probabilities

For an event indicator $I_i \sim \text{Bernoulli}(p)$,

$$\widehat{p}_n = \frac{1}{n}\sum_{i=1}^{n} I_i \xrightarrow{p} p.$$

The Monte Carlo recipe is to record $I_i = 1$ when the event occurs and $I_i = 0$ otherwise, then use the proportion of successes.

For the event that the game produces a positive payoff,

$$\begin{aligned}\mathbb{P}(\text{win}) &= \frac{3}{6}(0.6)(1) + \frac{3}{6}(0.4)(0) \\ &\quad + \frac{3}{6}(0.5)(0) + \frac{3}{6}(0.5)(1) \\ &= 0.30 + 0 + 0 + 0.25 = 0.55.\end{aligned}$$

The simulated running win rate in Figure 6.3 converges to 55%.

The same approach estimates quantities such as the probability of an overall profit, the probability of wealth exceeding a target after a fixed number of plays, or the probability of ruin before reaching a target. Classical gambler's-ruin models sometimes have closed forms, but Monte Carlo also handles uneven odds, path-dependent rules, and bankroll constraints.

6.8 Applying Monte Carlo to Option Pricing

6.8.1 The Black–Scholes Equation and Two Pricing Views

The Black–Scholes portfolio-replication argument constructs a risk-free portfolio from an option and an offsetting stock position. Requiring this portfolio to earn the risk-free rate produces the Black–Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0.$$

The Fundamental Theorem of Asset Pricing states that, in an arbitrage-free market, there is a risk-neutral measure $\mathbb{Q}$ under which the fair value of a derivative is its discounted expected payoff:

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\text{Payoff}(S_T)].$$

These are two representations of the same price. The Feynman–Kac theorem connects them:

$$\boxed{V(t,S) = \mathbb{E}^{\mathbb{Q}}\Big[e^{-r(T-t)}\,\text{Payoff}(S_T) \mid S_t = S\Big].}$$

Thus one may either solve the PDE analytically or numerically, or compute the risk-neutral expectation analytically or by Monte Carlo simulation. The expectation approach is often more convenient for complicated products.

6.8.2 Why Simulation Is Valid

Simulation-based option pricing combines three ideas.

1. Replication, no-arbitrage pricing, and Feynman–Kac identify the theoretically correct risk-neutral expectation.
2. The Law of Large Numbers ensures that averages of independent simulated discounted payoffs converge to this expectation when the payoff is integrable.
3. A discretization scheme such as Euler–Maruyama approximates paths of an SDE when an exact sampling formula is unavailable.

The LLN controls sampling error for the model being simulated. A time discretization may introduce a separate numerical bias, which must be controlled by refining the time grid or using a higher-order method.

6.8.3 Euler–Maruyama Discretization

Computers require a discrete time grid. For the geometric Brownian-motion SDE

$$\mathrm{d}S_t = \mu S_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}W_t,$$

Euler–Maruyama uses

$$\boxed{S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma S_t \Delta W_t, \qquad \Delta W_t \sim \mathcal{N}(0, \Delta t).}$$

At each step the price receives a deterministic drift contribution and a Gaussian shock whose standard deviation is proportional to $\sqrt{\Delta t}$. Repeating the update until T generates one approximate path. The method extends to stochastic-volatility and multifactor models for which exact path sampling is unavailable.

6.8.4 General Monte Carlo Pricing Recipe

1. Select a stochastic model for the underlying asset, such as arithmetic Brownian motion, geometric Brownian motion, or Heston stochastic volatility.
2. Simulate the process to maturity T, using an exact transition law or an appropriate discretization.
3. Compute the payoff Ψ for the simulated path.
4. Discount it:
$$Y = e^{-rT}\Psi.$$
5. Repeat independently and average:
$$\widehat{V}_N = \frac{1}{N}\sum_{i=1}^{N} Y_i.$$

By the LLN, $\widehat{V}_N$ converges to the risk-neutral price whenever the discounted payoff has a finite expectation.

6.8.5 Deriving the Black–Scholes Formula

Consider a European call $V(S,t)$. Form the portfolio
$$\Pi = V - \Delta S.$$
Over a short interval,
$$\mathrm{d}\Pi = \mathrm{d}V - \Delta\ \mathrm{d}S.$$
Itô's lemma gives
$$\mathrm{d}V = \frac{\partial V}{\partial t}\,\mathrm{d}t + \frac{\partial V}{\partial S}\,\mathrm{d}S + \frac{1}{2}\frac{\partial^2 V}{\partial S^2}(\,\mathrm{d}S)^2.$$
Under risk-neutral GBM,
$$\mathrm{d}S = rS\ \mathrm{d}t + \sigma S\ \mathrm{d}W, \qquad (\,\mathrm{d}S)^2 = \sigma^2 S^2\ \mathrm{d}t.$$
Therefore
$$\mathrm{d}\Pi = \left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2}\right)\mathrm{d}t + \left(\frac{\partial V}{\partial S} - \Delta\right)\mathrm{d}S.$$
Choosing
$$\Delta = \frac{\partial V}{\partial S}$$
eliminates the random term. The resulting riskless portfolio must satisfy
$$\mathrm{d}\Pi = r\Pi\ \mathrm{d}t.$$
Substitution yields
$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} = rV.$$
With the terminal condition
$$V(S,T) = \max(S-K,0),$$
the European call price is
$$\boxed{C(S_0,K,T) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2),}$$
where
$$d_1 = \frac{\ln(S_0/K) + (r+\sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

6.8.6 Monte Carlo European Call: Numerical Example

Instead of solving the PDE, approximate the risk-neutral expectation

$$C(S_0, K, T) = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\max(S_T - K, 0)].$$

For geometric Brownian motion under $\mathbb{Q}$, the terminal price can be sampled exactly:

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}\,Z\right], \qquad Z \sim \mathcal{N}(0,1).$$

Generate independent Z_i, calculate the corresponding terminal prices $S_T^{(i)}$, and use

$$\boxed{\widehat{C}_N = \frac{e^{-rT}}{N}\sum_{i=1}^{N}\max(S_T^{(i)} - K, 0).}$$

The Strong Law of Large Numbers gives

$$\frac{1}{N}\sum_{i=1}^{N}\max(S_T^{(i)} - K, 0) \xrightarrow{\text{a.s.}} \mathbb{E}^{\mathbb{Q}}[\max(S_T - K, 0)].$$

For

$$S_0 = 100, \qquad K = 100, \qquad T = 1, \qquad r = 5\%, \qquad \sigma = 20\%,$$

the analytical Black–Scholes value is approximately

$$\$10.4506,$$

while a representative Monte Carlo estimate from 100,000 simulations is

$$\$10.4436.$$

The difference is about seven cents for an option worth roughly ten dollars, and the sampling error decreases as the number of paths increases.

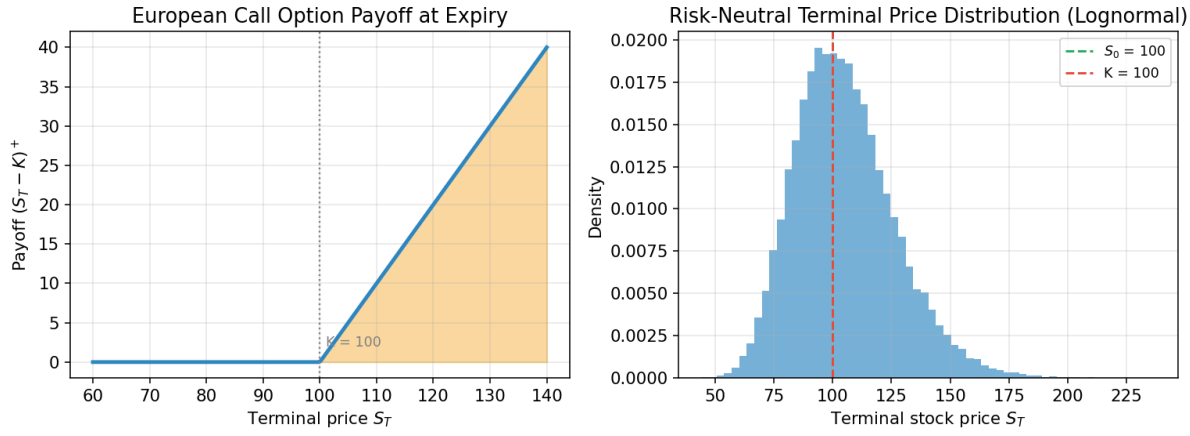


Figure 6.4: European call payoff and the risk-neutral lognormal distribution of the terminal stock price.

The payoff has the familiar hockey-stick shape: it is zero below the strike and increases one-for-one above it. The terminal stock-price distribution is right-skewed and strictly positive because GBM exponentiates a normal random variable.

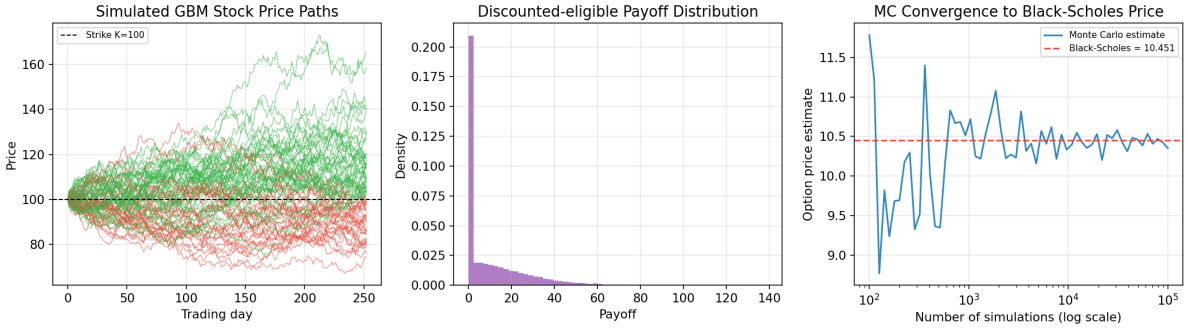


Figure 6.5: Simulated GBM paths, discounted-payoff distribution, and Monte Carlo convergence to the Black–Scholes call price.

In Figure 6.5, paths finishing above the strike produce positive payoffs, while out-of-the-money paths produce the point mass at zero visible in the payoff distribution. The final panel plots the running Monte Carlo estimate against the number of simulations on a logarithmic scale. It settles around the analytical Black–Scholes value as the sample grows.

6.8.7 Path-Dependent and Exotic Payoffs

For a derivative whose payoff depends on an additional path statistic A_t, such as a running average or maximum, a schematic pricing PDE may be written

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV + \mathcal{G}[V] = 0,$$

where $\mathcal{G}$ contains operators associated with the extra state variable. The Feynman–Kac theorem gives the corresponding conditional expectation:

$$V(t, S, A) = \mathbb{E}^{\mathbb{Q}}\Big[e^{-r(T-t)}\Psi(S_T, A_T) \mid S_t = S,\ A_t = A\Big].$$

Monte Carlo does not require a simple algebraic payoff. Once the relevant path features are simulated, the same discount-and-average procedure applies to Asian, barrier, lookback, and other path-dependent contracts.

Best-of rainbow option under stochastic volatility.
Suppose a two-asset option pays

$$\Psi = \max\Big(\max(S_T^{(1)}, S_T^{(2)}) - K, 0\Big).$$

If each asset follows a Heston model, its variance is itself a random, mean-reverting process. Important parameters include

- κ: the speed of variance mean reversion;
- θ: the long-run average variance;
- ξ: volatility of variance, or vol of vol;

- ρ: correlation between the asset and variance shocks, commonly negative for equities because of the leverage effect;
- v_0: initial variance.

The asset shocks may also be correlated with one another. This dependence is essential because the best-of payoff depends on their joint movement.

There is no simple Black–Scholes-style formula for this payoff under general correlated Heston dynamics. Monte Carlo nevertheless follows the same recipe: simulate the correlated price and variance paths, calculate the best-of payoff at maturity, discount it, repeat, and average. This illustrates the method's principal advantage: model and payoff complexity often changes the simulation details without changing the valuation framework.

6.9 Analytical and Monte Carlo Methods

	Analytical solutions	**Monte Carlo methods**
Speed	Exact results are immediate once the formula has been derived.	Can be slow: error typically decreases as $O(n^{-1/2})$, so quadrupling the simulation count only halves the standard error.
Insight	Exposes mathematical relationships, such as the dependence of value on volatility and time.	Produces numerical estimates rather than an explicit pricing formula.
Repeated use	Extremely efficient once a reusable formula exists.	A new parameter set or valuation may require another simulation run.
Flexibility	A tractable formula may not exist for exotic, path-dependent, or high-dimensional problems.	Handles nearly any distribution that can be sampled and scales naturally to path-dependent and multi-asset models.
Uncertainty quantification	Often presented as a single exact point value within the assumed model.	Naturally provides standard errors and confidence intervals from the dispersion of simulated payoffs.

The practical rule is to use a reliable closed form when one exists, because it is faster and more informative, and to use Monte Carlo when payoff structure, dimensionality, or stochastic dynamics make analytical or PDE methods impractical.

6.10 Learning Fast Pricing Functions from Simulation

A modern workflow combines the flexibility of Monte Carlo with the speed of a closed-form approximation.

1. Monte Carlo generates a training data set of model parameters and corresponding prices.
2. A neural network learns the mapping from parameters to prices.

3. After training, the network returns near-instant approximate prices for new parameter combinations.

This approach is valuable when no analytical solution exists but a full simulation is too expensive for repeated real-time use. The trained model acts as a surrogate for the pricing engine. Its accuracy, however, is limited by the quality and coverage of the simulated training data and must be validated carefully outside the calibration sample.

6.11 Challenges in Real-World Finance

All Monte Carlo conclusions are conditional on the assumed data-generating model. In practice:

- financial applications involve distributions not only of returns, but also of sentiment, momentum, mean reversion, volatility, liquidity, and dependence;
- a fixed distribution is mathematically convenient, while observed market distributions often change through time;
- model parameters must be estimated from finite and noisy data;
- every pricing or risk estimate inherits model-specification and parameter-estimation error;
- more simulations reduce sampling error but do not correct a wrong model.

These limitations do not prevent useful decisions under uncertainty. Even an imperfect simulation can reveal whether a decision has a sufficiently large edge, just as repeated trials in the casino example expose the effect of an entry fee without requiring perfect foresight about any single outcome.

Further topics include the failure modes of Value-at-Risk and related risk measures, discretization schemes for difficult stochastic processes, rough volatility, and the distinction between generating a global path and constructing a process properly adapted to a filtration.

6.12 Summary

- A random variable describes a distribution of possible outcomes, not a single predictable value.
- The Weak and Strong Laws of Large Numbers justify estimating expectations, moments, probabilities, and distribution functions from repeated samples.
- Monte Carlo simulation generates independent scenarios, evaluates a quantity in every scenario, and averages the results.
- The same convergence argument applies to ensembles of independent stochastic-process paths, including Brownian motion and asset-price models.
- In derivative pricing, no-arbitrage theory and Feynman–Kac equate the PDE solution with a discounted risk-neutral expectation. Monte Carlo approximates this expectation.
- Euler–Maruyama provides a general method for discretizing SDE-driven models, while exact transition sampling should be used when available.
- Monte Carlo handles complex, path-dependent, stochastic-volatility, and multi-asset payoffs with little change to the basic valuation recipe.

- Its principal weaknesses are computational cost and the slow $O(n^{-1/2})$ convergence rate.
- The most important practical limitation is model uncertainty: additional paths reduce Monte Carlo sampling noise but cannot identify or repair an incorrect distributional assumption.

Chapter 7

Modelling Stock Prices

7.1 Why Stock Prices Need a Stochastic Model

Accurate models of stock-price behavior matter for investment strategy, risk management, and derivative pricing. Market prices typically move in an irregular, unpredictable way: a time-series plot displays continual small fluctuations, periods of trend, and occasionally a large structural shock. Brownian motion provides a foundational mathematical description of this random variation.

7.1.1 Visual Motivation

Consider the share price of Marks & Spencer plotted against time. The series wiggles around, sometimes trends upward or downward, and may display a large dip during an exceptional event such as the coronavirus pandemic.

> **Key observation.** A share price is a time-indexed process whose value changes unpredictably. A useful model must represent both its systematic movement and its random fluctuations.

7.2 Brownian Motion: Recap

7.2.1 Definition and Historical Origin

Brownian motion, also called the **Wiener process**, is a continuous-time stochastic process originally motivated by the irregular motion of microscopic particles. Robert Brown reported the phenomenon in 1827, Einstein explained its physical mechanism in 1905, and Norbert Wiener placed the process on a rigorous mathematical foundation in work beginning in 1918.

Brownian motion is commonly denoted by $B(t)$, B_t, or W_t, where t represents time. A standard Brownian motion $W = (W_t)_{t\geq 0}$ satisfies:

1. $W_0 = 0$;
2. increments over disjoint intervals are independent;
3. for $0 \leq s < t$,
$$W_t - W_s \sim \mathcal{N}(0, t-s);$$
4. sample paths are almost surely continuous and nowhere differentiable.

> These properties make Brownian motion a natural model of behavior that is continuous but irregular at every time scale.

7.2.2 Single and Multiple Sample Paths

A plotted Brownian path is one realization of the random process. A second realization follows a different jagged trajectory even though it obeys the same probabilistic law. Plotting many paths reveals that all begin at zero and fan outward as time increases.

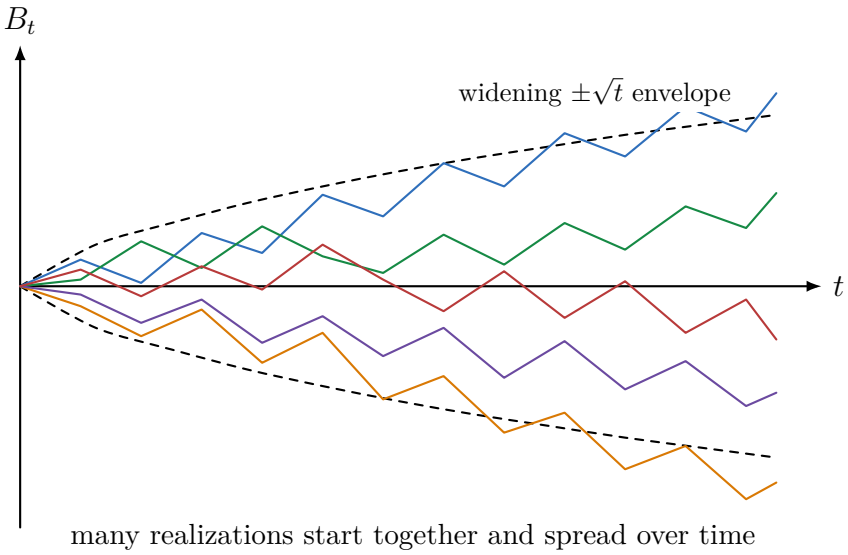


Figure 7.1: Schematic Brownian sample paths and their increasing dispersion.

The diagram highlights four observations:

- every path starts at zero;
- different paths are different realizations of the same process;
- the dispersion grows with the forecast horizon;
- longer-horizon values are consequently more uncertain.

7.2.3 Distribution at a Fixed Time

At time 1,

$$B_1 \sim \mathcal{N}(0, 1).$$

More generally,

$$\boxed{B_t \sim \mathcal{N}(0, t).}$$

Thus the variance grows linearly with time and the standard deviation grows like $\sqrt{t}$, explaining the fan shape in Figure 7.1.

For any $t_0 < t_1$, the gap between two observations satisfies

$$\boxed{B_{t_1} - B_{t_0} \sim \mathcal{N}(0, t_1 - t_0).}$$

The increment has mean zero and variance equal to the elapsed time.

7.3 Why Standard Brownian Motion Is Not a Stock-Price Model

Standard Brownian motion cannot be used directly as a realistic share-price model for two fundamental reasons:

1. it is centered at zero,
$$\mathbb{E}[B_t] = 0,$$
whereas stock prices may exhibit a persistent positive or negative trend;
2. it can take negative values, while a limited-liability stock price cannot be negative.

The first problem motivates Brownian motion with drift. The second motivates exponentiation, producing geometric Brownian motion.

7.4 Brownian Motion with Drift

7.4.1 Constructing the Model

Begin with the deterministic line $y = \mu t$, where μ may be positive, negative, or zero, and superimpose a scaled Brownian fluctuation:

$$\boxed{Y_t = \mu t + \sigma B_t.}$$

Equivalently, an arithmetic Brownian motion can be written in differential form as

$$\boxed{\mathrm{d}X_t = a\ \mathrm{d}t + b\ \mathrm{d}W_t.}$$

Here a is the drift rate and b is the volatility coefficient. A larger $|b|$ produces greater dispersion, while the sign of a determines the direction of the average trend.

7.4.2 Distribution

Since $B_t \sim \mathcal{N}(0, t)$,
$$X_t = X_0 + at + bW_t$$
has distribution
$$\boxed{X_t \sim \mathcal{N}(X_0 + at, b^2 t).}$$
Thus
$$\mathbb{E}[X_t] = X_0 + at, \qquad \mathrm{Var}(X_t) = b^2 t.$$

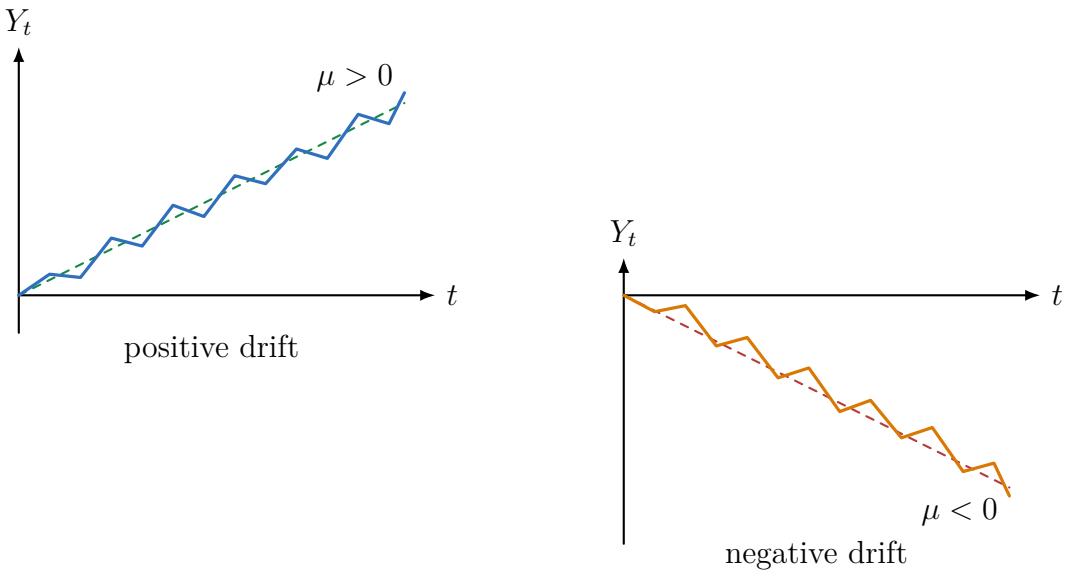


Figure 7.2: Brownian fluctuations around positive and negative linear trends.

Brownian motion with drift solves the missing-trend problem, but it still has Gaussian support on the whole real line. It therefore assigns positive probability to negative prices.

7.5 Geometric Brownian Motion

7.5.1 Exponentiation and Positivity

The exponential function is positive for every real input. If

$$Y_t = \alpha t + \sigma B_t,$$

then

$$\exp(Y_t) > 0 \qquad \text{almost surely.}$$

Since $B_0 = 0$,

$$\exp(\alpha \cdot 0 + \sigma B_0) = 1.$$

Multiplying by an arbitrary initial price $S_0 > 0$ gives

$$\boxed{S_t = S_0 \exp(\alpha t + \sigma B_t),}$$

which starts at S_0 and remains strictly positive.

7.5.2 The GBM Stochastic Differential Equation

Applying Itô's lemma to the exponential representation gives

$$\mathrm{d}S_t = \left(\alpha + \frac{1}{2}\sigma^2\right) S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}B_t.$$

Quantitative finance usually denotes the SDE drift by μ. Setting

$$\alpha = \mu - \frac{1}{2}\sigma^2$$

produces the standard GBM formulas

$$\boxed{\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}B_t,}$$

and

$$\boxed{S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right) t + \sigma B_t\right].}$$

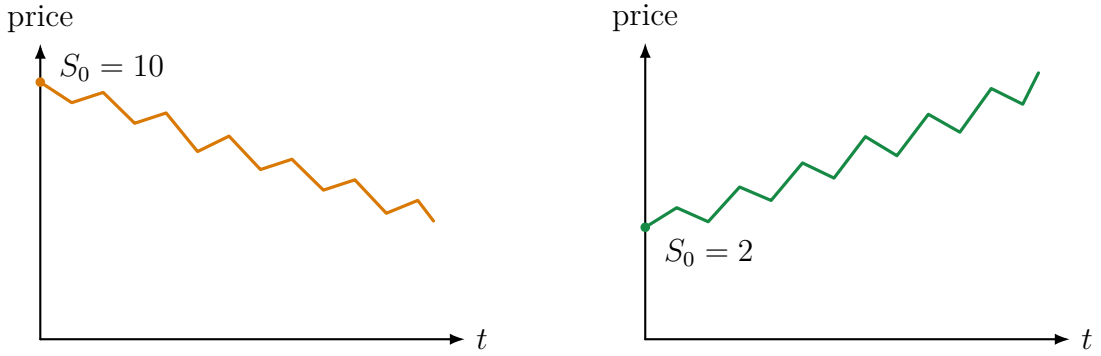
The parameters have direct interpretations:

- S_t is the stock price;
- μ is the instantaneous expected return or drift;
- σ is the volatility;
- $\mathrm{d}B_t$ is the Brownian innovation.

Both the drift and diffusion terms are proportional to the current price. GBM therefore describes multiplicative, percentage-based moves rather than fixed-dollar moves.

7.5.3 Illustrative Paths

Different realizations may rise or fall, but all remain positive.



a decreasing realization an increasing realization

Figure 7.3: Two possible geometric Brownian-motion price paths. Both stay strictly above zero.

GBM combines randomness, guaranteed positivity, upward or downward average movement, an arbitrary starting price, and explicit control of fluctuation through σ.

7.5.4 Distribution of Log Returns

For $0 \le s < t$,

$$\log\left(\frac{S_t}{S_s}\right) = \left(\mu - \frac{1}{2}\sigma^2\right)(t-s) + \sigma(B_t - B_s).$$

Since $B_t - B_s \sim \mathcal{N}(0, t-s)$,

$$\boxed{\log\left(\frac{S_t}{S_s}\right) \sim \mathcal{N}\left(\left(\mu - \frac{1}{2}\sigma^2\right)(t-s), \sigma^2(t-s)\right).}$$

Normal log returns are a central implication of GBM and provide a direct route for estimating its parameters from observed prices.

7.6 Estimating GBM Parameters from Data

Suppose observed prices $S_0, S_1, \ldots, S_n$ are modeled by

$$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma B_t\right].$$

Define the log-drift

$$g = \mu - \frac{1}{2}\sigma^2.$$

The parameters can be estimated from consecutive log-price differences.

7.6.1 Taking Logarithms

Taking natural logarithms converts multiplication into addition:

$$\log S_t = \log S_0 + gt + \sigma B_t.$$

At the previous unit time,

$$\log S_{t-1} = \log S_0 + g(t-1) + \sigma B_{t-1}.$$

7.6.2 Eliminating the Initial Price

Subtracting eliminates the constant $\log S_0$:

$$\begin{aligned}\log S_t - \log S_{t-1} &= \Big[\log S_0 - \log S_0\Big] + g\Big[t-(t-1)\Big] \\ &\quad + \sigma(B_t - B_{t-1}) \\ &= g + \sigma(B_t - B_{t-1}).\end{aligned}$$

Thus the observable log difference

$$R_t = \log S_t - \log S_{t-1}$$

is expressed using the two unknown parameters and a Brownian increment with a known distribution.

7.6.3 Mean and Variance

Because

$$B_t - B_{t-1} \sim \mathcal{N}(0,1),$$

linearity of expectation gives

$$\boxed{\mathbb{E}[R_t] = g.}$$

Adding a constant does not change variance, while multiplying by σ multiplies variance by σ^2. Hence

$$\boxed{\mathrm{Var}(R_t) = \sigma^2.}$$

The complete distributional result is

$$\boxed{R_t = \log S_t - \log S_{t-1} \sim \mathcal{N}(g, \sigma^2).}$$

For observations separated by a general interval Δt,

$$\log\Big(\frac{S_{t+\Delta t}}{S_t}\Big) \sim \mathcal{N}(g\Delta t, \sigma^2\Delta t).$$

7.6.4 Sample Estimators

Let

$$r_i = \log S_i - \log S_{i-1}, \qquad i = 1, \dots, n,$$

and let

$$\overline{r} = \frac{1}{n}\sum_{i=1}^{n} r_i.$$

For unit time steps, estimate

$$\boxed{\widehat{g} = \overline{r}}$$

and

$$\boxed{\widehat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(r_i - \overline{r})^2, \qquad \widehat{\sigma} = \sqrt{\widehat{\sigma}^2}.}$$

The corresponding SDE drift estimate is

$$\boxed{\widehat{\mu} = \widehat{g} + \frac{1}{2}\widehat{\sigma}^2.}$$

Both parameters of the GBM model can therefore be estimated using only successive log differences of an observed price series.

7.6.5 Marks & Spencer Workflow

A practical fitting procedure is:

1. obtain the original Marks & Spencer share-price series;
2. calculate the natural logarithm of every price;
3. form consecutive differences

$$r_t = \log S_t - \log S_{t-1};$$

4. plot the differences through time;
5. inspect a histogram or normal diagnostic plot;
6. calculate the sample mean and variance.

If the plotted log differences fluctuate around a roughly constant center and their histogram is approximately bell-shaped, the normal-increment assumption of GBM may be a reasonable first approximation.

7.6.6 Illustrative Parameter Estimates

The course example reports a unit-period sample mean and variance

$$\widehat{g} = 0.0004, \qquad \widehat{\sigma}^2 = 0.0004, \qquad \widehat{\sigma} \approx 0.02.$$

In notation that uses μ for the drift inside the exponential, the first estimate is written $\widehat{\mu} = 0.0004$. Under the standard SDE convention $\mathrm{d}S_t = \mu S_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}B_t$, the corresponding expected return estimate is

$$\widehat{\mu}_{\mathrm{SDE}} = 0.0004 + \frac{1}{2}(0.0004) = 0.0006$$

per observation interval.

7.6.7 Validation by Simulation

Substitute the fitted parameters into the GBM formula and generate several independent sample paths from the observed initial price. Compare the simulated paths with the real series in terms of:

- overall direction and scale;
- day-to-day variability;
- distribution of log returns;
- drawdowns and extreme movements.

Some simulated paths may resemble the observed price history closely, while others differ substantially. This is expected: a stochastic model describes a distribution of plausible paths rather than reproducing one historical path exactly.

7.6.8 Limitations of GBM

Stock prices combine systematic structure with random movement, but GBM is not a perfect model. In particular, it can fail to represent:

- sudden jumps and structural shocks;
- volatility clustering and changing volatility;

- heavy-tailed or skewed return distributions;
- regime changes and time-varying parameters.

Large exceptional movements, such as those associated with the COVID-19 market shock, motivate extensions including jump-diffusion, regime-switching, and stochastic-volatility models.

> Despite these limitations, GBM remains a useful benchmark for share-price behavior and is foundational to derivative pricing through the Black–Scholes–Merton framework.

7.7 Financial Interpretation of Log Differences

The same quantity used to estimate GBM has a direct interpretation in finance.

7.7.1 From a Difference to a Ratio

The logarithm identity gives

$$\boxed{\log S_t - \log S_{t-1} = \log\left(\frac{S_t}{S_{t-1}}\right).}$$

The ratio S_t/S_{t-1} is today's price divided by the previous price. Writing it as one plus a change gives

$$\log\left(\frac{S_t}{S_{t-1}}\right) = \log\left[1 + \left(\frac{S_t}{S_{t-1}} - 1\right)\right].$$

7.7.2 Taylor Approximation

For $-1 < x < 1$,

$$\log(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots.$$

This follows from the Taylor expansion

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2}x^2 + \cdots$$

applied to $f(x) = \log(1+x)$.

Keeping only the first-order term gives

$$\log(1+x) \approx x.$$

With

$$x = \frac{S_t}{S_{t-1}} - 1,$$

we obtain

$$\boxed{\log\left(\frac{S_t}{S_{t-1}}\right) \approx \frac{S_t - S_{t-1}}{S_{t-1}}.}$$

7.7.3 Simple and Log Returns

If a share is bought at $t-1$ for S_{t-1} and sold at t for S_t, the monetary profit is

$$S_t - S_{t-1}.$$

Relative to the initial investment, the **simple return** is

$$\boxed{R_t = \frac{S_t - S_{t-1}}{S_{t-1}} = \frac{S_t}{S_{t-1}} - 1.}$$

The **log return** is

$$\boxed{r_t = \log\left(\frac{S_t}{S_{t-1}}\right) = \log(1 + R_t).}$$

For small price changes,

$$r_t \approx R_t.$$

The log differences arising naturally from geometric Brownian motion are therefore first-order approximations to ordinary relative returns. This connects the stochastic model directly to the language of trading and investment performance.

7.8 Summary of the Stock-Price Modelling Journey

1. Standard Brownian motion models continuous random fluctuations but has zero mean and can take negative values.
2. Adding a drift term incorporates an upward or downward trend, but arithmetic Brownian motion still permits negative prices.
3. Exponentiating a Brownian motion with drift produces geometric Brownian motion, which remains strictly positive and models percentage-based price changes.
4. The standard GBM SDE is

   $$\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}B_t,$$

   with explicit solution

   $$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right) t + \sigma B_t\right].$$

5. Consecutive log returns are Gaussian under the model. Their sample mean and variance estimate the log drift and volatility.
6. Simulation provides a visual and statistical validation tool, but similarity between one simulated path and one historical path does not prove that the model is correct.
7. GBM omits jumps, volatility clustering, heavy tails, and regime changes, yet remains a valuable benchmark and the foundation of the Black–Scholes–Merton pricing framework.

Chapter 8

Itô's Lemma and Girsanov's Theorem

8.1 Motivation

Suppose that a state variable, such as a stock price, evolves randomly over time. In quantitative finance one is usually interested not only in the state variable itself, but also in a function of it: for example, the value of an option as a function of the underlying stock price and time. Ordinary calculus provides a chain rule for smooth paths; Brownian paths require a modified rule.

> **Core idea.** Itô's lemma is the stochastic chain rule. Besides the usual time and first-derivative terms, it contains a second-derivative, or curvature, correction caused by the nonzero quadratic variation of Brownian motion.

Itô's lemma is one of the principal tools for transforming and solving stochastic differential equations (SDEs), and it is essential in the derivation of the Black–Scholes equation.

8.2 From Taylor's Formula to Stochastic Calculus

8.2.1 The Ordinary Taylor Expansion

For a twice continuously differentiable function f, Taylor's formula gives

$$f(x+\Delta x) - f(x) = f'(x)\Delta x + \frac{1}{2}f''(x)(\Delta x)^2 + o\Big((\Delta x)^2\Big)\,.$$

In differential notation this is often written formally as

$$\mathrm{d}f = f'(x)\ \mathrm{d}x + \frac{1}{2}f''(x)(\,\mathrm{d}x)^2 + \cdots\,.$$

For a smooth deterministic path, the squared increment is of smaller order than the first-order increment and disappears in the limiting chain rule.

8.2.2 An Ordinary-Calculus Example

For $f(x) = x^2$, one has $f'(x) = 2x$ and $f''(x) = 2$, so

$$(x+\Delta x)^2 - x^2 = 2x\Delta x + (\Delta x)^2.$$

If $x = 1$ and $\Delta x = 0.01$, the linear approximation gives 0.02, whereas the exact change is 0.0201. The quadratic term is negligible at first order for a smooth path.

8.2.3 Why Brownian Motion Is Different

Over a short interval of length Δt, a Brownian increment satisfies

$$\Delta W_t \sim \mathcal{N}(0, \Delta t).$$

Thus ΔW_t has typical magnitude $\sqrt{\Delta t}$, and its square has magnitude Δt. Consequently, the second-order Taylor term survives in the limit.

This fact is made precise by the quadratic variation of Brownian motion. If $0 = t_0 < t_1 < \cdots < t_n = T$ is a partition whose mesh tends to zero, then

$$\sum_{k=1}^{n} \left(W_{t_k} - W_{t_{k-1}}\right)^2 \longrightarrow T$$

in probability. The corresponding Itô multiplication rules are

$$\boxed{(\mathrm{d}t)^2 = 0, \qquad \mathrm{d}t\ \mathrm{d}W_t = 0, \qquad (\mathrm{d}W_t)^2 = \mathrm{d}t.}$$

The last identity is shorthand for quadratic variation, not an ordinary algebraic identity between infinitesimals.

8.3 Itô's Lemma

Theorem 8.3.1 (Itô's lemma)**.** *Let X satisfy*

$$\mathrm{d}X_t = a(t, X_t)\ \mathrm{d}t + b(t, X_t)\ \mathrm{d}W_t,$$

and let $f \in C^{1,2}$, meaning that f is once continuously differentiable in time and twice continuously differentiable in its spatial variable. Then

$$\boxed{\begin{aligned}\mathrm{d}f(t, X_t) = &\left[\frac{\partial f}{\partial t} + a(t, X_t)\frac{\partial f}{\partial x} + \frac{1}{2}b^2(t, X_t)\frac{\partial^2 f}{\partial x^2}\right]\ \mathrm{d}t \\ &+ b(t, X_t)\frac{\partial f}{\partial x}\ \mathrm{d}W_t.\end{aligned}}$$

All derivatives are evaluated at (t, X_t).

Derivation from the Itô rules. Apply the second-order Taylor expansion:

$$\mathrm{d}f = f_t\ \mathrm{d}t + f_x\ \mathrm{d}X_t + \frac{1}{2}f_{xx}(\mathrm{d}X_t)^2.$$

Since

$$(\mathrm{d}X_t)^2 = \left(a\ \mathrm{d}t + b\ \mathrm{d}W_t\right)^2 = b^2\ \mathrm{d}t,$$

substitution and collection of the $\mathrm{d}t$ and $\mathrm{d}W_t$ terms gives the stated formula. □

The formula separates the change in f into three effects: direct time dependence, exposure to the drift and random movement of X_t, and the additional curvature correction $\frac{1}{2}b^2 f_{xx}\ \mathrm{d}t$.

8.3.1 Example: Squared Brownian Motion

Take $X_t = W_t$ and $f(x) = x^2$. Here $a = 0$, $b = 1$, $f_x = 2x$, and $f_{xx} = 2$. Itô's lemma gives

$$\boxed{\mathrm{d}(W_t^2) = 2W_t\ \mathrm{d}W_t + \mathrm{d}t.}$$

Equivalently,

$$W_t^2 = t + 2\int_0^t W_s\ \mathrm{d}W_s.$$

The extra $\mathrm{d}t$ term has no counterpart in the ordinary chain rule.

	Ordinary calculus	**Stochastic calculus**
Function	$f(x) = x^2$	$f(W_t) = W_t^2$
Result	$\mathrm{d}(x^2) = 2x\ \mathrm{d}x$	$\mathrm{d}(W_t^2) = 2W_t\ \mathrm{d}W_t + \mathrm{d}t$
Second-order term	Vanishes in the first-order limit	Survives through $(\mathrm{d}W_t)^2 = \mathrm{d}t$

8.4 Solving Geometric Brownian Motion

Suppose that a stock price follows

$$\mathrm{d}S_t = \mu S_t\ \mathrm{d}t + \sigma S_t\ \mathrm{d}W_t, \qquad S_0 > 0.$$

Apply Itô's lemma to $f(S) = \log S$. Since

$$f_S = \frac{1}{S}, \qquad f_{SS} = -\frac{1}{S^2},$$

we obtain

$$\begin{aligned}\mathrm{d}\log S_t &= \frac{1}{S_t}\ \mathrm{d}S_t + \frac{1}{2}\left(-\frac{1}{S_t^2}\right)\sigma^2 S_t^2\ \mathrm{d}t \\ &= \left(\mu - \frac{1}{2}\sigma^2\right)\mathrm{d}t + \sigma\ \mathrm{d}W_t.\end{aligned}$$

Integrating from 0 to t yields

$$\log\frac{S_t}{S_0} = \left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t,$$

and therefore

$$\boxed{S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right].}$$

The term $-\sigma^2/2$ is precisely the Itô curvature correction. It would be missed by applying the ordinary chain rule.

For observations separated by a time step Δt, the log-return is

$$\log\frac{S_{t+\Delta t}}{S_t} \sim \mathcal{N}\left(\left(\mu - \frac{1}{2}\sigma^2\right)\Delta t, \sigma^2\Delta t\right).$$

Thus the sample mean of log-returns estimates the log-growth rate $\mu - \sigma^2/2$, while the sample variance estimates $\sigma^2\Delta t$.

8.5 Girsanov's Theorem

8.5.1 Changing the Probability Measure

Girsanov's theorem explains how the drift of a diffusion changes when the underlying probability measure is changed. The sample paths themselves are not altered; instead, their probabilities are reweighted. In finance this provides the mathematical bridge from the real-world measure $\mathbb{P}$ to a risk-neutral measure $\mathbb{Q}$.

Theorem 8.5.1 (Girsanov's theorem, Brownian version)**.** *Let $W^{\mathbb{P}}$ be Brownian motion under $\mathbb{P}$, and let θ_t be an adapted process satisfying Novikov's condition*

$$\mathbb{E}^{\mathbb{P}}\left[\exp\left(\frac{1}{2}\int_0^T \theta_s^2 \, \mathrm{d}s\right)\right] < \infty.$$

Define

$$Z_T = \exp\left(-\int_0^T \theta_s \, \mathrm{d}W_s^{\mathbb{P}} - \frac{1}{2}\int_0^T \theta_s^2 \, \mathrm{d}s\right), \qquad \left.\frac{\mathrm{d}\mathbb{Q}}{\mathrm{d}\mathbb{P}}\right|_{\mathcal{F}_T} = Z_T.$$

Then

$$\boxed{W_t^{\mathbb{Q}} = W_t^{\mathbb{P}} + \int_0^t \theta_s \, \mathrm{d}s}$$

is Brownian motion under $\mathbb{Q}$ on $[0, T]$.

8.5.2 Risk-Neutral Stock Dynamics

Under the real-world measure, suppose

$$\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}W_t^{\mathbb{P}},$$

and let r be the continuously compounded risk-free rate. Choose the market price of risk

$$\theta = \frac{\mu - r}{\sigma}.$$

Because $\mathrm{d}W_t^{\mathbb{P}} = \mathrm{d}W_t^{\mathbb{Q}} - \theta \, \mathrm{d}t$, substitution gives

$$\boxed{\mathrm{d}S_t = rS_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}W_t^{\mathbb{Q}}.}$$

The unknown real-world drift μ has disappeared. Under standard integrability assumptions, the discounted stock price $e^{-rt}S_t$ is a martingale under $\mathbb{Q}$. Consequently, a claim with payoff H at time T has the risk-neutral value

$$\boxed{V_t = \mathbb{E}^{\mathbb{Q}}\left[e^{-r(T-t)}H \mid \mathcal{F}_t\right].}$$

Remark 8.5.2. Girsanov's theorem changes drift, not instantaneous volatility. It does not by itself prove that a risk-neutral measure exists or is unique; those facts depend on the absence of arbitrage and market completeness.

8.6 How the Ideas Connect

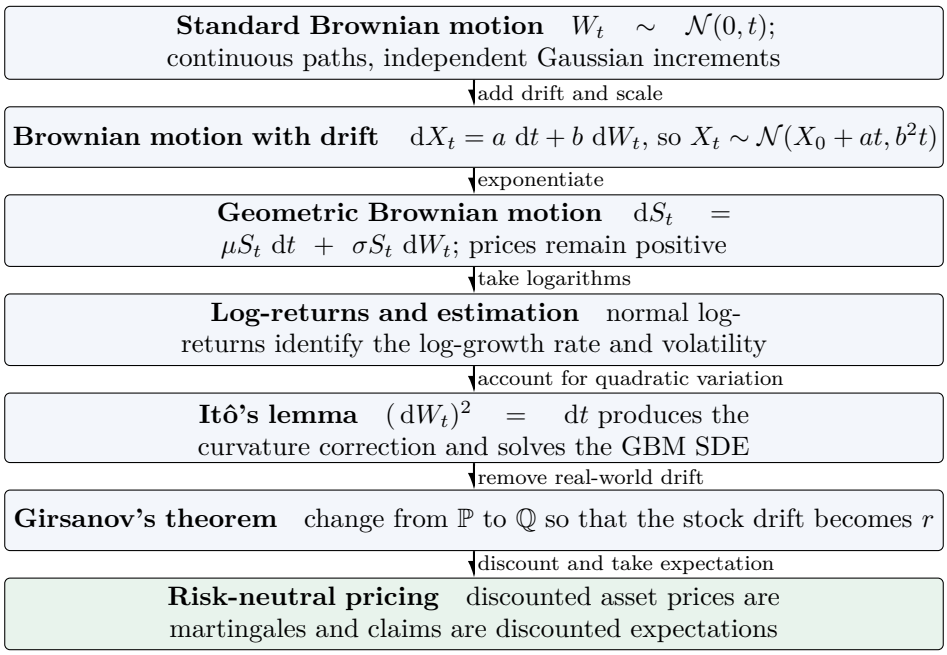


Figure 8.1: From Brownian motion to risk-neutral derivative pricing.

8.7 Key Formula Reference

Concept	Formula
Standard Brownian motion	$W_t \sim \mathcal{N}(0,t)$, with $W_0 = 0$
Brownian increment	$W_t - W_s \sim \mathcal{N}(0, t-s)$ for $t > s$
Brownian motion with drift	$\mathrm{d}X_t = a\,\mathrm{d}t + b\,\mathrm{d}W_t$
GBM SDE	$\mathrm{d}S_t = \mu S_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}W_t$
GBM solution	$S_t = S_0 \exp[(\mu - \frac{1}{2}\sigma^2)t + \sigma W_t]$
Log-return distribution	$\log(S_{t+\Delta t}/S_t) \sim \mathcal{N}((\mu - \frac{1}{2}\sigma^2)\Delta t, \sigma^2\Delta t)$
Simple and log returns	$\log(S_t/S_{t-1}) \approx (S_t - S_{t-1})/S_{t-1}$ for small returns
Itô multiplication rules	$(\mathrm{d}t)^2 = 0,\ \mathrm{d}t\,\mathrm{d}W_t = 0,\ (\mathrm{d}W_t)^2 = \mathrm{d}t$
Itô example	$\mathrm{d}(W_t^2) = 2W_t\,\mathrm{d}W_t + \mathrm{d}t$
Itô's lemma	$\mathrm{d}f = (f_t + af_x + \frac{1}{2}b^2 f_{xx})\,\mathrm{d}t + bf_x\,\mathrm{d}W_t$
Girsanov density	$Z_T = \exp[-\int_0^T \theta_s\,\mathrm{d}W_s^{\mathbb{P}} - \frac{1}{2}\int_0^T \theta_s^2\,\mathrm{d}s]$
Risk-neutral dynamics	$\mathrm{d}S_t = rS_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}W_t^{\mathbb{Q}}$
Risk-neutral value	$V_t = \mathbb{E}^{\mathbb{Q}}[e^{-r(T-t)}H \mid \mathcal{F}_t]$

8.8 Core Takeaways

1. Standard Brownian motion is a continuous pure-noise process whose variance grows linearly with time.

2. Adding drift produces a trend but does not prevent negative values; exponentiating the process produces the positive geometric Brownian motion model used for stock prices.
3. Brownian increments are of order $\sqrt{\mathrm{d}t}$, so their squares are of order $\mathrm{d}t$. This is why second-order terms survive in stochastic calculus.
4. Itô's lemma adds the curvature term $\frac{1}{2}b^2 f_{xx}\,\mathrm{d}t$ to the ordinary chain rule and produces the $-\sigma^2/2$ correction in the explicit GBM solution.
5. Girsanov's theorem reweights path probabilities so that drift can be changed consistently while volatility remains unchanged.
6. Under the risk-neutral measure, discounted tradable asset prices are martingales, allowing derivatives to be valued as discounted conditional expectations of their future payoffs.

Chapter 9

Factors Affecting Option Prices

9.1 Option Price Fundamentals

9.1.1 The Option Price Equation

When you purchase an option — whether a Call or a Put — you pay a **premium**. This premium is not a single monolithic number; it is composed of two distinct components:

$$\boxed{\text{Option Price (Premium)} = \text{Intrinsic Value} + \text{Extrinsic Value}}$$

Understanding what drives each component is the foundation of options pricing.

9.1.2 Intrinsic Value

Intrinsic value is the "built-in" or "real" worth of an option — the amount you would gain if you exercised the option *right now*, at the current market price. It is sometimes called the option's **immediate exercise value**.

Think of it as the option's objective, factual value based purely on where the underlying price stands relative to the strike price at this exact moment.

For a Call Option (the right to *buy* the underlying at strike price K):

$$\text{Intrinsic Value (Call)} = \max(S - K,\ 0)$$

- If $S > K$ (In-The-Money, ITM): Intrinsic Value $= S - K$ (you could buy at K and immediately sell at S for a profit)
- If $S \leq K$ (At-The-Money or Out-of-The-Money): Intrinsic Value $= 0$

For a Put Option (the right to *sell* the underlying at strike price K):

$$\text{Intrinsic Value (Put)} = \max(K - S,\ 0)$$

- If $K > S$ (ITM): Intrinsic Value $= K - S$ (you could buy at S and immediately sell at K for a profit)
- If $K \leq S$ (ATM or OTM): Intrinsic Value $= 0$

> **Critical Rule:** Only **in-the-money (ITM)** options have intrinsic value. At-the-money (ATM) and out-of-the-money (OTM) options have zero intrinsic value.

Example:

- Call option with strike K = Rs. 90, current stock price S = Rs. 100
- Intrinsic Value = 100 − 90 = **Rs. 10**
- If the option trades at Rs. 12.50, then: Extrinsic Value = 12.50 − 10 = **Rs. 2.50**

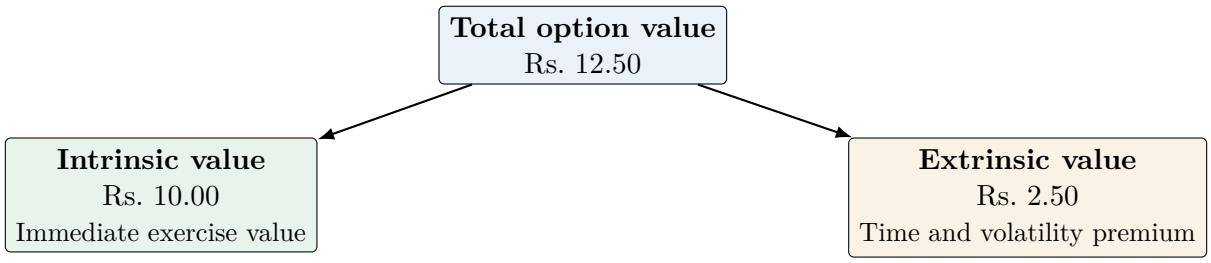


Figure 9.1: Decomposition of the option premium.

9.1.3 Extrinsic Value (Time Value)

Extrinsic value — also called **time value** — is the additional premium beyond intrinsic value that option buyers pay for *optionality*: the probability that the option could become more valuable (or move from OTM to ITM) before expiration.

An option carries extrinsic value **as long as there is time remaining** before it expires. At expiration, extrinsic value = 0, and the option price collapses to its intrinsic value (or zero if OTM).

Extrinsic value is affected by two things:

1. **Time until expiration** — More time remaining = more opportunity for favorable price movement = higher extrinsic value
2. **Implied Volatility** — Greater expected price swings = higher chance of the option ending ITM = higher extrinsic value

Key insight: Even a deeply OTM option (zero intrinsic value) can have significant extrinsic value if there is a lot of time remaining and/or volatility is high. The option represents a *possibility* of profit, and that possibility has a price.

9.2 The Six Key Factors Affecting Option Prices

9.2.1 Underlying Price (Related Greek: Delta)

The price of the underlying asset is the most direct driver of option value. Its effect is **directionally opposite** for calls vs. puts.

Scenario	Call Option	Put Option
Underlying price ↑	Value ↑	Value ↓
Underlying price ↓	Value ↓	Value ↑

Why? A call is a bet that the price will rise. When it rises, the call moves deeper ITM (or from OTM toward ITM), increasing its value. The opposite holds for puts.

Example: Mr. XYZ buys a **Call** and Mr. ABC buys a **Put** on "Bright Industries," both with strike K = Rs. 50:

Scenario 1 — Stock rises to Rs. 70:

- Mr. XYZ (Call):
  (
  checkmark
) Gains (call is now Rs. 20 ITM; intrinsic value = Rs. 20)
- Mr. ABC (Put):
  (
  times
) Loses (put is OTM; intrinsic value = 0)

Scenario 2 — Stock falls to Rs. 40:

- Mr. XYZ (Call):
  (
  times
) Loses (call is OTM; intrinsic value = 0)
- Mr. ABC (Put):
  (
  checkmark
) Gains (put is Rs. 10 ITM; intrinsic value = Rs. 10)

The **Delta (Δ)** Greek measures this sensitivity:

- For calls: $\Delta \in (0, 1)$ — a delta of 0.6 means the call's price rises by Rs. 0.60 for every Rs. 1 rise in the stock
- For puts: $\Delta \in (-1, 0)$

9.2.2 Strike Price

The strike price K is fixed at the time the options contract is written. It determines whether and how much intrinsic value the option has, and directly affects the option's premium.

For Call Options:

- **Lower strike K** → Option is more likely to be ITM (or deeper ITM) → Higher intrinsic value → **Higher premium**
- **Higher strike K** → Option further OTM → Lower probability of profit → **Lower premium**

For Put Options:

- **Higher strike K** → Option more likely to be ITM → **Higher premium**
- **Lower strike K** → Option further OTM → **Lower premium**

Example: A call option with K = Rs. 90, S = Rs. 100:

- Intrinsic Value = Rs. 100 − Rs. 90 = Rs. 10
- Total premium = Rs. 12.50 = Rs. 10 (intrinsic) + Rs. 2.50 (extrinsic)

As the option moves deeper ITM, the premium grows (intrinsic value component increases). As it goes deeper OTM, only extrinsic value remains, which eventually approaches zero.

9.2.3 Time Until Expiration (Related Greek: Theta)

Time is one of the most powerful forces in options pricing. Options are **wasting assets** — their extrinsic (time) value erodes continuously as expiration approaches. This erosion is called **time decay** or **theta decay**.

Key relationships:

- **Longer time to expiration** → Higher probability that OTM option turns ITM → Higher extrinsic value → **Higher premium** (for both calls and puts)
- **As expiration approaches** → Extrinsic value decays, and the rate of decay *accelerates* near expiration
- **At expiration** → Extrinsic value = 0; option worth only intrinsic value (or zero if OTM)

Intuition — The Insurance Analogy: Options function like insurance policies. A 1-year car insurance policy costs more than a 1-month policy because there is a greater window of time during which an accident can occur. As the coverage period shrinks, the premium drops. Similarly, the longer an option has until expiration, the more time there is for a favorable price move — hence higher premium.

Analogy — Perishable Goods: Think of options like perishable goods in a grocery store. As the expiration date approaches, the store discounts the items to move them before they become worthless. An option approaching expiry is similarly discounted toward its intrinsic value.

ATM options are most sensitive to time decay: Because ATM options have zero intrinsic value, their entire premium is extrinsic. This premium erodes fully to zero by expiration.

Theta (Θ) measures the rate of time decay — specifically, the dollar amount an option loses per day from time passing alone (all else equal). Theta is always negative for long options.

9.2.4 Volatility of the Underlying (Related Greek: Vega)

Volatility is arguably the **most critical and nuanced** factor in options pricing — and the only one that is not directly observable in the market. It must be estimated or implied from other market data.

Definition: Volatility (σ) is the **standard deviation of the logarithmic returns** of the underlying asset over a given period. Intuitively, it measures how much the asset's price swings around its average path.

Why high volatility benefits option buyers: Options provide asymmetric payoffs — you profit on favorable moves, and your maximum loss is limited to the premium paid. High volatility means large moves in *either* direction are possible:

- For call buyers: large upward moves → large profits
- For put buyers: large downward moves → large profits
- The maximum loss is always just the premium

Therefore:

$$\begin{aligned}\text{Higher volatility} &\Rightarrow \text{higher probability of large moves} \\ &\Rightarrow \text{higher call and put premiums.}\end{aligned}$$

The insurance analogy again: Options are insurance contracts. A neighborhood with high crime rates has expensive home insurance — because the probability of a claim

is higher. Similarly, a highly volatile stock has expensive options, because the chance of large price moves (claims on the option) is higher.

Practical observation: Before major events (RBI monetary policy announcements, quarterly earnings, Budget Day), volatility spikes and option premiums become expensive. After the event, the uncertainty resolves and volatility — along with option premiums — collapses. This post-event collapse is called a **volatility crush**.

Vega (ν) measures how much the option price changes for a 1% change in implied volatility. A Vega of 0.10 means the option price rises by Rs. 0.10 for every 1% increase in volatility.

Vega and time to expiration: Vega decreases as expiration approaches. With more time remaining, the implied volatility has a longer window to affect the option outcome — so the premium is more sensitive to volatility changes. An option with 1 day to expiry has near-zero Vega because there is almost no time for volatility to matter.

How volatility is measured:

1. **Magnitude of price changes** — how large were the daily moves?
2. **Speed of changes** — how quickly did those changes occur?
3. **Historical price change patterns** — do large moves tend to follow large moves? (volatility clustering)
4. **Expected future events** — are there upcoming announcements that could cause large moves?

9.2.5 Dividends

Dividends have a small but real effect on option prices, transmitted through their impact on the underlying stock price.

Mechanism: When a stock goes ex-dividend, its price drops approximately by the dividend amount on the ex-dividend date (shareholders receive the dividend in cash, so the stock's total value is lower by that amount).

Effect on options:

Event	Effect on Stock Price	Effect on Call	Effect on Put
Dividend paid	Stock price ↓	Call value ↓	Put value ↑

Why: Since the stock price drops (all else equal), calls — which benefit from price increases — become less valuable. Puts — which benefit from price decreases — become more valuable.

This effect is small for low-dividend stocks but can be significant for high-dividend-paying stocks or for long-dated options. The Black-Scholes formula (to be discussed) assumes no dividends in its basic form; adjusted versions (e.g., Black-Scholes-Merton) incorporate a continuous dividend yield.

9.2.6 Interest Rates (Related Greek: Rho)

Interest rates affect option prices through the cost of carry — the opportunity cost of holding or financing a position.

Effect on Call Options:

- Higher interest rates → Higher call option values

The reasoning: When you buy a call instead of the underlying stock directly, you pay a smaller premium (vs. buying the stock outright) and retain the remaining cash. At higher interest rates, that retained cash earns more interest — making the call option relatively more attractive than owning the stock. This increased attractiveness is reflected in higher call premiums.

Effect on Put Options:

- Higher interest rates → Lower put option values (opposite direction)

Rho (ρ) measures option price sensitivity to interest rate changes.

Practical note: For short-dated equity options, Rho is typically the smallest of all the Greeks — its effect is modest. It becomes more important for:

- Long-dated options (LEAPs)
- Interest rate derivatives (caps, floors, swaptions), where interest rates *are* the underlying

9.3 Volatility — A Deep Dive

Since volatility is the most complex and influential factor, it warrants a much deeper treatment.

9.3.1 Volatility as a Measure of Risk

In quantitative finance, understanding any asset requires tracking two quantities:

- **Mean (μ):** The expected return — the direction you expect the asset to move on average
- **Variance (σ^2):** The expected risk — how much the price fluctuates around the mean

$$\text{Mean} = \text{Expected Return} \quad \text{(What we hope to gain)}$$

$$\text{Variance} = \text{Expected Risk} \quad \text{(How uncertain that gain is)}$$

Both must be predicted before investing or trading. An asset may have a high expected return but also high variance — meaning the actual return could be very different from the expectation. Volatility quantifies this uncertainty.

The **annualized volatility formula** for scaling across time periods:

$$\sigma_{\text{annualized}} = \sigma_{\text{daily}} \times \sqrt{T}$$

where T is the number of trading periods in a year (typically 252 trading days). For a 10-day horizon:

$$\sigma_{\text{10-day}} = \sigma_{\text{daily}} \times \sqrt{10}$$

> **Convention:** In Indian markets (NSE), 252 trading days per year is the standard assumption. Global practice sometimes uses 250 or 260. The choice affects the annualized estimate, so consistency is important.

9.3.2 Historical Volatility vs. Implied Volatility

These two concepts appear throughout options trading and quant finance literature. Understanding the distinction is essential.

Historical Volatility (HV) — Also called Realized Volatility:

- Computed from *past* price data over a chosen lookback window (e.g., 30 days, 60 days, 1 year)
- Formula: Standard deviation of daily log returns $\ln(S_t/S_{t-1})$ over the window
- Answers: "How volatile *was* this asset over the past period?"
- Assumption: Past volatility patterns will continue — which may or may not hold

Implied Volatility (IV):

- *Forward-looking* — it is the market's collective expectation of future volatility
- Not directly observable; it is *back-calculated* ("implied") from observed option prices using a pricing model
- **The mechanism:** In any option pricing model (e.g., Black-Scholes), volatility is the *only* input that cannot be observed directly. All other inputs (stock price, strike, time, interest rate) are known. If you take an observed market option price and solve for the volatility that makes the model price equal to the market price — *that* volatility is the implied volatility.
- Answers: "What does the market believe volatility will be going forward?"

Relationship:

- Historical volatility *informs* implied volatility — traders compare the two to assess whether options are "cheap" or "expensive"
- When IV > HV: market expects more volatility than history suggests — options are "expensive" (e.g., before earnings)
- When IV < HV: market expects less volatility — options are "cheap" (e.g., after a volatility crush)

9.3.3 Computing Historical Volatility: Step-by-Step

Step 1: Compute log returns

$$r_t = \ln\left(\frac{S_t}{S_{t-1}}\right)$$

Why log returns? They are symmetric, approximately normal, handle compounding correctly, and are bounded even for large price changes.

Step 2: Compute the mean return

$$\bar{r} = \frac{1}{n}\sum_{t=1}^{n} r_t$$

Step 3: Compute the variance

$$s^2 = \frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2$$

Step 4: Compute the standard deviation (volatility)

$$\sigma = \sqrt{s^2}$$

Step 5: Annualize

Data Frequency	Annualization
Daily	$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{N}$ where N = 252 (or 250/260)
Weekly	$\sigma_{\text{annual}} = \sigma_{\text{weekly}} \times \sqrt{52}$
Monthly	$\sigma_{\text{annual}} = \sigma_{\text{monthly}} \times \sqrt{12}$

> **Why $\sqrt{N}$?** Under the assumption that log returns are i.i.d. (independent and identically distributed), variance scales linearly with time: $\sigma_T^2 = T \cdot \sigma_1^2$. Therefore, the standard deviation scales with $\sqrt{T}$.

The assumption behind this: Daily return changes are uncorrelated with each other (no serial correlation). If serial correlation exists, this scaling is incorrect and more sophisticated methods are needed.

Two ways to measure rate change:

1. **Absolute rate change:** The raw change in basis points (e.g., rate moves from 4.50% to 4.56% → absolute change = 6 bps). Reported in basis points.
2. **Percentage rate change:** $\ln(r_t/r_{t-1}) \times 100$. Analogous to log returns for stock prices. More appropriate for comparing volatility across different rate levels (a 1 bp move when rates are 1% is proportionally much larger than a 1 bp move when rates are 10%).

Numerical Example (from textbook: 1-month LIBOR, weekly data 2004–2005):

Measure	Weekly Std Dev	Annualized ($\times \sqrt{52}$)
Absolute rate change	2.62 basis points	**18.89 basis points**
Percentage rate change	1.33%	**9.62%**

Trading days in a year — practical impact:

Days Assumed	$\sqrt{\text{Days}}$
250	15.81
252	15.87
260	16.12

For daily data, this choice of trading day convention can produce meaningfully different annualized volatility estimates — consistency in methodology matters.

9.4 A History of Volatility Models

Understanding where modern volatility models came from reveals *why* they are structured the way they are and what problems each was designed to solve.

9.4.1 Pre-Black-Scholes: Bachelier's Model (1900)

Louis Bachelier presented "The Theory of Speculation" in 1900 — the first known option pricing model, developed for pricing options on French government bonds.

Key characteristics:

- Uses **Arithmetic Brownian Motion (ABM):** The price change itself (not the percentage change) is normally distributed
- $dS_t = \mu, dt + \sigma, dZ_t$ where dZ_t is a Wiener process
- Similar structure to Black-Scholes in probability framework and payoff analysis
- Designed for options on *spreads* (on futures contracts), not on absolute price levels

Why ABM is problematic for stocks: Under arithmetic BM, cumulative changes can theoretically bring the price below zero — which is possible for a spread but impossible for a stock price. This limited the Bachelier model's applicability to standard equity options.

Bachelier's surprising revival: The model never became obsolete. In **April 2020**, WTI crude oil futures collapsed to **-$40 per barrel** (negative prices!) due to storage constraints during COVID-19. CME Group switched its pricing model to Bachelier to accommodate negative strikes — demonstrating the model's relevance 120 years after its creation. Similarly, in negative interest rate environments (EUR, JPY after the GFC), the Bachelier model is used because standard log-normal models cannot handle negative rates.

9.4.2 Black-Scholes Model (1973) — The Cornerstone

Fischer Black and Myron Scholes published their landmark paper in 1973. Myron Scholes received the Nobel Prize in Economics in 1997 (Fischer Black passed away in 1995, before the prize was awarded posthumously to Scholes; Robert Merton also received the Prize for related work).

The key advancement over Bachelier: B&S uses **Geometric Brownian Motion (GBM)** instead of ABM:

$$dS_t = \mu S_t \, dt + \sigma S_t \, dZ_t$$

Or equivalently, the continuous GBM simulation formula:

$$dS_t = S_t \mu, dt + S_t \sigma \varepsilon \sqrt{\Delta t}, \quad \varepsilon \sim N(0,1)$$

Why GBM solves the negativity problem: Because the volatility term is σS_t (proportional to price), when $S_t \to 0$, the stochastic term $\to 0$ as well. The price can approach zero but never cross it. This means **securities cannot change sign** (prices stay positive), which is economically correct for stocks.

GBM log-return distribution: GBM implies that *log-returns* $\ln(S_t/S_0)$ are normally distributed:

$$\ln\left(\frac{S_t}{S_0}\right) \sim N\left(\left(\mu - \frac{\sigma^2}{2}\right) t, \ \sigma^2 t\right)$$

And thus the price itself follows a **log-normal distribution**.

The Monte Carlo illustration: Running 10,000 GBM simulations from a starting price shows how the possible future price paths fan out over time — with some paths rising dramatically and others falling, but none going negative. The spread of paths (the width of the "cone") represents volatility.

Black-Scholes Assumptions

#	Assumption	Status
1	European options — exercised only at expiration	Reasonable
2	Risk-free asset exists (zero-coupon bond) for borrowing/investing	Reasonable
3	No dividends or accrued interest during option's life	Approximate
4	No transaction costs for trading or delta hedging	Approximate
5	Underlying follows **continuous GBM**; returns are normally distributed; **no jumps**	**Empirically violated**
6	Volatility is **constant** across all strikes and maturities	**Most problematic**

Assumptions 5 and 6 are the most critical violations:

Assumption 5 (No jumps): Real markets experience discontinuous jumps — stock prices gap significantly on earnings surprises, macro events, geopolitical shocks. GBM assumes continuous paths, meaning large moves must happen gradually. In reality, they don't.

Assumption 6 (Constant volatility): This is the **single most important limitation**, as it is definitively wrong empirically. Volatility changes over time and varies across strike prices and maturities. This violation gave rise to the entire field of volatility modeling.

9.4.3 The Volatility Surface: Theory vs. Reality

Under Black-Scholes, since volatility is constant, the implied volatility extracted from *any* option — regardless of strike or expiration — should be the same value σ. The "volatility surface" (implied volatility plotted against strike and maturity) should be a **flat plane**.

B&S Theoretical Volatility Surface:

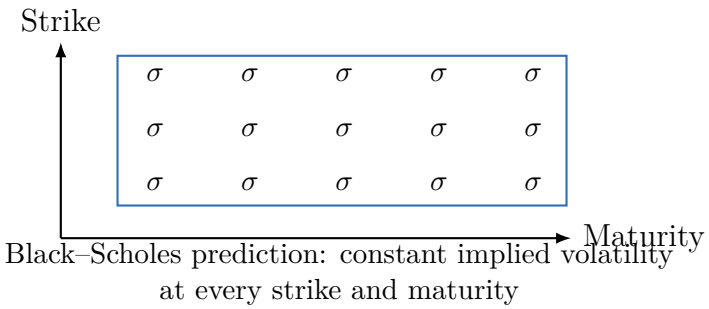


Figure 9.2: A flat theoretical Black–Scholes volatility surface.

Reality (S&P500 Volatility Surface): The actual surface is curved and tilted. It shows:

- **Volatility Skew/Smile:** IV varies across strikes
  - For equity indices: IV is much higher for lower strikes (downside puts) than for higher strikes (upside calls) — this is the **put skew** or "volatility skew"

  - For FX markets: roughly symmetric smile (higher IV for both very low and very high strikes)

- **Term Structure:** IV varies across maturities

Why the skew exists for equities:

1. **Leverage Effect (Fischer Black, 1976):** When stock prices fall, companies become more levered (debt/equity rises), increasing default probability → higher volatility. This creates a *negative correlation* between stock returns and volatility.
2. **Demand for downside protection:** Portfolio managers buy puts to hedge their equity portfolios → elevated demand for OTM puts → higher IV for low strikes
3. **Crash risk:** Investors fear sudden market crashes (fat left tail) and pay premium for protection

9.4.4 The Stochastic Volatility Era (Post-1973)

The critical insight: If volatility is not constant, model it as *random*. Just as stock prices follow a random process, so does volatility.

Key milestone — 1976: Latane and Rendelman published "Standard Deviations of Stock Price Ratios Implied in Option Prices" — the first paper to suggest that **implied volatility should be derived from traded options** (called I.S.D. — Implied Standard Deviation) rather than assumed as a constant input.

Black Monday, October 19, 1987: This event definitively demonstrated that the flat volatility surface assumption fails catastrophically. After Black Monday, out-of-the-money puts became persistently more expensive (higher IV) than at-the-money options — the volatility skew became a permanent feature of equity markets.

Stochastic Volatility: Core Idea A stochastic volatility model assumes that **volatility itself follows a random process**, typically correlated with the returns of the underlying asset. Mathematically, the system becomes:

$$dS_t = \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^1 \quad \text{(asset price)}$$
$$dv_t = \kappa(\theta - v_t)\,dt + \xi\sqrt{v_t}\,dW_t^2 \quad \text{(variance process)}$$

Where:

- $v_t = \sigma_t^2$ is the instantaneous variance (also random!)
- κ = mean reversion speed of variance
- θ = long-run mean variance
- ξ = volatility of volatility (vol-of-vol)
- dW_t^1 and dW_t^2 are correlated: $\text{corr}(dW^1, dW^2) = \rho_{SV}$ (typically negative for equities)

Heston (1993): The foundational paper. Offered a *closed-form* (exact) solution for option pricing under stochastic volatility. This is the model from which virtually all modern stochastic volatility research descends.

9.4.5 Asset Class Specific Volatility Models

Each asset class has adopted models calibrated to its unique behavior:

Interest Rates

- **Standard models:** Vasicek (1977), Hull-White (1990) — see Sections 8 and 9
- **Modern standard: SABR model** (Hagan, Kumar, Lesniewski, Woodward — 2002)
  - A two-factor stochastic model where both the forward rate and its volatility follow random processes
  - Parameters:
    - α **(alpha):** Controls the overall level of volatility (the scale)
    - β **(beta):** Controls the variance and shape of the volatility distribution — specifically the "vol-of-vol." When $\beta = 0$: normal/Bachelier dynamics; when $\beta = 1$: log-normal dynamics
    - ρ **(rho):** Correlation between the spot rate and its volatility — controls the slope of the volatility skew
  - **Special case — Negative Rates:** Many interest rates now trade negative (EUR, CHF, JPY). Since Black-Scholes assumes log-normal dynamics (and prices/rates can't be negative), practitioners use the **Bachelier model** for negative-rate instruments. Example: After WTI crude oil hit -$40 in April 2020, CME immediately switched its pricing model to Bachelier.

Foreign Exchange (FX)

- FX options are largely OTC (over-the-counter), enabling more customized structures
- **Standard: Vanna-Volga (VV) pricing** (Malz 1997; Lipton & McGhee 2002; Wystup 2003)
  - Designed to handle exotic options: barrier options, digital payouts (one/no-touch, European digital)
  - **Core idea:** Quantify the hedging cost of second-order volatility derivatives (Greeks):
    - **Vanna** $= \frac{\partial \text{Vega}}{\partial S} = \frac{\partial^2 V}{\partial S \partial \sigma}$ — sensitivity of Vega to spot price changes
    - **Volga** $= \frac{\partial \text{Vega}}{\partial \sigma} = \frac{\partial^2 V}{\partial \sigma^2}$ — sensitivity of Vega to volatility changes (vol-of-vol)
  - The model assumes quoted option strategies reflect the implied risk-neutral probability:
    - **Risk Reversal / Collar** $\rightarrow$ captures Vanna
    - **Butterfly spread** $\rightarrow$ captures Volga

Equities and Commodities

- **The Leverage Effect (Fischer Black, 1976):** Stocks can be thought of as call options on the underlying company assets (Merton, 1974). When stock prices fall, the company's debt/equity ratio rises $\rightarrow$ company becomes more leveraged $\rightarrow$ default probability increases $\rightarrow$ this creates a **negative correlation between stock returns and volatility**.
  Practical implication: Markets fall fast and hard (high vol when prices drop), but rise slowly (lower vol on the way up). This asymmetry is the foundation of the volatility skew in equity markets.
- **CEV Model (Constant Elasticity of Variance):**

- Originally developed by John Cox (1975)
- Extended by Linetsky and Mendoza (2009) to handle jumps, discontinuities, and a wide range of asset types including credit spreads
- Volatility scales as a power of the asset price: $\sigma(S) = \sigma_0 \cdot S^{\gamma-1}$
- Captures the leverage effect through the parameter $\gamma < 1$ (when $\gamma = 0$: Bachelier/normal; when $\gamma = 1$: Black-Scholes log-normal)

- **Local Volatility Models (Derman & Kani 1994; Dupire 1994):**
  - Instead of parameterizing volatility dynamics, *extract* volatility directly from the observed option market
  - **Process:** Given traded options at different strikes and maturities, calculate implied volatility for each → interpolate to create a smooth continuous "local vol surface" $\sigma_{\text{local}}(S, t)$
  - The local volatility is a *deterministic* function of the current asset price and time (not stochastic)
  - **Advantage:** Perfectly calibrated to current market option prices by construction
  - **Key limitation:** Local vol models are notoriously poor at capturing the *dynamics* of the vol surface — they tend to predict a flat smile in the future even when the current surface is skewed
  - More liquid option chain → more accurate local vol surface (better interpolation quality)

9.4.6 Rough Volatility — The Frontier

As volatility modeling matured, researchers began questioning whether GBM's fundamental assumptions hold for volatility itself.

GBM's key property — The Markov property: GBM assumes the process has **no memory**: each increment is independent of all prior increments. Formally, this means volatility increments have *zero autocorrelation*. The process grows proportional to $\sqrt{T}$ (the square root of time):

$$dS_t = S_t\mu\, dt + S_t\sigma\varepsilon\sqrt{\Delta t}$$

The empirical question: Do volatility increments actually have zero autocorrelation?

The answer is NO. Empirical evidence shows that volatility increments are *negatively correlated* — large jumps in volatility tend to be followed by reversals (rough, jagged behavior). This property cannot be captured by standard GBM.

Fractional Brownian Motion (fBM)

To model memory in the process, we use **fractional Brownian motion**, developed by Mandelbrot and Van Ness (1968). Unlike standard BM, fBM allows increments to be correlated.

The Hurst Exponent H ∈ (0, 1) characterizes the degree of autocorrelation:

Value	Increments	Behavior	Process Type
$H = 0.5$	Uncorrelated	Standard Brownian Motion (GBM)	Memoryless
$H < 0.5$	Negatively correlated	Rough, jagged time series; **mean-reverting**	Persistent anti-persistence
$H > 0.5$	Positively correlated	Smooth time series; **trending**	Persistent

For equity volatility, empirical studies find $H \approx 0.1$ to 0.2 — very rough, strongly mean-reverting behavior.

Fractality: An important property of fBM is self-similarity across scales. Zooming in or out on the time series, the shape looks approximately the same (like snowflakes, coastlines, or cauliflower). In volatility analysis: intraday volatility patterns look similar to daily patterns, which look similar to weekly patterns.

Gatheral, Jaisson, and Rosenbaum (2014): This paper brought rough volatility to mainstream finance. Key findings:

- Equity index volatility has $H \approx 0.13$ on long horizons
- This means volatility exhibits **strong mean reversion in normal markets**
- During periods of financial stress, H rises significantly toward $0.5 \rightarrow$ can use H as a **regime change indicator**

Applications of Rough Volatility:

1. **Volatility Regime Detection:** Track the Hurst index over time. H is typically low (rough, mean-reverting) in calm markets and rises during crises (Lehman bankruptcy, Flash Crash, Greek debt crisis, Brexit vote all appear as spikes in H time series). Use H as an early warning system.
2. **Volatility Arbitrage (Glasserman & He, 2018):** Strategy: Long positions in stocks with *rough* volatility (high mean reversion) + short positions in stocks with *smooth* volatility (trending vol) $\rightarrow$ earned excess risk-adjusted returns.
3. **S&P500/VIX Joint Smile Calibration (Julien Guyon, 2020):** The most significant achievement in rough volatility research. No previous model could simultaneously calibrate the volatility smiles of both S&P500 and VIX. Guyon demonstrated that rough volatility assumptions successfully solve this long-standing problem.

Why rough volatility models are computationally intensive: Standard models (B&S, Heston, SABR) are Markovian — they have analytical or semi-analytical solutions. Rough volatility models are **non-Markovian** (they have memory), so no closed-form solution exists. **Monte Carlo simulation** is required, which is computationally expensive — especially for practitioners needing real-time prices.

Market microstructure basis: The roughness of volatility is believed to stem from order flow dynamics:

- Buy orders generate more buy orders; sell orders generate more sell orders (autocorrelated order flow)
- Sell orders have disproportionately large impact on prices relative to buy orders (liquidity asymmetry / leverage effect)

- Most market activity is between professional trading firms (not end-users), amplifying these dynamics
- The proliferation of HFT (high-frequency trading) makes the phenomenon more pronounced

9.5 Volatility Modeling Techniques

9.5.1 Historical Approaches (Backward-Looking)

These methods estimate future volatility from past price data, assuming historical patterns repeat.

Simple Historical Volatility:

- Select a lookback window (e.g., 30 days, 60 days, 252 days)
- Compute daily log returns, then their standard deviation
- Use this as the volatility estimate going forward
- **Advantage:** Simple; implementable in Excel
- **Disadvantage:** Treats all past observations equally; slow to adapt to regime changes

Simple Moving Average (SMA):

- Compute the standard deviation over a rolling window of fixed length
- As time passes, add the newest observation and drop the oldest
- Captures the evolution of volatility over time better than a single fixed estimate
- **Disadvantage:** Still treats all observations within the window equally; sharp cutoff at the window boundary

Exponential Moving Average (EMA):

- Gives exponentially declining weight to older observations — more recent data matters more

$$\sigma^2_{t,\text{EMA}} = (1-\lambda)\cdot r_t^2 + \lambda \cdot \sigma^2_{t-1,\text{EMA}}$$

  where $\lambda \in (0,1)$ is the **decay factor** (e.g., $\lambda = 0.94$ for daily data — the RiskMetrics standard)
- More responsive to recent volatility spikes or drops
- Still backward-looking

Common limitation of all historical methods: The assumption that historical patterns will repeat is strong and often violated. These methods tend to **overfit historical data** — they work well in-sample but can badly mislead out-of-sample if the volatility regime changes. Always perform rigorous out-of-sample testing.

When to use: Historical methods are appropriate when computational resources are limited, or when a quick estimate is needed. Suitable for non-research contexts.

9.5.2 Geometric Brownian Motion (GBM) as a Volatility Framework

GBM, introduced earlier, provides the mathematical foundation for option pricing and volatility modeling. It is most commonly used in **asset pricing** and **option pricing** (as opposed to ARCH/GARCH which are more common in **econometric forecasting**).

The GBM SDE:

$$dS = \mu S, dt + \sigma S, dz$$

Components:

- $\mu S, dt$ = Deterministic drift term (predictable growth)
- $\sigma S, dz$ = Stochastic component (random fluctuations), where $dz \sim N(0, dt)$

Key properties:

- Returns dS/S are normally distributed: $dS/S \sim N(\mu, dt,\ \sigma^2, dt)$
- Price S itself follows a **log-normal distribution**
- Volatility σ is a *parameter* — constant in the basic GBM framework

Simulation (Monte Carlo): For option pricing when closed-form solutions are unavailable:

$$S_{t+\Delta t} = S_t \cdot \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)\Delta t + \sigma\varepsilon\sqrt{\Delta t}\right], \quad \varepsilon \sim N(0,1)$$

Use cases:

- Black-Scholes option pricing
- Monte Carlo simulation for complex derivatives
- Asset path generation for risk management
- Regulatory stress testing

9.6 The ARCH Model

9.6.1 Motivation: The Problem with Constant Variance

Standard time series models — AR (Autoregressive), MA (Moving Average), and ARIMA — model the **conditional mean** of a time series. These models assume that the **residuals (error terms) have constant variance** (homoskedasticity).

However, in financial time series, this assumption is routinely violated. Consider daily returns of the S&P 500 or the BP/Dollar exchange rate: periods of large volatility cluster together, followed by periods of calm. **Volatility clusters.**

High-volatility regime: large moves tend to follow large moves,
Low-volatility regime: small moves tend to follow small moves.

When variance changes systematically over time — becoming larger in some periods and smaller in others — we call the series **heteroskedastic** (unequal variance). If this changing variance is itself predictable from past information, we need a model that captures it.

ARCH (AutoRegressive Conditional Heteroskedasticity) was designed precisely for this.

> **Context:** Standard AR/ARIMA models cannot model changing variance. ARIMA models the conditional mean and assumes error terms have constant variance. ARCH instead models the *variance* of those error terms as an autoregressive function of past squared errors.

Real-world applications where ARCH is used: Stock prices, oil prices, bond prices, inflation rates, GDP, unemployment rates, exchange rates — any financial or economic time series where volatility changes over time.

9.6.2 The ARCH Framework

ARCH = AutoRegressive Conditional Heteroskedasticity Introduced by **Robert Engle (1982)** — for which he was awarded the Nobel Prize in Economics in 2003.

Decomposing the name:

- **Autoregressive:** Current variance depends on (is a function of) past values — there is serial dependence
- **Conditional:** The variance is *conditional on past information* — not unconditional. It changes based on what happened in prior periods
- **Heteroskedasticity:** The variance of the series is non-constant — it varies over time

Key insight: ARCH distinguishes between:

- **Unconditional variance:** The overall long-run variance — constant
- **Conditional variance:** The variance at time t given all information up to t-1 — time-varying

When to apply ARCH: ARCH should only be applied to a series that:

1. Has **no remaining trends or seasonal effects** — the conditional mean is already modeled (i.e., residuals look like white noise in the mean)
2. Has **mean of residuals equal to zero** (or close to it)

Standard workflow:

1. Fit an ARIMA (or similar) model to the raw time series to model the conditional mean
2. Examine the residuals: confirm they look like white noise (ACF/PACF of residuals show no serial correlation)
3. **Square the residuals**: if the ACF/PACF of *squared* residuals show significant patterns → evidence of ARCH effects → apply ARCH to model the variance

9.6.3 ARCH(1): The Basic Model

Model specification:

The time series of residuals $\varepsilon(t)$ is written as:

$$\varepsilon(t) = w(t) \cdot \sigma(t)$$

Where:

- $w(t)$ is **white noise**: $w(t) \sim N(0,1)$, i.i.d. — pure random shocks
- $\sigma(t)$ is the **time-varying standard deviation** (conditional volatility)

The conditional variance evolves as:

$$\sigma^2(t) = \alpha_0 + \alpha_1 \cdot \varepsilon^2(t-1)$$

Combining:

$$\boxed{\varepsilon(t) = w(t) \cdot \sqrt{\alpha_0 + \alpha_1 \cdot \varepsilon^2(t-1)}}$$

Constraints (to ensure valid, non-negative variance and stationarity):

- $\alpha_0 > 0$ — base variance must be positive
- $\alpha_1 \geq 0$ — non-negative weight on past shock
- $\alpha_1 < 1$ — stability condition; if $\alpha_1 \geq 1$, the variance grows without bound (explosive)

Intuition — Volatility Clustering:

- If $|\varepsilon(t-1)|$ is large (big shock at t-1) $\rightarrow \sigma^2(t) = \alpha_0 + \alpha_1\varepsilon^2(t-1)$ is large $\rightarrow |\varepsilon(t)|$ tends to be large $\rightarrow$ **high volatility follows high volatility**
- If $|\varepsilon(t-1)|$ is small (quiet period at t-1) $\rightarrow \sigma^2(t)$ is small $\rightarrow$ **calm follows calm**

This is exactly the volatility clustering pattern observed in financial markets.

Why does ARCH model volatility? From the formula:

$$\text{Var}(\varepsilon(t)) = \alpha_0 + \alpha_1 \cdot \text{Var}(\varepsilon(t-1))$$

The variance at time t is a **linear combination of the variance at t-1** — so variance is time-varying and predictable from past variance.

9.6.4 ARCH(2) and ARCH(p): Higher Order Models

ARCH(2):

$$\varepsilon(t) = w(t) \cdot \sqrt{\alpha_0 + \alpha_1\varepsilon^2(t-1) + \alpha_2\varepsilon^2(t-2)}$$

ARCH(p) — General Form:

$$\boxed{\varepsilon_t = w_t \cdot \sqrt{\alpha_0 + \sum_{i=1}^{p} \alpha_i \varepsilon_{t-i}^2}}$$

The conditional variance:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{p} \alpha_i \varepsilon_{t-i}^2$$

Parameters:

- p = order of the model = number of lagged squared residual terms included
- $i = 1, 2, \ldots, p$ indexes the lag periods
- Constraints: $\alpha_0 > 0$, $\alpha_i \geq 0$ for all i, and $\sum_{i=1}^{p} \alpha_i < 1$ for stationarity

Interpretation:

1. If the error was high in period (t-1), the expected error at period (t) is also higher $\rightarrow$ volatility persistence
2. Conversely, a low error at (t-1) implies lower expected error at (t)
3. $\alpha_1 \geq 0$ ensures positive conditional variance at all times
4. $\alpha_1 < 1$ prevents the process from exploding over time

9.6.5 Configuring ARCH: ACF and PACF of Squared Residuals

To determine the appropriate order p for the ARCH model:

1. **Fit a mean model** (e.g., ARIMA) to the raw series; obtain residuals
2. **Square the residuals**: compute ε_t^2
3. **Plot ACF and PACF** of ε_t^2 (the squared residuals)
4. **Interpret:**
   - Significant ACF at lag 1 only → ARCH(1) likely appropriate
   - Significant ACF at lags 1 and 2 → ARCH(2) possibly
   - The pattern is analogous to choosing the AR order in an AR model for the mean
5. The ACF and PACF patterns of squared residuals tell you how many past shocks significantly predict today's variance

   Note: The raw (unsquared) residuals should look like white noise — no pattern. But the *squared* residuals may show significant autocorrelation, revealing the ARCH structure. This is why ARCH is described as modeling variance rather than the mean.

9.6.6 Python Implementation: ARCH(1)

```python
import numpy as np
import pandas as pd
from arch import arch_model

np.random.seed(13)

# ARCH(1) parameters
a0 = 2    # base variance (must be > 0)
a1 = 0.5  # weight on lagged squared error (must be in [0,1) for stability)

# Simulate 1000 observations
y = w = np.random.normal(size=1000)
Y = np.empty_like(y)

for t in range(len(y)):
    Y[t] = w[t] * np.sqrt(a0 + a1 * y[t-1]**2)  # ARCH(1) formula
```

Observation from simulating and plotting:

- The raw ARCH(1) series looks like white noise (ACF of Y $\approx$ 0 at all lags)
- But the *squared* ARCH(1) series shows significant ACF at lag 1 → evidence of ARCH structure
- This confirms: volatility is autocorrelated even when the level series appears uncorrelated

9.7 The GARCH Model

9.7.1 From ARCH to GARCH: Adding Persistence

GARCH = Generalized ARCH, introduced by **Tim Bollerslev (1986)** in the Journal of Econometrics.

The extension: GARCH adds **past variance terms** (σ^2_{t-j}) to the ARCH specification, creating a model analogous to ARMA (vs. ARCH's AR-only structure):

Model (Mean)	Model (Variance)
AR (autoregressive)	ARCH
ARMA (autoregressive + moving average)	GARCH

Why the moving average component helps:

- ARCH alone requires many lags (large p) to capture slowly decaying volatility persistence
- The GARCH moving average (β) term captures this persistence parsimoniously — with just one β term you can capture long-lived volatility clusters
- GARCH permits a wider range of behavior and more realistic persistence in volatility

9.7.2 GARCH(1,1) — The Workhorse

Model specification:

$$\varepsilon(t) = w(t) \cdot \sigma(t)$$

$$\boxed{\sigma^2(t) = \alpha_0 + \alpha_1 \varepsilon^2(t-1) + \beta_1 \sigma^2(t-1)}$$

Where:

- $\alpha_0 > 0$: long-run base variance (intercept)
- $\alpha_1 \geq 0$: ARCH component — weight on the most recent squared shock
- $\beta_1 \geq 0$: GARCH component — weight on the most recent variance estimate
- $\alpha_1 + \beta_1 < 1$: stationarity condition (otherwise, variance becomes non-stationary / explosive)

Interpreting GARCH(1,1):

- α_1 **(ARCH term):** How quickly variance responds to new shocks. Large $\alpha_1 \rightarrow$ current volatility reacts sharply to recent news
- β_1 **(GARCH term):** How persistent the variance is over time. Large $\beta_1 \rightarrow$ today's volatility is highly correlated with yesterday's $\rightarrow$ long memory in volatility
- $\alpha_1 + \beta_1$ **(persistence):** Values close to 1 indicate highly persistent volatility — shocks take a long time to decay
- **Mean reversion speed of variance:** $(1 - \alpha_1 - \beta_1)$ — the smaller this is, the slower variance reverts to its long-run mean

Long-run (unconditional) variance:

$$\bar{\sigma}^2 = \frac{\alpha_0}{1 - \alpha_1 - \beta_1}$$

This is the level around which the conditional variance oscillates.
Limiting cases:

- $\beta_1 = 0$: Reduces to ARCH(1) — no persistence in variance
- $p = q = 0$: $\varepsilon(t)$ is simply white noise — constant variance

9.7.3 GARCH(p,q) — General Form

A time series $\varepsilon(t)$ with $\varepsilon(t) = w(t) \cdot \sigma(t)$ and conditional variance:

$$\boxed{\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2}$$

Constraints: $\alpha_0 > 0$, $\alpha_i \geq 0$ for $i = 1, \ldots, q$, $\beta_j \geq 0$ for $j = 1, \ldots, p$
Interpretive notes:

1. A large β_1 causes $\sigma(t)$ to be highly correlated with $\sigma^2(t-1) \rightarrow$ long-term persistence in conditional standard deviation
2. If $p = 0$: reduces to ARCH(q) — no variance lag terms
3. If $p = q = 0$: $\varepsilon(t)$ is simply white noise
4. The moving average component allows the model to capture both conditional increases and decreases in variance

9.7.4 GARCH Configuration: ACF and PACF

Diagnostic process:

1. Fit appropriate mean model $\rightarrow$ obtain residuals (ensure mean is zero, no serial correlation)
2. Square the residuals: ε_t^2
3. Plot ACF and PACF of ε_t^2
4. Interpret:
   - **ACF cuts off after lag 1, PACF decays slowly** $\rightarrow$ MA structure in variance $\rightarrow$ ARCH-like
   - **Both ACF and PACF show extended autocorrelation** $\rightarrow$ Both AR and MA components needed $\rightarrow$ **GARCH model appropriate**
5. The ARIMA orders for the best-fitting mean model can guide GARCH order selection (e.g., ARIMA(3,0,2) residuals $\rightarrow$ try GARCH(3,2))

9.7.5 Heavy Tails in GARCH

A notable property: even when the innovations $w(t)$ are assumed to be **Gaussian** (normally distributed), GARCH processes produce **heavy tails** in the return distribution. This happens because the time-varying variance creates occasional very large returns (large σ_t at some time steps).

This is a desirable property because:

- Empirical financial returns have heavier tails than a Gaussian distribution (known as "fat tails" or excess kurtosis)
- GARCH naturally accommodates this even with normal innovations
- For even heavier tails, use non-Gaussian innovations (e.g., Student's t-distribution, GED)

9.7.6 Python Implementation: GARCH(1,1)

```
import numpy as np
from arch import arch_model

np.random.seed(1)

# GARCH(1,1) parameters
a0 = 0.2   # base variance
a1 = 0.5   # ARCH component (alpha_1)
b1 = 0.3   # GARCH component (beta_1)
# Note: a1 + b1 = 0.8 < 1 -> stationary

n = 10000
w = np.random.normal(size=n)
eps = np.zeros_like(w)
sigsq = np.zeros_like(w)

for i in range(1, n):
    sigsq[i] = a0 + a1 * (eps[i-1]**2) + b1 * sigsq[i-1]  # GARCH(1,1)
    eps[i] = w[i] * np.sqrt(sigsq[i])
```

Fitting a GARCH model to real data (SPY returns):

```
from arch import arch_model

am = arch_model(eps)      # default: GARCH(1,1) with constant mean
res = am.fit(update_freq=5)
print(res.summary())
```

Practical workflow for real financial data:

1. Get log returns: `lrets = np.log(data / data.shift(1)).dropna()`
2. Find best ARIMA(p,d,q) model by iterating over orders, minimizing AIC
3. Examine ACF/PACF of squared residuals from ARIMA
4. Fit GARCH using the ARIMA orders as a guide: e.g., ARIMA(3,0,2) $\rightarrow$ try GARCH(3,2)

5. Evaluate: residuals should look like white noise; squared residuals should show no remaining autocorrelation

Example output for SPY returns (2012–2015), best model ARIMA(3,0,2) → GARCH(3,2): The fitted GARCH coefficients (omega = α_0, alpha[1] = α_1, beta[1] = β_1) confirm statistically significant volatility clustering in equity returns.

9.7.7 GARCH Family — Extensions

Model	Key Feature
ARCH-M	Return modeled as a function of its own variance (risk-return tradeoff)
TGARCH (Threshold GARCH)	Asymmetric response: negative shocks affect variance more than positive (leverage effect)
EGARCH	Exponential GARCH; no non-negativity constraints; captures asymmetry naturally
GJR-GARCH	Similar to TGARCH; indicator function captures leverage effect

9.7.8 ARCH vs. GARCH vs. GBM: When to Use Which

Method	Best Application	Sophistication Needed
Simple historical vol	Quick estimates, limited computing	Low (Excel)
SMA / EMA	Monitoring volatility regimes	Low
ARCH(p)	Modeling short-memory vol clustering	Moderate (stats software)
GARCH(p,q)	Modeling persistent vol clustering (research)	Moderate–High
GBM / B&S	Option pricing, asset simulation	Moderate (math)
Vasicek / CIR	Interest rate dynamics	Moderate–High (SDEs, regression)
Heston / SABR	Stochastic vol surface calibration	High
Rough Volatility	Frontier vol research, regime detection	Very High (Monte Carlo)

Practical guideline: Historical approaches (SMA/EMA) are used by practitioners without sophisticated computing resources. ARCH/GARCH are used in research-driven financial firms for forecasting. GBM and SDE-based models (Vasicek, CIR, Hull-White) are used in asset pricing, derivatives, and regulatory modeling.

9.8 Interest Rate Modeling

9.8.1 Why Interest Rates Need Their Own Models

Interest rates influence option prices through the Rho Greek — but they are also fundamental underlying assets in the fixed income world. To price:

- Bond portfolios and their risk
- Interest rate derivatives: caps, floors, swaptions, options on bonds

- Mortgage-backed securities
- Credit risk capital (via Vasicek credit model)

. . . you need a model for how interest rates evolve over time. This is an entirely separate challenge from equity modeling because interest rates behave differently from stock prices in several important ways:

- Interest rates are **mean-reverting** (they don't drift up or down indefinitely)
- They tend toward a long-run equilibrium tied to economic fundamentals
- The entire **term structure** (yield curve across maturities) must be consistent

9.8.2 Properties of Interest Rate Processes

Any sound interest rate model must capture three statistical properties:

1. Drift (b) The expected direction of change in interest rates. Captures the predictable component of rate movement — including mean reversion toward equilibrium.

2. Volatility (σ) The magnitude of random fluctuations in interest rates. Captures the uncertainty and randomness in rate movements — analogous to σ in a stock price model.

3. Mean Reversion The tendency for interest rates to return to a long-run equilibrium level $\bar{r}$. This is one of the most economically important properties:

- Very high rates $\rightarrow$ discourage borrowing $\rightarrow$ reduce economic activity $\rightarrow$ policy pressure to reduce rates
- Very low rates $\rightarrow$ stimulate borrowing, potentially causing inflation $\rightarrow$ eventual pressure to raise rates
- Central bank inflation targeting anchors the long-run rate
- Economic business cycles create cyclical rate dynamics

Mean reversion is the feature that distinguishes interest rate models from stock price models. Stock prices have no economic reason to revert to any level — but interest rates do.

9.8.3 Stochastic Differential Equations (SDEs) — The Mathematical Tool

The standard mathematical tool for modeling continuous-time interest rate dynamics is the **Stochastic Differential Equation (SDE)**.

Basic SDE for the short rate:

$$dr = b, dt + \sigma, dz$$

Where:

- dr = infinitesimal change in the short rate
- b = **drift term** (expected direction of rate change)
- σ = **volatility term** (standard deviation of rate changes)
- dz = random **Wiener process** (or Brownian motion) increment: $dz \sim N(0, dt)$

Interpretation: The change in the short rate = a predictable component (b, dt) + a random shock (σ, dz).

Key statistical properties under this SDE:

- Expected change: $E[dr] = b$ (the drift)
- Variance of change: $\text{Var}(dr) = \sigma^2, dt$
- For a fixed interval of length T (with drift = 0, $\sigma = 1$): Var = T, so standard deviation = $\sqrt{T}$
- Rate changes in different time periods are **independent** (Markov property)

Standard Wiener Process: The special case $b = 0$, $\sigma = 1$ gives $dr = dz$ — a **standard Wiener process**. This is the building block of all continuous-time finance models.

The Itô Process (Generalized SDE):

$$dr = b(r, t), dt + \sigma(r, t), dz$$

Here the drift $b(r, t)$ and volatility $\sigma(r, t)$ can both depend on the current rate level r and time t. This is the most general single-factor interest rate model form.

Note: SDEs underlie not just interest rate models but also the Black-Scholes formula (described in Chapter 30 of the textbook) and all derivative pricing.

9.8.4 Specifying the Drift: Mean Reversion

The mean-reversion drift is specified as:

$$b(r, t) = -\alpha(r - \bar{r})$$

Where:

- $\alpha > 0$ = **speed of adjustment** (must be positive for stability)
- $\bar{r}$ = **long-run stable mean** (equilibrium level)

Mechanism:

- If $r > \bar{r}$: $b = -\alpha(r - \bar{r}) < 0 \rightarrow$ drift is negative $\rightarrow$ rate is pulled downward toward $\bar{r}$
- If $r < \bar{r}$: $b = -\alpha(r - \bar{r}) > 0 \rightarrow$ drift is positive $\rightarrow$ rate is pushed upward toward $\bar{r}$
- If $r = \bar{r}$: $b = 0 \rightarrow$ no drift (rate is at equilibrium)

The larger α, the faster the rate reverts to $\bar{r}$.

9.8.5 Specifying the Volatility: The CEV Framework

The dynamics of the volatility term can be specified using the **Constant Elasticity of Variance (CEV)** model:

$$\sigma(r, t) = \sigma \cdot r^{\gamma}$$

The parameter γ determines how volatility scales with the level of interest rates:

γ	**Volatility Specification**	**Model Name**	**Key Property**
0	$\sigma(r,t) = \sigma$	**Vasicek** (Normal model)	Volatility independent of rate level; **negative rates possible**
1	$\sigma(r,t) = \sigma r$	**Dothan** (Proportional vol)	Volatility proportional to rate; no negative rates
½	$\sigma(r,t) = \sigma\sqrt{r}$	**CIR** (Square-root model)	Volatility proportional to $\sqrt{r}$; **no negative rates**

Normal models ($\gamma = 0$): Volatility is constant in *basis points* — independent of the rate level. This corresponds to the Vasicek model. The rate changes are normally distributed. A drawback is that negative rates are theoretically possible (though in the Vasicek model, this probability is small for typical parameters).

Lognormal models ($\gamma = 1$): Volatility is proportional to the rate level — interest-rate changes are proportional to the current level of rates. This corresponds to the Dothan model. Negative rates are impossible (as rate approaches zero, volatility also approaches zero). This is analogous to GBM for stock prices.

The CIR specification ($\gamma = ½$): Intermediate behavior — a square-root relationship between rate level and volatility. Eliminates negative rates while being more tractable than the full lognormal.

Which model to use? — Empirical evidence: Research by Cheyette (examining Treasury yields from 1977 to 1996) and Levin (10-year Treasury and swap rates, 1980–2003) found:

- For rates **below 10%**: volatility is approximately **independent of the rate level** → normal model (Vasicek, Hull-White) is appropriate
- For rates **above 10%** (e.g., the high-rate environment of 1979–1982): positive correlation between vol and rate level → lognormal models may be more appropriate
- **Practical conclusion:** Since rates have remained below 10% in most developed markets since the 1980s, **normal models are the dominant choice in practice**

On negative rates: While normal models admit the theoretical possibility of negative rates, empirical testing shows that the probability of generating negative rates is very small for typical parameter values and initial term structures. Tests of option pricing with normal models (Cheyette, Levin) confirm that negative rates have **minimal impact on pricing accuracy** — so this concern should not be overstated.

9.8.6 One-Factor Interest Rate Models

Definition: A one-factor model assumes that **all interest rates across all maturities** are driven by a **single stochastic factor** — the short-term interest rate (the "short rate" r).

Once the short rate r is specified at time t, the rates for all other maturities are determined through no-arbitrage arguments. The short rate is the *only* random variable — everything else is a deterministic function of it.

The standard one-factor model SDE:

$$dr = b(r,t), dt + \sigma(r,t), dz$$

Why one-factor models in practice:

1. **Mathematical tractability:** One-factor models often yield closed-form (or semi-closed-form) solutions for bond and option prices
2. **Parsimony:** A single factor reduces the number of parameters to estimate
3. **Empirical justification:** The correlation between rate changes across different maturities is very high. A **level shift** in the yield curve (parallel shift up or down) explains the *majority* of yield curve variation. One-factor models capture this dominant movement well.

Limitation: One-factor models cannot capture the *shape* of the yield curve changing (twists, butterflies). They should **not** be used for instruments whose payoff depends on the shape of the term structure (e.g., dual-index floaters, yield curve options, steepener/flattener trades).

Multi-factor models: Two-factor models (short rate + long rate as factors) are richer but significantly harder to implement. In practice, the difficulty of implementing even a two-factor model means one-factor models dominate.

9.8.7 The Full One-Factor Mean-Reverting Model

Combining mean-reverting drift with the CIR square-root volatility:

$$\boxed{dr = -a(r - \bar{r})\,dt + \sigma\sqrt{r}\,dz}$$

This is the **Mean-Reverting Square-Root Model** — also known as the **CIR (Cox-Ingersoll-Ross) Model**. It is an equilibrium model with:

- Mean-reverting drift pushing toward $\bar{r}$
- Volatility proportional to $\sqrt{r}$ → no negative rates

9.8.8 Arbitrage-Free vs. Equilibrium Models

Interest rate models fall into two fundamental categories based on how they are designed:

Equilibrium Models:

- Describe the *dynamics* of interest rates using fundamental economic variables and assumptions
- Parameters are estimated from historical data (e.g., using OLS or MLE on historical yield data)
- Derive closed-form solutions for bond and option prices
- **Key limitation:** They are *not* calibrated to current market prices. The model may produce prices inconsistent with what you can actually buy or sell in the market today — potentially creating arbitrage opportunities.
- Examples: **Vasicek (1977), CIR (1985), Brennan-Schwartz, Longstaff-Schwartz**

Arbitrage-Free Models (No-Arbitrage Models):

- Begin with **observed market prices** of benchmark instruments (e.g., Treasury bonds)
- Assume these observed prices are correct (fairly priced)
- Find the interest rate process that *reproduces* these prices exactly

- Any non-benchmark instrument is then priced using this calibrated process
- **Key property:** By construction, the model matches current market prices → no arbitrage opportunity relative to the benchmarks
- Examples: **Ho-Lee (1986), Hull-White (1990), KWF, Black-Karasinski, BDT, HJM**

Key arbitrage-free models:

Model	Mean Reversion	Volatility	Notes
Ho-Lee (1986)	No	Normal (constant)	First arbitrage-free model; ϕ(t) time-varying drift
Hull-White (1990)	Yes	Normal (constant)	First arbitrage-free + mean-reverting normal model; widely used
KWF	No	Lognormal	Models ln(r); used for bonds with embedded options
Black-Karasinski	Yes	Lognormal	Lognormal generalization of KWF
BDT (Black-Derman-Toy)	Yes (endogenous)	Lognormal	Mean reversion determined by market conditions
HJM (Heath-Jarrow-Morton, 1992)	—	General	Multi-factor, continuous time; models forward rates; many models are special cases

Hull-White Model is particularly important in practice:

- It extends Ho-Lee by adding mean reversion
- It is the first arbitrage-free, mean-reverting, normal model
- It is widely used for MBS (mortgage-backed securities) pricing (validated by Levin for this application)

HJM Framework: Instead of modeling the short rate, HJM models the *entire forward rate curve* simultaneously. It requires only a specification of the volatility structure of forward rates — no utility function assumptions needed. Many other models (Ho-Lee, Hull-White, etc.) are special cases of HJM.

Why arbitrage-free models are preferred in practice:

- Equilibrium models may produce prices inconsistent with what can be observed in the market today → creates apparent arbitrage opportunities
- Arbitrage-free models are calibrated to current market prices → consistent with observable reality
- However, equilibrium models provide better economic insight and are more useful for long-horizon forecasting

9.9 The Vasicek Interest Rate Model

9.9.1 Overview

The **Vasicek model**, developed by **Oldřich Vašíček (1977)**, was one of the first rigorous analytical models for interest rate dynamics. It is:

- A **one-factor model**: driven by the short-term interest rate
- An **equilibrium model**: parameters estimated from historical data
- A **mean-reverting model**: economically grounded in the stability of long-run equilibrium rates
- A **normal model**: volatility is independent of the rate level ($\gamma = 0$ in CEV framework)

It represents the application of the **Ornstein-Uhlenbeck (OU) process** from physics (used to model the velocity of a particle under friction) to interest rates.

9.9.2 The Vasicek SDE

$$\boxed{dr_t = a(b - r_t)\,dt + \sigma\,dW_t}$$

Parameters and their roles:

Symbol	Parameter	Description	Constraint
a	Speed of mean reversion	How quickly the rate reverts to equilibrium	$a > 0$
b	Long-term mean (equilibrium)	The level rates gravitate toward in the long run	—
r_t	Current short rate	The interest rate at time t	—
σ	Volatility	Standard deviation of random interest rate changes	$\sigma > 0$
W_t	Wiener process	The source of randomness (market risk factor)	—

The model has **two components**:

1. **Deterministic drift:** $a(b - r_t), dt$ — pulls the rate toward equilibrium b
2. **Stochastic component:** σ, dW_t — random market shock

9.9.3 Mean Reversion Mechanics

The drift term $a(b - r_t)$ acts like a spring — the further the rate deviates from equilibrium, the stronger the restoring force:

Condition	Drift Sign	Market Behavior
$r_t > b$ (rate above equilibrium)	Negative (< 0)	Rate pulled downward → forms **resistance level**
$r_t < b$ (rate below equilibrium)	Positive (> 0)	Rate pushed upward → forms **support level**
$r_t = b$ (at equilibrium)	Zero	No systematic drift

Condition	Drift Sign	Market Behavior
$dW_t = 0$ (no market shocks)	—	Rate equals long-term mean exactly

Resistance and support:

- When the rate increases above b and then mean-reverts back down to b: this forms a **resistance level** at the high point
- When the rate decreases below b and bounces back to b: this forms a **support level** at the low point

Stability condition: a must be strictly positive ($a > 0$). If $a \leq 0$, the drift would amplify deviations from equilibrium rather than damping them → unstable process.

9.9.4 Discrete Form and OLS Estimation

The continuous SDE is hard to estimate directly. We convert it to a **discrete form** suitable for Ordinary Least Squares (OLS) regression.

Discretizing the Vasicek SDE: Replace the continuous derivative $\frac{dr}{dt}$ with the first difference $\Delta r_t = r_t - r_{t-1}$ and time step Δt:

$$\begin{aligned}\Delta r_t &= a(b - r_{t-1})\Delta t + \sigma\varepsilon_t \\ &= (ab)\Delta t - a \cdot r_{t-1} \cdot \Delta t + \sigma\varepsilon_t\end{aligned}$$

For weekly data ($\Delta t = 1$ week), this becomes a simple linear regression:

$$\boxed{\Delta r_t = \alpha_0 + \alpha_1 \cdot r_{t-1} + \varepsilon_t}$$

Where:

- $\alpha_0 = a \cdot b$ (intercept estimates the product of speed and equilibrium)
- $\alpha_1 = -a$ (slope estimates the *negative* of the speed of mean reversion)
- $\varepsilon_t \sim N(0, \sigma^2)$ i.i.d. (in the Vasicek model)

Extracting the structural parameters from OLS:

$$a = -\hat{\alpha}_1 \quad \text{(speed of reversion)}$$

$$b = \frac{\hat{\alpha}_0}{a} = \frac{\hat{\alpha}_0}{-\hat{\alpha}_1} \quad \text{(long-run equilibrium)}$$

$$\sigma = \hat{\sigma}_\varepsilon \quad \text{(residual standard error from OLS)}$$

Implementation steps:

1. Compute first differences: $\Delta r_t = r_t - r_{t-1}$
2. Create lagged rate column: r_{t-1}
3. Regress Δr_t on constant + r_{t-1} using OLS (LINEST in Excel, or `np.linalg.lstsq` / `statsmodels.OLS` in Python)
4. Test significance: compute t-statistics (coefficient ÷ std error); check p-values
5. Extract a, b, σ

Real-world Example (US Weekly Mortgage Rates, 2010–present, 587 observations):

- Regression confirms: **both coefficients statistically significant at 5%**
- Estimated slope $\hat{\alpha}_1 \approx -0.01$ per week $\rightarrow a \approx 0.01$ **per week**
  - Interpretation: each week the rate closes approximately 1% of its gap from equilibrium
- $b \approx 3.05\%$ (long-run equilibrium mortgage rate)
- **Economic sanity check:** With a 2% long-run US inflation target, a 3.05% nominal mortgage rate implies a real rate of 1%, which is economically reasonable

9.9.5 Forecasting Mean with the Vasicek Model

Given current rate r_t, the expected interest rate T periods in the future:

$$\boxed{E[r_{t+T}] = r_t \cdot e^{-aT} + b \cdot (1 - e^{-aT})}$$

Mathematical intuition:

- **First term** $r_t \cdot e^{-aT}$: The current rate's influence, which decays exponentially at rate a over time
- **Second term** $b \cdot (1 - e^{-aT})$: The long-run equilibrium's contribution, which grows as the current rate's influence fades
- The two terms sum to a weighted average of r_t and b, with weights e^{-aT} and $(1-e^{-aT})$ respectively

Long-run limit: As $T \rightarrow \infty$: $e^{-aT} \rightarrow 0$, so $E[r_{t+T}] \rightarrow b$ — the forecast converges to the long-run equilibrium regardless of where rates are today.

Short-run limit: For small T: the current rate r_t dominates (the $e^{-aT} \approx 1$ for small aT).

Example:

- $r_t = 3.63\%$, $a = 0.01$ (weekly), $b = 3.05\%$, $T = 52$ weeks

$$E[r_{t+52}] = 3.63 \times e^{-0.52} + 3.05 \times (1 - e^{-0.52})$$

$$= 3.63 \times 0.595 + 3.05 \times 0.405 \approx 2.16 + 1.24 \approx 3.37\%$$

- Long-run forecast: as $T \rightarrow \infty$, rate gravitates toward $b = 3.05\%$

9.9.6 Volatility Forecasting with the Vasicek Model

The Vasicek model provides a **closed-form formula** for the variance of the interest rate over a future period T:

$$\text{Var}(r_{t+T}) = \frac{\sigma^2}{2a} \cdot \left(1 - e^{-2aT}\right)$$

Volatility (standard deviation):

$$\text{Vol}(r_{t+T}) = \sigma \cdot \sqrt{\frac{1 - e^{-2aT}}{2a}}$$

Long-run ($T \to \infty$) limits: As $T \to \infty$: $e^{-2aT} \to 0$, so:

$$\text{Var}_\infty = \frac{\sigma^2}{2a} \qquad \text{(long-run variance)}$$

$$\text{Vol}_\infty = \frac{\sigma}{\sqrt{2a}} \qquad \text{(long-run volatility)}$$

Effect of parameters on volatility:

- **Larger σ:** Higher instantaneous randomness → higher variance at all horizons (direct, proportional effect)
- **Smaller a (slower convergence):** The rate can deviate from equilibrium for a longer time before being pulled back → larger long-run variance. Intuition: a weak spring (small a) allows larger swings.
- **Larger a (faster convergence):** Shocks are absorbed quickly → smaller long-run variance. Intuition: a stiff spring (large a) quickly snaps back, limiting excursions.

Summary: $\text{Vol}_\infty = \frac{\sigma}{\sqrt{2a}}$ — long-run volatility is determined by the ratio of instantaneous vol to the square root of mean reversion speed. Higher shock intensity AND slower reversion → higher uncertainty.

Example:

- $\sigma = 0.003$ (weekly vol from regression), $a = 0.01$
- 52-week variance: $\frac{0.003^2}{2\times 0.01}(1 - e^{-2\times 0.01\times 52}) \approx 0.00045 \times 0.648 \approx 0.000292$
- 52-week volatility: $\sqrt{0.000292} \approx 0.017 =$ **1.7% (or about 36 basis points)**
- Long-run volatility: $\frac{0.003}{\sqrt{0.02}} \approx \frac{0.003}{0.141} \approx 0.021 =$ **2.1% (≈ 43.5 bps)**

Interpretation: Over 1 year, the expected mortgage rate is 3.37%, with an average deviation of about 36 basis points from this forecast. In the very long run, rates are expected to hover near 3.05% with a volatility of about 43.5 basis points.

9.9.7 Applications of the Vasicek Model

1. **Interest Rate Forecasting:** Generate mean forecasts and confidence intervals for future rate levels, informing investment decisions in fixed income
2. **Bond Pricing and Risk:** Derive the full term structure of interest rates from the estimated parameters; quantify the volatility and risk of fixed-rate bonds over a holding period
3. **Derivatives Pricing:** Price interest rate options (caps, floors, bond options) using the derived rate distribution
4. **Regulatory Capital — Credit Risk:** Used by regulators (Basel II/III) to calculate the worst-case default rate for loan portfolios and hence the capital requirement (see Section 10)
5. **Risk Management:** For a fixed-rate bond portfolio, use the Vasicek-derived interest rate distribution to quantify risk (Value-at-Risk, stress tests)

9.9.8 Limitations of the Vasicek Model

1. Single-Factor: Only one source of randomness (the market risk factor dW_t). Real interest rates are affected by multiple factors — monetary policy surprises, inflation expectations, credit risk, global capital flows. The model cannot capture the relative movement of different points on the yield curve (twists, butterflies).

2. Negative Interest Rates: Since σ is constant (not proportional to r), the model allows rates to go negative with some probability. In standard rate environments this probability is very small, and empirical evidence suggests this has minimal impact on pricing — but it is a theoretical limitation. The CIR model avoids this.

3. Constant Parameters: a, b, and σ are assumed constant over time. In reality:

- The long-run equilibrium b can shift (e.g., from one monetary policy regime to another)
- Volatility σ can change (e.g., during crises)
- The speed of adjustment a may vary

4. Equilibrium vs. Arbitrage-Free: As an equilibrium model, the Vasicek model is calibrated to historical data, not to current market prices. This means it may produce theoretical bond prices that differ from observable market prices, potentially implying arbitrage opportunities. In practice, arbitrage-free models (Hull-White) are preferred for derivatives pricing.

9.10 The Vasicek Model for Credit Risk

9.10.1 The Problem: Portfolio-Level Default Risk

The Vasicek model has an important application beyond interest rate modeling: **calculating regulatory capital for bank loan portfolios** under Basel II/III's Internal Ratings-Based (IRB) approach.

The central question: A bank has 2,000 corporate loans. Each loan has some individual probability of default. But the *portfolio*'s worst-case outcome is not simply the sum of individual probabilities — because defaults are **correlated**. When the economy turns bad, many borrowers default simultaneously.

Why correlation matters:

- If each loan's default is **independent** of all others: the Law of Large Numbers applies; the portfolio default rate converges to the average PD. No capital needed beyond expected loss provisions.
- If defaults are **positively correlated** (systemic risk): bad economic conditions cause many defaults simultaneously. The portfolio can experience catastrophic losses far exceeding the average PD. **Capital must be held to absorb this.**

The Vasicek framework provides an *analytical* (closed-form) solution for the worst-case portfolio default rate, making it far more tractable than Monte Carlo simulation for regulatory purposes.

9.10.2 The Gaussian Copula and Factor Structure

Fundamental modeling approach: Each borrower's "creditworthiness" can be represented by a latent variable. Even if individual loan loss distributions are not normal,

we can transform them to standard normal distributions. The joint distribution is then modeled using a **Gaussian copula** — a mathematical tool for constructing a joint distribution from individual (possibly non-normal) marginals.

Factor decomposition: Each borrower's transformed variable U_i is decomposed into:

$$\boxed{U_i = \sqrt{\rho} \cdot F + \sqrt{1-\rho} \cdot \varepsilon_i}$$

Where:

- $F \sim N(0,1)$: **Systemic (common) factor** — represents macro-economic conditions (e.g., GDP growth, unemployment, interest rates). Affects ALL firms simultaneously. Think of it as "the state of the economy."
- $\varepsilon_i \sim N(0,1)$: **Idiosyncratic factor** — firm-specific risk unique to borrower i (e.g., product launch success, regulatory approval, management changes). Independent across firms.
- ρ: **Pairwise correlation** between any two borrowers (assumed the same for all pairs in the portfolio — the simplifying assumption)
- F and ε_i are independent standard normals

Interpreting the formula:

- U_i is the sum of two independent normal distributions $\rightarrow U_i \sim N(0,1)$ (Gaussian copula property)
- The correlation between any two borrowers U_i and U_j is exactly ρ (from the common factor F)
- When F drops drastically (severe economic recession), all U_i drop together $\rightarrow$ correlated defaults

Simplifying assumptions of the Vasicek credit model:

1. **All loans have the same pairwise correlation** ρ (homogeneous portfolio assumption)
2. **All loans have the same probability of default PD** (same average PD for all loans)

These assumptions simplify the model from a 2000-dimensional problem to a single-parameter problem, enabling an analytical solution.

9.10.3 Loss Distribution and Worst-Case Default Rate

Portfolio-level default rate: Given the state of the economy (systemic factor F), the default rate across the portfolio is:

$$DR(F) = N\left[\frac{N^{-1}(PD) - \sqrt{\rho} \cdot F}{\sqrt{1-\rho}}\right]$$

Where $N(\cdot)$ is the standard normal CDF and $N^{-1}(\cdot)$ is its inverse.

Intuition: When F is very negative (severe recession), the formula gives a very high portfolio default rate. When F is positive (good economy), the default rate is low.

9.10.4 The Worst-Case Default Rate (WCDR) Formula

For regulatory capital purposes, we want the default rate at an extreme confidence level. **Basel II/III uses 99.9% confidence** — meaning we look at the default rate that would occur in the worst 0.1% of economic scenarios (roughly once every 1,000 years).

In this worst case, the systematic factor F takes its 0.1th percentile value:

$$F_{\text{worst}} = N^{-1}(0.001) = -N^{-1}(0.999) \approx -3.09$$

Substituting this worst-case F:

$$\boxed{WCDR = N\left[\frac{N^{-1}(PD) + \sqrt{\rho} \cdot N^{-1}(0.999)}{\sqrt{1-\rho}}\right]}$$

Inputs to the formula:

Input	Description
PD	Average probability of default of individual loans
ρ	Pairwise correlation between loans (the transformed distributions)
$N^{-1}(0.999) \approx 3.09$	99.9th percentile of standard normal — the extreme economic scenario

Key insight — Effect of correlation ρ:

- If $\rho = 0$ (loans completely uncorrelated): the $\sqrt{\rho}$ terms vanish $\rightarrow$ $WCDR = N[N^{-1}(PD)] = PD$
  - No portfolio amplification! The worst-case portfolio default rate = individual default rate
- As ρ increases toward 1: WCDR increases dramatically — systemic risk dominates

9.10.5 Numerical Example

Inputs: $PD = 1\% = 0.01$, $\rho = 0.5$

Step 1: $N^{-1}(0.01) = -2.326$

Step 2: $\sqrt{0.5} \approx 0.707$, $N^{-1}(0.999) \approx 3.09$, $\sqrt{1-0.5} = \sqrt{0.5} \approx 0.707$

Step 3:

$$WCDR = N\left[\frac{-2.326 + 0.707(3.09)}{0.707}\right]$$
$$= N\left[\frac{-0.142}{0.707}\right] = N(-0.201) \approx 42\%.$$

Interpretation:

- Individual loan default probability: **1%** (on average, 1 in 100 loans defaults)
- Portfolio worst-case default rate (99.9% confidence): **42%** (in the worst 1-in-1,000-year scenario, 42% of loans default simultaneously!)

This 42× amplification is entirely driven by the correlation $\rho = 0.5$.

Quantity	Default rate
Individual probability of default	1%
Portfolio WCDR ($\rho = 0.5$)	42%
Portfolio WCDR ($\rho = 0$)	1%

What this means practically: Even if your loan book looks healthy on average (1% default rate per loan), systemic correlation means the *tail risk* is enormous. This is why banks must hold substantial capital — to survive the tail scenario.

9.10.6 Capital Requirement Calculation

The IRB Capital Formula:

$$\boxed{\text{Capital} = LGD \times EAD \times (WCDR - PD)}$$

Component definitions:

Variable	Full Name	Description	Example
LGD	Loss Given Default	Fraction of loan not recovered if default occurs	$= 1 -$ Recovery Rate. If 75% recovered $\rightarrow$ LGD = 25%
EAD	Exposure at Default	Outstanding loan balance at time of default	Rs. 10 million outstanding $\rightarrow$ EAD = Rs. 10M
$WCDR$	Worst-Case Default Rate	Portfolio default rate at 99.9% confidence	From Vasicek formula above
PD	Probability of Default	Average individual loan PD	1%

Why $(WCDR - PD)$ rather than $WCDR$?
The key distinction is between **expected loss** and **unexpected loss**:

$$\text{Expected Loss (EL)} = LGD \times EAD \times PD,$$

$$\text{Worst-Case Loss (WCL)} = LGD \times EAD \times WCDR,$$

$$\text{Unexpected Loss (UL)} = LGD \times EAD \times (WCDR - PD).$$

- **Expected Loss** is the average loss the bank anticipates. This should be built into the loan's interest rate spread and provisioned for in the P&L. It does not require regulatory capital.
- **Unexpected Loss** is the *excess* loss beyond what was expected in a tail scenario. This is what capital must cover.

Capital = Unexpected Loss = Coverage for the gap between the expected and worst-case scenario.

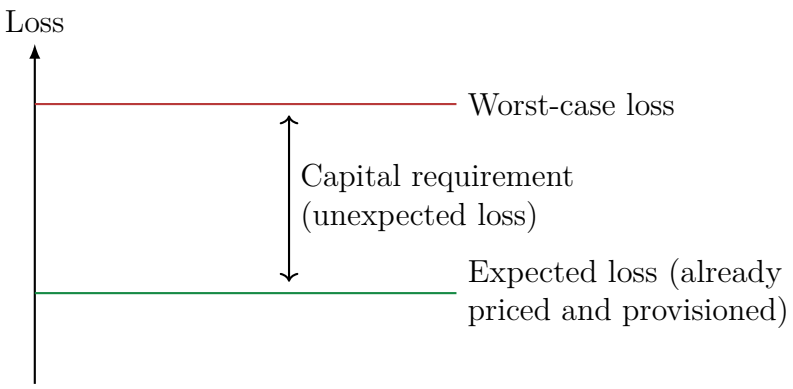


Figure 9.3: Regulatory capital covers the gap between expected and worst-case loss.

Full example:

- $PD = 1\%$, $\rho = 0.5$, $LGD = 25\%$, $EAD =$ Rs. 10 million per loan, 2,000 loans
- $WCDR = 42\%$ (from above)
- Capital per loan $= 0.25 \times 10M \times (0.42 - 0.01) = 0.25 \times 10M \times 0.41 = Rs.\ 1.025M$
- Total capital for portfolio = Rs. 1.025M $\times$ 2,000 = Rs. 2,050M = Rs. 2.05 billion

Contrast with zero-correlation case ($\rho = 0$):

- $WCDR = PD = 1\% \rightarrow$ Capital $= 0.25 \times 10M \times 0 = Rs.\ 0$!
- With zero correlation, diversification is perfect — no tail risk beyond the expected loss

This demonstrates why the assumption of correlation ρ is the most important parameter in credit portfolio risk. Higher correlation $\rightarrow$ dramatically higher capital requirements.

9.10.7 Summary of Vasicek Credit Model Assumptions

Assumption	Description	Effect if Violated
Gaussian Copula	Joint default distribution modeled as multivariate normal	Model may understate tail risk if true dependence is more extreme (e.g., tail dependence)
Same ρ for all pairs	All loans have identical pairwise correlation	Unrealistic; simplifies model but may misprice portfolios with heterogeneous correlations
Same PD for all loans	All loans have the same probability of default	Approximate; real portfolios have diverse credit qualities
Large/infinite portfolio	Assumes many loans (LLN applies to idiosyncratic risk)	For small portfolios, idiosyncratic risk matters and the formula overstates capital

9.11 Summary and Quick Reference

9.11.1 The Six Factors at a Glance

Factor	Call Option	Put Option	Greek
Underlying price ↑	↑	↓	Delta (Δ)
Strike price (lower K)	↑	↓	—
Time to expiration ↑	↑	↑	Theta (Θ) — negative
Volatility ↑	↑	↑	Vega (ν)
Dividend	↓	↑	—
Interest rate ↑	↑	↓	Rho (ρ)

9.11.2 Volatility Modeling Landscape

Model	Class	Use Case	Key Assumption
Simple historical vol	Backward-looking	Quick estimates	History repeats
SMA / EMA	Backward-looking	Regime monitoring	History repeats (with recency weighting for EMA)
ARCH(p)	Statistical (conditional variance)	Short-memory vol forecasting	Variance depends on past p squared errors
GARCH(p,q)	Statistical	Persistent vol forecasting	Variance depends on past errors + past variance
GBM	Stochastic process	Asset pricing, option pricing	Constant vol, log-normal returns
Vasicek / CIR	SDE models	Interest rate modeling	Mean reversion, one factor
Heston	Stochastic vol	Equity option surface	Variance follows CIR process
SABR	Stochastic vol	IR derivatives (swaptions)	Forward rate + vol both stochastic
Local vol (Dupire)	Deterministic vol surface	Exact surface fit	Vol is $\sigma(S,t)$ — no future dynamics
Rough vol (fBM)	Fractional process	Frontier research	Volatility has memory; Hurst H < 0.5

9.11.3 Interest Rate Models at a Glance

Model	Type	Mean Reversion	Negative Rates?	Calibration
Vasicek (1977)	Equilibrium	Yes	Possible	Historical data
CIR (1985)	Equilibrium	Yes	Impossible ($\sigma\sqrt{r}$)	Historical data
Ho-Lee (1986)	Arbitrage-free	No	Possible	Current market prices
Hull-White (1990)	Arbitrage-free	Yes	Possible	Current market prices
KWF	Arbitrage-free	No	Impossible (lognormal)	Current market prices
Black-Karasinski	Arbitrage-free	Yes	Impossible (lognormal)	Current market prices
HJM	Arbitrage-free	—	Depends on vol spec	Forward rate vol structure

9.11.4 Key Formulas Reference

Annualized Volatility:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252} = \sigma_{\text{weekly}} \times \sqrt{52} = \sigma_{\text{monthly}} \times \sqrt{12}$$

ARCH(p):

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{p} \alpha_i \varepsilon_{t-i}^2, \quad \alpha_0 > 0,\ \alpha_i \geq 0,\ \sum \alpha_i < 1$$

GARCH(p,q):

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2$$

Vasicek SDE:

$$dr_t = a(b - r_t), dt + \sigma, dW_t$$

Vasicek Mean Forecast:

$$E[r_{t+T}] = r_t e^{-aT} + b(1 - e^{-aT}) \xrightarrow{T\to\infty} b$$

Vasicek Variance Forecast:

$$\text{Var}(r_{t+T}) = \frac{\sigma^2}{2a}(1 - e^{-2aT}) \xrightarrow{T\to\infty} \frac{\sigma^2}{2a}$$

Vasicek WCDR (Credit Risk):

$$WCDR = N\left[\frac{N^{-1}(PD) + \sqrt{\rho} \cdot N^{-1}(0.999)}{\sqrt{1-\rho}}\right]$$

IRB Capital Requirement:

$$\text{Capital} = LGD \times EAD \times (WCDR - PD)$$

Sources.

SBI Securities (Option Pricing Factors), Harel Jacobson — "A Brief History of Volatility Models" (Medium), Ranjith Kumar K — "Time Series Models: ARCH and GARCH" (Medium), CFI — "Vasicek Interest Rate Model", Fabozzi — Chapter 17: Interest-Rate Models, Video transcripts on Volatility Modeling, Vasicek Interest Rate Model, and Vasicek for Credit Risk.

Chapter 10

Black–Scholes Model

10.1 Overview

The **Black-Scholes model** (also called the **Black-Scholes-Merton**, or **BSM**, model) is a mathematical equation used to calculate the fair, theoretical price of an options contract. It was developed by **Fischer Black**, **Myron Scholes**, and **Robert Merton** and published in 1973. It is one of the most important ideas in modern financial theory: it was the first widely used method for pricing options mathematically, and it transformed derivatives from an informally-priced, "bargained-over" product into a multi-trillion-dollar, precisely priced industry.

At its core, the model answers a simple question: **given a set of known variables about a stock and an option on that stock, what should that option be worth today?** It does this by assuming the underlying stock price moves randomly over time (a "random walk"/Brownian motion) and by building a risk-free hedged portfolio that must, in an efficient market, earn exactly the risk-free rate.

The model only prices **European-style options** — options that can only be exercised at expiration (as opposed to American-style options, which can be exercised any time up to expiration).

10.2 Historical Origins and Development

10.2.1 Options before Black–Scholes: Thales of Miletus (circa 600 BC)

The earliest known option contract dates back to around 600 BC. The Greek philosopher **Thales of Miletus** believed the coming olive harvest would be unusually large. Rather than buying olive presses outright (he didn't have the money), he paid press owners a small fee for the *right* — but not the obligation — to rent their presses at a fixed price later in the year. When the bumper harvest arrived, demand for presses (and their rental price) skyrocketed. Thales rented the presses at the pre-agreed low price and re-rented them at the new, much higher market rate, pocketing the difference. This was, in effect, the first recorded **call option**.

10.2.2 Louis Bachelier and the random walk (1900)

For centuries after Thales, options continued to be traded, but no one had a rigorous way to price them — traders simply haggled to reach an agreed price. The first person to bring real mathematical rigor to the problem was **Louis Bachelier**, a French mathematician born in 1870. After losing both parents at 18, he took over and later sold his father's wine business, moved to Paris to study physics, and took a job at the **Paris Stock Exchange (the Bourse)** to support himself. There, amid the chaotic shouting and hand-signaling of traders, he became fascinated by the pricing of options.

Bachelier proposed the mathematics of finance as his PhD topic to his advisor, the renowned mathematician **Henri Poincaré**, who — despite this being an unusual subject at the time — agreed.

Bachelier's key insight:

- Stock prices are driven by the constant "tug-of-war" between buyers and sellers, which is influenced by countless unpredictable factors (weather, politics, competition, innovation, etc.).
- Because these influences are effectively unpredictable, the best assumption is that at any moment, a stock price is equally likely to go up or down.
- Over time, this means stock prices follow a **random walk** — like a coin flip repeated over and over.
- This randomness is actually the hallmark of an **efficient market**: a market in which prices already reflect all available information, so that no one can reliably profit from predicting future price moves (the **Efficient Market Hypothesis**). If prices *were* predictable, everyone would trade on that prediction today, which would itself move the price today — eliminating the predictable move.

The Galton board analogy: A Galton board is a vertical board studded with rows of pegs, with balls dropped from the top. Each time a ball hits a peg, it has a 50/50 chance of bouncing left or right — a random walk. Any *single* ball's path is essentially unpredictable. But when thousands of balls are dropped, the *collection* of their final resting positions forms a predictable bell-shaped **normal distribution**, concentrated in the middle (since there are vastly more possible paths that lead to the center than to the far edges).

Bachelier modeled a stock price the same way: each moment in time is like an additional row of pegs. Over a short time, the price can only drift a little; over a longer time, a much wider range of outcomes becomes possible. He concluded that **the future price of a stock is described by a normal distribution centered on the current price, which spreads out (widens) as time passes.**

Remarkably, Bachelier realized that the equation describing this process was mathematically identical to the equation describing how **heat diffuses** from hot regions to cold regions — a phenomenon first described by **Joseph Fourier** in 1822. He called his discovery the "radiation of probabilities." Because he had framed it in terms of finance, the physics community ignored it — but the same mathematics would later resolve a genuine mystery in physics.

10.2.3 Brownian motion: Robert Brown and Einstein

In 1827, Scottish botanist **Robert Brown** observed under a microscope that pollen grains suspended in water moved in a random, jittery path. He ruled out biological explanations by testing inorganic particles (dust from lava and meteorite rock), which moved the same way. This unexplained random jittering became known as **Brownian motion**.

The cause remained a mystery for 80 years until 1905, when **Albert Einstein** explained it: Brownian motion is caused by countless invisible molecules randomly colliding with the suspended particle from every direction. Most of the time these collisions cancel out, but occasionally more hit from one side than another, causing the particle to visibly jump. Since these individual molecular collisions can't be observed, the particle must, at any instant, be assumed equally likely to move in any direction — exactly the same "random

walk" logic Bachelier had already applied to stock prices five years earlier, without knowing it. Einstein's explanation of Brownian motion also provided strong evidence that atoms and molecules physically exist.

Intuition for finance: Just as a pollen grain is constantly bombarded by water molecules from every direction, a stock's price is constantly bombarded by buy and sell orders from many market participants. The key difference: a pollen grain can move in any direction in 2D/3D space, while a stock price can only move up or down along one axis (it only "collides" with upward and downward pressure).

In the early 20th century, Japanese mathematician **Kiyoshi Itô** rigorously proved that financial asset prices do indeed follow Brownian motion, in what became known as **Itô's Lemma** — the mathematical tool the Black-Scholes model is built on (see Section 8).

10.2.4 Ed Thorp: from blackjack to Wall Street (1950s–1960s)

Ed Thorp, a physics PhD student in Los Angeles in the 1950s, applied mathematical thinking to blackjack in Las Vegas. Because casinos at the time used only a single deck, Thorp could mentally track which cards had already been played, letting him work out when the odds favored him. He bet more when the odds were good and less when they weren't — inventing **card counting**. This worked well until casinos responded by adding more decks to dilute the advantage.

Thorp then took his approach to what he called "the biggest casino on Earth" — the stock market. He started a hedge fund that delivered a 20% annual return for 20 years, the best track record seen at the time. His key innovation was a form of **hedging**: protecting against losses using balancing, offsetting transactions, applied mathematically rather than intuitively.

Example of Thorp's dynamic hedging (Bob and Alice): Suppose Bob sells Alice a call option on a stock, and the stock price rises so the option is now "in the money" for Alice. For every \$1 the stock rises further, Bob (the option seller) loses \$1. Bob can neutralize this risk by owning one unit of the underlying stock: if the price rises \$1, he loses \$1 on the option but gains \$1 back on the stock he owns. If the price later falls back out of the money, he sells the stock, since he no longer needs the hedge. This continuous adjustment of the hedge is called **dynamic hedging**. It effectively lets Bob "manufacture" the option position out of nothing through trading, while remaining close to risk-neutral.

The quantity of stock needed to hedge one option is called **delta** — mathematically, how much the option's price changes for a small change in the underlying stock price. As the example shows, delta itself changes as the stock price moves, which is why the hedge must be *dynamically* rebalanced.

Thorp wasn't fully satisfied with Bachelier's original model, because Bachelier's random walk ignored the fact that stocks tend to drift upward (or downward) over time depending on how the underlying business performs — not purely 50/50 randomness. Thorp built his own improved model incorporating this **drift** around 1967, and used it privately (and later with his investors) to buy underpriced options and short-sell overpriced ones according to his model — profiting from the mispricing more often than not.

10.2.5 Black, Scholes, and Merton (1973)

In 1973, this all changed when **Fischer Black** and **Myron Scholes** published an equation for option pricing, and **Robert Merton** independently published his own derivation using stochastic calculus (and is therefore also credited as a co-creator). Thorp had assumed he

would have the field to himself, but Black and Scholes got there first — with, in Thorp's own assessment, a more rigorous mathematical derivation than his.

The core idea, building on Thorp: if you can construct a risk-free portfolio made of the option and the underlying stock (exactly as Thorp was doing with delta hedging), then in an efficient/fair market, that risk-free portfolio should earn no more than the **risk-free rate** — i.e., what the same money would earn if invested safely (e.g., in U.S. Treasury bonds). This is a **no-arbitrage** argument: since the hedged portfolio carries no risk, it cannot be expected to earn more than the risk-free return, or a riskless profit opportunity (arbitrage) would exist.

To describe how the stock price evolves, Black, Scholes, and Merton used an upgraded version of Bachelier's model (like Thorp's) that included both a random component and a drift component. Combining this stock price model with the no-arbitrage hedging argument produced the now-famous **Black-Scholes equation** — a partial differential equation that, when solved, yields an explicit formula: plug in the known input parameters, and the formula directly outputs a price. This was one of the first times options traders had a concrete, usable number to trade on, rather than a vague negotiated price.

Impact: The same year the paper was published (1973), the **Chicago Board Options Exchange (CBOE)** was founded. The Black-Scholes formula was adopted industry-wide within just a couple of years — one of the fastest adoptions of an academic idea in the social sciences. The exchange-traded options market has since grown into a multi-trillion-dollar industry, with trading volume roughly doubling every five years (compared by some to a "financial Moore's Law"). Related derivative markets that also rest on Black-Scholes-Merton option-pricing logic — credit default swaps, OTC derivatives, securitized debt — have grown similarly explosively.

Nobel Prize: In 1997, **Myron Scholes** and **Robert Merton** were awarded the **Nobel Memorial Prize in Economic Sciences** for "a new method to determine the value of derivatives." **Fischer Black** had died in 1995, two years earlier; since the Nobel Prize is never awarded posthumously, he could not receive it, though the Nobel committee explicitly acknowledged his contribution.

Historical note: Merton helped edit Black and Scholes's original 1973 paper ("The Pricing of Options and Corporate Liabilities," published in the *Journal of Political Economy*) before separately publishing his own paper, "Theory of Rational Option Pricing," in *The Bell Journal of Economics and Management Science* the same year — in which he coined the term "Black-Scholes theory of options pricing."

10.2.6 Real-world use cases after Black-Scholes

- **Corporate hedging example:** An airline worried that rising oil prices will hurt profits can use Black-Scholes-Merton-based pricing to buy an option that tracks oil prices. If oil prices rise, the option pays off, offsetting the airline's higher fuel costs.
- **The GameStop short squeeze (2021):** Retail traders coordinated on Reddit's WallStreetBets to buy GameStop shares and drive up the price, punishing hedge funds that were short-selling the stock. Buying shares alone wasn't enough — because a dollar of cash buys only a dollar of stock, but a dollar of options (thanks to their built-in leverage) can control $10–$20 worth of stock exposure. Combining direct stock purchases with options purchases caused the price to spike very quickly, inflicting large losses on the short-selling hedge funds.
- **Scale of derivatives markets:** Global derivatives markets are estimated at several

hundred trillion dollars — multiples larger than the total value of the underlying securities they are based on. This is possible because options and derivatives let market participants slice one underlying asset into many different risk/reward "versions" tailored to different investors' preferences.

- **Effect on market stability:** During normal times, derivatives markets are a major source of liquidity and therefore stability. During periods of market stress, however, these instruments can all move in the same direction (typically down) simultaneously, which can amplify or exacerbate market crashes.

10.2.7 Jim Simons and Renaissance Technologies

With the Black-Scholes formula now public, simply knowing "the secret" of option pricing was no longer enough to reliably outperform the market — hedge funds needed new ways to find inefficiencies.

Jim Simons was a mathematician (not a trader) whose academic work on Riemann geometry contributed to knot theory, quantum field theory, and quantum computing; his **Chern-Simons theory** laid mathematical groundwork used in string theory, and he won the **Oswald Veblen Prize in Geometry** in 1976. He had also worked for the U.S. Institute for Defense Analysis breaking Russian codes during the Cold War — extracting patterns from large volumes of data.

In 1978, Simons founded **Renaissance Technologies**, applying a code-breaking, pattern-recognition mindset (using what would now be called machine learning) to financial markets, rather than fundamental analysis or traditional trading intuition. He deliberately hired people with **no finance background** — instead looking for people with strong PhDs in physics, mathematics, astronomy, or statistics, and a track record of good published research. One key hire was **Leonard Baum**, a pioneer of **hidden Markov models** — statistical tools for inferring hidden, unobservable factors from observable data (conceptually similar to how Einstein inferred the existence of invisible atoms from the visible, observable jittering of a pollen grain).

In 1988, Simons launched the now-famous **Medallion Fund**. Using hidden Markov models and other data-driven strategies, it became the highest-returning investment fund in history, delivering **66% average annual returns for roughly 30 years**. At that rate, $100 invested in 1988 would be worth about **$8.4 billion** today — making Simons the richest mathematician of all time. Economist Bradford Cornell (UCLA) discussed this performance in a paper titled *"Medallion Fund: The Ultimate Counterexample"*, arguing it challenges the Efficient Market Hypothesis. Simons himself published a 1988 paper testing the EMH on the U.S. stock market and concluded the hypothesis is false — i.e., there are real, exploitable predictabilities in the market, given the right models, training, data, and computational resources.

A cautionary counterexample — Isaac Newton (1720): Even brilliant mathematicians can fail badly at markets. In 1720, the 77-year-old Newton — already wealthy from his academic career and his role as Master of the Royal Mint (net worth then equivalent to roughly $6 million today) — invested in the **South Sea Company** (a business built on shipping enslaved Africans across the Atlantic). As the share price rose, Newton sold his initial position after it had doubled — but the price kept climbing, so he bought back in, and kept buying even near the peak. When the price began to fall, instead of selling he bought more, believing he was "buying the dip." There was no rebound, and he ultimately lost about a third of his fortune. His own explanation: *"I can calculate the motions of the heavenly bodies, but not the madness of people."* This illustrates that

mathematical skill alone doesn't guarantee market success — Simons succeeded in part because he built on more than a century of accumulated mathematical understanding of markets (Bachelier, Einstein/Itô, Thorp, Black-Scholes-Merton) that Newton simply didn't have access to.

10.3 Options Basics (Building Blocks)

Before the pricing formula itself, it helps to understand what an option actually is and how its payoff behaves.

- A **call option** gives the holder the right, but not the obligation, to **buy** an asset at a later date for a pre-agreed **strike price**. Useful when you expect the price to rise.
- A **put option** gives the holder the right, but not the obligation, to **sell** an asset at a later date for the strike price. Useful when you expect the price to fall.
- **Long** means you are the *buyer* of the option (you pay a premium up front for this right).
- **Short** means you are the *seller/issuer* of the option — you collect the premium now, but take on the risk that the buyer will exercise the option against you later. The short seller is effectively acting as an **insurer**.
- **American options** can be exercised at any time up to and including the expiration date. **European options** can only be exercised exactly at expiration. The Black-Scholes model in its standard form only prices **European** options, because it does not account for the extra value of being able to exercise early.

10.3.1 Worked numeric example (call option)

Suppose Apple stock currently trades at **\$100**, and you expect it to rise. You buy a call option for a **\$10 premium**, giving you the right to buy Apple stock in one year at a strike price of **\$100**.

- **If in one year the price has risen to \$130:** you exercise the option, buying at \$100 and immediately selling at \$130 — a \$30 gross gain. Subtracting the \$10 premium you paid, your **net profit is \$20**.
- **If in one year the price has fallen to \$70:** you simply don't exercise the option (it would make no sense to buy at \$100 something worth \$70 on the open market). You lose only the **\$10 premium** you paid — not the full drop in value.

Payoff diagram shape: If the stock ends below the strike price, your loss is capped at what you paid for the option (a flat line at $-\$10$). If the stock ends above the strike price, your profit rises linearly, dollar-for-dollar, with the stock price above the strike, minus the cost of the option.

10.3.2 Three advantages of options

1. **Limited downside.** Had you bought the stock outright instead of the option, a drop from \$100 to \$70 would have cost you \$30 (and in the extreme, you could lose up to the full \$100 if the stock went to zero). With the option, your maximum loss is capped at the \$10 premium.
2. **Leverage.** Buying the stock and seeing it rise from \$100 to \$130 is a 30% return. Buying the \$10 option and seeing the same move nets a \$20 profit — a **200%**

return on the capital actually put up. The flip side: on the downside, the stock only drops in proportion (e.g., −30%), while the option can lose its **entire value (−100%)**.

3. **Hedging.** Options can be used as a form of insurance — the original motivation for their existence was risk reduction. Once some market participants wanted to buy that "insurance," others were naturally willing to sell it for a profit, which is how these markets developed in the first place.

10.3.3 Options as insurance — long put and long call

- **Long put (insurance against a price drop):** Suppose you own a stock worth $100 and worry it could fall to $80 or $60. You buy a put option with a strike of $100. If the market price falls below $100, you can still sell at $100 using the option, offsetting your loss. This is effectively insurance against a decline.
- **Long call (insurance/participation for an upside you don't yet own):** Suppose the stock is at $100 and you believe it will rise, but you don't own it yet. Buying a call with a strike of $100 means that if the price rises to, say, $120, you can still buy at $100 — capturing the $20 difference as profit.

10.3.4 Short call example

When you are **short** a call (you sold/issued it), you collect the premium up front but bear the risk that the buyer exercises against you.

- If the stock price ends up **below** the strike at expiration, the buyer has no reason to exercise (a European holder cannot exercise early even if the option was briefly in-the-money along the way — only an American holder could). The option expires worthless, and as the seller you keep the full premium.
- If the stock price ends up **well above** the strike at expiration, the buyer will definitely exercise, and as the seller you must deliver — resulting in a loss for you.

At expiration, a call option's value ("payoff") is simply: $\max(S_T - K, 0)$ — i.e., stock price minus strike if positive, otherwise zero. This much is straightforward. The much harder question the Black-Scholes model answers is: **what is the option worth at any point *before* expiration**, while its final payoff is still uncertain? Answering this requires modeling the stochastic (random) behavior of the underlying stock over the remaining time — which is exactly what the mathematics in Sections 8–9 develops.

10.4 What the Black-Scholes Model Does

Framed simply: the Black-Scholes model is a **multivariate mathematical formula** that returns the fair price of a European stock option, given a defined set of input variables. The formula for a **call** option is different from the formula for a **put** option.

10.5 Key Input Variables

The Black-Scholes equation requires the following inputs:

1. **Volatility** (σ) of the underlying asset's returns
2. **Current price of the underlying asset** (S)
3. **Strike price of the option** (K)

4. **Time until the option's expiration** (t)
5. **Risk-free interest rate** (r)
6. **Type of option** (call or put)

With these six variables, an options seller can, in theory, set a rational, consistent price for the options they sell.

10.6 The Black-Scholes Formula

The Black-Scholes price of a **European call option** is calculated as:

$$\boxed{C_0 = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2)}$$

where:

- $d_1 = [\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T]/(\sigma\sqrt{T})$
- $d_2 = d_1 - \sigma\sqrt{T}$

and:

- **C** = call option price
- **S** = current price of the underlying stock
- **K** = strike price
- **r** = risk-free interest rate
- **t** = time to maturity (expiration)
- σ = volatility of the underlying asset
- **N( )** = the cumulative distribution function of the standard normal distribution
- **e** = Euler's number (the mathematical constant ≈ 2.71828), used here to discount the strike price back to present value continuously

Reading the formula intuitively: The call price is the current stock price weighted by a probability-like factor $N(d_1)$, minus the present value of the strike price (discounted continuously at the risk-free rate) weighted by another probability-like factor $N(d_2)$. Loosely, $N(d_2)$ can be thought of as related to the (risk-neutral) probability that the option will be exercised, and $S{\cdot}N(d_1)$ as the expected benefit of receiving the stock conditional on exercising.

For completeness, the corresponding European put price is

$$\boxed{P_0 = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1)}.$$

The call and put formulas are connected by put–call parity,

$$C_0 - P_0 = S_0 - Ke^{-rT}.$$

Note: the formula for a **put option** is structurally related but different from the call formula (the put-call relationship follows from a no-arbitrage identity called put-call parity, though the individual put formula isn't detailed in the source material here).

Practical note: you do not need to compute this by hand to use it. Most options trading platforms include built-in options calculators and analytical tools that compute Black-Scholes prices (and the associated Greeks — see Section 10) automatically from the six inputs.

10.7 Assumptions of the Black-Scholes Model

The model rests on a number of simplifying assumptions:

1. **Stock price dynamics:** The underlying price follows a **geometric Brownian motion** with **constant drift and volatility**. This implies the asset's *returns* are normally distributed, and its *price* is lognormally distributed (prices can never go negative, but returns — which can be negative — are assumed normal).
2. **No arbitrage:** There is no way to make a riskless profit with no investment.
3. **Efficient, frictionless markets:** All available information is already reflected in the price, and prices adjust instantly. There are no transaction costs or taxes, and assets are infinitely divisible. Short selling is permitted, with full use of proceeds.
4. **Constant, known risk-free rate:** The risk-free interest rate is constant and known for the life of the option.
5. **No dividends:** The underlying stock pays no dividends during the option's life (the standard model can be, and often is, extended/adapted to account for dividends, typically via the ex-dividend-date value of the stock).
6. **European exercise only:** The option can only be exercised at expiration, not before.
7. **Continuous trading:** Trading happens continuously in time, and hedge portfolios can be rebalanced instantly and at no cost.

Because many of these assumptions are unrealistic — volatility isn't really constant, real markets have transaction costs, risk-free rates fluctuate, and many stocks do pay dividends — the model has drawn substantial criticism (see Sections 9 and 11).

10.8 The Mathematics Behind the Model

10.8.1 Geometric Brownian Motion (GBM)

The Black-Scholes model assumes the stock price S follows **geometric Brownian motion**:

$$\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}W_t.$$

This has two parts:

- **Drift term: μS dt** — the deterministic, expected trend of the stock (its average expected growth rate μ).
- **Volatility (diffusion) term: σS dW** — the random component, driven by an increment of Brownian motion dW, scaled by the stock's volatility σ.

The problem: because this equation contains a Brownian motion term, it cannot be solved with ordinary calculus techniques. Solving it requires **Itô's Lemma**.

10.8.2 Warm-up: the deterministic case

Start by dividing through by S so all the S terms sit on the left: $\mathrm{d}S_t/S_t = \mu \, \mathrm{d}t + \sigma \, \mathrm{d}W_t$.

First, temporarily ignore the random (volatility) term and keep only the drift: $\mathrm{d}S_t/S_t = \mu \, \mathrm{d}t$. This is now an ordinary, deterministic differential equation. Expanding with a Taylor series and keeping only first-order terms (since in standard calculus, dt^2 and higher powers vanish as dt shrinks), integrating over time yields the familiar result: **exponential growth**, i.e., $S(t) = S_0 e^{\mu t}$.

10.8.3 Bringing back randomness: Itô's Lemma

To reintroduce the random term, a simple first-order Taylor expansion is no longer sufficient — we need Itô's Lemma. The intuition, built up step by step:

1. **Function of time only:** for a smooth function that depends only on time, laying successive-order Taylor expansions on top of the function shows the approximation improving, but at the infinitesimal scale, the first-order term is enough — i.e., terms involving dt^2 (and higher) can be treated as zero.
2. **Function of time *and* Brownian motion:** once the function also depends on a Brownian motion term, a first-order (straight-line) approximation is no longer enough to capture the local behavior — no matter what slope is chosen, it "misses." Second-order terms must be brought back in.
3. **Switch to an Itô process:** instead of raw Brownian motion, consider a process that is Brownian motion scaled by σ, plus a drift term (i.e., exactly the GBM setup above).
4. **List all Taylor terms up to second order**, then apply two key rules for how infinitesimal increments behave:
   - $\mathbf{dt \cdot dW = 0}$ — because the increment of Brownian motion over a small time step has mean zero and is independent of the past, so this cross term vanishes in expectation.
   - $\mathbf{dW \cdot dW = dt}$ — because Brownian motion accumulates "quadratic variation" proportional to time; this term does *not* vanish, which is precisely why the second-order term matters here (unlike in ordinary calculus).
   - The dt^2 term is negligible and dropped.

Putting these pieces together yields **Itô's Lemma**: for a function f(t, X) where X follows an Itô process, the change in f decomposes into a drift part and a diffusion (random) part that together properly account for the extra "curvature" contributed by the $dW^2 = dt$ term.

10.8.4 Solving Geometric Brownian Motion with Itô's Lemma

Apply Itô's Lemma to the function $\mathbf{f = \ln(S)}$, where S follows the GBM equation from Section 8.1. Working through the derivative terms of ln(S) and substituting into Itô's Lemma produces the solved equation for the logarithm of the stock price, which after integrating over time gives the closed-form solution:

$$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right) t + \sigma W_t\right].$$

This is the key result: it shows that under the GBM assumption, the stock price at any future time t is **lognormally distributed** — consistent with the model's core assumption in Section 7 that prices are lognormal (since a normally-distributed random variable, W(t), sits inside an exponential). Note the term $\mathbf{-\sigma^2/2}$ — this is a direct mathematical consequence of applying Itô's Lemma (it comes from the second-order term that ordinary calculus would have missed), and it means the median and almost-sure log-growth rate is $\mu - \sigma^2/2$, while $\mathbb{E}[S_t] = S_0 e^{\mu t}$.

10.9 Deriving the Black-Scholes Equation: Replicating Portfolios and Delta Hedging

10.9.1 The core idea

To price an option before expiration, think from the perspective of the option **seller**. Having sold (issued) the option, the seller is exposed to risk if it ends up "in the money" for the buyer at expiration. To protect against this, the seller wants to **hedge**: build a position that doesn't lose money no matter which way the stock moves.

The hedging tool is a **replicating portfolio**: hold some combination of (a) the underlying stock and (b) cash or a risk-free bond, such that this combination behaves identically to the option, cancelling out the seller's risk.

- If the stock is moving up and the seller is short a call, they are exposed — so they buy some stock to compensate.
- If the option currently has little sensitivity to the stock (e.g., it's far out of the money), less stock is needed, and more of the portfolio can sit in cash earning the risk-free rate.

The central question the model answers: **exactly how much stock vs. how much cash should be held at each moment, and therefore what should the option's fair price be, before expiration?**

10.9.2 Setting up the portfolio (formal version)

Let the replicating portfolio hold:

- an amount **A** of the stock, and
- an amount **B** of cash/bond.

Dynamics of each piece:

- The stock S follows geometric Brownian motion (drift + random part), as in Section 8.1.
- The bond/cash B has **no randomness** — it grows deterministically at the constant risk-free rate r.

Combining these lets us describe how the whole portfolio (A·S + B) evolves, and this can be split into its own **drift** (predictable) part and **diffusion** (random) part.

There is a second way to describe how the portfolio (or equivalently, the option value) evolves: applying **Itô's Lemma** directly to the option's value function V(S,t), exactly as in Section 8.3.

This produces **two separate expressions** for the portfolio's drift and diffusion — one derived from the stock-plus-bond composition, and one derived from Itô's Lemma applied to the option value. **Matching these two expressions to each other** and solving lets us solve for A and B.

10.9.3 Solving for A (delta) and B

- **Solving for A:** A turns out to equal how sensitive the option price is to the stock price — i.e., $A = \partial V/\partial S = \Delta$, the option's **delta**. If a small move in the stock produces a big move in the option price, more stock must be held to hedge; if the

option barely reacts to the stock (e.g., a deep out-of-the-money call), very little stock is needed.

- **Solving for B:** B is the amount of cash/bond needed to complete the hedge once A (the stock holding) is known.

10.9.4 The Black-Scholes equation and formula

Substituting the solved A and B back into the replicating portfolio equation produces the **Black–Scholes equation**:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0.}$$

For a European call, the terminal condition is

$$V(S,T) = \max(S - K, 0).$$

This boundary condition states that the option's terminal value equals its known payoff at expiration.

Solving this differential equation (the derivation itself is not carried out step-by-step in the source material) yields the **Black-Scholes formula** given in Section 6. Structurally, it contains two cumulative normal distribution terms, $\mathbf{N(d_+)}$ and $\mathbf{N(d_-)}$ (equivalent to $N(d_1)$ and $N(d_2)$ in Section 6), where d_+ is always slightly larger than d_-.

Visual intuition for $\mathbf{d_1}$/$\mathbf{d_2}$ ("sliding a bell curve"):

- When the stock price equals the strike price, both d_+ and d_- sit near the center of the standard normal curve.
- When the stock price drops, both slide to the left (their N() values shrink toward 0).
- When the stock price rises a lot, both slide far to the right, and their cumulative distribution values approach 1. In this case the option is **deep in the money** — exercise is essentially certain — and the call price formula simplifies to just **stock price minus the discounted strike price** (matching intuition: the option behaves almost exactly like owning the stock outright, minus the cost of paying the strike later, discounted back to today because money today is worth more than money in the future).

Behavior of the option value over time (path intuition):

- If the stock drops below the strike and stays there, the call option becomes worth less and less, fading toward zero as expiration approaches (less and less time left for it to recover).
- If the stock is in the middle of its path and still has time left, the option retains some value, because there's still a chance it moves back above the strike.
- If the stock rises well above the strike, the option's value increasingly tracks (stock price − discounted strike), converging to that relationship almost exactly near expiration.

10.9.5 Alternative framing: hedging a short option position directly

A second, equivalent way to derive the same hedge:

- If you are **short** an option (you sold/issued it), you face a potential future payment obligation if it ends up in the money. Represent this as a **negative** position: $-\mathbf{C}$ (C = option price).
- To offset this risk, hold some quantity Δ **(delta)** of the underlying stock, forming the combined portfolio: $-C + \Delta S$.
- Break the small changes in this portfolio into a **predictable (drift)** part and a **random (risk)** part, driven by the stock's volatility.
- The goal of hedging is to choose Δ so that the **random part cancels out entirely**.
- Solving for this condition shows that the correct Δ is exactly **how sensitive the option price is to the stock price** — i.e., the derivative of the option price with respect to the stock price. This is the same delta as in Section 9.3.

Two worked examples:

- **Deep in-the-money call, near expiration:** the option price moves almost one-for-one with the stock price, so delta is close to 1 (or close to 100%) — a large amount of stock must be held to hedge. In practice this often means *borrowing* cash to fund the stock purchase (cash balance goes negative) in order to hold that much stock.
- **Deep out-of-the-money call:** the stock price is well below the strike, the option is essentially worthless, and its sensitivity to the stock is near zero — so delta is near zero, and little or no stock needs to be held.

In both cases, if the hedge is constructed correctly, the **total value of the replicating portfolio (option + stock + cash) stays flat, close to zero, throughout the life of the trade** — even though the individual pieces move up and down. This flatness demonstrates that the strategy has successfully cancelled out risk: the hedger is neither reliably making nor losing money from the stock's random fluctuations; they are **risk-neutral** with respect to the stock's movement. This is the essence of **delta hedging**.

10.10 Option Greeks

Because the Black-Scholes model is a multivariate equation, institutional traders study its **partial derivatives** with respect to each input variable — collectively called the **Greeks**. These let traders decompose overall option risk into distinct, separately manageable and hedgeable types of risk, which is what makes it practical to trade profitably despite the model's unrealistic underlying assumptions.

Greek	What it measures
Delta	Exposure/sensitivity to the *direction* of the underlying's price (Sections 9.3–9.5)
Gamma	Exposure to price *fluctuations* in either direction (i.e., how much delta itself changes as the stock price moves)
Theta	Exposure to the passage of *time* (time decay of the option's value)
Vega	Exposure to changes in *implied volatility*
Rho	Exposure to changes in the *risk-free interest rate*

Key practical insight: institutional traders generally don't trade options purely to bet on price direction — they trade **volatility**, using the Black-Scholes framework and

its Greeks. This is because price *volatility* tends to be easier to predict (or model) than raw price *direction*, given the mathematical nature of how prices behave.

10.11 Volatility Skew

The Black-Scholes model assumes stock prices are **lognormally distributed** (since prices can never go below zero). Under this assumption, implied volatility should be roughly similar across different strike prices for options on the same underlying with the same expiration.

In practice, this is not what's observed:

- Equity-index returns commonly show significant **negative skewness** and **kurtosis (fat tails)** — large downward price moves happen more often than a normal distribution would predict.
- Since the 1987 market crash, **at-the-money options have consistently shown lower implied volatility than far out-of-the-money or deep in-the-money options**. This produces a "smile" or "skew" shape when implied volatilities across strikes (for a given expiration) are plotted on a graph — the market is effectively signaling a higher perceived chance of a large drop.

Empirical evidence of downside/upside asymmetry (COVID-19 crash): during the COVID-19 pandemic crash, the S&P 500 took only **32 days** to fall to its bottom, but took **153 days** — roughly five times as long — to climb back to its pre-crash level. This illustrates that downside volatility is typically much greater than upside volatility, a pattern seen in essentially all historical market crashes, and a real-world violation of the model's constant-volatility assumption. (This asymmetry is also what specialized **volatility-skewness trading strategies** are designed to exploit.)

Consequence: because of this skew, the Black-Scholes model, in its plain form, is **not efficient/accurate for calculating implied volatility** directly from market prices.

10.12 Benefits and Limitations

10.12.1 Benefits

- **Provides a structured framework:** gives investors and traders a tried-and-tested, defined methodology for determining a fair option price, rather than relying on ad hoc negotiation.
- **Enables risk management:** knowing an option's theoretical value helps investors understand and manage their exposure, and identify weaknesses or gaps in a portfolio.
- **Supports portfolio optimization:** provides a measure of expected returns and risk for different options, helping investors make choices aligned with their risk tolerance and return objectives.
- **Enhances market efficiency and transparency:** because traders share a common, transparent method for pricing, options markets have become more efficient and easier to price and trade.
- **Streamlines and standardizes pricing:** its wide acceptance among practitioners provides consistency and comparability of prices across different markets and jurisdictions.

10.12.2 Limitations

- **Limited to European options:** the standard model does not account for the possibility of early exercise, so it isn't directly suited to American-style options (which are more commonly traded in the U.S.). Firms instead often use **binomial or trinomial models**, or the **Bjerksund-Stensland model**, for pricing American-style options.
- **Lacks cash-flow flexibility:** the model assumes dividends and the risk-free rate are constant, which may not reflect real-world cash flows accurately, since the model is somewhat rigid.
- **Assumes constant volatility:** in reality, volatility fluctuates with supply and demand, which the standard model does not capture (see Section 11).
- **Relies on other simplifying assumptions that can deviate from reality:** including no transaction costs or taxes, a constant risk-free interest rate across all maturities, unrestricted short selling with full use of proceeds, and the absence of any risk-free arbitrage opportunities. Each of these, where violated in the real world, can cause the model's price to diverge from actual market prices.

Fast fact: The Black-Scholes model is often contrasted with other pricing approaches such as the **binomial model** or **Monte Carlo simulation**, which can more flexibly accommodate early exercise and non-constant assumptions.

10.13 Quick-Reference Summary Table: Key People

Person	Contribution
Thales of Miletus (∼600 BC)	Executed the earliest known call option (olive press rentals)
Robert Brown (1827)	Observed random particle motion under a microscope — "Brownian motion"
Louis Bachelier (1900)	First to model stock prices as a random walk; PhD thesis "Théorie de la Spéculation"; discovered the same math later used for Brownian motion and Black-Scholes
Albert Einstein (1905)	Mathematically explained Brownian motion via molecular collisions, proving atoms exist
Kiyoshi Itô	Proved financial asset prices follow Brownian motion; developed Itô's Lemma, the core mathematical tool behind Black-Scholes
Isaac Newton (1720)	Cautionary example: lost roughly a third of his fortune trading South Sea Company stock despite his mathematical genius
Ed Thorp (1950s–60s)	Invented blackjack card counting; pioneered dynamic delta-hedging in an options-based hedge fund; built an improved (drift-inclusive) option pricing model before Black-Scholes was published
Fischer Black, Myron Scholes, Robert Merton (1973)	Published the Black-Scholes(-Merton) option pricing equation; Scholes and Merton won the 1997 Nobel Prize (Black had died in 1995)

Person	Contribution
Jim Simons (1978–)	Founded Renaissance Technologies; built the Medallion Fund (∼66%/year returns for ∼30 years) using data-driven, pattern-based (machine-learning-style) strategies rather than fundamental trading
Leonard Baum	Pioneer of hidden Markov models, key hire at Renaissance Technologies

10.14 Summary

The Black-Scholes model emerged from a century-long convergence of ideas from physics (Brownian motion, heat diffusion), probability theory (random walks, the normal distribution), and finance (options as insurance, hedging, no-arbitrage pricing). Its central mechanism is the construction of a **risk-free replicating portfolio** of the underlying stock and a risk-free bond that mirrors the behavior of the option; the requirement that this riskless portfolio earn only the risk-free rate (no arbitrage) pins down a unique, fair option price. Despite resting on a number of assumptions that don't perfectly hold in real markets — constant volatility chief among them — its introduction was transformative: it gave the market its first standardized, computable way to price options, and its logical framework (particularly the Greeks and the concept of dynamic delta hedging) remains the backbone of how institutional derivatives desks manage risk today.

Chapter 11

Deriving the Black–Scholes Equation

11.1 Overview

There are two classic ways to arrive at the Black-Scholes pricing result, and this topic covers both:

1. **The PDE / replicating-portfolio approach** — build a riskless portfolio out of the option and the underlying stock, dynamically rebalanced, and use the no-arbitrage principle (a riskless portfolio must earn exactly the risk-free rate) to derive a partial differential equation (PDE) that the option price must satisfy.
2. **The risk-neutral pricing / expectation approach** — directly define the option's price today as the *discounted, risk-neutral expected value* of its future payoff, and use a change-of-measure result called **Girsanov's theorem** to justify computing this expectation.

Both approaches are mathematically equivalent and arrive at the same final pricing formula; they are simply two different lenses (PDE vs. probabilistic) on the same underlying idea: **construct something risk-free, and require it to earn only the risk-free rate.**

11.2 Derivation 1 — The PDE Approach (Replicating Portfolio / Delta Hedging)

This is the more "mechanical," step-by-step derivation, and it directly formalizes the intuitive hedging argument introduced in the previous notes (Section 9).

11.2.1 Step 1 — Stock Price Dynamics

Assume the underlying stock price S follows **Geometric Brownian Motion (GBM)**:

$$\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}W_t.$$

where:

- μ = the expected drift (return) of the stock
- σ = the volatility of the stock
- **dW** = a standard Brownian motion increment (the random component)

11.2.2 Step 2 — Defining the Derivative's Value

Let **V(S, t)** be the price of a derivative (e.g., a European call option) that depends on both the current stock price S and time t. Because V is a function of a stochastic (random) process, its own change over time, **dV**, must be found using **Itô's Lemma** (see previous notes, Section 8.3, for the derivation of Itô's Lemma itself):

$$\mathrm{d}V = \left(V_t + \mu S V_S + \frac{1}{2}\sigma^2 S^2 V_{SS}\right)\,\mathrm{d}t + \sigma S V_S\,\mathrm{d}W_t.$$

Reading this: the first (bracketed) term is the **deterministic/drift part** of the option's price change; the second term, multiplying dW, is the **random part**, whose size depends on σ, S, and how sensitive V is to S (i.e., ∂V/∂S).

11.2.3 Step 3 — Constructing a Riskless Portfolio

To eliminate the randomness coming from dW, build a portfolio Π consisting of:

- a **long** position of one unit of the derivative: $+\mathbf{V}$
- a **short** position of Δ units of the underlying stock: $-\Delta\cdot\mathbf{S}$

$$\Pi = V - \Delta S.$$

Over a small time interval dt, the change in this portfolio's value is:

$$\mathrm{d}\Pi = \mathrm{d}V - \Delta\,\mathrm{d}S.$$

Substituting in the expressions for dV (from Step 2, Itô's Lemma) and dS (from Step 1, GBM):

$$\begin{aligned}\mathrm{d}\Pi = &\left(V_t + \mu S V_S + \frac{1}{2}\sigma^2 S^2 V_{SS}\right)\,\mathrm{d}t + \sigma S V_S\,\mathrm{d}W_t \\ &- \Delta\left(\mu S\,\mathrm{d}t + \sigma S\,\mathrm{d}W_t\right).\end{aligned}$$

Grouping the deterministic (dt) terms and the random (dW) terms separately:

$$\mathrm{d}\Pi = \left(V_t + \mu S V_S + \frac{1}{2}\sigma^2 S^2 V_{SS} - \Delta\mu S\right)\,\mathrm{d}t + \sigma S(V_S - \Delta)\,\mathrm{d}W_t.$$

11.2.4 Step 4 — Eliminating Risk (Delta Hedging)

To make the portfolio **perfectly riskless**, the random dW term must vanish entirely. This happens if we choose:

$$\Delta = V_S = \frac{\partial V}{\partial S}.$$

— i.e., the hedge ratio Δ is set equal to the option's sensitivity to the stock price (exactly the same "delta" defined in the previous notes, Sections 9.3–9.5 and 10). Substituting Δ = ∂V/∂S back into the equation makes the coefficient of dW exactly zero (σS·∂V/∂S − (∂V/∂S)·σS = 0), leaving only the deterministic part:

$$\mathrm{d}\Pi = \left(V_t + \frac{1}{2}\sigma^2 S^2 V_{SS}\right)\,\mathrm{d}t.$$

Notice that μ **(the stock's real-world expected return/drift) has completely dropped out** of this equation. This is a crucial and somewhat surprising feature of the Black-Scholes framework: because risk has been fully hedged away, the option's price does **not** depend on how bullish or bearish investors are about the stock's future direction — only on its volatility, the risk-free rate, and the other known inputs.

11.2.5 Step 5 — Applying the No-Arbitrage Principle

Since the portfolio Π is now entirely risk-free, it must earn exactly the same rate of return as any other risk-free investment (such as a government bond) with interest rate **r**. If it earned more, investors would pile into building this exact riskless portfolio for a free profit; if it earned less, they would short it and invest in bonds instead — either way, in an efficient market, such a **riskless arbitrage opportunity cannot persist**. This gives:

$$\mathrm{d}\Pi = r\Pi \, \mathrm{d}t.$$

Substituting the definition of the portfolio, $\Pi = V - SV_S$, into this condition:

$$\mathrm{d}\Pi = r(V - SV_S) \, \mathrm{d}t.$$

11.2.6 Step 6 — The Black-Scholes Equation

We now have **two separate expressions** for dΠ — one from the hedging derivation (Step 4) and one from the no-arbitrage condition (Step 5). Setting them equal:

$$\left(V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) \mathrm{d}t = r(V - SV_S)\,\mathrm{d}t.$$

Dividing out dt (which appears on both sides) and rearranging all terms onto one side gives the final result — the **Black-Scholes partial differential equation**:

$$\boxed{V_t + rSV_S + \frac{1}{2}\sigma^2 S^2 V_{SS} - rV = 0.}$$

This single PDE, together with a **terminal (boundary) condition** specifying the derivative's known payoff at expiration (e.g., for a call: V(S,T) = max(S − K, 0)), is what defines the option's price at every earlier point in time. Solving this PDE for that boundary condition produces the explicit closed-form Black-Scholes formula covered in the previous notes and recapped in Section 4 below.

Why this derivation is considered elegant: it converts a problem involving an unpredictable, risky asset into a completely deterministic equation, purely by the trick of continuously rebalancing a hedge (Δ) so that all randomness cancels out at every instant. The price of risk (μ) never has to be estimated or agreed upon — the no-arbitrage argument alone pins down a unique price.

11.3 Derivation 2 — The Risk-Neutral Pricing / Expectation Approach

This is a different (but ultimately equivalent) route to the same destination, developed via probability rather than differential equations.

11.3.1 Recap: How the Underlying Evolves

From prior derivation (see also previous notes, Section 8.4), the price of the underlying asset at a future time **T**, given today's time is **t**, follows:

$$S_T = S_t \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)(T - t) + \sigma(W_T - W_t)\right].$$

where ΔW = W_T − W_t is the increment of Brownian motion between today and the future time T. The instantaneous dynamics behind this are, as before:

$$\mathrm{d}S_t = \mu S_t \, \mathrm{d}t + \sigma S_t \, \mathrm{d}W_t.$$

11.3.2 Two Approaches to Pricing: Risky vs. Risk-Neutral

There are, broadly, two ways to price any asset or derivative:

- **Risky pricing** — pricing that explicitly accounts for investors' risk preferences and required risk premiums.
- **Risk-neutral pricing** — a mathematically convenient shortcut in which we act *as if* all investors were indifferent to risk, and price the derivative purely as a **discounted expected payoff**.

This derivation takes the **risk-neutral pricing** approach.

11.3.3 The Risk-Neutral Pricing Formula

Under risk-neutral pricing, the price of any asset today (time t) is given by the **risk-neutral expectation of its future payoff**, discounted back to the present at the risk-free rate:

$$V_t = e^{-r(T-t)}\mathbb{E}^{\mathbb{Q}}[H(S_T) \mid \mathcal{F}_t].$$

Continuous compounding of the risk-free rate: in continuous time, the risk-free growth factor over a period is defined as $e^{r\tau}$. For example, if the annual risk-free rate is 5%, then the risk-free growth factor over half a year (time period = 0.5) is $e^{0.05(0.5)} = e^{0.025} \approx 1.025$.

Substituting this definition of the discount factor into the pricing formula:

Equivalently,

$$V_t = e^{-r(T-t)}\mathbb{E}^{\mathbb{Q}}[H(S_T)].$$

11.3.4 Expanding the Expectation

The derivative's payoff is some function of the underlying's future value: $H(S_T) = f(S_T)$. We already have a formula for S_T (Section 3.1), so we can substitute it in:

$$V_t = e^{-r\tau}\mathbb{E}\Big[f\Big(S_t e^{(\mu-\sigma^2/2)\tau+\sigma\Delta W}\Big)\Big], \qquad \tau = T - t.$$

To actually compute this expectation, we need the **distribution** of the only random component here, ΔW. As established previously, Brownian motion increments are **normally distributed with mean 0 and variance equal to the elapsed time**: ΔW Normal(0, T−t).

A useful scaling trick: if Y is a **standard** normal random variable (mean 0, variance 1), then multiplying it by $\sqrt{}$(T−t) produces a normal random variable with mean 0 and variance (T−t) — exactly matching the distribution of ΔW. (This scaling property holds generally: multiplying a standard normal variable by $\sqrt{}$k produces a normal variable with variance k.) So we can write:

$\Delta W = \sqrt{T-t}\,Y$, where $Y \sim \mathcal{N}(0,1)$, where Y Standard Normal(0, 1)

11.3.5 Writing the Expectation as an Integral

The expectation of a function of a normal random variable is, by definition, the integral of that function weighted by the variable's probability density function (PDF). For a standard normal variable Y, the PDF is:

$$\phi(y) = \frac{1}{\sqrt{2\pi}} e^{-y^2/2}.$$

So the expectation becomes an explicit integral over all possible values of y, from $-\infty$ to ∞:

$$\mathbb{E}[f(S_T)] = \int_{-\infty}^{\infty} f\Big(S_t e^{(\mu-\sigma^2/2)\tau+\sigma\sqrt{\tau}\,y}\Big)\,\frac{e^{-y^2/2}}{\sqrt{2\pi}}\,\mathrm{d}y.$$

This integral form is completely general: whatever function f (the payoff structure) is plugged in, this integral — once evaluated — gives that derivative's expected future payoff.

11.3.6 The Problem: This Is a "Normal" Expectation, Not a Risk-Neutral One

There's a subtlety: the expression above, using the *real-world* drift μ, is a **normal (real-world / physical-measure) expectation** — not the **risk-neutral** expectation the pricing formula in Section 3.3 actually calls for. To correctly switch from one to the other requires a result called **Girsanov's Theorem**.

Girsanov's Theorem (used here, not derived): this is an advanced result from stochastic calculus. Recall the underlying's dynamics were written as $\mathrm{d}S_t/S_t = \mu\,\mathrm{d}t + \sigma\,\mathrm{d}W_t^{\mathbb{P}}$. Girsanov's theorem tells us that we can compute the **risk-neutral** expectation using exactly the same machinery as a normal expectation — the same integral, the same normal-distribution logic — **provided we simply replace the real-world drift μ with the risk-free rate r** in the underlying's dynamics:

$\mathrm{d}S_t/S_t = r\,\mathrm{d}t + \sigma\,\mathrm{d}W_t^{\mathbb{Q}}$ (under the risk-neutral measure)

In other words: to go from a "physical world" expectation to a "risk-neutral" expectation, swap μ for r everywhere it appears, and otherwise carry out the same calculation. (The rigorous proof of Girsanov's theorem is advanced and outside the scope of these notes.)

11.3.7 The Resulting Master Pricing Equation

Applying this substitution ($\mu \to$ r) to the integral from Section 3.5, and plugging back into the discounted-expectation pricing formula from Section 3.3, gives what is, in effect, the **Black-Scholes pricing equation** in its general, implicit integral form:

$$\boxed{V_t = e^{-r\tau} \int_{-\infty}^{\infty} f\Big(S_t e^{(r-\sigma^2/2)\tau+\sigma\sqrt{\tau}\,y}\Big)\,\frac{e^{-y^2/2}}{\sqrt{2\pi}}\,\mathrm{d}y.}$$

This single formula can price **any** European-style derivative on this underlying — all that changes is the payoff function f plugged in. For example, if a derivative's payoff were the *square* of the underlying's value at time T, you would substitute f(x) = x^2 into this integral and evaluate it. For a standard **call option**, the payoff function is f(S_T) = max(S_T − K, 0); evaluating the integral for this specific payoff function (a separate calculus exercise not carried out in this transcript) is exactly what produces the closed-form Black-Scholes call formula recapped in Section 4 below.

11.4 Recap: The Black-Scholes-Merton Formula

Both derivation routes above converge on the same explicit pricing formula for a **European call option** on a **non-dividend-paying** asset:

$$\boxed{C(S_t, t) = S_t\Phi(d_1) - Ke^{-r\tau}\Phi(d_2).}$$

where:

- $\mathbf{S_0}$ = the stock price today
- $\mathbf{C(S_0,\ t)}$ = the price of the call option, as a function of the stock price and time
- $\mathbf{K}$ = the exercise (strike) price
- $\mathbf{(T\ -\ t)}$ = time to maturity — the exercise date T, minus the current time t (typically expressed in years; e.g., one month = 1/12 ≈ 0.0833 years)
- $\mathbf{r}$ = the risk-free interest rate (the return available on an assumed risk-free investment, like a T-bill)
- σ = the underlying's volatility (the standard deviation of its log returns)
- $\mathbf{N(d_1)}$ and $\mathbf{N(d_2)}$ = cumulative distribution functions (CDFs) of the **standard normal distribution**, computed as:

$$d_1 = \frac{\ln(S_t/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

$$d_2 = d_1 - \sigma\sqrt{\tau} = \frac{\ln(S_t/K) + (r - \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

(The formula for a European **put** option is structurally similar, differing in which terms are added vs. subtracted and which probability terms are used — related to the call formula through *put-call parity.*)

Important caveat on notation: this is the notation standard to the finance industry, but it is not, strictly, standard mathematical notation. What is loosely called "a normal distribution" here is technically the **cumulative distribution function of a log-normal distribution** — built from an underlying standard normal distribution (mean 0, standard deviation 1) that is assumed but rarely stated explicitly.

11.5 Understanding Volatility in the Model

What volatility measures: volatility is a measure of how much, and how quickly, an asset's price changes — fundamentally, a measure of *uncertainty.* If everyone were certain of an asset's true worth, it would trade in a very tight range around that value (buyers wouldn't pay above it, sellers wouldn't accept below it). Highly *uncertain* assets, by contrast, trade across a much wider range of prices. **Implied volatility** — the specific volatility value used inside the Black-Scholes-Merton formula — represents the market's own implied estimate of the underlying's volatility.

11.5.1 Worked Comparison: Oil ETF vs. Cow (Live Cattle) ETF

To make this concrete, consider historical data comparing an **Oil ETF** and a **Cow (Live Cattle) ETF** over a three-year window (Nov 2013 – Nov 2016):

- The **Oil ETF** is visibly much more volatile: its price not only trends downward more steeply (a steeper slope in its overall trend line) but also fluctuates much more sharply around that trend line on a day-to-day basis.
- The **Cow ETF**, by contrast, shows a gentler downward trend and smaller day-to-day fluctuations.

Quantifying this with standard deviation of daily returns:

- **Cow ETF:** mean price ≈ **$26.85**; standard deviation of daily returns ≈ **3.7**, or about **13.8%** of the mean. Since one standard deviation in a normal distribution captures **68.2%** of outcomes, this means about 68.2% of the Cow ETF's daily prices fell within roughly **$23.13 to $30.56**.
- **Oil ETF:** mean price ≈ **$13.58**; standard deviation of daily returns is much larger — about **7.5**, or **55.4%** of the mean. So about 68.2% of the Oil ETF's daily prices fell within a much wider band: roughly **$6.03 to $21.01** — a far broader (both in absolute dollar terms and in percentage terms) range than the Cow ETF.

Why volatility increases an option's value: because of how a call option works, its buyer only profits if the underlying finishes *above* the strike price — and they don't care *how far* below the strike it might have gone, since their loss is capped at the premium paid either way. But they care a great deal about *how far above* the strike the price ends up, since that directly determines their profit. As a result, a **more volatile underlying** — one more likely to swing to extreme high (or low) values — makes the call option **more valuable**, since it raises the chance of a large favorable move without increasing the cost of an unfavorable one beyond the already-capped premium.

11.5.2 Volatility as Both an Implied Input and a Standard Deviation

Technically, in the Black-Scholes-Merton model, **implied volatility is the annualized standard deviation of the asset's returns**, expressed as a decimal or percentage. The single symbol σ in the formula plays a dual role: it is simultaneously the market's *implied* volatility estimate and the *statistical standard deviation* of the (log-normal) distribution of possible future prices — because a more volatile asset, by definition, has a wider spread (standard deviation) of possible future returns.

11.6 High-Level, Intuitive Reading of the Formula

$$\boxed{C(S_t, t) = S_t\Phi(d_1) - Ke^{-r\tau}\Phi(d_2).}$$

Roughly speaking, this formula computes:
[**an investor's expected return**] minus [**the cost of the option**]

- $\mathbf{S_0 \cdot N(d_1)}$ — the "return" side: today's stock price, multiplied by a probability-like weighting $N(d_1)$ that is close to 0 if the stock is likely to finish below the strike price, and represents the stock's expected contribution to value if it finishes above the strike.
- $Ke^{-r(T-t)}\Phi(d_2)$ — the "cost" side: the strike price, **discounted back to its present value**, multiplied by the risk-neutral exercise-probability term $\Phi(d_2)$.

11.6.1 Discounting to Present Value

The factor $e^{-r(T-t)}$ discounts the strike price K back to today's dollars using continuous compounding. This adjustment reflects the return available from investing at the risk-free rate over the same period.

11.6.2 The Probability Terms

These probability weightings come from the standard normal probability distribution (a bell curve). The Black-Scholes-Merton model assumes that if you plotted the periodic (e.g., daily) returns of the underlying, the resulting distribution would be **normal** (bell-shaped) — and it further assumes future returns will continue to follow that same normal distribution.

N(d_1) is, roughly, equivalent to **N(z-score)** — the area under the standard normal bell curve up to a particular **z-score**, representing the (risk-adjusted) probability that the future price will finish above the strike price at expiration. Recall the standard definition of a z-score:

$z = (x - \mu)/\sigma$

Mapping this general definition onto the Black-Scholes d_1 formula:

$$d_1 = \frac{\ln(S_t/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

- The **numerator**, $\ln(S_0/K) + (r + \sigma^2/2)(T-t)$, plays the role of "$\mathbf{x} - \mu$": $\ln(S_0/K)$ is the *value above the strike price* being measured (in log terms), while $(r + \sigma^2/2)(T-t)$ acts as the **mean** of the underlying normal distribution — reflecting the risk-free rate plus the spread of possible option outcomes accumulated over time.
- The **denominator**, $\sigma\sqrt{(T-t)}$, plays the role of "σ" (the standard deviation) — here, a function of volatility scaled by the square root of time remaining.

Both **N(d_1)** and **N(d_2)** are, mechanically, just **calculations of area under this bell curve** up to a given point. If the curve represents the full normal distribution of probabilities for the option's outcome, then specifically **N(d_2) represents the percentage probability that the option will expire in the money** (i.e., that it will actually be exercised).

11.7 Worked Numerical Example

Problem: You're considering buying a European call option with the following parameters. Using the Black-Scholes-Merton formula, what is the theoretical fair price of this option — i.e., the maximum you should be willing to pay for it to be worth buying?

- Stock price (S_0): **\$50**
- Strike price (K): **\$45**
- Time to expiration: **80 days**
- Risk-free interest rate (r): **2%**
- Implied volatility (σ): **30%**

Step 1 — Compute d_1:

$$d_1 = \frac{\ln(50/45) + (0.02 + 0.3^2/2)(80/365)}{0.3\sqrt{80/365}}.$$

Here $\ln(50/45) \approx 0.105$, the drift contribution is approximately 0.014, and the denominator is approximately 0.140.

$d_1 \approx 0.851$

Step 2 — Compute d_2:

$$d_2 = d_1 - \sigma\sqrt{T-t} = 0.851 - 0.3\sqrt{80/365}.$$

$d_2 \approx 0.711$

Step 3 — Look up $N(d_1)$ and $N(d_2)$ from a standard normal z-score table:

- $\mathbf{N(d_1) = N(0.851) \approx 0.8023}$ ($\approx 80\%$)
- $\mathbf{N(d_2) = N(0.711) \approx 0.7611}$ ($\approx 76\%$)

Step 4 — Plug into the formula:

$$\begin{aligned} C_0 &= 50(0.8023) - 45e^{-0.02(80/365)}(0.7611) \\ &\approx 40.12 - 34.10 = \boxed{6.02}. \end{aligned}$$

Interpretation: The stock's expected contribution to value (weighted by the 80% probability term) is about \$40.12, out of a \$50 stock price. The strike price of \$45, discounted to its present value of about \$44.80, weighted by the 76% probability the option finishes in the money, contributes a "cost" of \$34.10. The difference — **\$6.02** — is the theoretical fair price of the call option: you should pay less than roughly \$6.02 for it to be worth buying.

11.8 Practice Problem: Backing Out Implied Volatility

The formula can also be run "in reverse": given a known market price for an option, solve for the implied volatility (σ) that the market must be assuming, since σ is the only input not directly observable in the market.

Problem setup: Suppose you want to know what the market thinks the implied volatility of the S&P 500 is today.

- Assume every other investor is using the Black-Scholes-Merton formula for pricing.
- The SPY ETF (which tracks the S&P 500) is priced today at **\$216**.
- A European call option has a strike price of **\$210**.
- There are **30 days** left until expiration.
- The risk-free interest rate is **1.8%**.
- The market is currently pricing this call option at **\$7.93**.

Question: what is the implied volatility?

Because σ cannot be isolated algebraically from the Black-Scholes formula, this type of problem is typically solved **numerically** — e.g., using a tool like Excel's "Goal Seek" feature to iteratively adjust σ until the formula's output matches the observed market price of \$7.93. (The candidate answer choices given alongside this practice problem were:

1.0%, 10.0%, 15.7%, 16.5%, 25.2%, 25.4%, and 50.6% — numerical root-finding gives $\sigma \approx 0.1647$, so the closest listed answer is 16.5%.)

11.9 Delta Hedging — Eliminating Risk in Practice

Once an option has been priced, the next practical step is to actually **hedge** the position by buying and selling the underlying asset in just the right proportions, so as to "eliminate risk." This is called **delta hedging** (or **dynamic hedging**) — the same mechanism formally derived in Section 2.4 above.

Goal: maintain a portfolio with **zero net delta** at all times, where delta (Δ) — one of the "option Greeks" (see previous notes, Section 10) — measures the sensitivity of the option's price to changes in the underlying asset's price. This is a fairly complex, continuous hedging exercise, principally carried out by large investment institutions (investment banks, hedge funds, private equity funds, etc.).

Formal definition:

$$\Delta = V_S = \frac{\partial V}{\partial S}.$$

— the first derivative of the option's value with respect to the underlying asset's price. In practice, delta hedging means continually buying or selling units of the underlying asset in response to changes in its price, in order to keep the *combined* position's sensitivity to price (∂S) effectively static at zero — even though the underlying's value itself is constantly changing.

Important clarification — what risk is (and isn't) being eliminated: delta hedging is designed to offset small continuous moves in the underlying. It does not remove volatility, gamma, transaction-cost, discrete-rebalancing, or jump risk; sharp price gaps are therefore among the risks that remain — the "tail" events in the return distribution where an asset's price jumps sharply in a single day and then partially or fully corrects in the days that follow. Also worth remembering: **risk is never actually fully "eliminated"** in practice — that's the theoretical goal of the model, but real hedged positions still carry residual risks that are harder to control for, such as **default risk** (the risk that a counterparty fails to honor the contract).

11.10 Criticisms of the Black-Scholes-Merton Model

The model, while foundational, has drawn substantial and well-documented criticism — most notably from **Nassim Nicholas Taleb** (author of the 2007 bestseller *"Black Swan"*, about unpredictable events in financial markets) together with **Espen Gaarder Haug**, who have argued the model is **"fragile to jumps and tail events"** and can only handle **"mild randomness."**

Specific, well-known challenges include:

- **Fragility to tail risk / extreme randomness:** in reality, asset returns do not perfectly follow a normal distribution. Statistical testing (the Anderson-Darling normality test) applied to S&P 500 returns yields a p-value of 0.000, indicating market returns are actually **leptokurtic** — more concentrated around the mean, but with **fatter tails** (more extreme outliers) than a true normal distribution would predict.

- **The model's structure doesn't reflect present-day market realities:** the standard model assumes a market of **European**-style call options, but most options actually traded today are **American**-style (exercisable at any time before expiration). It also does not natively account for **dividends**, which are common in real options markets.
- **The risk-free interest rate assumption is imperfect:** a truly theoretical risk-free rate is hard to pin down. In practice, investors substitute proxies such as a maturity-matched government or overnight-index-swap rate — but this treats Treasury bonds as genuinely "risk-free," when a more accurate characterization is that they're simply the investment the market currently regards as *least* risky.
- **The assumption of costless trading is unrealistic:** real trading involves exchange fees, transaction costs for buying/selling stocks and options, and time costs (the delay between placing an order and its execution can itself allow the market price to move). These costs can be managed in practice but are not built into the model itself.
- **Gap risk:** the model assumes continuous trading, but real markets close overnight (and on weekends/holidays) and can reopen at significantly different prices once new information arrives — a discontinuity the model does not capture.

Despite these issues: empirically, pricing discrepancies between Black-Scholes-Merton predictions and real market prices occur *more often* than would be expected if returns were truly log-normal. Nonetheless, the model **continues to be widely used** in practice — it is simple, computationally easy, well understood, and can be adjusted or extended to compensate for many of its inadequacies.

11.10.1 The Volatility Smile

One of the most common and visible criticisms of the model is the existence of the **volatility smile**. The Black-Scholes-Merton model assumes **constant volatility** and log-normal return distributions — implying that, for options at different strike prices (but the same underlying and expiration), the implied volatility used to price them should be roughly the same. In reality, implied volatility **varies substantially by strike price**. Specifically, options whose strike prices are **"deep in the money"** or **"far out of the money"** (i.e., strikes further away from the current underlying price) tend to command **higher prices** than a flat, constant-volatility assumption would predict — meaning their *implied* volatility, backed out from their market price, is higher. When implied volatility is plotted against strike price, the resulting curve is often U-shaped or "smile"-shaped (also discussed as the volatility skew in the previous notes, Section 11).

11.11 Advanced Technical Notes (Notation and Modeling Choices)

These points, while more advanced, help clarify exactly what the model is (and isn't) doing mathematically:

- **The notation used in finance is not strictly standard mathematical notation** — it is a convention specific to the finance industry.
- The functions $\Phi(d_1)$ and $\Phi(d_2)$ are values of the **standard normal CDF**; the stock price itself is lognormally distributed under GBM. This relies on an underlying

standard normal distribution (mean 0, standard deviation 1) that is *assumed* but often not explicitly stated.

- **Why a log-normal distribution is used at all:** the model is fundamentally describing **compound interest**, which behaves as a power law (multiplicative growth). Taking the **logarithm** of growth factors converts this multiplicative process into something nearly **linear**, and the resulting distribution of log-returns becomes approximately **normal**. The parameters μ and σ in this context represent the expected growth factor (akin to an interest rate) and the expected standard deviation (volatility) for one time period — so values close to 0 are typically expected for the log-return parameters.
- **Continuous functions are used to approximate what are really discrete, periodic processes** (e.g., dividends and interest are, in reality, usually paid periodically — quarterly, annually — not continuously). Using continuous compounding simplifies the math considerably, though this simplification is often left unstated.
- **What's fundamentally being modeled is a random one-dimensional walk (or martingale).** Since a binomial distribution converges to (approximates) a normal distribution over a large number of trials — such as many small price changes accumulated over a year — modeling the stock's price path as normally/log-normally distributed is a reasonable large-sample approximation of an underlying discrete, binomial-like process (this connects to the binomial option pricing model as an alternative/complementary framework).
- **A technical constraint:** because the model takes the logarithm of the stock price, that argument must always be a **positive real number** — this is required to avoid undefined results (infinities or complex numbers), which is automatically satisfied since stock prices can never be zero or negative.

11.11.1 The Payoff Functions, Formally

For a call option, the payoff at expiration is only positive if the underlying finishes above the strike (otherwise the option is simply not exercised, since it would represent an immediate loss):

$$H_{\text{call}}(S_T) = \max(S_T - K, 0).$$

Symmetrically, for a put option, the payoff is only positive if the underlying finishes below the strike:

$$H_{\text{put}}(S_T) = \max(K - S_T, 0).$$

The option's price today is then the **discounted, risk-neutral expectation** of this payoff — matching exactly the general master formula derived in Section 3.7 above, just written out with the specific call/put payoff functions substituted in.

11.11.2 Closed-Form via the Error Function (erf)

As an alternative, mathematically equivalent way of writing the standard normal CDF used throughout this model, some technical references express **N(x)** using the **error function**, erf:

$\Phi(x) = \frac{1}{2}\operatorname{erfc}(-x/\sqrt{2})$, where $\operatorname{erfc}(z) = 1 - \operatorname{erf}(z)$, and $\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-u^2}\, du$

Using this notation, the full closed-form Black-Scholes call and put pricing formulas can be rewritten entirely in terms of erf instead of N( ) — this is purely a notational variant (common in some symbolic-math and computational tools, such as Mathematica) and represents exactly the same underlying pricing formula given in Section 4, just expressed with a different (but mathematically interchangeable) special function.

11.12 Summary

The Black-Scholes equation can be reached via two complementary derivations: (1) building a continuously-rebalanced, delta-hedged riskless portfolio and applying no-arbitrage logic to derive a PDE that the option's price must satisfy, or (2) directly computing the discounted, risk-neutral expected payoff of the option, using Girsanov's theorem to justify swapping the real-world drift μ for the risk-free rate r. Both routes converge on the identical closed-form pricing formula. Understanding *why* the formula works — the probability interpretation of $N(d_1)$ and $N(d_2)$, the role of discounting, and the meaning of volatility (σ) as both an implied-market estimate and a statistical standard deviation — matters just as much as being able to compute it, especially given the model's well-documented real-world limitations (fat tails, the volatility smile, unrealistic frictionless-market assumptions) discussed in Section 10.

Chapter 12

Option Pricing

12.1 Options Pricing — Overview

12.1.1 What Is Options Pricing?

Options pricing is the process of determining the fair value of an option, which is a financial derivative contract. An option gives its holder the right, but not the obligation, to buy or sell an underlying asset (such as a stock) at a specific price (the **strike price** or **exercise price**) on or before a specific date (the **expiry/expiration date**).

12.1.2 The Goal of Options Pricing

The objective is to work out how much this "right" is worth today. This depends on six main inputs:

- The current price of the underlying stock
- The strike price of the option
- The time remaining until expiration
- The volatility of the underlying stock
- Prevailing interest rates
- Dividends paid by the underlying (if any)

12.1.3 Common Option Pricing Models

1. Black-Scholes Model

- Used for pricing European options on non-dividend-paying stocks.
- Assumes the underlying stock price follows a lognormal distribution.

2. Binomial Model

- Breaks the time to expiry into discrete intervals and models the up/down movement of the underlying price at each interval.
- Can handle American options (which are exercisable before expiry), unlike the basic Black-Scholes model.

12.1.4 Recommended Study Approach

1. Watch the topic video first.
2. Then work through the supporting blogs alongside the CFA curriculum reading on the basics of derivatives pricing and valuation.
    - Forwards and futures pricing reference: AnalystPrep — *Pricing and Valuation Concepts*
    - Options pricing reference: PrepNuggets — *Pricing and Valuation of Options*

12.2 Foundational Concepts in Derivative Pricing

These ideas underpin the pricing of every derivative — forwards, futures, swaps, and options. Understanding them first makes the binomial option pricing model much more intuitive.

12.2.1 The Core Intuition: A Simple Analogy

Imagine two people agree today (with just a handshake, i.e., no cash changes hands) to trade a specific item — say, an old putter worth $100 today (its **spot price**) — six months from now for an agreed price of $120 (the **forward price**). This is the simplest possible derivative contract: a **forward contract**.

- The person who agreed to **buy** the putter in the future is **long** the contract — they benefit if the item's price rises.
- The person who agreed to **sell** the putter in the future is **short** the contract — they benefit if the item's price falls.

If, before the six months are up, one party wants to exit ("break") the contract, the question becomes: **what is this contract worth right now?** The answer depends on how the spot price of the putter has moved relative to the agreed forward price. This is the essence of derivatives pricing and valuation — and the same logic scales up to gold, oil, stock indices, or anything else that can be bought or sold in the future.

12.2.2 Arbitrage

Arbitrage means buying an asset in one market and simultaneously selling it (or an equivalent) in another market at a different price, capturing a profit with **no risk**.

Conditions for genuine arbitrage:

- Requires **no net investment** of your own money.
- Involves **no risk**.
- Generates a **positive, riskless return**.

In real markets, arbitrage opportunities are rare and short-lived, because well-capitalized traders (often backed by large institutions, using algorithms) act on them instantly, causing the two prices to converge. This is why derivative prices in liquid markets rarely deviate from their no-arbitrage value for long.

12.2.3 Replication

Replication means constructing a portfolio of other assets that reproduces the exact payoff of a given asset or derivative.

Example: If you **own** the putter (long the spot asset) and simultaneously **agree to sell it** in the future via a forward contract (short the forward), the combination behaves exactly like an investment in a risk-free asset (e.g., a Treasury bill maturing on the same date as the forward). This is because:

- Owning the spot asset benefits you when prices rise (long).
- Being short the forward benefits you when prices fall.
- Combined, the position is insulated from price moves — it behaves like a riskless bond.

Replication is one of the two pillars (along with arbitrage) that pin down the *unique* no-arbitrage price of any derivative.

12.2.4 Risk Neutrality

At first this sounds strange — real investors are *not* indifferent to risk. However, decades of research have shown that when **pricing derivatives**, we can assume the underlying asset grows at the risk-free rate, and this "risk-neutral" assumption gives the correct derivative price. This works because the derivative's payoff can always be replicated using the underlying asset and a risk-free bond (arbitrage + replication), so the *actual* real-world probabilities of the asset going up or down turn out to be irrelevant to the derivative's price. Risk-neutral pricing is a *modeling convenience* built directly on the arbitrage and replication arguments above — not a claim that real investors don't care about risk.

Summary of the three pillars:

> We can form a risk-free portfolio consisting of the underlying asset and a derivative (forward, future, option, or swap) on it. This portfolio must earn the risk-free rate. There is only **one** derivative price consistent with that condition — and that price is the one implied by arbitrage and replication.

12.2.5 Value vs. Price — A Critical Distinction

This distinction is heavily tested and is central to every derivative pricing topic:

Point in time	Value of the derivative	Price of the derivative
At initiation (t = 0)	**Zero** (for forwards, futures, swaps — no cash/assets exchanged at signing, other than an option's premium)	The agreed forward/exercise price is set
During the contract's life (0 < t < T)	**Not zero** — changes as the spot price moves	The forward price agreed at initiation stays fixed
At maturity/expiration (t = T)	Equal to the difference between the spot price at expiry and the agreed forward/exercise price	N/A — contract settles

Options are the one exception at initiation: because the buyer pays the seller a **premium** up front, an option does have value (its time value) at t = 0, even though forwards/futures/swaps have zero value at initiation.

General valuation logic (forward contract, no interim cash flows on the underlying):

```
Value during life = Spot price at time t  -  PV(Forward price agreed at t=0)
                  = St - F0(T) / (1+r)^(T-t)
```

This is exactly analogous to bond and stock pricing — **the value of a derivative is always a present value calculation.**

At maturity, whichever party is "in the money" collects the difference between the spot price and the agreed price. In practice, most derivatives are **cash-settled** rather than requiring physical delivery of the underlying asset.

12.2.6 Monetary vs. Non-Monetary Benefits (and Costs) of Holding the Underlying

- **Monetary benefits**: coupon payments (bonds) and dividend payments (stocks) — cash received simply for owning the underlying asset. You do **not** receive these if you only hold the derivative instead of the asset itself.
- **Convenience yield**: a *non-monetary* benefit that accrues to whoever physically holds the underlying asset (rather than a paper claim on it) — e.g., being able to immediately capture value if the item's spot price unexpectedly spikes, or (for commodities) not being caught short of an input needed for production.
- **Costs of holding the underlying**: storage costs, insurance costs, and other opportunity costs incurred by physically holding the asset until the forward's maturity.

12.2.7 The Cost-of-Carry Model

Because benefits (dividends, interest, convenience yield — denoted γ, "gamma") and costs (storage, insurance, financing — denoted θ, "theta") affect how expensive it is to "carry" the underlying asset to the future delivery date, they must be built into the forward price:

- **Costs increase** the forward price.
- **Benefits decrease** the forward price.

Formula (costs/benefits expressed as currency amounts, compounded to maturity):

```
F0(T) = [S0 - (gamma - theta)] x (1 + rf)^T
```

Worked example: Spot = \$55, net cost of carry = +\$3 (net cost, i.e., costs exceed benefits), risk-free rate = 3%.

```
F0(T) = (55 - 3) x (1.03)^1 = $53.56
```

Note that here the forward price (\$53.56) is actually **below** the spot price (\$55) — there is no rule that forward prices must exceed spot prices. Commodities typically have forward prices *above* spot (since storage costs dominate and there's little "dividend-like" benefit to holding a physical commodity), but for financial assets like equity indices the relationship can go either way depending on the balance of dividends vs. financing costs, and can flip during periods of market stress (e.g., 2020 crash) or high inflation (e.g., 2022).

12.3 Forward and Futures Contracts — Detailed Pricing and Valuation

Deeper, formula-driven treatment of forward/futures pricing — the natural stepping stone before options and the binomial model, since options pricing reuses the same no-arbitrage logic.

12.3.1 Key Definitions

- **Forward commitment**: a derivative contract that obligates one party to buy, and the other to sell, an underlying asset at a predetermined price at a future date.

- **Price** of a forward/futures contract = the prespecified price the long pays the short at expiration/settlement.
- **Value** of a forward/futures contract = its current monetary worth, which changes after inception as the spot price moves.

12.3.2 The No-Arbitrage Approach

Two rules define arbitrage-free pricing:

1. **Do not use your own money** — the arbitrageur borrows to buy the underlying, and invests proceeds from any short sale at the risk-free rate.
2. **Do not take any price risk** — all market price risk from the underlying and the derivative must be eliminated (fully hedged).

No-arbitrage principle: it is impossible to generate riskless profit today with no future liability. This implies:

- **Law of one price**: two portfolios with identical future cash flows (regardless of what happens) must have the same price today.
- A portfolio that generates a certain (riskless) payoff must earn exactly the risk-free rate.

Illustrative example: An investor goes short a September E-Mini S&P 500 futures contract on 30 October 2020 at \$3,225 (initial value = \$0, as expected). The next trading day, the futures price rises to \$3,324. The short position's value is now **−\$99** (the short loses as the price they're obligated to sell at falls behind the market).

12.3.3 Forward Pricing (No Interim Cash Flows)

The forward price is the price that prevents riskless arbitrage in a frictionless market — it equals the spot price compounded forward at the risk-free rate over the contract's life:

```
F0(T) = S0 x (1 + rf)^T          or equivalently          S0 = F0(T) / (1 + rf)
   ^T
```

Where: F0(T) = forward price; S0 = spot price at contract initiation; rf = risk-free rate; T = term of the contract.

Example: Asset priced at \$130, risk-free rate 4% annually. One-year forward price:

```
F0(1) = 130 x (1.04)^1 = $135.20
```

12.3.4 Forward Mispricing and Arbitrage

An arbitrage opportunity exists whenever `F0(T)` $\neq$ `FV(S0)`, where `FV(S0) = S0` $\times$ `(1+rf)^T`.

- If **F0(T) > FV(S0)** → the forward is **overpriced**. The arbitrageur **shorts the forward and longs the underlying** ("carry arbitrage"). This drives the forward price down and the spot price up until they converge.
- If **F0(T) < FV(S0)** → the forward is **underpriced**. The arbitrageur **buys the forward and shorts the underlying** ("reverse carry arbitrage"), driving the forward price up to equal the future value of the underlying.

a) Carry Arbitrage (forward overpriced)

Example: Asset priced at \$130, risk-free rate 4%. Quoted forward price F0(T) = \$140 (overpriced, since fair value is \$135.20).

Cash flows at initiation:

- Short the forward contract at \$140.
- Borrow +\$130 at the risk-free rate.
- Purchase the underlying using the loan: −\$130.
- Borrow the (discounted) arbitrage profit:

```
PV[F0(T) − S0(1+rf)^T] = PV[$140 − $135.20] = $4.8 / 1.04 = +$4.615
```

- **Initial cash flow = −\$130 + \$130 + @@I0@@4.62** (riskless profit today).

Cash flows at time T:

- Deliver the underlying to the short-forward counterparty, receive +\$140.
- Repay the loan: (130 + 4.62) × 1.04 = −\$140.
- **Net cash flow at T = −\$140 + @@I0@@0.**

The arbitrageur pockets \$4.62 today and has zero net exposure at T — a genuine riskless profit.

b) Reverse Carry Arbitrage (forward underpriced)

Example: Asset priced at \$130, risk-free rate 4%. Quoted forward price F0(T) = \$115 (underpriced).

At contract initiation:

- Long the forward contract at \$130... *(fair value benchmark)* — actually enter long forward at the quoted underpriced rate.
- Short-sell the underlying at \$130.
- Lend the \$130 proceeds from the short sale.
- Borrow the arbitrage profit: `PV[FV(S0) − F0(T)] = PV[$135.20 − $130] = $5.20/1.04 = +$5.00`
- **Cash flow at time 0: +\$130 − \$130 + @@I0@@5.00.**

Cash flows at time T:

- Collect the loan: (130)(1.04) = +135.20.
- Repay the borrowed arbitrage profit: −5.00(1.04) = −5.20.
- The long forward position delivers −\$130 to acquire the underlying.
- Use the underlying to cover (close out) the short position.
- **Net cash flow at T: +\$135.20 − \$5.20 − @@I0@@0.**

Again, riskless profit (\$5.00) is captured today, with zero net exposure later.

12.3.5 Valuing a Forward Contract (No Cash Flows on Underlying)

At initiation (t = 0), the value to the **long** position:

```
V0 = S0 - F0(T)/(1+rf)^T          (this equals $0 by construction)
```

During the contract's life, the value to the long position is the discounted difference between the *current* forward price for a matching maturity and the *original* forward price locked in:

```
Vt(T) = [Ft(T) - F0(T)] / (1+i)^(T-t)
```

Example: Long one-year forward purchased at $130. Three months later, the contract's (new) price is $132. Risk-free rate = 4%.

```
Vt(T) = ($132 - $130) / (1.04)^(9/12) = $1.9420
```

12.3.6 Equity Forward Contracts

With Discrete Cash Flows (dividends)

Dividends (γ, carry benefit) and costs like interest/storage (θ, carry cost) adjust the forward pricing formula. Costs *increase* the burden of carry (added); benefits *decrease* it (subtracted):

```
F0(T) = [S0 - (gamma - theta)] x (1 + rf)^T
```

Example 1: Stock priced at $50, expected to pay a $1 dividend in 30 days and another $1 in 90 days. Interest rate = 4%. Price a 70-day forward:

- Only the 30-day dividend falls within the 70-day contract term.
- γ = 1 × (1.04)^(−30/365) = $0.9968
- F0(T) = [$50 − ($0.9968 − 0)] × (1.04)^(70/365) = $49.37

Example 2: Spot = $100, asset pays a $2 dividend in 6 months, risk-free rate = 8%. Price the one-year forward:

- γ = 2 × (1.08)^(−6/12) = $1.9245, θ = 0
- F0(T) = [100 − (1.9245 − 0)] × (1.08)^1 = $105.92

With Continuous Cash Flows

For stock indices with many constituent stocks (e.g., S&P 500), dividends are typically modeled as continuously compounded:

```
F0(T) = S0 x e^[(rc + theta - gamma) x T]
```

Where rc = continuously compounded risk-free rate.

Example: Dow Jones (DJ30) spot = $30,125. Continuous dividend yield = 3%. Continuously compounded risk-free rate = 2%. Six-month forward price:

```
F0(T) = 30,125 x e^[(0.02 + 0 - 0.03) x 0.5] = $29,974.75
```

Equity Forward Valuation (Discrete Payments)

```
Vt(T) = St - (gamma - theta)(1+r)^t - F0(T) x (1+r)^(-(T-t))
```

Example: Stock priced at $50, expected to pay dividends of $1 in 30 days and 90 days, risk-free rate = 4%. After 40 days, the stock price is $40. Value the forward to the long position:

Step 1 — Recompute the (original) forward price using only the dividend that falls within the contract's term (30-day dividend; the 90-day dividend arrives after the 70-day contract already expired):

```
gamma = 1 x (1.04)^(-30/365) = 0.9968
F0(T) = [$50 - 0.9968] x (1.04)^(70/365) = $49.37
```

Step 2 — Value the contract 40 days in (30 remaining days to expiry):

```
Vt(t) = 40 - 49.37/(1.04)^(30/365) = -$9.21
```

The forward's value to the **long** is $-$**\$9.21**; correspondingly, its value to the **short** is $+$**\$9.21**.

12.3.7 Forward and Future Contracts on Fixed-Income Bonds

The mechanics mirror dividend-paying stocks, except the periodic cash flows are **coupon payments** rather than dividends.

Price of a forward on a fixed-income security:

```
F0(T) = (S0 - PVC0) x (1 + rf)^T
```

Where PVC0 = present value of coupon payments expected before the forward expires.

Example: Forward contract duration = 200 days. Bond: \$100 par, 4% coupon (semi-annual), spot price = \$110, risk-free rate = 8%, next coupon in 160 days.

```
Coupon payment = 4% x 0.5 x 100 = $2
PV(coupon) = 2 / (1.08)^(160/365) = $1.93
Forward Price = (110 - 1.93) x (1.08)^(200/365) = $112.72
```

Value of a fixed-income forward at time t:

```
Vt(Long) = (St - PVCt) - F0(T)/(1+rf)^(T-t)
```

or, if given a current forward price Ft(T) with the same maturity:

```
Vt(Long) = PV(t,T)[Ft(T) - F0(T)]
```

Example: \$100 par, 4% semi-annual coupon bond, spot = \$100, matures in 200 days, just paid a coupon (next coupon due in 160 days). After 120 days, the bond's value is \$127. Risk-free rate = 8%. Value the forward to the long:

- Only one coupon (due in 40 more days, i.e., 160−120) remains before the contract's maturity (in 80 more days, i.e., 200−120).

```
PVCt = 2 / (1.08)^(40/365) = $1.9832
V120(Long) = $127 - 1.9832 - 112.72/(1.08)^(80/365) = $14.18
```

12.3.8 Futures Contracts on Fixed-Income Bonds

Bond prices are often quoted **clean** (excluding accrued interest since the last coupon):

```
Clean price = Full price - Accrued Interest (AI)
```

```
AI = (Days since last coupon / Days between coupons) x Coupon amount
```

Because multiple bonds may be eligible for delivery against a bond future, a **cheapest-to-deliver (CTD)** bond emerges once prices are adjusted by each bond's **conversion factor (CF)**. The buyer pays the futures price × the conversion factor at expiration:

```
F0(T) = QF0(T) x CF(T)
```

where QF0(T) is the *quoted* futures price.

Price of a bond futures contract (carry arbitrage model — futures price equals the future value of the underlying, net of carry costs/benefits):

```
F0(T) = FV0,T(S0) - AIT - FVCI0,T
```

The conversion-factor-adjusted quoted futures price:

```
QF0(T) = [1/CF(T)] x (FV0,T[B0(T+Y) + AI0] - AIT - FVCI0,T)
```

Where AIT = accrued interest at futures maturity; FVCI0,T = future value of any coupons received and reinvested during the futures' life.

Example: Six-month Euro bond futures, contract value €50,000. Underlying: 5% bond quoted at €110, accrued interest of €1.00. No coupon payments occur before expiration. Conversion factor = 0.65, accrued interest at expiration = €3.00, risk-free rate = 4%.

```
T = 6/12,  CF(T) = 0.65,  B0(T+Y) = EUR 110,  FVCI0,T = 0,  AI0 = EUR 1.00,
   AIT = EUR 3.00,  r = 4%

QF0(T) = [1/0.65] x [(110 + 1) x (1.04)^0.5 - 3 - 0] = EUR 169.54
```

Value of a bond future: because futures are marked to market daily, their value resets to **zero** at the end of each trading day once gains/losses are settled — the value only accumulates *within* a single day (the change in price since the previous settlement).

12.3.9 Interest Rate Forward and Futures Contracts

- **LIBOR** (London Interbank Offered Rate): the classic reference rate for interbank borrowing, historically quoted as an annualized rate on a 360-day-year basis.
- **Eurodollar deposits**: deposits at banks outside the US, denominated in USD.

Forward Rate Agreements (FRAs)

An FRA is a forward contract on an interest rate (e.g., LIBOR). The counterparties agree today on a fixed interest payment, to be exchanged at a future date for an unknown (floating) interest payment.

- **Short (fixed receiver / "lender")**: receives the fixed rate, pays the floating rate.
- **Long (floating receiver / "borrower")**: receives the floating rate, pays the fixed rate — benefits when rates rise.
- FRAs are **cash-settled** — no principal is actually lent or borrowed.

Notation — "X × Y" convention: X = months until the FRA expires (and the underlying loan starts); Y = months until the underlying loan ends.

Example: a **6 × 9 FRA**

- Expires in 6 months.
- The underlying loan settles in 9 months (i.e., runs for a further 3 months after the FRA expires).
- Notional loan period = 3 months.
- Payoff is based on 3-month LIBOR observed when the FRA expires (at month 6).

Equivalently written **FRA(0, h, m)**: fixed forward rate set at time 0, expiring at time h, with the underlying LIBOR deposit maturing m days later. A 6×9 FRA = FRA(0, 180, 90).

FRA settlement conventions:

- **Advanced set**: the reference rate is set (observed) at the FRA's expiration date h — this is when the underlying loan notionally begins.
- **Advanced settled**: the cash settlement itself occurs at time h.
- **Settled in arrears**: interest is paid at time h + m (when the underlying loan would have matured).
- FRAs are usually **advanced set, advanced settled**; swaps and interest rate options are usually **advanced set, settled in arrears**.

Pricing an FRA — find the fixed rate that gives the FRA zero value at inception:

```
FRA(0,h,m) = { [1 + L0(h+m) x t(h+m)] / [1 + L0(h) x t(h)]  -  1 } / tm
```

This is read as: the compounded value of $1 invested at the longer-term LIBOR for (h+m) days, divided by the compounded value of $1 invested at the shorter-term LIBOR for h days, minus 1, then annualized.

Steps to price an FRA:

1. Lay out a timeline (0 → h → h+m).
2. De-annualize the fixed rate: `[1+L0(h+m)t(h+m)] / [1+L0(h)t(h)] − 1`
3. Annualize by multiplying by `360/m`.

Example: Current 30-day LIBOR = 2%, 180-day LIBOR = 3.5% (30/360 convention). Price the 1×6 FRA:

```
FRA(0,30,150) = { [1+0.035x(180/360)] / [1+0.02x(30/360)]  -  1 } / (150/360)
   = 3.79%
```

Valuing an existing FRA — done by comparing it to an *offsetting* FRA priced at the *current* market rate for the same remaining exposure:

```
Vg(0,h,m) = { [FRA(g,h-g,m) - FRA(0,h,m)] x tm } / [1 + Dg(h+m-g) x t(h+m-g)]
```

Process:

1. Compute the new FRA rate as of "today" (time g): `FRA(g,h−g,m)`.
2. Compute the payoff at $h + m$:

$$\text{Payoff} = (\text{FRA}_g - \text{FRA}_0)\left(\frac{m}{360}\right) \times \text{Notional}.$$

3. Discount that payoff back to time g.

Worked example: ABC Bank entered a 6×9 FRA three months ago as the **receive-floating** party. FRA rate = 0.75%, notional = $300,000. Now (3 months later, so g = 90 days): 90-day USD LIBOR = 0.80%, 180-day LIBOR = 0.85%. At the *original* FRA's expiration, 3-month LIBOR = 1.00%, 6-month LIBOR = 1.10%. Discount rate for settlement cash flows = 1%. (Note: since the FRA was entered 3 months ago and is a 6×9, it now has 3 months left until it expires.)

Step 1 — compute the new FRA rate, FRA(90, 90, 90):

```
FRA(90,90,90) = { [1+0.0085x(180/360)] / [1+0.0080x(90/360)]  -  1 } x
    (360/90) = 0.8982%
```

Step 2 — payoff at h+m:

```
Payoff = (0.8982% - 0.75%) x (90/360) x $300,000 = $111.15
```

Step 3 — discount back to time g (90 days), at 1%:

```
V(0,180,90) = $111.15 / [1 + (0.01 x 180/360)] = $110.60
```

12.3.10 Currency Forward and Futures Contracts

Foreign currency is quoted in units of the **domestic** currency. The **covered interest rate parity** relationship is the carry-arbitrage model applied to FX — it gives the no-arbitrage forward price of one unit of foreign currency (F), in terms of the domestic currency (D):

```
FT(D/F) = S0(D/F) x [(1+rD)^T / (1+rF)^T]
```

Continuous-compounding version:

```
FT(D/F) = S0(D/F) x e^[(rD - rF) x T]
```

Convention: if the spot quote is expressed as *base currency per unit of quote/price currency*, the base-currency interest rate is the numerator and the quote-currency rate is the denominator (e.g., in GBP/USD, GBP is the base and USD is the quote currency).

Example: Spot = $0.06757 per South African Rand (ZAR). US risk-free rate = 4%, South African risk-free rate = 6%. Price a 90-day forward (365-day year):

```
FT(USD/ZAR) = 0.06757 x [(1.04)^(90/365) / (1.06)^(90/365)] = $0.067
```

Valuing a currency forward after inception:

```
Vt(T) = PVt,T x [Ft(T) - F0(T)]
```

Note: the present value must be computed using the **quote (price) currency's** interest rate.

Example: A trader bought A$500,000 against USD at F0($/A) =0.76. Four months to expiry remain; the current forward price is Ft($/A) =0.70. USD rate = 1.5%, AUD rate = 1%. Value the long position:

```
Vt(T) = (0.70 - 0.76) / (1.015)^(4/12) = -$0.0597 per AUD
Total value = -($0.0597/A$) x A$500,000 = -$59,702.97
```

(Note: the source example scales this to a notional of A$1,000,000 in its final line, giving −$59,702.97 — the value is negative to the long trader because the AUD has weakened relative to the contracted forward rate.)

12.3.11 Futures Contracts — General Note

The same forward pricing and valuation formulas generally apply to futures:

```
Forward pricing:    F0(T) = FV0,T(S0 + theta0 - gamma0)
Forward valuation:  Vt(T) = PVt,T[Ft(T) - F0(T)]
```

The key structural difference: futures are **marked to market daily**, so a futures contract's value resets to **zero** at the end of each trading day (gains and losses are settled in cash daily), whereas a forward's value simply accumulates until the contract is closed out or matures.

12.3.12 Practice Questions (Forward Valuation)

Q1. An investor purchased a one-year Eurobond forward contract six months ago with a notional value of €50,000 at a price of 100. The current forward price for a contract with six months remaining to maturity is 110. Risk-free rate = 3%. Find the value of the forward position.

Solution:

```
Vt(T) = PVt,T[Ft(T) - F0(T)] = (110 - 100) x (1.03)^(-0.5) = 9.853 per EUR 100
    par
Position value = 0.09853 x EUR 50,000 = EUR 4,926.50
```

Q2. XYZ Inc. sold $50,000 forward against EUR at a forward rate of EUR 0.78 per $1 at time 0. At time t, the spot rate is EUR 0.72 per $1; annually compounded risk-free rates are 2% (EUR) and 5% (USD). At time t, the forward contract has three months to expiration. Find the forward price at time t.

Solution:

```
FT(EUR/USD) = S0(USD) x [(1+rEUR)^T / (1+rUSD)^T]
            = 0.7200 x [(1.02)^(3/12) / (1.05)^(3/12)] = 0.7148
```

12.4 Options — Basic Concepts, Moneyness, and Valuation

12.4.1 Recap of Basic Option Concepts

- A **call option** gives the holder the **right, but not the obligation, to buy** the underlying asset at a given (exercise/strike) price.
- A **put option** gives the holder the **right, but not the obligation, to sell** the underlying asset at a given price.
- Options can be:
  - **American-style**: exercisable **at any time** up to and including the expiration date.
  - **European-style**: exercisable **only on** the expiration date.

Four basic option positions:

Position	Right/Obligation
Long Call	Has the right to **buy** at a given price (pays the option premium)
Short Call	**Required to sell** at a given price if the long exercises
Long Put	Has the right to **sell** at a given price (pays the option premium)
Short Put	**Required to buy** at a given price if the long exercises

The buyer of an option always pays a **premium** to the seller (writer) at initiation for the right they are acquiring. The Level I curriculum focuses primarily on **European** options.

12.4.2 Moneyness

Even though a European option can't be exercised early, it still has value before expiration — this concept is called **moneyness**:

- **In-the-money (ITM)**: immediate exercise would generate a **positive payoff**.
- **Out-of-the-money (OTM)**: immediate exercise would generate a **negative payoff** (so rationally, you wouldn't exercise).
- **At-the-money (ATM)**: the spot price equals the exercise price — immediate exercise generates **no payoff**.

Summary of moneyness by position (St = spot at time t, PV(X) = present value of strike):

	Condition	Moneyness
Call	St > PV(X)	In-the-money
Call	St = PV(X)	At-the-money
Call	St < PV(X)	Out-of-the-money
Put	St > PV(X)	Out-of-the-money
Put	St = PV(X)	At-the-money
Put	St < PV(X)	In-the-money

Value at expiration (T):

```
Call:  CT = max(0, ST - X)
Put:   PT = max(0, X - ST)
```

Where X = exercise/strike price, ST = spot price at expiration.

12.4.3 Intrinsic Value and Time Value

An option's total value (its premium) is the sum of two components:

```
Option Value (Ct)  =  Intrinsic Value  +  Time Value (speculative value)
```

Intrinsic value — the value the option would have if exercised right now. Importantly, under the current CFA curriculum (a change from the pre-2023 treatment), intrinsic value

before expiration is based on the **present value of the strike price** (discounted at the risk-free rate), not the raw strike price:

```
Intrinsic value (call) = max(0, St - PV(X))
Intrinsic value (put)  = max(0, PV(X) - St)
where PV(X) = X / (1+Rt)^(T-t)
```

(Before 2023, the curriculum defined intrinsic/exercise value before expiration *without* this time-value-of-money discounting — this is a nuance worth remembering for exam purposes.)

Worked example: A one-year European put on ABC stock, strike = \$80. Three months later, ABC trades at \$78; risk-free rate = 5%. Find the intrinsic value.

```
PV(X) - St = 80/(1.05)^0.75 - 78 = -$0.87
```

Since the intrinsic value to the long position can never be negative, the intrinsic value here is **\$0** (the put is technically out-of-the-money on a present-value basis, even though the raw spot (\$78) is below the raw strike (\$80)).

Time value (speculative value): the remainder of the premium beyond intrinsic value — it reflects the *possibility* that the option becomes more valuable before expiration. Options (unlike forwards/futures/swaps) have **positive value at initiation** — this initial value is pure time value (since there's no intrinsic value yet if at-the-money). As the contract progresses, time value decays, and **at expiration, the option's entire value is its intrinsic value** (time value = 0 at T).

12.4.4 Theoretical Bounds for Option Values

At any time, option values are bounded as follows:

	Lower Bound	**Upper Bound**
Call option	Intrinsic value	Spot price
Put option	Intrinsic value	Strike price

Worked example: XYZ stock trades at \$120. A call option with 3 months to expiration and strike \$105 trades at \$18. Risk-free rate = 4%.

(a) Time value of the option:

```
Et = max(0, St - PV(X)) = 120 - 105/(1.04)^0.25 = $16.02
Time value = Ct - Et = $18 - $16.02 = $1.98
```

(b) Theoretical bounds:

```
Lower bound (Ct) = Intrinsic value Et = $16.02
Upper bound (Ct) = Spot price St = $120
```

12.4.5 Six Key Factors Affecting European Option Values

The first two factors affect **intrinsic value**; the remaining four affect **time value**.

1. Spot Price

- Call options: higher spot price → greater intrinsic value → higher option value.

- Put options: higher spot price → lower option value.

2. **Strike Price**
   - Call options: higher strike price → lower intrinsic value.
   - Put options: higher strike price → higher intrinsic value.
3. **Time to Expiration**
   - Generally, longer time to expiration → higher time value for both calls and puts.
   - Shorter time to expiration → lower time value.
   - Caveat: this relationship reliably holds for calls, but may **not always hold for European puts** in certain exceptional cases.
4. **Risk-Free Rate**
   - Higher risk-free rate → higher value for European calls, lower value for European puts.
   - Lower risk-free rate → lower value for European calls, higher value for European puts.
5. **Volatility of the Underlying**
   - Higher volatility → higher time value (for both calls and puts) — more volatility means a greater chance of a large favorable move, and losses are capped at the premium either way.
   - Lower volatility → lower time value.
6. **Costs and Benefits of Holding the Underlying Asset**
   - Call options: expected **costs** of holding the underlying (e.g., storage) **increase** the call's value; expected **benefits** (e.g., dividends) **decrease** it.
   - Put options: expected **costs decrease** the put's value; expected **benefits increase** it.

Intuition: a call option holder effectively "misses out" on carrying costs the actual asset owner would pay, so those costs get built into a higher call premium as compensation to the seller/reflection of forgone drag; conversely the call holder also misses out on the benefits the owner enjoys, which reduces relative call value. The logic mirrors, in reverse, how costs/benefits shift the forward price up or down in Section 2.7.

12.5 Put-Call Parity

12.5.1 The Replication Argument

Put-call parity is a direct application of the **replication** principle (Section 2.3) to options.

Strategy 1 — Protective Put: own the underlying asset (long, benefits when prices rise) **and** buy a put option on it (long put, benefits when prices fall). Together, this hedges the position — you own the asset but are protected against downside.

```
Value at inception = P0 + S0
```

Strategy 2 — Fiduciary Call: buy a call option (benefits when prices rise) **and** buy a risk-free zero-coupon bond whose maturity value equals the options' exercise price.

```
Value at inception = C0 + PV(Bond)
```

If the call and put share the same exercise price and expiration, and the bond's maturity value equals that exercise price, then **these two portfolios produce identical payoffs at expiration** under every possible future scenario. By the law of one price, they must therefore have the same value (cost) today:

```
C0 + PV(X) = P0 + S0
```

This equation is put-call parity.

12.5.2 Rearranging: Synthetic Positions

Because put-call parity is an equality, it can be rearranged to construct any of the four components synthetically from the other three:

```
Synthetic Call:  C0 = P0 + S0 - PV(X)     (buy put + buy stock + sell/short a
   risk-free bond)
Synthetic Put:   P0 = C0 - S0 + PV(X)
Synthetic Bond:  PV(X) = C0 - P0 + S0 (rearranged)
```

An **arbitrage opportunity is created whenever put-call parity does not hold** — i.e., whenever the actual market prices of C, P, S, and the bond deviate from this equality.

12.5.3 Worked Example

Given: exercise price X = \$50 (same for both put and call), option expires in 120 days, underlying asset price S0 = \$52, risk-free rate = 4.5%, put price P0 = \$3.80. Find the call price.

```
C0 = P0 + S0 - PV(X)
   = 3.80 + 52 - 50/(1.045)^(120/365)
   = $6.52
```

If the actual quoted call price differs from \$6.52 (e.g., \$7.00), an arbitrage opportunity exists — the mispricing can be captured either by trading directly in whichever leg is mispriced, or by constructing the full replication strategy.

12.5.4 Put-Call-Forward Parity

The same relationship can be restated by substituting a **forward contract** in place of the underlying stock position:

```
C0 + PV(X) = P0 + PV(F0(T))
```

By setting the call equal to the synthetic protective-put-on-a-forward, we get **put-call parity for options on forward contracts** — the same core idea, one layer removed (an option on a forward is, in a loose sense, a "derivative of a derivative").

Chapter 13

Binomial Model

13.6 The Binomial Option Pricing Model — Core Theory

13.6.1 What Is the Binomial Model, and Why Use It?

Valuing options is inherently difficult: unlike a bond or a stock, an option's value depends on several interacting variables — the underlying price, strike price, time to expiration, interest rates, and (critically) volatility — combined with genuine uncertainty about the future. The **binomial model** tackles this by radically simplifying the problem:

> At any given moment/step, the underlying price can only do one of **two** things: move **up** by some factor, or move **down** by some factor.

This single simplification — repeated over many small discrete time steps — builds up a **binomial tree**: a branching diagram of every possible path the underlying's price could take between now and the option's expiration. Although reality is more continuous and complex, chaining enough small up/down steps together lets the tree closely approximate real-world price behavior.

Developed by **John Cox, Stephen Ross, and Mark Rubinstein in 1979**, the binomial model offered an alternative to the **Black-Scholes model** (Fischer Black and Myron Scholes, 1973). Black-Scholes uses continuous-time mathematics and closed-form equations, but:

- It assumes the underlying follows a smooth, continuous price path.
- It assumes volatility stays **constant** over the option's life.
- It is designed for **European** options and struggles to handle the early-exercise feature of **American** options.

The binomial model addresses these limitations directly: it is built from **discrete** steps (so it can naturally check, at every step, whether early exercise is optimal — making it well-suited to American options), and it can accommodate **changing volatility** over time (different u/d factors at different steps, if desired).

13.6.2 Key Inputs and Notation

Symbol	Meaning
S (or S0, X)	Current stock/spot price
K (or X)	Strike/exercise price of the option
T	Time to expiration (in years)
N	Number of discrete time steps in the tree

Δt	Length of each step = T / N
r (or Rf)	Risk-free interest rate
u	"Up" factor — the multiple by which the price rises in an up move (u > 1); commonly modeled as u = $e^{\sigma\sqrt{\Delta t}}$, where σ = volatility
d	"Down" factor — the multiple by which price falls (0 < d < 1); often set as d = 1/u
p (or π, or q)	**Risk-neutral probability** of an up move (different course materials use different letters for the same concept — see note below)

Important note on notation across sources: different instructors/materials in this course use different symbols for the same risk-neutral probability (π, p, q) and the same growth factor (1+Rf, R, e^(rt)). Throughout this section, whichever symbol a given worked example uses is kept consistent *within* that example, and the underlying concept is always the same: **the probability under which the underlying's expected growth exactly equals the risk-free rate.**

This risk-neutral probability does **not** represent real-world (subjective) odds of the price going up or down. It is a purely theoretical construct, engineered specifically to enforce **no-arbitrage pricing** — the bedrock principle of modern derivatives theory (Section 2.2–2.4).

13.6.3 The One-Step Binomial Tree — Setup

Consider a stock priced at **S0** today. Over one period (e.g., one year), assume the stock can only move to one of two nodes:

```
                    ????  Su  = S0 x u     (probability p)
       S0  ??????????
                    ????  Sd  = S0 x d     (probability 1 - p)
```

For a **European call option** with strike K (assume Sd < K < Su, i.e., the strike sits between the two possible future prices):

```
Payoff if price goes up:    Cu = max(Su - K, 0)
Payoff if price goes down:  Cd = max(Sd - K, 0)   [often 0, if Sd < K]
```

13.6.4 Deriving the Risk-Neutral Probability (Full Algebra)

We want the underlying, growing at the risk-free rate under the risk-neutral measure, to have an *expected* value equal to what compounding at (1 + risk-free rate) would give:

```
S0 x R  =  p x Su  +  (1 - p) x Sd          where R = 1 + Rf
```

Substitute Su = u × S0 and Sd = d × S0:

```
S0 x R = p x u x S0 + (1 - p) x d x S0
```

S0 cancels from every term:

```
R = pxu + (1-p)xd
```

Expand:

```
R = pxu + d - pxd
```

Isolate the p terms:

```
R - d = pxu - pxd = p(u - d)
```

Solve for p:

```
p = (R - d) / (u - d)
```

This is the master formula for the risk-neutral "up" probability in the discrete one-step binomial world (R = 1 + risk-free rate). The probability of a "down" move is simply (1 − p).

13.6.5 Worked Numeric Example (One-Step Tree, Discrete Compounding)

Stock priced at 100 today. Strike (K) = 110. Expected up level = 120 → **u = 1.2**. Expected down level = 80 → **d = 0.8**. Risk-free rate = 8% → **R = 1.08**.

Step 1 — risk-neutral probability:

```
p = (R - d)/(u - d) = (1.08 - 0.8)/(1.2 - 0.8) = 0.28/0.4 = 70%
```

Step 2 — call payoffs at each node:

```
If price rises to 120: Call value = 120 - 110 = 10
If price falls to 80:  Call value = 0  (expires worthless -- out of the money)
```

Step 3 — probability-weighted expected payoff:

```
Expected payoff = p x 10 + (1 - p) x 0 = 0.70 x 10 = 7
```

Step 4 — discount to present value at the risk-free rate:

```
Call value today = 7 / 1.08 = 6.481   (~ $6.48)
```

13.6.6 Alternative Method: The Replicating Portfolio (Delta-Hedging) Approach

Instead of computing a risk-neutral probability, we can directly build a portfolio of the **underlying stock** and a **risk-free bond** that exactly replicates the option's payoff in both the up and down states. Since the portfolio and the option have identical payoffs everywhere, they must have the same value today (no-arbitrage / law of one price).

Setup: Go **long Δ (delta) shares** of stock and **short a bond** worth B at expiration (i.e., borrow the present value of B today).

Payoff equations (using the same numbers as above: S0=100, Su=120, Sd=80, Cu=10, Cd=0, R=1.08):

```
Up state:    Delta x Su  -  B  =  Cu     ->     Deltax120 - B = 10
Down state:  Delta x Sd  -  B  =  Cd     ->     Deltax80  - B = 0
```

Subtract the second equation from the first to eliminate B:

```
Deltax120 - Deltax80 = 10 - 0
Delta x 40 = 10
Delta = 0.25
```

So the replicating portfolio is **long 0.25 shares** (a quarter-share) of stock for every option sold/hedged.

Substitute $\Delta = 0.25$ back into the down-state equation to solve for B:

```
0.25 x 80 - B = 0  ->  B = 20
```

This B = 20 is the amount **owed at expiration** (i.e., you must repay 20 at expiration on what you borrowed today).

Present value of the replicating portfolio today:

```
Value of shares today = Delta x S0 = 0.25 x 100 = 25
Present value of the bond owed = B / R = 20 / 1.08 = 18.41 (rounded)
Portfolio value today = 25 - 18.41 = 6.48 (approx -- matches exactly with
    rounding: 25 - 18.5185 = 6.48)
```

This exactly matches the $6.48 found via the risk-neutral probability method above — confirming that both approaches (risk-neutral valuation and portfolio replication) are two routes to the *same* no-arbitrage answer. This is the direct, hands-on demonstration of the "replication" principle introduced in Section 2.3.

13.6.7 An Equivalent One-Period Model — Alternate Notation (as taught in the derivatives-basics lecture)

The same one-period logic is sometimes presented with slightly different labels: c_0 for today's call price, c^+ and c^- for the call's payoff in the up/down states, S^+ and S^- for the up/down stock prices, and π (pi) for the risk-neutral up-probability:

```
pi = (1 + r - d) / (u - d)
c0 = [pi x c+  +  (1 - pi) x c-] / (1 + r)
```

Worked example: Stock = 50, *strike* =52, risk-free rate = 5%, u = 1.2, d = 0.75.

```
S+ = 50 x 1.2 = 60          S- = 50 x 0.75 = 37.50
c+ = max(60 - 52, 0) = 8    c- = max(37.50 - 52, 0) = 0   (out of the money --
     expires worthless)

pi = (1.05 - 0.75) / (1.2 - 0.75) = 0.30/0.45 = 0.67  (67%)
1 - pi = 0.33  (33%)

c0 = [0.67 x 8 + 0.33 x 0] / 1.05 = 5.36/1.05 ~ $5.10
```

So the fair price to pay today for this call option (giving the right, but not the obligation, to buy the stock at $52) is about **$5.10**.

Key intuitive takeaway repeated throughout this derivation: the price of an option, just like the price of a bond or a stock, is fundamentally a **present value** calculation — you take an expected future payoff and discount it back at the risk-free

rate. The subtlety unique to options/derivatives is that the "expectation" must be taken using **risk-neutral probabilities**, not real-world subjective probabilities.

13.6.8 American vs. European Options in the Binomial Framework

- For a **European** option, the payoff can only be realized at the single expiration date, so the binomial calculation above (discount the expected terminal payoff) is sufficient.
- For an **American** option, at *each* node in the tree, the model must compare the value of **holding** the option (its computed continuation value from later nodes) versus **exercising immediately** (the intrinsic value at that node), and takes the greater of the two.
- **American call prices equal European call prices only when the underlying pays no dividends/interest** (i.e., options on non-dividend-paying stocks, commodities, or zero-coupon bonds). It is *only* the existence of dividends or interest payments on the underlying that can ever create an incentive to exercise a call early — otherwise, holding the option is always at least as good as exercising it early.

13.7 The Binomial Model — Full Generalization and Multi-Step Trees

13.7.1 Comparing Black-Scholes and the Binomial Model

Feature	Binomial	Black-Scholes
Price process	Discrete (up/down moves)	Continuous
Volatility	Can accommodate **changing** volatility	Assumes **constant** volatility
Option styles	Can value **American and European**	Designed for **European** only
Intuition	Relatively intuitive and easy to understand	Based on more complex mathematics
Computation	Can be computationally intensive for many steps	Less computationally intensive
Flexibility	More flexible, adaptable to real-world conditions	More restrictive assumptions

In practice, many practitioners use **both**: Black-Scholes for a quick estimate, then the binomial model to refine or handle more complex features (e.g., early exercise, changing volatility, dividends) — and sometimes the binomial model is even used simply to **cross-check** a Black-Scholes result.

13.7.2 Why the Actual Probability of Up/Down Doesn't Matter — A Clean Illustration

Suppose a stock trades at \$100. A one-year, at-the-money call option has strike \$100. Assume the stock will either rise to \$110 or fall to \$90 in a year. Two traders disagree on the real-world odds:

- **Trader A** believes there's a 60% chance the stock rises to $110.
- **Trader B** believes there's only a 40% chance the stock rises to $110.

Intuitively, you might expect the more "optimistic" Trader A to value the call higher. But under the binomial (risk-neutral) model, **both traders arrive at the exact same fair price of approximately $7.14** (assuming a 5% risk-free rate), regardless of their differing real-world beliefs.

Why? Because risk-neutral pricing does not use actual/subjective probabilities at all. What matters is purely the *relationship* between the possible future prices (110, 90) and the current price (100) and the risk-free rate — not what anyone personally believes will happen. This is the practical payoff of everything built up in Sections 2.2–2.4 (arbitrage, replication, risk-neutrality).

13.7.3 Binomial Calculation by Replicating Portfolio

Using the $100/$110/$90 example above, risk-free rate = 5%:

1. **Create a "risk-free" portfolio**: find a combination of stock and the option that gives the *same* outcome regardless of which way the stock moves.
2. **Build the portfolio**: buy some number of shares ("d" — note: this source's notation for the hedge ratio, equivalent to Δ above) and sell (short) one call option.
3. **Consider both scenarios:**
   - If the stock rises to $110: shares worth 110×d; the option written costs you the $10 it's now in-the-money by: total = 110d − 10.
   - If the stock falls to $90: shares worth 90×d; the option expires worthless (costs nothing): total = 90d.
4. **Make the portfolio risk-free** by setting both outcomes equal:

```
110d - 10 = 90d
```

1. **Solve for d:**

```
20d = 10   ->   d = 0.5
```

You need half a share of stock for every option sold, to create a fully hedged, risk-free position.

1. **Calculate the (certain) portfolio value at expiration**: using the down-state value, `90 × 0.5 = 45`.
2. **Apply the risk-free rate** to discount this certain future value back to the present:

```
PV = 45 / (1 + 0.05) = 42.86
```

1. **Back out the option price**: the present value of the portfolio (42.86) must equal half a share's current value minus the option's price:

```
42.86 = 0.5 x 100 - Option price
Option price = 50 - 42.86 = $7.14
```

Fair price of the call option = $7.14.

This approach never required guessing at the real-world probability of the stock going up or down. The logic is: if any obvious profit *were* available, arbitrage traders would immediately act on it and eliminate that profit — so the price *must* settle at the level that removes all riskless profit opportunities.

Where is volatility in this calculation? It's implicit — baked directly into the choice of the two future price levels ($110 and $90, i.e., ±10% either way in this example). A wider spread between the up and down outcomes corresponds to higher assumed volatility.

13.7.4 Full Mathematical Generalization

Let:

- **X** = current market price of the stock.
- **X**×**u** and **X**×**d** = the future prices after time t years (u > 1 for the up move; 0 < d < 1 for the down move).
- **Pup** and **Pdn** = the option's payoffs in the up and down states, respectively, at expiry.

Build a portfolio: buy **s** shares of stock today, and short (write) one call option. After time t:

```
Value if up move:    VUM = s x X x u - Pup
Value if down move:  VDM = s x X x d - Pdown
```

For a genuinely risk-free (hedged) portfolio, these must be equal:

```
s x X x u - Pup = s x X x d - Pdown
```

Solving for **s** (the hedge ratio / number of shares per option):

```
s = (Pup - Pdown) / [X x (u - d)]
```

The present-day value is obtained by discounting the (certain) future portfolio value at the risk-free rate:

```
PV = e^(-rt) x [s x X x u - Pup]
```

This present value must equal the current holding of s shares minus the call premium (s×X − c). Solving for **c** (the call price):

```
c = e^(-rt) x [(e^(rt) - d)/(u-d) x Pup  +  (u - e^(rt))/(u-d) x Pdown]    [
   continuous-compounding version]
```

Introducing q — the risk-neutral probability (continuous-compounding form):

```
q = (e^(-rt) - d) / (u - d)
```

Rearranging the option pricing formula into its cleanest form:

```
c = e^(-rt) x [q x Pup + (1 - q) x Pdown]
```

This is structurally identical to the discrete-time formula from Section 6.4 (`p = (R−d)/(u−d)`), just expressed with continuous compounding (e^(−rt) in place of 1/(1+r)).

Why "q" is a genuine probability: substituting q back in and simplifying shows that, under this measure, the *expected* stock price at time t grows exactly at the risk-free rate:

```
Expected Stock Price = q x X x u + (1-q) x X x d  =  X x e^(rt)
```

In other words, under the risk-neutral measure, the stock behaves *as if* it were a risk-free asset — investors are (in this modeling framework) indifferent to risk. This is precisely the **risk-neutral model** referenced conceptually back in Section 2.4.

13.7.5 Multi-Step (Two-Step, Three-Step, ...) Binomial Trees

Real markets don't move in just one discrete jump before an option expires — realistic pricing typically requires **many small steps**. The extension is conceptually simple:

- Divide the time to expiry into N small steps.
- At **each** step, the price can again only move up (×u) or down (×d), compounding upon the *previous* node's price.
- This produces a full **binomial tree**: at each additional step, the number of distinct terminal price paths grows (e.g., 3 steps → $2^3 = 8$ total paths, though many collapse onto the same terminal price levels since "up-then-down" reaches the same node as "down-then-up").

Working backward through the tree (the general algorithm):

1. **At expiration** (the final column of the tree), compute the option's exact payoff at *every* terminal node: for a call, `max(0, Price − Strike)`; for a put, `max(0, Strike − Price)`.
2. **Step back one period.** At each node in the second-to-last column, compute the expected value of the option one period ahead using the risk-neutral probability q (weighting the "up-child" and "down-child" nodes' already-known values).
3. **Discount** that expected value back one period at the risk-free rate — this gives the option's value at that node in the *present* period (relative to that step).
4. **Repeat**, working backward node-by-node, column-by-column, until you reach the single starting node at t = 0. That final number is the model's estimate of the option's fair value today.

Illustrative tree, matching the article's numeric example — starting value 10, with the price evolving down two consecutive branches: 10 → 8 → 6.4 → 5.12, and up two consecutive branches: 10 → 12 → 14.4 → 17.28, with the two "mixed" middle paths converging on 9.6 and 11.52 respectively at intermediate steps — demonstrating how paths recombine onto shared nodes rather than exploding independently.

Note: multi-step examples in this course typically assume the **same** u and d factors apply at every step (i.e., they compound in an identical, repeated fashion at each node) — though the model's flexibility does allow different u/d values at different steps if volatility is expected to change over time.

13.7.6 Worked Example #1 — Two-Step Binomial Tree (Put Option)

Setup: Put option, strike = \$110, underlying spot = \$100, expires in 1 year. Annual risk-free rate = 5%. The price is expected to move **+20% or −15% every six months** (so two steps of 6 months each): u = 1.2, d = 0.85, t = 0.5 (per step).

Step 1 — risk-neutral probability (continuous-compounding formula):

```
q = (e^(-rt) - d)/(u - d)          ... using the given values ...
q = 0.358 (i.e., 35.8%)
```

Step 2 — terminal (t = 1 year) underlying prices and put payoffs, via all combinations of the two 6-month moves:

```
Up-Up:    100 x 1.2 x 1.2  = $144.00  ->  Pupup = max(110-144, 0) = $0
Up-Down:  100 x 1.2 x 0.85 = $102.00  ->  Pupdn = max(110-102, 0) = $8
Down-Down: 100 x 0.85 x 0.85 = $72.25 ->  Pdndn = max(110-72.25, 0) = $37.75
```

(The "Up-Down" and "Down-Up" paths recombine to the same \$102 node.)

Step 3 — value at the intermediate (6-month) nodes, discounting back one step:

```
Node 2 (after an up-move):    p2 = e^(-rt) x [qxPupup + (1-q)xPupdn]
                                  = 0.975 x [0.358x0 + (1-0.358)x8]  =
   5.00897 ~ $5.01

Node 3 (after a down-move):   p3 = e^(-rt) x [qxPupdn + (1-q)xPdndn]
                                  = 0.975 x [0.358x8 + (1-0.358)x37.75]  =
   $26.43
```

Step 4 — discount back to today (t = 0):

```
p1 = e^(-rt) x [qxp2 + (1-q)xp3]
   = 0.975 x [0.358x5.01 + (1-0.358)x26.43]
   ~ $18.29
```

Value of the put option today = \$18.29.

13.7.7 Worked Example #2 — Three-Step Binomial Tree (European Put)

Setup: European put, 9 months to expiry, strike = \$12, underlying spot = \$10. Risk-free rate = 5% for all periods. Price moves ±20% every 3 months → u = 1.2, d = 0.8, t = 0.25 per step, giving a **3-step** tree (9 months ÷ 3 months = 3 steps).

Step 1 — risk-neutral probability:

```
q = 0.531 (53.1%)
```

Step 2: Using q and the terminal payoff values at t = 9 months (the final column of the tree), compute the corresponding intermediate values at t = 6 months by working backward one step at a time (same method as the two-step example above).

Step 3: Using the t = 6-month values, compute the values at t = 3 months; then

using those, compute the single value at t = 0.

Result: the present-day value of the put option = $2.18 — reasonably close to the value obtained from the Black-Scholes model for the same inputs, which is **$2.30**. (This closeness illustrates that as the binomial tree is built with more steps, it converges toward the continuous-time Black-Scholes result.)

13.7.8 When Is the Binomial Model the Right Choice?

- **American options** that can be exercised before expiry — the binomial model can check, at every single node, whether early exercise is optimal.
- Situations with **changing volatility** over the option's life (different u/d factors can be applied at different steps).
- Underlyings with **dividend payments**, since the model's discrete, node-by-node structure can accommodate cash flows occurring partway through the option's life.

13.7.9 Effectiveness and Limitations of the Binomial Model

Effectiveness: As the number of steps (N) increases (i.e., as each Δt shrinks), the binomial tree's discrete up/down paths increasingly resemble the continuous price movement observed in real markets — the model converges toward the smoother, continuous-time behavior captured by models like Black-Scholes.

Handling risk aversion: The clever mechanism is in how **u**, **d**, and the risk-neutral probabilities (**p**/**q**, **1**−**p**) are chosen/calibrated — based on the underlying's observed volatility and the risk-free rate. This effectively "builds in" the risk premium investors would demand, *without* the model needing to explicitly specify or model investors' individual risk preferences. This elegant shortcut is precisely what is meant by **risk-neutral valuation**.

Limitations:

- Requires **many computations**, especially for options with long times to expiry (more steps needed for accuracy) — this can become computationally intensive.
- Still relies on **simplifying assumptions** — typically that the risk-free rate and volatility are **constant** over the option's life (even though the model *can* be adapted to allow them to vary, doing so adds complexity).
- The finer the time intervals used, the harder it becomes to specify/predict the exact payoffs at each node with precision — accurately forecasting future price levels remains an inherent limitation of any binomial approach.

Despite these constraints, the binomial model remains a valuable and widely-used tool, striking a practical balance between the computational intensity of more advanced numerical methods and the restrictive assumptions of simpler closed-form models like Black-Scholes. Values computed via the binomial model closely match those from Black-Scholes, which supports its accuracy and usefulness — and it can be adapted flexibly to a trader's specific needs as an alternative (or complement) to Black-Scholes.

13.7.10 Other Applications of the Binomial Model (Beyond Plain Options)

Because the binomial model is fundamentally a general framework for valuing anything with sequential, uncertain decision points, it extends naturally to:

- **Valuing projects or investments** with a high degree of uncertainty.
- **Capital budgeting and resource-allocation decisions.**

- Projects with **multiple periods** or an **embedded option to continue or abandon** the project at specific decision points (this is often called "real options" analysis).

Example: complex investment decisions such as **oil drilling projects** can be modeled with the binomial approach by breaking the overall decision into a series of decision points, each with different possible outcomes — allowing a company to assess risk, quantify potential rewards, and retain the flexibility to adapt as new information becomes available over time.

13.7.11 Embedded Options

Embedded options are option-like features woven directly into the terms of certain financial securities, giving either the issuer or the holder the right to alter the security's cash flows or value under pre-defined conditions. Unlike a standalone, separately-traded option, an embedded option is an integral part of the underlying security itself. They are most common in **fixed-income securities**.

- **Callable bonds**: give the **issuer** the right to redeem (buy back) the bond before its stated maturity date — typically exercised when interest rates fall (the issuer can refinance more cheaply). This benefits the issuer but exposes the bondholder to **reinvestment risk** (having to reinvest the returned principal at now-lower prevailing rates).
- **Putable bonds**: give the **bondholder** the right to sell the bond back to the issuer before maturity — providing the investor a form of **downside protection**.

13.8 Consolidated Formula Cheat-Sheet

No-Arbitrage / Forward Pricing

```
F0(T) = S0 x (1+rf)^T                                          [basic forward price,
    no cash flows]
F0(T) = [S0 - (gamma-theta)] x (1+rf)^T                                  [with
   discrete carry benefits/costs]
F0(T) = S0 x e^[(rc+theta-gamma)xT]                                      [continuous
    carry benefits/costs]
Vt(T) = [Ft(T) - F0(T)] / (1+i)^(T-t)                          [value of forward,
   no cash flows]
Vt(T) = St - (gamma-theta)(1+r)^t - F0(T)(1+r)^(-(T-t))                  [equity
   forward value, discrete divs]
```

Fixed-Income Forwards/Futures

```
F0(T) = (S0 - PVC0) x (1+rf)^T                                  [bond forward price
   ]
Vt(Long) = (St - PVCt) - F0(T)/(1+rf)^(T-t)                     [bond forward value
   ]
F0(T) = QF0(T) x CF(T)                                           [bond futures
   price]
Clean price = Full price - Accrued Interest
```

FRAs

```
FRA(0,h,m) = {[1+L0(h+m)t(h+m)]/[1+L0(h)th] - 1} / tm
Payoff = (FRAg - FRA0) x (m/360) x Notional
```

Currency Forwards

```
FT(D/F) = S0(D/F) x [(1+rD)^T / (1+rF)^T]                    [covered interest
   rate parity]
```

Put-Call Parity

```
C0 + PV(X) = P0 + S0
```

Binomial Model — One Step

```
p (or pi or q) = (R - d)/(u - d)              [discrete: R = 1+rf]
q = (e^(-rt) - d)/(u - d)                    [continuous compounding]
c = e^(-rt) x [qxPup + (1-q)xPdown]
s (hedge ratio) = (Pup - Pdown)/[Xx(u-d)]
```

Option Moneyness / Intrinsic Value

```
Intrinsic value (call) = max(0, St - PV(X))
Intrinsic value (put)  = max(0, PV(X) - St)
CT = max(0, ST - X);   PT = max(0, X - ST)
```

13.9 Sources Referenced in These Notes

- Course reading: *Options Pricing* (course module notes)
- Course reading: *Binomial Model* (course module notes)
- CFA Level I video lecture — Basics of Derivatives Pricing and Valuation (forwards, futures, swaps, FRAs, options, put-call parity, one-period binomial model)
- Video lecture — Binomial Option Pricing Model (one-step tree derivation, risk-neutral probability, replicating portfolio, Excel walkthrough)
- Investopedia — *Binomial Option Pricing Model: A Simple Guide With Examples* (Adam Hayes)
- AnalystPrep — *Pricing and Valuation Concepts* (CFA Level II, Derivatives)
- PrepNuggets — *Pricing and Valuation of Options* (CFA Level I, Derivatives)

Chapter 14

Option Greeks

Course notes — Introduction to Quantitative Finance (Reading Course)

14.1 Why We Need the Greeks

An option's price (its **premium**) is not a fixed number — it is a function of several inputs that are all constantly changing:

$$V = f(S, K, r, \sigma, T)$$

where

Symbol	Meaning
S	Current price of the underlying asset
K	Strike (exercise) price
r	Risk-free interest rate
σ	Volatility of the underlying (usually *implied* volatility)
T	Time remaining to expiration

The **Option Greeks** are simply the **partial derivatives** of the option price with respect to each of these inputs — i.e., they tell you *how much the option's price will move if one input changes, holding everything else constant.* They are the sensitivity measures (and, collectively, the risk-management dashboard) of an option position.

> **Intuition:** if the stock market is a moving system with several "dials" (price, time, volatility, interest rate), the Greeks tell you how hard the option's price gets tugged when each dial is turned by one unit.

Each Greek is a first derivative of price with respect to one variable — **except Gamma**, which is a *second* derivative (the derivative of Delta, which is itself a derivative of price). This is why Gamma is sometimes described as "delta's delta," or the *acceleration* of the option price.

Greek	Dependent variable	Independent variable	Order
Delta (Δ)	Option price	Underlying asset price	1st derivative of V w.r.t. S
Gamma (Γ)	Delta	Underlying asset price	2nd derivative of V w.r.t. S
Vega (ν)	Option price	Volatility	1st derivative of V w.r.t. σ

Theta (Θ)	Option price	Time to maturity	1st derivative of V w.r.t. T
Rho (ρ)	Option price	Interest rate	1st derivative of V w.r.t. r

These five — **Delta, Gamma, Theta, Vega, Rho** — are the primary Greeks. Others exist (Vanna, Charm, Volga, etc., "second-order Greeks") but are outside the scope here.

14.1.1 Quick refresher on options

- A **call option** gives the buyer the right (not obligation) to *buy* the underlying at the strike price before/at expiry.
- A **put option** gives the buyer the right (not obligation) to *sell* the underlying at the strike price before/at expiry.
- The price paid for this right is the **premium**. The buyer (long) pays the premium to the writer (short); the writer collects the premium and profits if the option expires worthless/unexercised.
- Moneyness classification: **ITM** (in-the-money — has intrinsic value), **ATM** (at-the-money — strike $\approx$ spot), **OTM** (out-of-the-money — no intrinsic value, pure time value).

14.1.2 Context: the Black–Scholes–Merton (BSM) model

The Greeks are formally derived from the BSM option-pricing model, which extended a simple option-pricing framework by adding volatility as an input. Given S, K, r, σ, T, BSM defines:

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{\sigma^2}{2}\right) T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}$$

Interpretation of d_1: the numerator splits into two pieces —

- $\ln(S/K)$: the **return attributable to intrinsic value** (how far the stock is from the strike, in log terms), plus
- $\left(r + \frac{\sigma^2}{2}\right) T$: the **return attributable to time value**,

—all scaled by $\sigma\sqrt{T}$ (the total standard deviation of the stock's return over the option's life). This "intrinsic value + time value" decomposition is the intuitive core of the whole BSM framework, and every Greek formula below is built from d_1, d_2, and the standard normal CDF/PDF, $N(\cdot)$ and $N'(\cdot)$.

$$N'(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad \text{(standard normal density)}$$

Because Greeks come from a model with idealized assumptions (constant volatility, continuous trading, no jumps, etc.), they are **theoretical estimates**, not guarantees — real option prices can move differently from what the Greeks predict, especially around unexpected events. In practice, virtually no one calculates these by hand while trading; every options trading platform/broker terminal displays live Greeks per strike, computed automatically.

14.2 Delta (Δ) — Directional Sensitivity

14.2.1 Definition

$$\Delta = \frac{\partial V}{\partial S}$$

Delta measures how much an option's price changes for a **1-unit (e.g. Rs. 1 or \$1) change in the underlying's price**, holding everything else constant.

14.2.2 Sign and range

	Range	**Sign**
Call option	$0 \leq \Delta \leq 1$	Positive (option gains value as stock rises)
Put option	$-1 \leq \Delta \leq 0$	Negative (option gains value as stock falls)

The **closer Delta is to ± 1, the deeper in-the-money** the option is (it starts behaving almost like the stock itself). The closer Delta is to 0, the deeper out-of-the-money (the option barely reacts to the stock).

14.2.3 BSM formula

For a **non-dividend-paying** stock:

$$\Delta_{\text{call}} = N(d_1), \qquad \Delta_{\text{put}} = N(d_1) - 1$$

For a **dividend-paying** stock with continuous dividend yield q:

$$\Delta_{\text{call}} = e^{-qT} N(d_1), \qquad \Delta_{\text{put}} = e^{-qT}\Big(N(d_1) - 1\Big)$$

The e^{-qT} term discounts $N(d_1)$ slightly — e.g. if the non-dividend delta would be $N(d_1) = 0.50$, a modest dividend yield might pull it down to something like \$0.48\$. Since $e^{-qT} < 1$ always (for $q > 0$), a dividend-paying stock's call delta is always a little *smaller* than the non-dividend equivalent.

An **at-the-money option's delta is always close to 0.5** for a call (and -0.5 for a put) — this follows because when $S \approx K$, the d_1 term is small (near 0), and $N(0) = 0.5$.

Important edge-case behavior: as the stock price moves deep ITM, both $N(d_1)$ and $N(d_2)$ approach 1, and delta approaches 1 (or -1 for puts). At this extreme, BSM essentially collapses into the *simple* (pre-volatility) option-pricing model — this is part of why BSM does a relatively poor job pricing deep-ITM or deep-OTM options: volatility stops mattering much once the outcome is nearly certain either way.

> **Technical nuance:** delta ($N(d_1)$) is often used casually as "the probability the option finishes in-the-money," but strictly speaking the **risk-neutral probability of finishing ITM is @@I1@@**, not $N(d_1)$. They are close in value (especially for short-dated, low-volatility options), which is why the rule-of-thumb persists, but they are not mathematically identical.

14.2.4 Delta as a hedge ratio

Delta tells you **how many shares of the underlying are needed to offset one option's price risk**. If you know an option's delta, you can hedge your position by buying or shorting (delta × number of shares underlying the option) of the underlying.

14.2.5 Worked examples

Example (CFI/course style): A call option has Delta = 0.6. If the stock rises by Rs. 1, the option's price rises by about Rs. 0.60.

Example (Plain Bagel): An option has Delta = 0.2. If the underlying stock rises $1, the option rises 20 cents.

Example (course, ATM): Nifty is at 25,500 (ATM strike), call premium = Rs. 100, Delta = 0.55. If Nifty moves up by 1 point → premium moves by 0.55 → premium ≈ Rs. 100.55.

Example (FRM): Stock $S = \$100$, strike $K = \$90$, 6-month call, premium ≈ $16 (from BSM), Delta = 0.5. If the stock rises to $101, the call price rises by about 50 cents (delta × $1 move), i.e. to roughly $16.50 — **not** a full $1 move, because the option is not a 1-for-1 proxy for the stock.

14.2.6 Portfolio Delta

The Delta of a portfolio of options is simply the **weighted average of the individual deltas**, weighted by position size (number of contracts × multiplier). E.g., if you write options with deltas of 0.2, 0.4, 0.6, 0.7, 0.8 (in some position-weighted mix) and the weighted average works out to 0.45, then a Rs. 1 move in the underlying changes your total book value by about Rs. 0.45 per unit — even though the individual legs move by very different amounts.

14.2.7 Delta of Forwards and Futures

- **Forward contracts:** Delta ≈ **1**. Forwards have thin/no secondary markets, so their price tracks the underlying almost exactly.
- **Futures contracts:** Delta is **slightly less than 1** (e.g., 0.94–0.96), because futures are marked-to-market daily; the futures price and spot price can diverge slightly day-to-day even though they must converge by the maturity date. The correlation between changes in the futures price and changes in the spot price is not perfectly 1, which is why futures delta is marginally below 1.

14.2.8 Covered vs Naked Positions — Why Hedging Matters

Before getting into hedging mechanics, it's worth understanding **why** an option writer needs to hedge at all.

Say you (the writer) sell someone a call option: stock at $100, strike $90, premium collected = $16. If the buyer exercises (stock price rises above $90), they can demand a share at $90 from you.

- **Covered position:** you already own the underlying share. You simply hand it over — no extra risk.
- **Naked position:** you do *not* own the share. You must go buy it in the market at the *current* (higher) price and sell it to the option holder for $90 — realizing the

loss immediately. In practice, options are usually cash-settled rather than requiring this physical scramble, but the economic loss is identical.

The danger: if the stock price at expiration is \$110 or \$120, your loss is the difference. If it's \$500 or \$1,000 (hypothetically), your loss keeps growing — **a naked call writer has theoretically unlimited loss potential**, since a stock price has no upper bound.

Solution: buy enough of the underlying stock *in proportion to Delta* so that gains/losses on the stock offset gains/losses on the written option. This is **Delta hedging**.

> **Analogy:** think of the standard deviation (risk) of your unhedged position as a piece of metal in a vise. Delta hedging is cranking the vise — squeezing that variability down. A perfect hedge can, in theory, drive the variability to (near) zero, by holding **offsetting long and short positions**.

Practically, writers often start naked and only begin hedging once there's real risk of the option moving in-the-money with momentum — if an OTM option looks likely to stay OTM, the incentive to hedge is low; but predicting this is inherently uncertain, which is exactly why the Greeks-based framework exists.

14.2.9 Worked Delta-Hedging Examples

Example 1 — hedging with stock (simple): You are **long 1 call option** (each option contract = 100 shares of underlying) with Delta = 0.6. Total position delta = $0.6 \times 100 = 60$. To become delta-neutral, **short 60 shares** of the underlying stock. (If instead you *wrote* — went short — the call, you'd do the opposite: **buy** 60 shares. If long a put, you'd need to be long the corresponding number of shares to hedge.)

Example 2 — dynamic rebalancing (FRM): Non-dividend stock at \$200. You've **written** (are short) call options on 200 shares (i.e., 2 contracts) with strike \$202, Delta = 0.5 (nearly ATM).

Q1: What do you need to do right now to be delta-neutral? Total delta from the short calls = $-0.5 \times 200 = -100$. To offset this, **buy 100 shares** of stock (delta = $+100$ from the shares cancels the -100 from the calls).

Q2: The stock falls to \$195 — the option is now more OTM, and Delta falls to 0.4. What now? New required hedge = $0.4 \times 200 = 80$ shares. You are currently holding 100 shares, so you must **sell 20 shares** to bring your hedge down to 80.

This illustrates the central fact about Delta hedging: **it is not a one-time action.** As the stock price moves, Delta itself moves, so the hedge ratio must be continuously recalculated and rebalanced — this is called **dynamic hedging**.

Example 3 — hedging with another option (instead of stock): If you hold a position with a net delta of $+5$, you can instead find another option with a delta of -5 and combine the two so the deltas cancel — e.g., pairing a call and a put with matching (opposite-sign) deltas. This is not typically how long-horizon portfolios are managed, but it's a valid short-term hedging tool available to any options trader.

14.3 Gamma (Γ) — The Acceleration of Delta

14.3.1 Definition

$$\Gamma = \frac{\partial \Delta}{\partial S} = \frac{\partial^2 V}{\partial S^2}$$

Gamma measures how much **Delta itself changes** for a 1-unit move in the underlying. It is the **second derivative** of the option price with respect to the stock price — hence "the delta of delta," or, informally, the option's *acceleration*.

BSM formula (same for calls and puts):

$$\Gamma = \frac{N'(d_1)}{S\sigma\sqrt{T}}$$

14.3.2 Behavior

- **Long options (calls or puts) always have positive Gamma.**
- Gamma is **maximum when the option is at-the-money** (strike $\approx$ spot).
- Gamma **decreases** as the option moves deep ITM or deep OTM.
- Gamma tends to **rise as expiration approaches** for ATM options (the option's fate becomes more binary/urgent).

14.3.3 Why Gamma matters

If you're Delta-hedging, Gamma tells you **how fast your hedge will go stale** — i.e., how often you'll need to rebalance. High-Gamma positions require frequent adjustment; low-Gamma positions are more "set and forget."

Options with the highest Gamma are the **most responsive to changes in the underlying's price** — a small stock move can meaningfully re-rate the whole position's sensitivity.

14.3.4 Worked Examples

Example (Plain Bagel): A \$2 option has Delta = 0.2 and Gamma = 0.1.

- Stock rises \$1 $\rightarrow$ option rises to \$2.20 (delta effect); simultaneously, Delta rises by the gamma amount, from 0.2 to 0.3.
- **Next** time the stock rises \$1, the option now gains **30 cents** (not 20), because its Delta has grown.

This is the compounding/"inception" nature of Gamma: Delta tells you the *current* rate of change, but Gamma tells you how that rate of change is itself evolving.

Example (course): Delta = 0.50, Gamma = 0.02. After a 1-point move in the underlying, the new Delta = 0.50 + 0.02 = **0.52**. The *next* point move will therefore move the option price by 0.52, not 0.50 — the option price is now accelerating upward faster than a purely linear (delta-only) estimate would suggest.

14.3.5 Gamma-Neutral Hedging

Just as we can build a Delta-neutral position, we can build a **Gamma-neutral** position — useful because a Delta-neutral position can still have substantial Gamma risk (it will *stop* being delta-neutral the moment the stock moves, precisely because of Gamma).

Formula — the number of hedging options needed:

$$N_{\text{hedge}} = -\frac{\Gamma_{\text{portfolio}}}{\Gamma_{\text{hedging option}}}$$

Worked example (FRM): You hold a **short option position that is already Delta-neutral**, with portfolio Gamma = **−800**. A tradable option is available with Delta = 0.8, Gamma = 2.

Step 1 — Gamma-hedge: buy $N = 800/2 = 400$ of this option. This neutralizes the portfolio's Gamma.

Step 2 — but this introduces new Delta: buying 400 options with delta 0.8 each adds $400 \times 0.8 = 320$ of delta to the portfolio (which was previously delta-neutral).

Step 3 — re-hedge Delta: short **320 shares** of the underlying stock to bring the portfolio back to delta-neutral, *while remaining gamma-neutral.*

This two-step process (hedge Gamma with another option, then re-hedge the Delta that the Gamma-hedge introduced, usually with stock) is the standard technique for building a position that is neutral to *both* first- and second-order price risk.

14.3.6 The Delta–Theta–Gamma Relationship (BSM PDE)

The three "price-related" Greeks are not independent — they are linked by an identity derived from the Black-Scholes-Merton differential equation:

$$rV = \Theta + rS\Delta + \tfrac{1}{2}\sigma^2 S^2 \Gamma$$

Reading the two sides:

- **Left side:** the risk-free rate multiplied by the option's price — roughly, "the dollar return you should expect to earn on the option's value, at the risk-free rate."
- **Right side:** the *same* quantity decomposed into contributions from time decay (Θ), the underlying's drift weighted by rate and price ($rS\Delta$), and convexity from Gamma weighted by price-squared and variance ($\frac{1}{2}\sigma^2 S^2\Gamma$).

Think of this as a "budget constraint" among the Greeks: an option's assumed fair return has to be accounted for by *some combination* of time decay, directional exposure, and convexity — you can't get a return "for free."

14.4 Theta (Θ) — Time Decay

14.4.1 Definition

$$\Theta = \frac{\partial V}{\partial T} \quad \text{(commonly quoted as the negative of this — value lost per day)}$$

Theta measures how much an option's price changes with the **passage of one day**, holding everything else constant. Because time can only move in one direction (an option never "gains" time back), this is called **time decay**.

BSM formulas (non-dividend paying stock):

$$\Theta_{\text{call}} = -\frac{S\sigma N'(d_1)}{2\sqrt{T}} - rKe^{-rT}N(d_2)$$

$$\Theta_{\text{put}} = -\frac{S\sigma N'(d_1)}{2\sqrt{T}} + rKe^{-rT}N(-d_2)$$

14.4.2 Sign convention

- For a **long** option position (you bought it), Theta is (almost always) **negative** — the position loses value each day, all else equal. (It can occasionally be positive for some European options in unusual configurations, but negative is the standard case.)
- For a **short/written** option position, Theta works in your favor — you gain as the option decays, since you profit if it expires unused.

14.4.3 Behavior

- Theta is **most negative when the option is at-the-money** — the option's ultimate fate (ITM or OTM) is least certain, so each passing day matters most.
- Theta becomes more pronounced (accelerates) as **expiration approaches**.

14.4.4 Worked Examples

Example (course): Theta = -0.05. All else equal, the option loses Rs. 0.05 in value per day.

Example (course, in an option-chain context): Theta = -10 (often written as "-10" but interpreted as reducing price by 10). If a hypothetical ATM option is priced Rs. 100 today (6th of the month) and nothing else in the market changes by the next day (7th), the option's price will automatically fall by Rs. 10, to Rs. 90.

Key mental model: Options are **wasting assets** — if you're long an option and simply hold it without the underlying moving in your favor, time alone erodes your position's value every single day.

14.5 Vega (ν) — Volatility Sensitivity

14.5.1 Definition

$$\nu = \frac{\partial V}{\partial \sigma}$$

Vega measures how much an option's price changes for a **1 percentage-point change in implied volatility (IV)**. Unlike the other Greeks, Vega is expressed as a **money amount**, not a decimal ratio.

BSM formula (same for calls and puts):

$$\nu = S\sqrt{T}\,N'(d_1) \quad (\text{or } Se^{-qT}\sqrt{T}N'(d_1) \text{ with a dividend yield})$$

14.5.2 Implied Volatility vs. Historical Volatility

This distinction matters a great deal for Vega, since Vega is defined with respect to **implied** volatility specifically:

- **Historical volatility** is backward-looking — computed directly from a stock's past price data (e.g., standard deviation of past returns).
- **Implied volatility (IV)** is forward-looking — it is *backed out* of current option market prices, and represents the market's collective estimate of how much the stock is likely to swing going forward. IV tends to be **higher** for stocks in high demand and **during bear markets** (when future moves are assumed to be larger/riskier).

14.5.3 Behavior

- **An increase in volatility raises the price of both calls and puts** — a more volatile stock is more likely to swing far enough to make *either* type of option profitable before expiry, so both become more valuable.
- Vega is **highest for at-the-money options with longer time to expiry**.

Intuitive illustration (Plain Bagel): imagine choosing between two options with identical strikes — one on a volatile stock, one on a stable stock, both starting at the same price. All else equal, the option on the *more volatile* stock is more likely to reach (or exceed) the strike before expiry, and is therefore worth more, even though "more volatile" sounds riskier in a conventional (buy-and-hold) sense.

14.5.4 Worked Examples

Example (CFI/course): Vega = 0.20. If IV rises by 1 percentage point, the option's price rises by about Rs. 0.20.

Example (course): IV rises from 8% to 9% (a 1-point change) with Vega = 10. The option's premium rises from Rs. 100 to Rs. 110.

Traders use Vega to **bet on, or hedge against, changes in market volatility itself** — independent of which direction the underlying stock moves.

14.6 Rho (ρ) — Interest Rate Sensitivity

14.6.1 Definition

$$\rho = \frac{\partial V}{\partial r}$$

Rho measures how much an option's price changes for a **1 percentage-point change in the risk-free interest rate** (e.g., the relevant government T-bill rate).

BSM formulas:

$$\rho_{\text{call}} = KTe^{-rT}N(d_2), \qquad \rho_{\text{put}} = -KTe^{-rT}N(-d_2)$$

14.6.2 Sign convention and behavior

- **Calls:** Rho is **positive** — call options become more valuable as rates rise.
- **Puts:** Rho is **negative** — put options become less valuable as rates rise.

- Rho is **larger (more significant) for options with longer time to maturity** — this ties back to the "time value" component of d_1: longer-dated options carry more embedded time value, and interest rates directly affect the discounting of that time value.

14.6.3 Practical significance

Rho is generally considered the **least important of the five Greeks**, because option prices are far less sensitive to interest-rate changes than to changes in the underlying price, time, or volatility. Interest rates typically move slowly and in small increments (especially in a stable-rate regime), so Rho's real-world impact on short-dated options is often negligible and is frequently ignored by retail traders. It becomes more relevant for **long-dated options** and in **high-rate / rapidly-changing-rate environments**.

14.7 Summary Table of the Five Greeks

Greek	Measures sensitivity of...	...to a unit change in	Call sign	Put sign	Peaks when
Delta (Δ)	Option price	Underlying price	0 to 1	-1 to 0	Deep ITM ($\to \pm 1$)
Gamma (Γ)	Delta	Underlying price	positive	positive	At-the-money
Theta (Θ)	Option price	Time (1 day passing)	usually negative (long)	usually negative (long)	At-the-money, near expiry
Vega (ν)	Option price	Implied volatility (1%)	positive	positive	ATM + longer maturity
Rho (ρ)	Option price	Interest rate (1%)	positive	negative	Longer maturity

14.8 Practical Risk Management With the Greeks (Applied Trading Framework)

This section moves from the mathematics to how practitioners actually use the Greeks day-to-day when trading options (as opposed to computing them once and forgetting them).

14.8.1 The core mental model: "Risk = Greeks"

Central idea: whenever the underlying's price moves, *all* the Greeks of your position move simultaneously — and that collective change **is** your risk. Managing your Greeks *is* managing your risk. Practically, this means an active options trader should be watching a **Greeks dashboard**, not a raw profit-and-loss (P&L) screen — the Greeks tell you *why* your P&L is about to change, before it changes.

14.8.2 Two pre-trade steps, before you ever place an order

1. **Money management:** decide, before trading, the *maximum total amount* you are willing to lose overall.

2. **Risk management:** decide, based on lot size / position size, exactly where your **stop-loss** will sit for this specific trade.

Only after both are fixed should you move on to selecting *which* trade (which strike) to take.

14.8.3 Choosing a strike: let current Implied Volatility decide

When you open an option chain, there are usually several strikes to choose from — ATM, OTM, ITM. **There is no universally "best" strike** — the right choice depends on the *current volatility regime*, which you can read directly off the option chain's IV column:

- **If IV is currently HIGH** → OTM strikes become reasonable, because a highly volatile underlying has a realistic chance of swinging far enough to bring an OTM option into the money.
- **If IV is currently LOW** → avoid OTM strikes; a low-volatility underlying is unlikely to move far enough for an OTM option to become profitable before time decay erodes it. ATM/ITM becomes the more sensible choice.

Because volatility itself changes *after* you've entered a trade, the "best" strike at entry may stop being ideal — which is exactly why ongoing Greek monitoring (not a one-time decision) is necessary.

14.8.4 Greek behavior and biggest concern, by moneyness

Moneyness	Delta	Gamma / Theta / Vega	Biggest concern	What you want to happen
OTM	Low	All low	**Volatility must rise** — otherwise all Greeks decay toward zero as expiry nears, and the position simply bleeds out	IV increase, so Delta starts rising (OTM → ATM)
ATM	≈ 0.5	All **high** (Gamma, Theta, Vega peak here)	**Theta** — as expiry approaches, time decay accelerates fastest here, unless a large move happens soon	Delta should rise quickly (ATM → ITM) before decay eats the premium
ITM	High ($\to 1$ for deep ITM)	Lower (less time value left)	**Delta must not fall** — a falling Delta means the option is sliding back toward ATM/OTM	Delta should keep rising toward 1 (ITM → Deep ITM)

The common thread across all three cases: whether you're trying to move OTM→ATM, ATM→ITM, or ITM→Deep ITM, what you need in every case is for

Delta to increase — and Delta increases *only* when volatility increases. This is why experienced/professional options traders often say they "trade volatility," not a specific strike or even a specific direction: volatility is the underlying engine that drives Delta, which drives everything else.

14.8.5 Worked example: managing an ATM position as conditions drift

Setup: Nifty at 25,500 (ATM), you buy a call for Rs. 100. At entry: Delta = 0.55, Gamma = 0.018, Theta ≈ -11, Vega ≈ 11 — all four Greeks arrive "attached" to the position the moment you take the trade.

Now suppose Nifty **doesn't trend strongly** — it oscillates back toward its mean rather than moving decisively in one direction. This signals **low volatility** in the market. Consequences:

- With low/falling IV and the passage of time (Theta), **Delta starts drifting down** — say, from 0.55 to 0.40 over some days.
- A Delta of 0.40 now **behaves like an OTM option**, even though you originally entered at what looked like an ATM strike (ATM delta ≈ 0.55). The position has effectively "become" something you didn't intend to hold.
- Since your original comfort/target was Delta ≈ 0.55, your job now is to bring the position's Delta **back** to 0.55 — i.e., add $0.55 - 0.40 = 0.15$ of delta.
- You do this by **buying an additional option whose own delta is around 0.15**, adding it to your existing position. Combined, the position's delta returns to ≈ 0.55.

  Important distinction: this is emphatically **not** the same thing as going Delta-neutral (Delta = 0). Here, the goal is to restore the position to its *original target/comfort-zone* exposure, not to eliminate directional exposure entirely. "Delta-neutral" is a specific hedging strategy (see §2.9, §3.5); "returning to your comfort-level Delta" is a broader risk-management habit that applies to *any* directional position, hedged or not.

General workflow this illustrates:

1. Enter a trade based on your read of current conditions (open interest, IV, sentiment, FII/DII activity, etc.).
2. The moment the trade is live, the Greeks attach to it — this is your risk.
3. Monitor the Greeks (not just P&L). When market conditions shift, your risk profile shifts with them.
4. Adjust the position (usually by adding/removing option legs, or a stock hedge) to bring the risk profile back to a comfortable level.
5. Repeat — this is continuous, not a one-time setup.

14.8.6 A cautionary tale on *not* doing this

The practical trading source material includes a real anecdote of a trader who, for a long stretch, made decisions purely by looking at whether Nifty's candle was "red or green" on a given day — buying a large quantity of puts on red days and calls on green days, with no reference to the option chain, open interest, or Greeks, and with no defined

risk-management or exit process (simply watching the running loss and squaring off only after losses had already piled up). Losses escalated over time (from roughly Rs. 12,000, to Rs. 20,000, past Rs. 1 lakh, and eventually into several lakhs), including a period after the trader briefly stopped and then resumed once external pressure (e.g., having the trading app removed) was lifted.

The lesson drawn from this: trading options based only on naive directional cues (candle color), without understanding open interest and without monitoring the Greeks that actually describe your risk, is a recipe for large, compounding, poorly-controlled losses. Structured risk management — defined money management, a defined stop-loss, and ongoing Greek monitoring — is what separates disciplined options trading from gambling on direction.

14.9 What Actually Drives Volatility: Supply, Demand, and Market Participants

Since §8.4 concluded that "everything comes down to volatility," it's worth understanding *what moves volatility itself*. Like any market, options prices (and hence implied volatility) are governed by **supply and demand**.

14.9.1 Hedgers

Investors/institutions holding an underlying portfolio (stocks, bonds, mutual funds, futures contracts) who want to **protect** it from a decline will buy protective options (e.g., puts against a long stock portfolio). When hedgers are actively buying protection — especially heavily, e.g. ahead of a perceived risk event — this **increases demand for options**, which pushes premiums (and implied volatility) **up**.

14.9.2 Speculators

Traders with a directional view often prefer buying an **option** over buying the underlying stock outright, because it requires a smaller premium outlay for similar (leveraged) directional exposure. A bullish speculator buys calls; a bearish speculator buys puts. Either way, this is **additional demand** for options, again pushing volatility **up**.

14.9.3 Arbitrageurs

Arbitrageurs try to **eliminate directional exposure** entirely and trade in a Delta-neutral fashion (see §2.9). As the underlying moves and their position's Delta drifts away from zero, they continuously buy/sell options to bring Delta back to zero (essentially performing the delta-neutral rebalancing described in §2.9's worked examples, repeatedly). This constant rebalancing activity also **adds to options demand** in aggregate.

Put together: when hedgers, speculators, and arbitrageurs are all actively *buying* options, demand rises, and the market experiences **higher implied volatility**.

14.9.4 What causes volatility to fall

The reverse also holds. If there is no major perceived risk on the horizon (no anticipated earnings surprise, no macro event), hedgers reduce their protective buying. Speculators, seeing weak buying demand, may shift toward **selling/writing** options instead (to collect premium/income, effectively harvesting Theta) rather than buying them. When **sellers outnumber buyers**, supply increases relative to demand, and implied volatility **falls**.

14.9.5 Greed, fear, and news

- **Rumors of good news** → greed increases → more people rush to *buy* the stock/option → demand rises → volatility (and price) rises. This is described colloquially as **"buying the rumor."**
- **Bad or fake news** (e.g., a scandal at a company) → fear increases → people rush to **sell** → supply rises. This is described as **"selling the news."**

14.9.6 "Buying" and "Selling" volatility — trader shorthand

Because everything ultimately traces back to volatility, experienced traders often talk in terms of volatility itself rather than in terms of specific option legs:

- **"Buying volatility"** = buying options (calls or puts) — you benefit if implied volatility (or realized movement) increases.
- **"Selling volatility"** = writing/selling options — you benefit if implied volatility stays low or falls, and from time decay.

Practically:

- When **IV is high** (lots of demand/buying) → favor **selling** volatility (write/short options).
- When **IV is low** (lots of supply/selling) → favor **buying** volatility (go long options) *if you expect it to rise* — otherwise avoid, per §8.3.

14.10 Classifying Options Strategies by Direction-Dependence

Strategies traded in the market can broadly be grouped into three categories based on how their P&L depends on the underlying's direction. (Learn them in this order, per the source material — direction-dependent strategies first, since they're conceptually simplest.)

1. **Directional-bias trades** — both profit *and* loss depend fully on the underlying moving in the anticipated direction. Simple long option positions fall here: **Long Call, Long Put** (pure directional bets — net *buying* of options, i.e. "buying volatility" with a directional view).
2. **Directional-neutral trades** — the position does not necessarily *lose* regardless of which way the market goes, but it only *profits* if the market moves in one particular intended direction (e.g., certain hedged/limited-risk directional constructions).
3. **Direction-indifferent trades** — the position can **profit regardless of which direction** the market moves. Examples include:
    - **Non-directional / net-selling strategies** ("selling volatility"): **Short Call, Short Put, Covered Call, Short Straddle, Short Strangle, Ratio Call Spread** — these tend to profit when the market stays range-bound (low realized movement), regardless of which side of the range it drifts to.
    - Pure **direction-indifferent, big-move strategies** (net *buying*, "buying volatility"): **Long Straddle, Long Strangle** — profit from a large move in *either* direction.

Strategies that can fall into more than one bucket, depending on construction: Bull Call Spread, Bear Call Spread, Bull Put Spread, Bear Put Spread, Calendar Spread, Butterfly, Iron Butterfly, Broken Wing Butterfly, Condor, Diagonal, Double Diagonal, Collar. These combine both buying and selling legs, so they can be structured to either buy or sell volatility depending on the trader's view — they don't sit purely in one category.

The recommended learning path (per the source): understand pure directional-bias strategies first (simplest — one leg, direction determines everything), then move to strategies with capped risk/direction-dependent profit, then finally to fully direction-indifferent multi-leg strategies (straddles, strangles, and the spread/condor/butterfly family), which require the deepest understanding of how all the Greeks interact simultaneously.

14.11 Key Takeaways

1. **The Greeks are the sensitivities of an option's price** to the five BSM inputs: Delta (price), Gamma (delta's rate of change), Theta (time), Vega (volatility), Rho (rate) — each is a partial derivative of option price with respect to one variable (Gamma is second-order).
2. **Delta ≈ hedge ratio** and (loosely) a probability-of-finishing-ITM proxy; it is the single most-watched Greek in practice.
3. **Gamma tells you how fast your Delta hedge goes stale** — high Gamma near ATM strikes means frequent rebalancing is required.
4. **Theta is the enemy of long option holders** — options are wasting assets, and decay accelerates as expiry nears, especially ATM.
5. **Vega ties option value to *implied* (forward-looking) volatility**, not historical volatility — both calls and puts gain value as IV rises.
6. **Rho matters least day-to-day**, but grows more relevant for long-dated options and in high/rising-rate environments.
7. **All the Greeks change together** whenever the underlying moves — this collective shift *is* your risk, which is why practitioners describe "managing the Greeks" and "managing risk" as the same activity.
8. **Delta increasing is the common goal** across every moneyness bucket (OTM→ATM→ITM→Deep ITM) — and Delta increases only when volatility rises, which is why professionals frame their trading around volatility itself rather than a fixed strike or pure direction call.
9. **Volatility itself is driven by supply and demand**, rising with buying by hedgers, speculators, and arbitrageurs (and with greed-driven rumor-buying) and falling with increased selling (and fear-driven news-selling).
10. **Disciplined risk management (position sizing + stop-loss + continuous Greek monitoring) is what separates structured options trading from directional gambling** — the Greeks dashboard, not the raw P&L screen, is the right thing to watch while a trade is live.

Sources synthesized: CFI "Option Greeks" article; course primer text; Plain Bagel YouTube explainer; FRM Part I (Valuation & Risk Management) lecture on the Greek letters; practical options risk-management lecture (Hindi-language trading course).

Machine Learning in Financial Markets

Chapter 15

Sentiment Analysis: Opinion Mining

15.1 Introduction

Opinions are central to almost all human activities and strongly influence our behaviour. Our beliefs, perceptions of reality, and choices are conditioned to a considerable degree by how others see and evaluate the world. Consequently, both individuals and organizations often seek the opinions of others before making decisions.

Sentiment analysis and opinion mining study opinions and related concepts, including sentiments, evaluations, attitudes, and emotions.

15.2 Growth and Applications

The emergence and rapid growth of sentiment analysis coincided with the rise of social media on the Web, including reviews, forum discussions, blogs, microblogs, and social networks. For the first time, an enormous volume of opinionated material became available in digital form.

Since the early 2000s, sentiment analysis has become one of the most active research areas in natural language processing. It is also studied extensively in:

- data mining;
- Web mining; and
- text mining.

The field has spread beyond computer science into management and the social sciences because of its importance to businesses and society. Industrial activity has also expanded: numerous start-ups have emerged, and many large corporations have developed in-house sentiment-analysis capabilities. Today, such systems are applied across a broad range of business and social domains.

15.3 Books and General Resources

- Bing Liu's *Sentiment Analysis and Opinion Mining* provides a foundational introduction to the subject.
- *Deep Learning-Based Approaches for Sentiment Analysis* surveys developments and techniques in deep-learning-based sentiment analysis.
- *Sentiment Analysis in Social Networks* examines natural-language processing, big-data methods, and statistical techniques for social-media analysis.

15.4 Selected Research Papers

The following papers provide entry points into several sentiment-analysis subtopics:

- Bing Liu, *Sentiment Analysis and Subjectivity*.
- Carlos Iglesias and Antonio Moreno, *Sentiment Analysis for Social Media*.
- Apoorv Agarwal et al., *Sentiment Analysis of Twitter Data*.
- Shanshan Yi and Xiaofang Liu, *Machine Learning Based Customer Sentiment Analysis for Recommending Shoppers and Shops Based on Customers' Reviews*.
- Arwa A. Al Shamsi, Reem Bayari, and Said Salloum, *Sentiment Analysis in English Texts*.

15.5 Training Resources

The following online resources provide practical introductions to sentiment analysis and natural-language processing:

- Udemy: *Sentiment Analysis, Beginner to Expert*.
- Udemy: *Natural Language Processing (NLP) in Python*, including practical Python implementations.
- BuildByPython's YouTube playlist on NLP and sentiment analysis.
- Stanford Online lectures on YouTube: Lecture 1 and Lecture 2, the latter focusing on sentiment analysis.

Chapter 16

Datasets for Self-Practice

Suitable datasets are useful when beginning sentiment-analysis experiments. The following freely available resources provide review, social-media, and financial-language data for self-practice.

16.1 Review and Social-Media Datasets

- **Amazon Reviews Dataset:** Millions of Amazon reviews supplied in `fastText` format.
- **Reddit Bitcoin Comments Dataset:** Reddit comments containing references to Bitcoin from 2009 to 2019.
- **Booking.com Reviews Dataset:** Thousands of hotel reviews suitable for opinion-mining experiments.

16.2 Sentiment-Analysis Lexicons

Rule-based approaches require sentiment lexicons: lists of words that have already been labelled by sentiment or related linguistic categories.

- **Sentiment Lexicons for 81 Languages:** Multilingual sentiment word lists for experimentation across languages.

16.2.1 Loughran–McDonald Master Dictionary (1993–2023)

The Loughran–McDonald Master Dictionary is designed for financial text and is available in the following formats:

- **CSV format:** Download CSV.
- **XLSX format:** Download XLSX.

Baseline Dictionary

The base dictionary is derived from release 4.0 of `2of12inf`, a general-purpose dictionary oriented towards common words. It contains word inflections but excludes abbreviations, acronyms, British English, hyphenated words, names, and phrases.

Financial Extensions

The `2of12inf` baseline is extended with words found in 10-K documents and earnings calls but absent from the original word list. These additions were identified by examining tokens from the full EDGAR 10-K archive and earnings calls from CapIQ. Added entries are either inflections of common words or terms appearing in more than a trivial number of documents.

Reported Metrics

The dictionary reports counts, proportion of total, average proportion per document, standard deviation of proportion per document, document count (the number of documents containing at least one occurrence), seven sentiment-category identifiers, complexity, number of syllables, and the source of each word. The source is either `2of12inf` or the year in which the word was added.

Sentiment Categories

The categories are negative, positive, uncertainty, litigious, strong modal, weak modal, and constraining. Sentiment words are flagged with a number indicating the year in which they were added to the list.

> **Note.** A year preceded by a negative sign indicates the year or version in which the word was removed from the sentiment category.

The best way to improve at this stage is to practise through competitions or by solving real-world problems. Best of luck!

Chapter 17

Time Series Analysis: Handling Volatility

17.1 Why Should We Care About Modelling Volatility?

Volatility is a financial metric that reflects the degree of variation or fluctuation in the price of an asset (stocks, bonds, currencies, commodities, etc.) over time. It is widely used as a proxy for **risk**, since it captures how unpredictable an asset's price movements are.

- **High volatility** → the asset's price can change dramatically over a short period (large risk, large potential reward).
- **Low volatility** → the asset's price movements are more stable and less dramatic (lower risk, typically lower potential reward).

Understanding volatility matters to investors because it directly affects both the **risk** and the **potential return** of an investment:

- High volatility creates opportunities for significant gains, but also exposes the investor to the risk of substantial losses.
- By assessing volatility, investors can make informed decisions about their **risk tolerance**, **portfolio diversification**, and **investment strategy**, helping them navigate market uncertainty more effectively.

This is why an entire branch of time-series modelling (ARCH/GARCH-family models) exists specifically to model and forecast *volatility* rather than the price/return itself.

17.2 How Do We Measure Volatility?

Volatility and **variance** are closely related but distinct concepts:

Concept	Definition
Variance	A statistical measure that quantifies the spread of data points in a dataset — the average *squared* deviation of returns from their mean over a window of time.
Volatility	Proportional to the **square root of variance** (i.e., analogous to a standard deviation). It is more intuitive than variance because it is expressed in the *same units* as the asset's returns.

Key relationship:

```
Volatility proportional to sqrt(Variance)
```

Because raw variance is in "squared return" units (hard to interpret intuitively), volatility (its square root) is preferred by practitioners since it maps back onto the same scale as returns themselves.

This is the conceptual bridge to ARCH/GARCH models: since we cannot reliably predict the *direction* of returns (see the Efficient Market Hypothesis below), we instead try to model and predict the **conditional variance** — i.e., volatility — of the return series.

17.3 Background for ARCH/GARCH Models

17.3.1 The Efficient Market Hypothesis (EMH)

The EMH states that the price of a stock reflects an **efficient market** — i.e., all available information about the company/asset is already incorporated into its current price.

Implications of EMH:

- No essentially riskless (or minimally riskless) profit-taking can exist, because there are no market inefficiencies causing a gap between the current price and the "true" value → **no arbitrage**.
- It is impossible to *consistently* beat the market using only information about a security's price up to time t, since all such information is already public knowledge by time t.

Mathematical statement:

```
E[X_t | X_t-1, X_t-2, ...] = 0
```

where X_t is the current asset price/return, conditioned on all previous prices/returns. In words: the *expected* return, given everything we know so far, is zero (i.e., our best forecast of tomorrow's return is "no change," consistent with a **random walk**).

Important nuance: This does *not* mean there's no way to make a profit — it means there's no *riskless* way to profit (arbitrage). Riskier investments still create opportunities for profit *through* that risk. This is where **volatility modelling** comes in: since we can't forecast the *direction* of returns without violating EMH, we instead forecast the *risk* (variance/volatility) of returns, which is still valuable and does not contradict EMH.

17.3.2 Returns vs. Prices

Rather than modelling raw prices, financial time-series models typically model **returns**, since returns tend to be more stationary and comparable across time and assets.

Simple return at time t:

```
R_t = (P_t - P_t-1) / P_t-1
```

where P_t is the asset price at time t and P_{t-1} is the price at the previous period.

Log return at time t (more commonly used):

```
r_t = log(P_t / P_t-1) = log(P_t) - log(P_t-1)
```

Why log returns are preferred: Log returns have an *additive* property that simple returns do not — the multi-period log return is simply the sum of single-period log returns. This makes them more convenient for statistical modelling.

Python implementation:

```
# Convert prices to log returns
df['Return'] = np.pad(np.diff(np.log(df['Close'])) * 100, (1, 0), 'constant')
```

17.3.3 Volatility Clustering

Volatility clustering refers to the empirical observation that financial time series alternate between **periods of high volatility** and **periods of low volatility** — large price swings tend to be followed by more large swings, and calm periods tend to be followed by more calm periods.

- Example: the S&P 500 shows a visibly "jagged" (high-volatility) pattern during 2010–2012, followed by a relatively calm (low-volatility) period from 2013–2015.
- ARCH and GARCH models are specifically designed to **capture volatility clustering** — allowing us to model risk/volatility in a way that is consistent with the Efficient Market Hypothesis (since we're not forecasting the direction of returns, just their conditional variance).

17.4 Heteroskedasticity — The Core Problem

Traditional autoregressive models (AR, ARIMA, SARIMA) and other common regression-style models (linear regression, ARMA) assume **homoskedasticity** — i.e., that the variance of the error terms is **constant** over time.

However, real financial time series frequently display **heteroskedasticity** — a systematically *changing* (non-constant) variance over time. In finance, this manifests as increasing/decreasing volatility.

- If this changing variance is itself **correlated over time** (i.e., it follows a predictable pattern), it can be modeled using an autoregressive process applied to the variance — this is exactly what **ARCH** does.

Real-world series where ARCH-type effects are commonly observed: **stock prices, oil prices, bond prices, inflation rates, GDP, unemployment rates**, etc.

17.5 The ARCH Model

17.5.1 What Does "ARCH" Mean?

ARCH = Autoregressive Conditional Heteroskedasticity, introduced by Engle (1982). Its core idea is that it explicitly distinguishes between the **unconditional variance** and the **conditional variance**, allowing the conditional variance to change over time as a function of past errors.

Breaking down the name:

Term	Meaning

Autoregressive	The current value can be expressed as a function of previous values — i.e., they are correlated.
Conditional	The variance at time t depends on (is "conditional on") past errors.
Heteroskedasticity	The series displays unusual/varying (non-constant) variance.

17.5.2 Why Use an ARCH Model?

- Standard autoregressive models such as AR, ARIMA, and SARIMA can capture trend and seasonality in the *mean* of a series. However, they assume **constant error variance** and cannot model changes in variance over time.
- ARCH models fill this gap by modelling the **conditional variance** as a function of past squared errors or residuals.
- **Practical use case:** ARCH is typically used to model the expected variance of the *residuals* after another model (e.g., ARMA) has already been fit to remove any trend/seasonality/autocorrelation from the mean.

When is ARCH appropriate?

- ARCH should be applied only to a series that has **no trend, no seasonality, and no evident serial correlation** in the mean (i.e., it looks like discrete white noise once any ARMA/ARIMA structure has been removed).
- We check appropriateness by **squaring the residuals** and examining their **correlogram** (ACF/PACF) — we also need the mean of the residuals to be (near) zero.
- Typical workflow: fit ARIMA/SARIMA first → check if ARCH effects remain in the squared residuals → if yes, ARCH/GARCH may be a good fit for modelling the remaining variance structure.

17.5.3 ARCH Model Statement and Derivation

A time series $\{\epsilon(t)\}$ is defined at each time step as:

```
epsilon(t) = w(t) * sigma(t)
```

where **w(t)** is white noise with zero mean and unit variance.

Conditional variance equation (ARCH(1)):

```
Var(x(t)) = sigma^2(t) = alpha0 + alpha1 * sigma^2(t-1)
```

or, expressed directly in terms of the previous **squared error** (more standard form):

```
sigma^2(t) = alpha0 + alpha1 * epsilon^2(t-1)

epsilon(t) = w(t) * sqrt(alpha0 + alpha1 * epsilon^2(t-1))
```

Parameter constraints: $\alpha_0 > 0$ and $\alpha_1 \geq 0$, which ensures the conditional variance is always positive.

This is called an **autoregressive conditional heteroskedastic model of order unity**, denoted **ARCH(1)**.

ARCH(2):

```
epsilon(t) = w(t) * sqrt(alpha0 + alpha1*epsilon^2(t-1) + alpha2*epsilon^2(t
    -2))
```

General ARCH(p):

```
epsilon(t) = w(t) * sqrt( alpha0 + sum_i=1^p alpha_i * epsilon^2(t-i) )
```

where:

- **p** = the number of lagged squared residual errors included in the model.
- **i = 1, 2, 3, . . . , p** indexes the lagged periods of the squared error.

17.5.4 Interpretation of the ARCH Model

1. If the error was **high** during period (t−1), it is more likely that the error at period (t) will also be **high** (volatility persists/clusters).
2. Conversely, if the error was **low** at (t−1), the term inside the square root is smaller, resulting in a smaller (lower-volatility) error at time (t).
3. $\alpha_1 \geq 0$ is required to guarantee positive variance.
4. **Stability condition:** $\alpha_1 < 1$ is required, otherwise $\epsilon(t)$ becomes explosive (grows without bound over time).
5. ARCH(1) should only ever be applied to a series that has already had an appropriate mean model fitted, such that its residuals resemble discrete white noise.

Why does this model "volatility"?
From the variance formula, we can derive:

```
Var(epsilon(t)) = alpha0 + alpha1 * Var(epsilon(t-1))
```

This shows that the variance of the series is simply a **linear combination** of the variance of the *previous* element in the series — i.e., today's volatility depends linearly on yesterday's volatility (via the squared error term).

17.5.5 Formal Model Statement (Alternative Notation)

An equivalent and commonly used notation for ARCH(P):

```
X_t = sigma_t*epsilon_t,   with   sigma_t^2 = b0 + sum_j=1^p b_j*X_t-_j^2
```

where ε_t is IID(0,1) and $b_0, b_1, \ldots, b_p > 0$.

- X_t can represent price, returns, or log returns — for simplicity, X_t is usually taken to be the **log returns** of an asset.
- **X_t^2 is (in expectation) the marginal variance of the log returns.**

Derivation — why X_t^2 represents marginal variance:
Recall the identity:

```
Var(X) = E[X^2] - E[X]^2
```

Because of the Efficient Market Hypothesis, expected returns are trivially zero: $\mathbf{E[X_t] = 0}$. Therefore $E[X_t]^2 = 0$, and we are left with:

```
sigma^2 = E[X_t^2]
```

which is **constant** and does not depend on t — this is the marginal (unconditional) variance.

We build on this concept of a constant marginal variance to introduce **heteroskedasticity**: the **conditional variance at time t (σ_t^2)** — i.e., volatility — is modeled as being directly influenced by the *squared* log return $\mathbf{X_t^2}$.

Intuition: If X_t swings rapidly (the log return increases or decreases sharply), then ${X_t}^2$ is large, and therefore the volatility ${\sigma_t}^2$ at time t is high. This is precisely how ARCH captures the impact of large price swings on the volatility of an asset.

17.6 The GARCH Model

17.6.1 What Does "GARCH" Mean?

GARCH = Generalized Autoregressive Conditional Heteroskedasticity. It is an extension of ARCH that adds a **moving-average component** on top of the autoregressive component. It was introduced by **Bollerslev (1986, Journal of Econometrics)**, generalizing Engle's original ARCH model.

Adding the moving-average component allows the model to:

1. Model the **conditional change in variance** over time.
2. Capture **time-dependent changes in variance** (e.g., conditional increases and decreases in variance) more flexibly than ARCH alone.

Conceptually: GARCH is the **"ARMA equivalent" of ARCH**, since ARCH only has an autoregressive component (on the variance), whereas GARCH adds a moving-average-like component. This lets GARCH capture a wider range of behavior, including more **persistent volatility**.

17.6.2 GARCH Model Statement and Derivation

GARCH(1,1): a single autoregressive lag + a single "moving average" lag on the variance:

```
epsilon(t) = w(t) * sigma(t)

epsilon(t) = w(t) * sqrt( alpha0 + alpha1*epsilon^2(t-1) + beta1*sigma^2(t-1)
   )
```

General GARCH(p,q):

A time series $\{\epsilon(t)\}$ is given at each instance by:

```
epsilon(t) = w(t) * sigma(t)
```

with the conditional variance given by:

```
sigma^2_t = alpha0 + sum_i=1? alpha_i*epsilon^2(t-i) + sum_j=1^p beta_j*sigma
   ^2(t-j)
```

where α(i) and β(j) are the model's parameters, and:

```
alpha0 > 0,   alpha_i >= 0 for i = 1,...,q,   beta_j >= 0 for j = 1,...,p
```

(these constraints ensure the conditional variance is always positive).

Interpretation of the moving-average addition: $\sigma^2(t)$ now depends not only on past *squared errors* (ARCH effect) but also on past *conditional variances* $\sigma^2(t-j)$ themselves (GARCH effect).

17.6.3 Interpretation of GARCH Coefficients

1. A **large value of** β_1 causes $\sigma(t)$ to be highly correlated with $\sigma^2(t-1)$, giving the conditional standard deviation process a **long-term persistence** — much more so than under a pure ARCH model.
2. **Special case — p = 0:** the process reduces to a pure **ARCH(q)** process.
3. **Special case — p = q = 0:** $\epsilon(t)$ is simply **white noise**.

17.6.4 Alternative Notation (used in the second source article)

```
X_t = sigma_t*epsilon_t,   with   sigma_t^2 = b0 + sum_j=1^p b_j*X_t-_j^2 +
   sum_j=1^n a_j*sigma^2_t-_j
```

where ε_t is IID(0,1), and $b_0, b_1, \ldots, b_p, a_0, a_1, \ldots, a_n \neq 0$.

GARCH(P,Q) is the generalized ARCH model that adds the extra term above: it allows the conditional variance to depend on **lagged versions of itself** (via $a_0, a_1, \ldots, a_n$), in addition to lagged squared returns.

We can decompose a GARCH model's effects into two components:

1. **ARCH effects** — attributable to lagged squared log returns ${X_t}^2$.
2. **GARCH effects** — attributable to lagged conditional variances ${\sigma_t}^2$.

This additional flexibility is what allows GARCH models to better capture the **persistence** of volatility observed in real financial data.

17.7 Model Identification: Using ACF and PACF

17.7.1 General Procedure

The configuration (choice of order p, q) for ARCH/GARCH models is best understood via **ACF and PACF plots of the variance** of the time series:

1. Subtract the mean from each observation and **square the result** (or simply square the observation if you're already working with white-noise residuals from another fitted model).
2. Interpret the resulting **ACF and PACF plots** to estimate values for p and q — in a manner analogous to how ARMA model orders are chosen.

17.7.2 Worked Example (S&P 500 Returns)

Steps followed on real S&P 500 data (2010–2020):

1. Plot a **trace plot** of the returns over time to visually inspect for volatility clustering (e.g., visibly higher highs/lower lows in 2010–2012 vs. a calmer 2013–2015 period).
2. Plot **ACF and PACF of the raw returns**. In the worked example, these plots showed **no significant spikes** outside the confidence bands — suggesting the *raw* returns series looks close to independent, and would not, on its own, indicate ARCH/GARCH effects.

3. To more rigorously test for independence, repeat the ACF/PACF analysis on a **transformation** of the series — either **squared returns** or **absolute returns**. Absolute returns are generally preferred in financial data because they **diminish the effect of outliers** compared to squared returns.
4. After transforming to **absolute returns**, a clear pattern emerges: ACF and PACF values are now significantly outside the confidence bands at multiple lags — confirming the presence of ARCH/GARCH effects. In the example, the PACF does not show a sharp drop-off until around **lag 4**, suggesting an order of P, Q up to 4 might be appropriate. For simplicity, the analysis proceeds first with a **GARCH(1,1)** model before testing more complex specifications.

```
# Plot ACF and PACF of raw returns
diff_ts = df['Return'].iloc[1:]
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
plot_acf(diff_ts, ax=ax1, lags=10)
plot_pacf(diff_ts, ax=ax2, lags=10)

# Plot ACF and PACF of absolute returns (more robust to outliers)
abs_returns = diff_ts.abs()
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
plot_acf(abs_returns, ax=ax1, lags=10)
plot_pacf(abs_returns, ax=ax2, lags=10)
```

17.8 Formal Testing for ARCH/GARCH Effects: The McLeod-Li Test

The **McLeod-Li test** formally tests for white-noise behavior in the residuals of a series (i.e., whether ARCH effects are present).

Hypotheses:

- **H_0:** There are **no ARCH effects** in the time series.
- **H_a:** There **are ARCH effects** in the time series.

Workflow: First fit a GARCH(1,1) model, then apply the McLeod-Li test to the residuals.

Worked example result: p-value $< 0.001 \rightarrow$ reject $H_0 \rightarrow$ conclude that ARCH effects **are** present in the time series.

```
def mcleod_li_test(x, k):
    """
    Calculates the McLeod-Li test statistic for a time series with k lags.
    Returns the test statistic and its p-value.
    """
    n = len(x)
    x_sq = x ** 2
    x_sum = np.sum(x_sq)
    x_lag_sum = np.sum(x_sq[:-k])
    test_stat = n * (n + 2) * x_lag_sum / (x_sum ** 2)
    df = k
    p_value = 1 - chi2.cdf(test_stat, df)
```

```
    return test_stat, p_value

test_stat, p_value = mcleod_li_test(abs_returns, 50)
# Result: test statistic ~ 2929.75, p-value ~ 0.0 -> ARCH effects present
```

17.9 Simulating ARCH and GARCH Processes in Python

Simulating these processes is a useful way to build intuition for how the models behave.

Setup:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from arch import arch_model
import warnings
warnings.filterwarnings("ignore")
```

17.9.1 Simulating an ARCH(1) Process

```
np.random.seed(13)
# a0 and a1 are constants and should be > 0
a0 = 2
a1 = .5

y = w = np.random.normal(size=1000)   # random number generator, size 1000
Y = np.empty_like(y)

for t in range(len(y)):
    Y[t] = w[t] * np.sqrt((a0 + a1 * y[t-1]**2))   # ARCH(1) formula

# simulated ARCH(1) series -- looks like white noise
tsplot(Y, lags=30)
```

Observation on the raw simulated series: the ARCH(1) series itself visually resembles white noise, and its ACF/PACF show no significant lags.

17.9.2 Squared ARCH(1) Series

```
tsplot(np.square(Y), lags=30)   # squared Y
```

Observation: Once the series is **squared**, the ACF and PACF now show significance at **lag 1**, indicating that an **AR(1) model for the variance** may be appropriate — this is exactly the ARCH(1) structure we simulated, confirming the model captures the intended dependency in the *variance*, even though the raw series looks like white noise.

17.9.3 Simulating a GARCH(1,1) Process

```
np.random.seed(1)
# Constants
a0 = 0.2
a1 = 0.5
b1 = 0.3

n = 10000
w = np.random.normal(size=n)
eps = np.zeros_like(w)
sigsq = np.zeros_like(w)

for i in range(1, n):
    sigsq[i] = a0 + a1*(eps[i-1]**2) + b1*sigsq[i-1]
    eps[i] = w[i] * np.sqrt(sigsq[i])     # GARCH formula

tsplot(eps, lags=30)
```

Observation: Overall, this process again closely resembles white noise when plotted raw.

17.9.4 Squared GARCH(1,1) Series

```
tsplot(np.square(eps), lags=30)
```

Observation: The squared series now shows **clear autocorrelation**, with significant lags in *both* the ACF and PACF — indicating that we need **both AR and MA components** (i.e., both α and β terms) to properly model this variance process, which matches the GARCH structure used to simulate it.

17.9.5 Fitting a GARCH(1,1) Model to the Simulated Series

```
# Fit a GARCH(1,1) model to our simulated EPS series using the `arch` package
from arch import arch_model

am = arch_model(eps)
res = am.fit(update_freq=5)
print(res.summary())
```

Example fitted output (from the simulated series):

```
Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:                            y   R-squared:
    -0.000
Mean Model:                  Constant Mean   Adj. R-squared:
    -0.000
Vol Model:                           GARCH   Log-Likelihood:
    -12199.1
Distribution:                       Normal   AIC:
    24406.3
```

```
Method:                 Maximum Likelihood   BIC:
    24435.1
                                             No. Observations:
    10000
                                             Df Residuals:
     9996
                                             Df Model:
        4
                              Mean Model
============================================================================
                 coef    std err          t      P>|t|        95.0% Conf. Int.
mu           -5.97e-07   6.742e-03    -8.86e-05      1.000     [-1.321e-02,
    1.321e-02]
                            Volatility Model
============================================================================
              coef    std err          t      P>|t|        95.0% Conf. Int.
omega       0.2157    1.014e-02     21.282   1.678e-100    [0.196, 0.236]
alpha[1]    0.4939    1.960e-02     25.199   4.134e-140    [0.455, 0.532]
beta[1]     0.2781    1.841e-02     15.107   1.452e-51     [0.242, 0.314]
============================================================================
```

Note the recovered parameters (omega ≈ 0.22, alpha[1] ≈ 0.49, beta[1] ≈ 0.28) are close to the true simulation parameters (a0=0.2, a1=0.5, b1=0.3) used to generate the data — confirming the model correctly recovers the underlying GARCH structure.

17.10 Applying GARCH to Real Data (S&P 500)

17.10.1 Data Import

```
spy = yf.Ticker("SPY")
hist = spy.history(start="2010-01-04", end="2020-02-01")
df = pd.DataFrame(hist, columns=['Close'])
```

17.10.2 Fitting GARCH(1,1) on Absolute Returns

Workflow: split into an 80/20 train/test set (test set = more recent data), then fit using `arch_model` with mean set to zero, GARCH volatility, and p=q=1:

```
y_train, y_test = train_test_split(abs_returns, train_size=0.8)

garch_mod = arch_model(y_train, mean="Zero", vol='Garch', p=1, q=1, rescale=
    False)
res_garch = garch_mod.fit()
```

Resulting fitted model equation:

```
sigma_t^2 = 0.0309 + 0.14*X^2_t-1 + 0.82*sigma^2_t-1
```

where $\sigma_t{}^2$ is the conditional variance at time t, X^2_{t-1} is the squared return at time $t-1$, and σ^2_{t-1} is the conditional variance at time $t-1$.

Parameter estimates: b_0 (omega) = 0.0309, b_1 (alpha[1]) = 0.14, a_1 (beta[1]) = 0.82.

```
Zero Mean - GARCH Model Results
==============================================================================
Dep. Variable:                      Return   R-squared:
   0.000
Mean Model:                     Zero Mean   Adj. R-squared:
   0.000
Vol Model:                          GARCH   Log-Likelihood:
   -2403.69
Distribution:                       Normal   AIC:
   4813.38
Method:                Maximum Likelihood   BIC:
   4830.23
                                            No. Observations:
    2028
                            Volatility Model
==============================================================================
               coef      std err         t          P>|t|      95.0% Conf. Int.
omega        0.0309    8.422e-03      3.667      2.450e-04   [0.0144, 0.0474]
alpha[1]     0.1432    2.548e-02      5.621      1.900e-08   [0.0933, 0.193]
beta[1]      0.8196    2.771e-02      29.575     3.116e-192  [0.765, 0.874]
==============================================================================
```

Interpretation: The high β_1 ($\approx$0.82) indicates strong persistence of volatility — consistent with the "volatility clustering" phenomenon discussed earlier.

17.11 Forecasting Volatility

Once fitted, a GARCH model lets us forecast σ_t^2 (volatility) at future times — these are non-trivial predictions (unlike return forecasts, which are trivially zero under EMH). There are two main forecasting approaches:

17.11.1 N-Step-Ahead Forecast

- The model accesses log-return data X_t, computes $\sigma^2{}_{t+1}$, and then **extends** this forecast forward to t+1, t+2, ..., t+n **without updating** X_t as time progresses.
- In other words: all future volatility forecasts are based solely on information available at the current date — the model does *not* incorporate new actual data as it becomes available.

```
yhat = res_garch.forecast(horizon=y_test.shape[0], reindex=True)

plt.plot(diff_ts[-y_test.shape[0]:])
plt.plot(y_test.index, np.sqrt(yhat.variance.values[-1, :]))
plt.title('S&P 500 Volatility Prediction')
plt.legend(['True Daily Log Returns', 'Predicted Volatility'])
```

Observation: The N-step-ahead forecast produces a relatively **flat/smoothed** volatility prediction over the forecast horizon, since it isn't updated with new information

as time progresses — it tends to converge toward the long-run average volatility level rather than tracking short-term spikes.

17.11.2 One-Step-Ahead Rolling Forecast

- A more pragmatic approach: only forecast **one step ahead** at a time, then **update** the model with the actual observed return once it becomes available, and repeat.
- In other words: with each passing day, X_t, X_{t+1}, X_{t+2}, ... are updated with the actual observed returns, and the model re-forecasts one step ahead at a time.

```
rolling_preds = []
for i in range(y_test.shape[0]):
    train = abs_returns[:-(y_test.shape[0]-i)]
    model = arch_model(train, p=1, q=1, rescale=False)
    model_fit = model.fit(disp='off')
    pred = model_fit.forecast(horizon=1, reindex=True)
    rolling_preds.append(np.sqrt(pred.variance.values[-1, :][0]))

rolling_preds = pd.Series(rolling_preds, index=y_test.index)

plt.plot(rolling_preds)
plt.title('S&P 500 Rolling Volatility Prediction')
```

Observation — strengths of the rolling approach:

1. It generates **spikes in predicted volatility** that line up well with periods of actual higher variance in daily returns.
2. It successfully captures **volatility clustering** — alternating periods of low and high predicted variance.
3. It tends to **forecast volatility spikes in advance** of the actual swings in daily returns, making it more useful in practice than the static n-step-ahead approach.

17.11.3 Comparison

Approach	Description	Behavior
N-step-ahead	Forecast horizon fixed at the current date; no updating with new data	Smoothed/flat forecast, converges to long-run average
One-step-ahead rolling	Re-forecast one step at a time, updating with each new observation	Tracks volatility clustering and spikes much more responsively

The rolling approach is generally the more **practical** choice for real-world volatility forecasting, since it continually incorporates the latest information.

17.12 Model Diagnostics

Model diagnostics are an important (and often overlooked) step to validate whether a fitted GARCH model is appropriate, and to identify possible next steps for improvement.

17.12.1 Jarque-Bera Test (Normality of Residuals)

The GARCH(P,Q) model assumes that, if correctly specified, the model's errors are **normally distributed** with mean 0 and constant variance. The **Jarque-Bera test** checks whether the standardized residuals are normally distributed.

Test statistic formula:

```
JB = (n/6) * [ S^2 + (1/4)(K - 3)^2 ]
```

where:

- **n** = sample size
- **S** = sample skewness (symmetry of the data relative to the mean)
- **K** = sample kurtosis (shape of the distribution, particularly the tails)

Reference point: the kurtosis of a normal distribution is **3**.

- $K > 3 \rightarrow$ the sample is **heavier-tailed** than normal.
- $K < 3 \rightarrow$ the sample is **lighter-tailed** than normal.

Hypotheses:

- $\mathbf{H_0}$**:** the data has a normal distribution.
- $\mathbf{H_a}$**:** the data does not have a normal distribution.

```
std_resid = res_garch.resid / res_garch.conditional_volatility

jb_test = jarque_bera(std_resid)
print("Jarque-Bera test statistic:", jb_test[0])
print("p-value:", jb_test[1])
# Result: JB statistic ~ 1950.91, p-value ~ 0.0
```

Conclusion from worked example: p-value $\approx 0 \rightarrow$ reject $H_0 \rightarrow$ the standardized residuals do **not** follow a normal distribution.

17.12.2 Implication — Heavy Tails in Financial Data

- Financial time series typically have **tails heavier** than what is implied by a GARCH process using Gaussian (normal) innovations $\{\epsilon(t)\}$.
- To address this, one can assume $\{\epsilon(t)\}$ follows a **heavy-tailed distribution** (e.g., **Student's t-distribution**) instead of a standard Gaussian white noise process.
- Interestingly, GARCH processes can exhibit heavy tails in their *outputs* even when the underlying innovations $\{\epsilon(t)\}$ *are* Gaussian — this is a natural byproduct of the conditional variance structure.
- **Practical takeaway:** Using GARCH models allows us to jointly model both (a) the conditional heteroskedasticity, and (b) the heavy-tailed nature of financial returns distributions — but for better-calibrated models, it's often worth specifying a heavier-tailed error distribution (like Student's t) rather than assuming Gaussian errors.

17.13 Model Extensions and Related Techniques

Several extensions/variants of ARCH/GARCH exist in the finance industry for more specialized use cases:

- **ARCH-M** — ARCH-in-Mean, where volatility feeds directly into the mean equation (e.g., to capture a risk premium).
- **GARCH-M** — GARCH-in-Mean, analogous extension of GARCH.
- **TGARCH** — Threshold GARCH, allows asymmetric responses to positive vs. negative shocks.
- **EGARCH** — Exponential GARCH, models the log of the conditional variance, allowing asymmetric effects without needing non-negativity constraints on coefficients.

17.13.1 Combining ARIMA with GARCH

A common and powerful real-world workflow combines a **mean model (ARIMA)** with a **variance model (GARCH)**:

1. Iterate over combinations of ARIMA(p, d, q) models to find the best fit for the mean of the time series (e.g., using AIC as the selection criterion).
2. Choose the GARCH model order(s) according to the best-fit ARIMA model.
3. Fit the GARCH(p, q) model to the time series (often on the ARIMA residuals).
4. Examine the model's residuals and *squared* residuals for remaining autocorrelation (to confirm the GARCH component adequately captures the volatility structure).

Worked example (SPY returns, 2012–2015):

```
def _get_best_model(TS):
    best_aic = np.inf
    best_order = None
    best_mdl = None
    pq_rng = range(5)  # [0,1,2,3,4]
    d_rng = range(2)   # [0,1]
    for i in pq_rng:
        for d in d_rng:
            for j in pq_rng:
                try:
                    tmp_mdl = smt.ARIMA(TS, order=(i, d, j)).fit(method='mle',
    trend='nc')
                    tmp_aic = tmp_mdl.aic
                    if tmp_aic < best_aic:
                        best_aic = tmp_aic
                        best_order = (i, d, j)
                        best_mdl = tmp_mdl
                except:
                    continue
    return best_aic, best_order, best_mdl

TS = lrets.SPY.ix['2012':'2015']
res_tup = _get_best_model(TS)
# Output: aic = -5255.56734 | order = (3, 0, 2)
```

Result: ARIMA(3,0,2) was found to best fit the mean of the SPY return series (lowest AIC). The next step is to fit a **GARCH(3,2)** model (using the ARIMA order as a starting point for the GARCH order) and examine the ACF/PACF of the squared residuals to check for remaining autocorrelation in the variance.

This combined **ARMA + GARCH** approach — modelling the mean with ARIMA and the variance with GARCH — is one of the most versatile and widely used approaches in practical volatility forecasting.

17.14 Key Takeaways / Summary

- **Volatility** measures the degree of price fluctuation in an asset and is a core proxy for financial **risk**; it is proportional to the square root of variance.
- Standard time-series models (ARIMA/SARIMA) assume **homoskedasticity** (constant variance) and cannot capture changing variance over time.
- **Heteroskedasticity** — systematically changing variance — is common in financial data, and when this changing variance is itself autocorrelated, it can be modeled with an autoregressive process on the *variance*.
- **ARCH(p):** models the conditional variance as a function of **p lagged squared errors/returns**. Requires $\alpha_0 > 0$, $\alpha_i \geq 0$, and a stability condition ($\Sigma\alpha < 1$) to avoid explosive variance.
- **GARCH(p,q):** extends ARCH by adding lagged **conditional variances** themselves (the "moving average" component on variance), giving it the ability to model more persistent, flexible volatility dynamics. GARCH is the "ARMA equivalent" of ARCH.
- Model **identification** uses ACF/PACF plots on the (squared or absolute) residuals/returns; absolute returns are preferred in practice to reduce the influence of outliers.
- The **McLeod-Li test** formally tests for the presence of ARCH/GARCH effects (H_0: no ARCH effects).
- **Two forecasting strategies:** n-step-ahead (static, no updating — smoother, less responsive) vs. one-step-ahead rolling forecasts (updates with new data — captures volatility clustering and spikes much more effectively).
- **Model diagnostics** (e.g., Jarque-Bera test) often reveal that GARCH residuals are not perfectly normal — financial data tends to have **heavy tails**, motivating the use of heavier-tailed error distributions (e.g., Student's t) in more advanced specifications.
- In practice, GARCH is frequently combined with an **ARIMA** mean model (ARIMA + GARCH) — ARIMA captures the mean/trend structure, while GARCH captures the residual volatility structure.
- Extensions such as **ARCH-M, GARCH-M, TGARCH, and EGARCH** exist to handle more specialized volatility dynamics (e.g., asymmetric responses to positive vs. negative shocks).

17.15 References

- Ranjith Kumar K, "Time Series Model(s) — ARCH and GARCH", Medium.
- Daniel Herrera, "Forecasting Volatility: Deep Dive into ARCH & GARCH Models", Medium.
- Engle, R. (1982). Original ARCH paper.
- Bollerslev, T. (1986). *Journal of Econometrics* — original GARCH paper.

Chapter 18

Sequence-to-Sequence Modelling in Finance

18.1 What is Sequence-to-Sequence (Seq2Seq) Modelling?

Sequence-to-sequence (Seq2Seq) modelling is a machine learning approach that transforms one sequence of data into another sequence. It is the standard framework behind machine translation, time-series analysis, and speech recognition.

A Seq2Seq model has two main components:

- **Encoder**: reads the input sequence, step by step, and compresses everything it has seen into a single fixed-size vector called the **context vector**. This vector is meant to summarize the essential information contained in the whole input sequence.
- **Decoder**: takes the context vector produced by the encoder and uses it to generate the output sequence, step by step.

Because the encoder and decoder are separate modules connected only through the context vector, the input and output sequences do **not** need to be the same length — this is what makes Seq2Seq so flexible (e.g., translating a 5-word sentence into a 7-word sentence, or using 60 days of stock prices to forecast the next 10 days).

Attention mechanism: Modern Seq2Seq models rarely rely on a single fixed context vector, since compressing an entire long sequence into one vector loses information (a bottleneck problem). Instead, they add an *attention mechanism*, which allows the decoder to "look back" at different parts of the input sequence and weigh their relevance differently while generating each element of the output. This significantly improves performance, especially on long sequences, and adds flexibility to what the model can learn.

18.1.1 How is Seq2Seq used in Time Series?

Use case	What happens	Example in finance
Forecasting	The encoder compresses a window of past time steps into a context vector; the decoder unrolls this into a sequence of predicted future values.	Predicting the next N days of a stock price or trading volume from the last M days.

Anomaly Detection	The model learns the "normal" pattern of a time series by learning to predict it well. When the actual observed values deviate significantly from what the model predicted, that deviation is flagged as a possible anomaly.	Flagging unusual market behaviour (e.g., a sudden price/volume spike that doesn't match the learned pattern), useful for spotting shocks or volatility clusters.
Pattern Recognition	Historical sequences are encoded and then decoded into predefined pattern classes rather than raw future values, i.e., the decoder outputs a classification instead of a forecast.	Classifying different types of temporal patterns in financial markets (e.g., recognizing a regime) or in industrial process monitoring.

Why this matters for volatility: all three use cases are directly relevant to a volatile financial series — forecasting the next values, catching abnormal swings early (anomaly detection), and recognizing which "regime" or pattern the market is currently in.

18.2 Recurrent Neural Networks (RNNs) — Foundations

18.2.1 From Feedforward Networks to Recurrence

A standard (feedforward) neural network has:

- An **input layer** — takes in the raw input.
- One or more **hidden layers** — apply an activation function to a weighted combination of inputs.
- An **output layer** — produces the final prediction.

In a *deep* neural network, there are multiple hidden layers, and **each hidden layer has its own distinct set of weights and biases**. Because each layer's weights differ, each layer behaves independently, and the layers cannot naturally be treated as a single repeating unit.

The key idea behind an RNN: if you force all the hidden layers to *share the same weights and biases*, they effectively become the same computational unit repeated over and over. Once every hidden layer uses identical weights/biases, they can be collapsed ("folded") into **one single recurrent layer** that is applied repeatedly — once per time step. This is exactly what a Recurrent Neural Network does: it reuses one hidden unit across every step of a sequence, and that unit's own previous output feeds back in as part of its next input.

18.2.2 Formal Definition

Given an input sequence:

```
x = (x1, x2, ..., xT)
```

an RNN maintains a **recurrent hidden state** `h_t`, whose value at each time step depends on the input at that step *and* on the hidden state from the previous step:

```
h_t = 0                          if t = 0
h_t = phi(h_(t-1), x_t)            otherwise
```

where ϕ is a nonlinear function (e.g., a sigmoid or an affine transform followed by a nonlinearity). The RNN may optionally also produce an output sequence `y = (y1, y2, ..., yT)`, which can itself be of variable length.

The classic ("vanilla") way to implement the hidden-state update is:

```
h_t = g(W*x_t + U*h_(t-1))
```

where `g` is a bounded, smooth nonlinearity such as a logistic sigmoid or `tanh`, `W` is the weight matrix applied to the current input, and `U` is the weight matrix applied to the previous hidden state.

Generative interpretation: An RNN can also be used to output a probability distribution over the *next* element of a sequence given everything seen so far. Using the chain rule of probability, the likelihood of an entire sequence can be decomposed as:

```
p(x1, ..., xT) = p(x1) * p(x2 | x1) * p(x3 | x1, x2) * ... * p(xT | x1, ...,
   x_(T-1))
```

with a special end-of-sequence symbol marking where the sequence terminates. Each conditional probability is modelled as `p(x_t | x1, ..., x_(t-1)) = g(h_t)`, where `h_t` comes from the recurrence above. This generative view is the foundation of sequence-modelling tasks such as language modelling and, by extension, time-series forecasting.

18.2.3 Visualizing the Recurrence ("Network Fold")

An RNN is often drawn as a single loop that gets "unfolded" across time steps:

```
      O                    O_(t-1)     O_t      O_(t+1)
      |                       |          |          |
      V                       V          V          V
  S (loop)(loop, weight W)  ==>  S_(t-1) -W-> S_t  -W-> S_(t+1)
      |                       |          |          |
      U                       U          U          U
      X                    X_(t-1)     X_t      X_(t+1)
```

- `X` = input at a time step
- `S` (or `h`) = hidden/recurrent state
- `O` = output at a time step
- `U` = weights from input to hidden state
- `W` = weights from the hidden state to itself (the recurrent connection, reused every step)
- `V` = weights from hidden state to output

The left side shows the *folded* (compact) representation — a single cell with a self-loop. The right side shows the same cell *unfolded* across time steps `t-1, t, t+1`, making explicit that the same weight matrices `U`, `V`, `W` are reused at every time step.

18.2.4 Types of RNNs (by Input/Output Shape)

Type	Description	Example
One-to-One	Single input → single output. Equivalent to a standard (vanilla) feedforward network — no real sequential behaviour.	Basic classification of a single fixed input.
One-to-Many	Single input → sequence of outputs.	Music generation from a single seed/style input.
Many-to-One	Sequence of inputs → single output.	Sentiment classification of a sentence; predicting the next word given prior words.
Many-to-Many	Sequence of inputs → sequence of outputs (this is the Seq2Seq case).	Language translation; multi-step financial time-series forecasting.

18.2.5 Applications of RNNs

- **Speech Recognition** — converting spoken audio into text, and translating speech across languages.
- **Language Translation** — combined with text mining and sentiment analysis for NLP tasks.
- **Image Recognition and Captioning** — analyzing sequences of visual activity and features.
- **Time Series Forecasting** — e.g., predicting stock prices over an upcoming period, which is the case most relevant to this course.

18.3 The Vanishing (and Exploding) Gradient Problem

Before building any RNN in practice, it's essential to understand *why* plain ("vanilla" / tanh) RNNs struggle to learn long-range dependencies.

18.3.1 What Is It?

The vanishing gradient problem was identified by **Sepp Hochreiter** (a German computer scientist who also contributed to the development of RNNs, including being a co-inventor of the LSTM). It is a difficulty that arises when training deep networks — and RNNs in particular — using gradient descent combined with backpropagation.

Why it happens in RNNs specifically: An RNN reuses the *same* weights at every time step, and the hidden state at time `t` depends on the hidden state at `t-1`, which depends on `t-2`, and so on. When backpropagating the error signal from a late time step back to an early one, the gradient must pass through many repeated multiplications (one per time step) involving the same weight matrix and the derivative of a bounded activation function (like `tanh` or sigmoid). Since these per-step derivatives are typically less than 1 in magnitude:

- If they are consistently **less than 1**, repeated multiplication shrinks the gradient **exponentially** as it is propagated backward through many time steps → the

gradient "vanishes," and the network effectively cannot learn dependencies that span a long time gap, because the earliest time steps receive almost no learning signal.

- If they are consistently **greater than 1**, the reverse can happen — gradients **explode**, becoming numerically unstable.

In short: the effect of long-term dependencies gets hidden/overwhelmed by the (exponentially larger, in comparison) effect of short-term dependencies, which confuses gradient-based optimization.

18.3.2 Approaches to Mitigate It

Two broad families of solutions exist:

1. **Better optimization algorithms**, rather than plain stochastic gradient descent:
   - **Gradient clipping** — rescale the gradient's norm to a fixed maximum whenever it exceeds a threshold, to explicitly prevent exploding gradients.
   - **Second-order optimization methods**, which can be less sensitive to this issue (though this is not guaranteed, since it depends on the growth pattern of second derivatives matching that of first derivatives).
   - **Weight initialization** — choosing a good initial value for the weights so that backpropagation doesn't immediately start assigning vanishingly small weights.
2. **More sophisticated recurrent units (gating mechanisms)** — instead of trying to patch the optimizer, redesign the hidden unit itself so it is naturally resistant to vanishing gradients. This is the approach behind **LSTM** and **GRU**, covered in detail next.

18.4 Gated Recurrent Units: LSTM and GRU

Both LSTM and GRU were designed to let a recurrent unit *selectively remember or forget* information, instead of being forced to overwrite its entire state at every time step (as the vanilla RNN does). This is achieved through **gates** — small neural network layers (typically a sigmoid) that output values between 0 and 1, acting as "how much information to let through" controls.

18.4.1 Long Short-Term Memory (LSTM)

Originally proposed by Hochreiter & Schmidhuber (1997). Each LSTM unit `j` maintains its own **memory cell** `c_t^j`, separate from the hidden output, at every time step `t`.

Output (activation) of the unit:

```
h_t^j = o_t^j * tanh(c_t^j)
```

where `o_t^j` is the **output gate**, which controls how much of the memory content is exposed to the rest of the network:

```
o_t^j = sigma(W_o x_t + U_o h_(t-1) + V_o c_t)^j
```

(σ = logistic sigmoid; `V_o` is a diagonal matrix, meaning the output gate for unit `j` only looks at cell `j`'s own memory, not other units' memories.)

Memory cell update — the cell content is a mix of "keep some of the old memory" plus "add some new memory content":

```
c_t^j = f_t^j * c_(t-1)^j + i_t^j * c_tilde_t^j
```

where the **candidate new memory content** is:

```
c_tilde_t^j = tanh(W_c x_t + U_c h_(t-1))^j
```

- `f_t^j` = **forget gate** — how much of the existing memory to keep/discard.
- `i_t^j` = **input gate** — how much of the newly computed candidate content to add in.

Both are computed the same way as the output gate (sigmoid over a combination of current input, previous hidden state, and previous cell state):

```
f_t^j = sigma(W_f x_t + U_f h_(t-1) + V_f c_(t-1))^j
i_t^j = sigma(W_i x_t + U_i h_(t-1) + V_i c_(t-1))^j
```

Intuition: Unlike a vanilla recurrent unit, which *overwrites* its content at every step, the LSTM can *decide* — via its gates — whether to keep its existing memory. If it detects an important feature early in a sequence, the forget gate can stay close to 1, letting that information be carried, largely undisturbed, over a long distance — capturing long-range dependencies that a vanilla RNN cannot.

LSTM cell diagram (conceptual):

```
                 +--------------------------+
       i (input  ->|                          |
         gate)     |       c (memory)         |-> OUT (via o, tanh)
       f (forget ->|     c = f*c_prev+i*c_tilde   |
         gate)     |                          |
                 +--------------------------+
                           ^
                  c_tilde (candidate memory)
                           ^
                 IN (x_t, h_(t-1))
       o (output gate) controls exposure of c -> h_t
```

18.4.2 Gated Recurrent Unit (GRU)

Proposed by Cho et al. (2014) as a simpler alternative that still adaptively captures dependencies at different time scales, but **without a separate memory cell** — the GRU only maintains the hidden activation `h_t^j`.

Activation is a linear interpolation between the previous activation and a candidate activation:

```
h_t^j = (1 - z_t^j)*h_(t-1)^j + z_t^j*h_tilde_t^j
```

where the **update gate** `z_t^j` decides how much of the unit's content gets updated:

```
z_t^j = sigma(W_z x_t + U_z h_(t-1))^j
```

This "linear sum between old and new state" idea is conceptually similar to the LSTM's

memory update. However, the GRU has **no mechanism to control how much of its state is exposed** — unlike LSTM's output gate, the GRU always exposes its full state.

Candidate activation:

```
h_tilde_t^j = tanh(W x_t + U(r_t * h_(t-1)))^j
```

($\odot$ = element-wise multiplication). When the **reset gate** `r_t^j` is close to 0, this effectively makes the unit behave as though it is reading the first symbol of a new sequence — i.e., it can "forget" the previously computed state:

```
r_t^j = sigma(W_r x_t + U_r h_(t-1))^j
```

GRU cell diagram (conceptual):

```
                  +--------------------------+
      z (update  ->|   h = (1-z)*h_prev+z*h_tilde   |-> OUT (full state
exposed)
        gate)      |                          |
      r (reset   ->|                          |
        gate)      +--------------------------+
                            ^
                    h_tilde (candidate activation,
                     computed using r to gate h_prev)
                            ^
                       IN (x_t, h_(t-1))
```

18.4.3 LSTM vs. GRU — Similarities and Differences

Shared design principle (the key innovation over vanilla RNNs): both units use an **additive** update — the new state is the *old* state plus/blended with a new candidate, rather than a full overwrite. This gives two major advantages:

1. **Persistence of important features** — once a gate (forget gate in LSTM, update gate in GRU) decides a feature is important, it is not overwritten but maintained as-is across time steps.
2. **Gradient shortcut paths** — the additive structure creates paths that let the error signal backpropagate across many time steps without passing through repeated bounded nonlinearities each time (which is exactly what causes vanishing gradients in vanilla RNNs). When a gate is close to saturation at 1, the gradient can flow through almost unchanged.

Key differences:

Aspect	LSTM	GRU
Separate memory cell	Yes (`c_t`, distinct from `h_t`)	No — only a single hidden state `h_t`
Control over state exposure	Yes — output gate `o_t` explicitly controls how much of memory content is exposed	No — GRU always exposes its full state

Control over incoming info from previous step	Forget gate `f_t` and input gate `i_t` operate *independently* of each other	Reset gate `r_t` controls the previous state's influence on the *candidate*, but the amount of that candidate added to the new state is tied to the same update gate `z_t` (not independently controlled)
Number of gates	3 (input, forget, output)	2 (update, reset)
Parameters	More (extra gate + separate memory cell)	Fewer, for a similar number of "units"

Based on similarities and differences alone, it isn't obvious which unit should perform better in general — this ambiguity is exactly what motivated the empirical study covered next.

18.5 Empirical Comparison: LSTM vs. GRU vs. tanh-RNN

(Based on: Chung, Gulcehre, Cho & Bengio — "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling", 2014, arXiv:1412.3555)

18.5.1 Goal

To fairly compare three types of recurrent units — the traditional **tanh unit**, the **LSTM** unit, and the **GRU** — on the general task of **sequence modelling**, i.e., learning a probability distribution over sequences by maximizing log-likelihood:

```
max_theta  (1/N) * sum_n sum_t  log p(x_t^n | x_1^n, ..., x_(t-1)^n; theta)
```

18.5.2 Datasets and Tasks

Polyphonic music modelling — 3 datasets used (from Boulanger-Lewandowski et al., 2012):

- Nottingham (93-dim binary vectors per symbol)
- JSB Chorales (96-dim)
- MuseData (105-dim)
- Piano-midi (108-dim)

(Each symbol is a high-dimensional binary vector; output units use a logistic sigmoid.)

Speech signal modelling — 2 internal datasets provided by Ubisoft, consisting of raw one-dimensional audio signals:

- **Ubisoft A**: sequence length 500, 7,230 sequences
- **Ubisoft B**: sequence length 8,000, 800 sequences

At each time step, the model looks at 20 consecutive audio samples to predict the following 10 consecutive samples. The output layer is a mixture of 20 Gaussians (since raw audio is continuous-valued, unlike the binary music data).

18.5.3 Model Setup

For each task, three RNN variants were trained — an LSTM-RNN, a GRU-RNN, and a tanh-RNN — each sized so that **all three models have approximately the same**

total number of parameters (models were deliberately kept small to avoid overfitting distracting the comparison):

Task	Unit	# of Units	# of Parameters
Polyphonic music	LSTM	36	≈ 19.8k
Polyphonic music	GRU	46	≈ 20.2k
Polyphonic music	tanh	100	≈ 20.1k
Speech signal	LSTM	195	≈ 169.1k
Speech signal	GRU	227	≈ 168.9k
Speech signal	tanh	400	≈ 168.4k

Training details:

- Optimizer: **RMSProp**
- **Weight noise** added, standard deviation fixed at 0.075 (a regularization technique)
- **Gradient norm clipping**: gradient rescaled to norm 1 whenever it exceeded 1, to prevent exploding gradients
- **Learning rate** selected (out of 10 log-uniform random candidates from U(−12, −6)) to maximize validation performance
- Validation set also used for early stopping

18.5.4 Results

Average negative log-probabilities (lower = better) on train/test sets:

Dataset	Split	tanh	GRU	LSTM
Nottingham	train / test	3.22 / 3.13	2.79 / 3.23	3.08 / 3.20
JSB Chorales	train / test	8.82 / 9.10	6.94 / 8.54	8.15 / 8.67
MuseData	train / test	5.64 / 6.23	5.06 / 5.99	5.18 / 6.23
Piano-midi	train / test	5.64 / 9.03	4.93 / 8.82	6.49 / 9.03
Ubisoft A	train / test	6.29 / 6.44	2.31 / 3.59	1.44 / 2.70
Ubisoft B	train / test	7.61 / 7.62	0.38 / 0.88	0.80 / 1.26

18.5.5 Key Findings

- On the **polyphonic music datasets**, GRU-RNN outperformed both LSTM-RNN and tanh-RNN on almost all datasets (except Nottingham), although all three models performed fairly closely to each other on this task overall.
- On the **Ubisoft speech datasets**, both gated units (GRU and LSTM) clearly and substantially outperformed the plain tanh-RNN. Between the two gated units: **LSTM was best on Ubisoft A**, while **GRU was best on Ubisoft B**.
- Looking at learning curves (both per-iteration and per wall-clock-time): the **GRU-RNN converges faster** than the others on the music datasets, in both number of updates and actual CPU time. On the Ubisoft datasets, the tanh-RNN required far less computation per update, but made very little progress per update and stalled at a much worse loss level — i.e., its computational cheapness did not translate into faster or better overall training.
- **Overall conclusion**: the gated units (LSTM and GRU) clearly outperform the traditional tanh unit — convergence is generally faster, and final solutions are

generally better. However, the comparison **between LSTM and GRU was inconclusive** — which one wins appears to depend heavily on the specific dataset and task. The authors describe these results as *preliminary*, noting that further, more thorough experiments are needed to isolate exactly how each gating mechanism contributes to the improvement (e.g., testing which specific gate in each architecture matters most).

18.6 Practical Implementation: RNN/LSTM for Time-Series Forecasting (Python)

(Based on: Analytics Vidhya — "Time Series Analysis Recurrence Neural Network in Python!")

This is a concrete, worked example of building an LSTM-based RNN to forecast **Google stock price** from historical data — a direct illustration of everything above applied to a financial series.

18.6.1 Step 1 — Load and Prepare the Data

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset_train = pd.read_csv('Google_Stock_Price_Train.csv')
training_set = dataset_train.iloc[:, 1:2].values   # extract the 'Open' price
    column as a 2D array
```

18.6.2 Step 2 — Feature Scaling (Normalization)

Neural networks (especially those using sigmoid/tanh activations, as in LSTM gates) train far more effectively when inputs are scaled into a bounded range. Here, `MinMaxScaler` rescales all values into `[0, 1]`:

```
from sklearn.preprocessing import MinMaxScaler
sc = MinMaxScaler(feature_range=(0, 1))
training_set_scaled = sc.fit_transform(training_set)
```

18.6.3 Step 3 — Build a Windowed (Sliding-Window) Dataset

To train a supervised model on a time series, the raw series must be converted into (input-window → next-value) pairs. Here, a **60-timestep lookback window** is used: each training example consists of 60 consecutive past days, with the target being the very next day's value.

```
X_train = []
y_train = []
for i in range(60, 1258):
    X_train.append(training_set_scaled[i-60:i, 0])   # past 60 days
    y_train.append(training_set_scaled[i, 0])          # next day (the target)
X_train, y_train = np.array(X_train), np.array(y_train)

```

```
# Reshape into the 3D shape Keras LSTM layers expect:
# (samples, timesteps, features)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

This windowing step is the practical realization of the "many-to-one" RNN structure described in Section 2.4: 60 input time steps map to a single predicted output value.

18.6.4 Step 4 — Build a Stacked LSTM Network

```
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout

regressor = Sequential()

# Layer 1
regressor.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape
    [1], 1)))
regressor.add(Dropout(0.2))

# Layer 2
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))

# Layer 3
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))

# Layer 4 (final LSTM layer -- no return_sequences, since only the last output
     is needed)
regressor.add(LSTM(units=50))
regressor.add(Dropout(0.2))

# Output layer -- a single continuous value (the predicted price)
regressor.add(Dense(units=1))
```

Notes on the architecture:

- **4 stacked LSTM layers**, each with 50 units — stacking LSTM layers lets the network learn hierarchical temporal representations (similar in spirit to stacking layers in any deep network).
- `return_sequences=True` on the first three LSTM layers means each of them outputs its *full sequence* of hidden states (one per input timestep), which is required so the *next* LSTM layer also has a sequence to process. The final LSTM layer omits this, since only the last time step's output is needed to feed into the final `Dense` (fully-connected) output layer.
- **Dropout(0.2)** after each LSTM layer randomly "drops" 20% of the units during training as a regularization technique, to reduce overfitting.

18.6.5 Step 5 — Compile and Train

```
regressor.compile(optimizer='adam', loss='mean_squared_error')
```

```
regressor.fit(X_train, y_train, epochs=100, batch_size=32)
```

- **Optimizer**: Adam (an adaptive learning-rate gradient descent variant).
- **Loss**: mean squared error — appropriate since this is a regression problem (predicting a continuous stock price), not classification.
- Training ran for 100 epochs, with the reported training loss steadily decreasing from ≈0.042 at epoch 1 down to ≈0.0014 by epoch 100 — indicating the model was successfully learning the patterns in the training window.

18.6.6 Step 6 — Prepare Test Data and Predict

To predict on the 2017 test period, the model needs the last 60 days *before* the test period begins (since it always needs 60 timesteps of lookback), so training and test data are concatenated before extracting windows:

```
dataset_test = pd.read_csv('Google_Stock_Price_Test.csv')
real_stock_price = dataset_test.iloc[:, 1:2].values

dataset_total = pd.concat((dataset_train['Open'], dataset_test['Open']), axis
    =0)
inputs = dataset_total[len(dataset_total) - len(dataset_test) - 60:].values
inputs = inputs.reshape(-1, 1)
inputs = sc.transform(inputs)   # use the SAME scaler fitted on training data
    -- never refit on test data

X_test = []
for i in range(60, 80):
    X_test.append(inputs[i-60:i, 0])
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

predicted_stock_price = regressor.predict(X_test)
predicted_stock_price = sc.inverse_transform(predicted_stock_price)   # undo
    scaling to get back real price units
```

Important detail: the scaler (`sc`) is *fit only once*, on the training data, and simply *applied* (`.transform`, not `.fit_transform`) to the test data. This avoids information leakage from the test period into the scaling process.

18.6.7 Step 7 — Visualize Results

```
plt.plot(real_stock_price, color='red', label='Real Google Stock Price')
plt.plot(predicted_stock_price, color='blue', label='Predicted Google Stock
    Price')
plt.title('Google Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Google Stock Price')
plt.legend()
plt.show()
```

The resulting plot showed the predicted price (blue) tracking the general **trend** of the real price (red) reasonably well, but — as is typical of this kind of simple LSTM regressor — it lagged behind and smoothed out sharp short-term movements/spikes rather than capturing them precisely. This is a very common limitation worth remembering: plain LSTM point-forecasters tend to under-react to sudden volatility, which is exactly the gap that more advanced architectures (attention, Fusion Transformers, TFT — see Sections 7–9) are designed to close.

18.7 Fusion Transformers — Combining Multiple Data Sources

18.7.1 Core Idea

A **Fusion Transformer** merges different sources or types of data (e.g., text, images, audio, or — in a financial setting — multiple correlated time series and indicators) into a single unified model that can exploit the complementary information each source provides.

Mechanism:

1. Each modality/source is processed **separately** through its own transformer layers/encoder.
2. The separate outputs are then **fused** into a shared representation space — typically via **cross-attention** (letting one modality's representation attend to another's) or simple **concatenation**.
3. This fusion lets the model capture complex interactions and dependencies *between* the different data types, not just within each one individually.

18.7.2 Applications to Time Series (and Finance Specifically)

Application	What it does	Financial example
Multi-Source Data Fusion	Combines disparate but related data streams into one model.	In a manufacturing setting, combining temperature, vibration, and production-metric sensors to predict equipment failure. Financially, combining price series, order-book data, and macroeconomic indicators.
Multi-Scale Temporal Analysis	Integrates information recorded at different time resolutions (e.g., minute-, hour-, and day-level) since different signal components behave differently across scales.	Combining intraday volatility patterns with daily/weekly trend signals.
Enhanced Forecasting	Uses additional context from related series or external time-dependent factors to improve prediction accuracy.	Combining historical stock prices with macroeconomic indicators *and* news sentiment to improve price forecasts.

Anomaly Detection	Combines features from multiple sources/scales to better flag anomalies than any single source could alone.	Using both historical trading data *and* news-sentiment analysis together to identify unusual/abnormal market behaviour — i.e., catching volatility events that are driven by external news rather than pure price action alone.

This is the conceptual bridge to why "handling volatility" in finance benefits from fusion-style architectures: volatility is rarely explained by price history alone — it is often driven by a mix of scheduled events (which can be *known in advance*, like holidays or earnings dates), external news/sentiment, and cross-asset or cross-market effects. A model that can only look at one univariate price series is blind to all of that context; a fusion/attention-based model is explicitly built to combine it.

18.8 The Temporal Fusion Transformer (TFT)

(Reference: Lim et al., Google Research — "Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting," arXiv:1912.09363)

18.8.1 Motivation

Real-world multi-horizon forecasting problems typically involve a **complex mix of input types**, without prior knowledge of how those inputs interact with the target:

- **Static (time-invariant) covariates** — attributes that never change across the series.
- **Known future inputs** — variables whose future values are already known ahead of time (e.g., calendar effects, scheduled holidays).
- **Exogenous time series observed only in the past** — variables that affect the target but whose future values are unknown at forecast time.

TFT is an **attention-based architecture** that combines high-performance multi-horizon forecasting with **interpretable insights** into the temporal dynamics driving the forecasts. It does this by:

- Using **recurrent layers** (LSTM-style) for **local** processing — short-range, sequential patterns.
- Using **interpretable self-attention layers** for **long-term dependencies** — capturing relationships across distant points in time.
- Using specialized **variable selection** components to automatically identify which input features are actually relevant.
- Using a series of **gating layers** to suppress components that turn out to be unnecessary for a given prediction.

18.8.2 Types of Inputs TFT Can Handle

Covariate type	Description	Example (retail/finance-style)

Static covariates	Do not change over the life of the series.	Country, store, product identity; or in finance, an asset's sector/exchange.
Time-varying, known in advance	Their future values are already known at forecast time.	Day of week, month, whether a date is a holiday.
Time-varying, unknown in advance	Only observed in the past; future values are not known ahead of time.	Stock level of a product (only known historically); analogously, realized trading volume or volatility itself.

18.8.3 Why This Architecture Suits Volatile Financial Series

Combining a **local recurrent component** (good at picking up short-term momentum/autocorrelation, similar to the LSTM/GRU units from Sections 4–6) with **long-range interpretable attention** (good at connecting a current volatility spike to a distant but relevant past event or a known future date like an earnings release) gives TFT strengths that neither a pure RNN nor a pure attention model has alone. The **variable selection and gating mechanism** additionally means the model can automatically learn to down-weight noisy/irrelevant inputs during calm periods and up-weight the inputs that matter during turbulent ones — directly useful for "handling volatility," where the relevant driving variables can change over time.

18.8.4 Architecture (Conceptual Summary)

At a high level, TFT is composed of:

1. An **input/feature-transformation stage** that embeds static, known, and unknown covariates.
2. **Variable selection networks**, which learn instance-wise weights for how much each input variable matters at each time step, giving interpretability into *which* covariates drive a given forecast.
3. An **LSTM-based encoder-decoder** ("sequence-to-sequence layer") for local temporal processing — directly analogous to the encoder/decoder structure from Section 1.
4. A **static enrichment layer**, which lets static covariates modulate temporal features.
5. A **temporal self-attention block**, which learns long-range dependencies and produces interpretable attention weights (showing which past time points mattered most for a given forecast).
6. **Gated Residual Networks (GRNs)** used throughout, which apply gating (similar in spirit to the LSTM/GRU gates from Section 4) to skip/suppress components that aren't useful for a particular input.
7. A final **quantile output layer**, producing not just a point forecast but a distribution of plausible outcomes (multiple quantiles) at each future time step — directly useful for expressing forecast *uncertainty*, which is central to volatility-aware forecasting.

18.9 TFT Implementation Walkthrough (PyTorch / pytorch-forecasting)

(Based on a Kaggle notebook applying TFT to the "Tabular Playground Series – Sep 2022" competition: forecasting daily book sales (`num_sold`) across combinations of country, store, and product.)

This section shows what an end-to-end TFT project looks like in code using the `pytorch-forecasting` library, which wraps the low-level PyTorch/TFT implementation behind an interface similar to a `scikit-learn` estimator — no manual `Dataset` class or attention-layer coding is required.

18.9.1 Setup and Imports

```
import pytorch_lightning as pl
from pytorch_lightning.callbacks import EarlyStopping, LearningRateMonitor
from pytorch_lightning.loggers import TensorBoardLogger
import torch

from pytorch_forecasting import Baseline, TemporalFusionTransformer,
    TimeSeriesDataSet
from pytorch_forecasting.data import GroupNormalizer, NaNLabelEncoder
from pytorch_forecasting.metrics import SMAPE, PoissonLoss, QuantileLoss
from pytorch_forecasting.models.temporal_fusion_transformer.tuning import
    optimize_hyperparameters
```

Key libraries: `pytorch_lightning` (training loop and trainer abstraction), `pytorch_forecasting` (the TFT implementation and specialized time-series `Dataset` class), and the `holidays` package (used to engineer holiday-related features).

18.9.2 Data and Exploratory Data Analysis (EDA)

The dataset: daily sales counts (`num_sold`) for combinations of **country, store, and product**, with `train` and `test` splits concatenated together for consistent preprocessing. Sanity checks confirm the data's structure:

```
data = pd.concat([train, test], axis=0, ignore_index=True)

# Every (country, store, product, date) combination is unique
assert len(data.drop_duplicates(['country', 'store', 'product', 'date'])) ==
    len(data)
# Every (country, store, product) group has exactly one row per date
assert len(data.drop_duplicates(['country', 'store', 'product'])) == len(data)
    //data['date'].nunique()
```

EDA performed:

- Count of missing values and unique values per column (comparing train vs. test — `num_sold` is naturally absent from test, since it's the prediction target).
- **KDE (density) plots** of `num_sold`, separately grouped by country, by store, and by product — to see how the sales distribution differs across each grouping.

- **Time series line plots** of total `num_sold` over time, again broken out by country/store/product, to visually inspect trend and seasonality.
- A **correlation heatmap** across all individual (country × store × product) time series, to see which series move together (masking out weak correlations below 0.7 for readability).

18.9.3 Feature Engineering

```
# A running integer time index required by TimeSeriesDataSet
data = data.merge((data[['date']].drop_duplicates(ignore_index=True)
                   .rename_axis('time_idx')).reset_index(), on=['date'])

# Calendar features (as categoricals -- pytorch-forecasting requires string categories)
data["day_of_week"] = data.date.dt.dayofweek.astype(str).astype("category")
data["week_of_year"] = data.date.dt.weekofyear.astype(str).astype("category")
data["month"] = data.date.dt.month.astype(str).astype("category")

# A log-transformed target (helps stabilize variance for a sales-count target)
data["log_num_sold"] = np.log(data.num_sold + 1e-8)

# Group-level average volume features -- extra unknown-in-advance covariates
data["avg_volume_by_country"] = data.groupby(["time_idx", "country"], observed=True).num_sold.transform("mean")
data["avg_volume_by_store"] = data.groupby(["time_idx", "store"], observed=True).num_sold.transform("mean")
data["avg_volume_by_product"] = data.groupby(["time_idx", "product"], observed=True).num_sold.transform("mean")
```

Holiday features — not just "is today a holiday," but also holiday *leads and lags* (is tomorrow/the day after a holiday, was yesterday/the day before a holiday). This is useful because sales patterns are often affected in the days surrounding a holiday, not just on the holiday itself:

```
unique_dates_country['is_holiday'] = unique_dates_country.apply(
    lambda x: x.date in holidays.country_holidays(x.country), axis=1).astype('category')
unique_dates_country['is_holiday_lead_1'] = ...   # is (date + 1 day) a holiday
unique_dates_country['is_holiday_lead_2'] = ...   # is (date + 2 days) a holiday
unique_dates_country['is_holiday_lag_1']  = ...   # was (date - 1 day) a holiday
unique_dates_country['is_holiday_lag_2']  = ...   # was (date - 2 days) a holiday
```

These leads/lags are a good general pattern for financial applications too — e.g., flagging the days around a scheduled macro announcement or earnings release, not just the announcement day itself.

18.9.4 Building the TimeSeriesDataSet

This is the core object that tells TFT how to interpret every column in the data:

```
max_prediction_length = 365          # forecast horizon: predict a full year ahead
max_encoder_length = train.date.nunique()   # how much history the encoder can look back on
training_cutoff = train["time_idx"].max() - max_prediction_length   # reserve the last year for validation

training = TimeSeriesDataSet(
    train[lambda x: x.time_idx <= training_cutoff].drop('row_id', axis=1),
    time_idx="time_idx",
    target="num_sold",
    group_ids=["country", "store", "product"],        # defines what an individual "series" is
    min_encoder_length=max_prediction_length,
    max_encoder_length=max_encoder_length,
    max_prediction_length=max_prediction_length,
    static_categoricals=["country", "store", "product"],
    time_varying_known_categoricals=[
        "month", "week_of_year", "day_of_week", "is_holiday",
        "is_holiday_lead_1", "is_holiday_lead_2",
        "is_holiday_lag_1", "is_holiday_lag_2"
    ],
    time_varying_known_reals=["time_idx"],
    time_varying_unknown_categoricals=[],
    time_varying_unknown_reals=[
        "num_sold", "log_num_sold", "avg_volume_by_country",
        "avg_volume_by_store", "avg_volume_by_product"
    ],
    target_normalizer=GroupNormalizer(
        groups=["country", "store", "product"], transformation="softplus"
    ),
    categorical_encoders={'week_of_year': NaNLabelEncoder(add_nan=True)},
    lags={'num_sold': [7, 30, 365]},     # explicit lagged versions of the target as extra features
    add_relative_time_idx=True,
    add_target_scales=True,
    add_encoder_length=True,
)

# Validation set: predict the held-out final `max_prediction_length` points for each series
validation = TimeSeriesDataSet.from_dataset(training, train, predict=True, stop_randomization=True)

train_dataloader = training.to_dataloader(train=True, batch_size=128, num_workers=0)
val_dataloader = validation.to_dataloader(train=False, batch_size=1280, num_workers=0)
```

Parameter-by-parameter meaning (this maps directly back to the covariate taxonomy in Section 8.2):

- `group_ids` — defines what constitutes one individual time series (here, each unique country-store-product combination is its own series).
- `static_categoricals` — the "static covariates" from Section 8.2 (country, store, product identity — never change within a series).
- `time_varying_known_categoricals` / `time_varying_known_reals` — the "known in advance" covariates (calendar features, holidays, and the time index itself — all knowable ahead of the forecast date).
- `time_varying_unknown_reals` — the "unknown in advance" covariates, including the target itself (`num_sold`) and derived features like the group-average volumes, since their *future* values are not available at prediction time.
- `target_normalizer=GroupNormalizer(...,transformation="softplus")` — normalizes the target separately *within each group* (since scale differs a lot across country/store/product) and applies a softplus transform (useful for strictly non-negative targets like sales counts).
- `lags` — explicitly provides lagged copies of the target (7, 30, 365 days back) as additional input features, giving the model direct access to weekly/monthly/yearly seasonality signals.
- `add_relative_time_idx`, `add_target_scales`, `add_encoder_length` — convenience flags that add extra useful derived features automatically (position within the encoder window, the per-group target scale, and the actual encoder length used).

18.9.5 Baseline Comparison

Before training the TFT, it's good practice to check how a trivial "naive" model performs, to have a reference point:

```
actuals = torch.cat([y for x, (y, weight) in iter(val_dataloader)])
baseline_predictions = Baseline().predict(val_dataloader)   # naive forecast (
    e.g., "last observed value")

sm = SMAPE()
print(sm.loss(actuals, baseline_predictions).mean(axis=1).median().item())
```

`SMAPE` (Symmetric Mean Absolute Percentage Error) is the evaluation metric used throughout — a scale-independent error metric well suited to comparing forecast accuracy across series with very different magnitudes (exactly the situation here, with many country-store-product combinations of different sizes).

18.9.6 Model Instantiation and Training

```
tft = TemporalFusionTransformer.from_dataset(
    training,
    learning_rate=0.03,
    lstm_layers=2,
    hidden_size=16,
    attention_head_size=2,
```

```
    dropout=0.2,
    hidden_continuous_size=8,
    output_size=1,          # (7 quantiles by default when using QuantileLoss;
     here set for SMAPE's single output)
    loss=SMAPE(),
    reduce_on_plateau_patience=4,
)

early_stop_callback = EarlyStopping(monitor="train_loss", min_delta=1e-2,
    patience=30, mode="min")
lr_logger = LearningRateMonitor()
logger = TensorBoardLogger("lightning_logs")

trainer = pl.Trainer(
    max_epochs=120,
    accelerator="gpu", devices=1,
    gradient_clip_val=0.25,          # same gradient-clipping idea from
    Section 3.2, applied here too
    callbacks=[lr_logger, early_stop_callback],
    logger=logger,
)

trainer.fit(tft, train_dataloaders=train_dataloader, val_dataloaders=
    val_dataloader)
```

Notice `lstm_layers=2` — confirming that TFT's "local processing" component (Section 8.4, point 3) is literally implemented as a stacked LSTM, directly built on everything covered in Sections 2 and 4. `gradient_clip_val=0.25` is the same gradient-clipping safeguard discussed in Section 3.2 for controlling exploding gradients.

Optional: automated hyperparameter tuning via Optuna, searching over ranges for gradient clipping, hidden sizes, attention heads, learning rate, and dropout:

```
study = optimize_hyperparameters(
    train_dataloader, val_dataloader,
    n_trials=50, max_epochs=50,
    gradient_clip_val_range=(0.01, 1.0),
    hidden_size_range=(8, 128),
    hidden_continuous_size_range=(8, 128),
    attention_head_size_range=(1, 4),
    learning_rate_range=(0.001, 0.1),
    dropout_range=(0.1, 0.3),
)
```

18.9.7 Evaluation and Interpretation

```
best_tft = TemporalFusionTransformer.load_from_checkpoint(trainer.
    checkpoint_callback.best_model_path)

predictions = best_tft.predict(val_dataloader, mode="prediction")
```

```
raw_predictions, x = best_tft.predict(val_dataloader, mode="raw", return_x=
    True)

sm = SMAPE()
print(sm.loss(actuals, predictions).mean(axis=1).median().item())

# Plot individual predicted-vs-actual forecast paths
for idx in range(raw_predictions.prediction.shape[0]):
    best_tft.plot_prediction(x, raw_predictions, idx=idx, add_loss_to_title=
    True)

# Plot prediction accuracy broken down by each input feature/variable --
# this is where TFT's built-in *interpretability* shows up directly
predictions_vs_actuals = best_tft.calculate_prediction_actual_by_variable(x,
    predictions)
for feature in predictions_vs_actuals['support'].keys():
    best_tft.plot_prediction_actual_by_variable(predictions_vs_actuals, name=
    feature)
```

The last step is worth highlighting: `calculate_prediction_actual_by_variable` / `plot_prediction_actual_by_variable` are exactly what makes TFT distinct from a plain LSTM forecaster (Section 6) — instead of only outputting a prediction, it lets you inspect **how prediction accuracy varies across each input feature**, giving direct insight into which covariates the model is relying on. This interpretability is precisely what the TFT paper's abstract emphasizes as a key contribution (Section 8.1).

18.10 Summary — Tying It Back to "Handling Volatility" in Finance

Model	Core mechanism	Strength	Limitation
Vanilla (tanh) RNN	Full state overwrite each step	Simple, cheap	Vanishing gradients → can't learn long-range dependencies; empirically the weakest of the three units tested
LSTM	Additive memory cell + input/forget/output gates	Strong long-range memory; controlled state exposure	More parameters/gates than GRU; empirically close to GRU, sometimes better on longer/harder sequences (e.g., Ubisoft A)
GRU	Additive hidden state + update/reset gates, no separate cell	Fewer parameters, often faster convergence	No independent control of state exposure; empirically close to LSTM, sometimes better (e.g., music data, Ubisoft B)

Seq2Seq (encoder-decoder, + attention)	Compress input into context vector, decode into output sequence, optionally attend back over inputs	Handles variable-length input/output; supports forecasting, anomaly detection, and pattern recognition uniformly	A single fixed context vector is a bottleneck without attention
Fusion Transformer	Separate per-modality encoders fused via cross-attention or concatenation	Combines heterogeneous data sources and time scales	More complex to design and train
Temporal Fusion Transformer (TFT)	LSTM encoder-decoder (local) + interpretable self-attention (long-range) + variable selection/gating + quantile outputs	Handles static, known-future, and unknown covariates together; interpretable; produces uncertainty (quantiles), not just point forecasts	Most complex architecture; more moving parts to tune

The throughline for volatile financial series: plain RNNs struggle with long dependencies (vanishing gradients) → gated units (LSTM/GRU) fix that with additive, gated memory → Seq2Seq lets these units handle full forecasting/anomaly-detection/pattern-recognition tasks with variable-length inputs and outputs → Fusion Transformers extend this by combining multiple relevant data sources (price, sentiment, macro indicators, multi-scale time resolutions) rather than a single univariate series → TFT packages all of this into one architecture, additionally providing **interpretability** (which variables/time points drove a given forecast) and **quantile-based uncertainty estimates** — both of which are exactly what's needed to reason about, and "handle," volatility rather than just predict a single expected value and ignore the surrounding uncertainty.

18.11 References / Further Reading (mentioned in course materials)

- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.* arXiv:1412.3555.
- Lim, B. et al. (Google Research). *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting.* arXiv:1912.09363.
- Analytics Vidhya — *Time Series Analysis Recurrence Neural Network in Python!* (Amruta, updated Oct 2024).
- Kaggle notebook — TFT applied to the "Tabular Playground Series – Sep 2022" competition using `pytorch-forecasting`.
- `pytorch-forecasting` official documentation / Stallion tutorial (referenced as the template the notebook's implementation was based on).